

3X-UI Panel Kullanım Kılavuzu

العربية · English · Español · فارسی · Bahasa Indonesia · 日本語 · Português · Русский · Türkçe · Українська · Tiếng Việt · 简体中文 · 繁體中文

3X-UI sürümü: 3.4.0. Bu kılavuz söz konusu sürüm esas alınarak hazırlanmış olup yalnızca bu sürüm için geçerlidir. 3.4.0'ın 3.3.1'e göre değişikliklerinin özeti [«3.4.0'daki Yenilikler»](#) bölümündedir.

3X-UI web panelinin (Xray-core yönetimi) ayrıntılı Türkçe kılavuzu: özellikler, yapılandırma ve işletim; arayüzdeki her alan ve geçiş açıklamasıyla birlikte.

Alan adları ve etiketler panel arayüzüne karşılık gelmektedir. *inbound* / *outbound* sözcükleri çevrilmez.

İçindekiler

- 3.4.0'daki Yenilikler
- 1. Giriş, Gereksinimler ve Kurulum
 - 1.1. 3X-UI Nedir
 - 1.2. Desteklenen İşletim Sistemleri ve Mimariler
 - 1.3. Kurulum Yöntemleri
 - 1.4. İlk Başlatma ve Varsayılan Kimlik Bilgileri
 - 1.5. Dosya Konumları
 - 1.6. `x-ui` Yönetim Komutu (Script Menüsü)
 - 1.7. `x-ui` Alt Komutları (İnteraktif Menü Olmadan)
 - 1.8. SQLite → PostgreSQL Geçişi
- 2. Panel Girişi ve Erişim Güvenliği
 - 2.1. Giriş Formu
 - 2.2. İki Faktörlü Kimlik Doğrulama (2FA / TOTP)
 - 2.3. Giriş Denemesi Sınırlaması (login limiter / kaba kuvvet koruması)
 - 2.4. Yönetici Kullanıcı Adı ve Parola Değişikliği
 - 2.5. Gizli Yol (URI yolu / webBasePath) ve Panel Portu
 - 2.6. Oturum Süresi (zaman aşımı)
 - 2.7. LDAP (Senkronizasyon ve Kimlik Doğrulama)
- 3. Genel Bakış / Gösterge Paneli
 - 3.1. Veri Toplama Genel İlkeleri
 - 3.2. İşlemci (CPU)
 - 3.3. Bellek (RAM)
 - 3.4. Takas Alanı (Swap)
 - 3.5. Disk (Storage)
 - 3.6. Sistem Çalışma Süresi (Uptime)
 - 3.7. Sistem Yüğü (Load average)
 - 3.8. Ağ: Hız ve Toplam Trafik
 - 3.9. Sunucu IP Adresleri
 - 3.10. TCP/UDP Bağlantıları
 - 3.11. Xray Durumu ve Süreç Yönetimi
 - 3.12. Panel Güncellemesi (3X-UI)
 - 3.13. Coğrafi Dosyaların Güncellenmesi (GeoIP / GeoSite)
 - 3.14. Veritabanı Yedekleme ve Geri Yükleme
 - 3.15. Ek Arayüz Öğeleri
- 4. Inbounds: Oluşturma ve Genel Parametreler
 - 4.1. Formun Genel Alanları
 - 4.2. Sniffing (Koklama)
 - 4.3. Allocate (Port Dağıtım Stratejisi)
 - 4.4. External Proxy (Dış Proxy)
 - 4.5. Fallbacks (Fallback'ler)
 - 4.6. Periyodik Trafik Sıfırlama
 - 4.7. Gelen JSON (gelişmiş)
 - 4.8. Inbound İşlemleri: QR / Edit / Reset / Delete ve İstatistikler

- 5. Protokoller
 - 5.1. Desteklenen Protokollerin Listesi
 - 5.2. Hangi Protokoller TLS / REALITY / Transport Destekler
 - 5.3. VLESS
 - 5.4. VMess
 - 5.5. Trojan
 - 5.6. Shadowsocks
 - 5.7. Dokodemo-door / Tunnel (Şeffaf Yönlendirici)
 - 5.8. SOCKS / HTTP (mixed protokolü)
 - 5.9. WireGuard (inbound)
 - 5.10. Hysteria (varsayılan v2)
 - 5.11. MTPProto (Telegram için proxy)
 - 5.12. Protokol Seçimi İçin Hızlı Başvuru Tablosu
- 6. Transport (Stream Settings)
 - 6.1. Ağ Aktarım Seçimi
 - 6.2. RAW / TCP (tcpSettings)
 - 6.3. mKCP (kcpSettings)
 - 6.4. WebSocket (wsSettings)
 - 6.5. gRPC (grpcSettings)
 - 6.6. HTTPUpgrade (httpupgradeSettings)
 - 6.7. XHTTP / SplitHTTP (xhttpSettings)
 - 6.8. Hysteria Transport (hysteriaSettings)
 - 6.9. İlgili Parametreler
- 7. Bağlantı Güvenliği: TLS, XTLS ve REALITY
 - 7.1. Fark Nedir: TLS vs XTLS vs REALITY
 - 7.2. «Yok» Modu (none)
 - 7.3. TLS Modu
 - 7.4. REALITY Modu
 - 7.5. Yapılandırma İçin Pratik Öneriler
- 8. İstemciler
 - 8.1. İstemci Alanları
 - 8.2. Inbound'a Bağlama
 - 8.3. İstemci İşlemleri
 - 8.4. Toplu İşlemler
 - 8.5. Arama, Filtreler ve Sıralama
 - 8.6. Simgeler ve Durumlar
- 9. İstemci Grupları
 - 9.1. İstemci Grubu Nedir ve Neden Gereklidir
 - 9.2. Grubun İstemcilerle, Inbound'lar, Düğümler ve Protokollerle İlişkisi
 - 9.3. Grup Rehberi ve «Boş» Gruplar
 - 9.4. Grup Alanları ve Sütunları
 - 9.5. Grup Oluşturma
 - 9.6. Grubu Yeniden Adlandırma
 - 9.7. Gruba İstemci Ekleme
 - 9.8. İstemcileri Gruptan Çıkarma (İstemcileri Silmeden)
 - 9.9. Grup Trafikini Sıfırlama

- 9.10. Grubu Silme ve Grup İstemcilerini Silme
- 9.11. «İstemciler» Sayfasıyla Bağlantı
- 9.12. API Endpoint'lerinin Özeti
- 9.13. Gruba Göre Trafik
- 10. Abonelikler (Subscription)
 - 10.1. subld Nedir ve Bağlantı Nasıl Oluşturulur
 - 10.2. Abonelik Sunucusu Ayarları
 - 10.3. Çıktı Biçimleri
 - 10.4. Abonelik Bilgi Sayfası ve QR Kodları
 - 10.5. Özel Abonelik Sayfası Şablonları
- 11. Xray: Yönlendirme, outbounds, DNS ve Uzantılar
 - 11.1. Düzenleyici Yapısı: Sekmeler/Modlar
 - 11.2. Genel Ayarlar (General)
 - 11.3. Yönlendirme Kuralları (routing)
 - 11.4. Outbounds (Giden Bağlantılar)
 - 11.5. Dengeleyiciler (Balancers)
 - 11.6. DNS
 - 11.7. Fake DNS
 - 11.8. WireGuard / WARP / NordVPN
 - 11.9. Ters Proxy ve TUN
 - 11.10. Günlükler ve İstatistikler (Stats, metrics)
 - 11.11. Kaydetme, Yeniden Başlatma ve Otomatik Dönüşümler
 - 11.12. Abonelikten Outbound (otomatik güncellemeyle)
 - 11.13. WARP'ta IP Rotasyonu
- 12. Düğümler (çok-panel, master/slave)
 - 12.1. Liste Başındaki Özet
 - 12.2. Düğüm Ekleme ve Düzenleme
 - 12.3. TLS Doğrulaması (https düğümleri için)
 - 12.4. Her Düğüm İçin Gösterilenler
 - 12.5. Düğüm İşlemleri
 - 12.6. Metrik Geçmişi
 - 12.7. Inbound'lar ve İstemcilerin Senkronizasyonu
 - 12.8. Düğüm Zincirleri (alt düğümler / geçiş düğümleri)
 - 12.9. Düğümler: 3.3.0'daki Yenilikler
- 13. Panel Ayarları
 - 13.1. Kaydetme ve Paneli Yeniden Başlatma
 - 13.2. Genel Ayarlar («Panel» sekmesi / *General*)
 - 13.3. Panel Erişimi: IP, Port, Yol, Alan Adı, Sertifika
 - 13.4. Oturum, Panel Proxy ve Güvenilir Proxy'ler («Proxy ve Sunucu» sekmesi / *Proxy and Server*)
 - 13.5. Telegram Botu («Telegram Bot» sekmesi / *Telegram Bot*)
 - 13.6. Tarih ve Saat («Tarih ve Saat» sekmesi / *Date and Time*)
 - 13.7. Dış Trafik ve Xray Davranışı («Dış Trafik» sekmesi / *External Traffic*)
 - 13.8. Diğer: Xray Yapılandırma Şablonu ve Doğrulama URL'si
 - 13.9. Yönetici Hesabı ve API Token'ları
 - 13.10. 3.3.0'daki API Değişiklikleri (entegrasyonlar için önemli)

- 14. Telegram Botu
 - 14.1. Botu Etkinleştirme ve Yapılandırma
 - 14.2. Ana Menü ve Düğmeler
 - 14.3. Bot Komutları
 - 14.4. İstemci Yönetimi (yalnızca yönetici)
 - 14.5. Bildirimler ve Raporlar
 - 14.6. Yedek ve Günlükler
 - 14.7. Çalışma Özellikleri
 - 15. Coğrafi Veritabanları (geoip / geosite ve özel)
 - 15.1. geoip.dat ve geosite.dat Nedir
 - 15.2. Standart Coğrafi Dosyalar ve Güncelleme
 - 15.3. Kullanıcı Tanımlı Coğrafi Kaynaklar (Custom GeoSite / GeoIP)
 - 15.4. Doğrulama ve Kısıtlamalar
 - 15.5. Panel Başlatılırken Otomatik Doğrulama
 - 15.6. Yönlendirme Kurallarında Coğrafi Veritabanlarının Kullanımı
 - 16. İşletim: Yedekler, Günlükler, Güncelleme, CLI
 - 16.1. Veritabanı Yedekleme ve Geri Yükleme
 - 16.2. Günlükleri Görüntüleme
 - 16.3. Xray Günlük Düzeyi ve Yapılandırması
 - 16.4. Xray Yönetimi: Durdurma ve Yeniden Başlatma
 - 16.5. Paneli Yeniden Başlatma ve Güncelleme
 - 16.6. Periyodik Görevler (cron)
 - 16.7. Konsol Menüsü ve CLI (`x-ui`)
 - 16.8. Paneli Kaldırma
 - 16.9. `x-ui migrateDB` Komutu
-

3.4.0'daki Yenilikler

Bu bölüm, **3.4.0** sürümünün 3.3.1'e göre panel kullanıcısı tarafından görülebilen değişikliklerini kılavuz bölümlerine göre gruplandırarak kısaca listeler. Her özelliğin ayrıntıları aşağıdaki ilgili bölümde verilmektedir.

3. Genel Bakış / Gösterge Paneli

- **Sistem metrik geçmişi: yeni 12s/24s/48s toplama aralıkları** – Sistem metrik geçmişi penceresinde ortalama alma aralıklarına 12s, 24s ve 48s değerleri eklendi; artık grafikler (CPU, RAM, trafik, paketler, bağlantılar, disk, çevrimiçi, yük) iki güne kadar olan süre için görüntülenebilmektedir.

4. Inbounds: Oluşturma ve Genel Parametreler

- **Inbound: ayrılmış Xray API portuyla çakışma uyarısı** – Bir inbound oluşturulurken veya değiştirilirken panel artık dahili Xray API'sinin ayrılmış portunu (varsayılan olarak 127.0.0.1:62789) kullanmaya izin vermiyor: loopback üzerinde bu porttaki yerel TCP inbound'u, port çakışması hatasıyla reddediliyor. Düğümlerde (nodes) kısıtlama geçerli değil – onların kendi Xray'leri var.
- **Tunnel/TPProxy: security anahtarı olmadan streamSettings kabulü** – streamSettings içinde security bloğu bulunmayan tunnel/TPProxy türündeki inbound'lar artık doğrulama hatası olmadan açılıp kaydedilebiliyor.
- **Inbounds: listede Speed (anlık hız) sütunu** – Inbound listesine her inbound için anlık trafik hızını (yükleme/indirme) gösteren Speed sütunu eklendi.

5. Protokoller

- **Shadowsocks-2022: farklı anahtar boyutuna sahip yöntem geçişte istemci PSK yeniden oluşturma** – Shadowsocks-2022 için: şifreleme yöntemi farklı anahtar boyutuna sahip bir yöntem (ör. aes-256 ile aes-128 arasında) değiştirildiğinde, istemci PSK'ları inbound kaydedilirken yeni uzunluğa göre otomatik olarak yeniden oluşturuluyor. Sonuç: etkilenen istemcilerin aboneliği yeniden alması gerekiyor (eski bağlantılar çalışmayı durduracak).
- **WireGuard: workers alanı kaldırıldı** – WireGuard formlarından (inbound ve outbound) workers alanı kaldırıldı: xray-core v26.6.22 artık bu alanı kullanmıyor. Önceden kaydedilmiş yapılandırmalar değişmeden çalışmaya devam ediyor – alan yalnızca yok sayılıyor.
- **VLESS+XHTTP: bağlantılar ve aboneliklerde xtls-rprx-vision akışı** – XHTTP üzerinden VLESS için xtls-rprx-vision akışı artık bağlantılara ve aboneliklere (XHTTP+REALITY ve Clash/Mihomo biçimi dahil) doğru şekilde ekleniyor. Önceden panel akışı kaydediyordu ancak istemciler vision olmadan yapılandırma alıyordu.

6. Transport (Stream Settings)

- **XHTTP: sessionID alanlarının yeniden adlandırılması + Session ID Table / Length** – XHTTP transportunda oturum alanları yeniden adlandırıldı: Session ID Placement ve Session ID Key (önceki adıyla – Session Placement / Session Key). İki yeni parametre eklendi. Session ID Table – oturum tanımlayıcıları oluşturmak için kullanılan karakter kümesi: önceden tanımlanmış bir küme (ALPHABET, Base62, hex, number vb.) seçilebilir ya da isteğe bağlı bir ASCII dizesi girilebilir; boş bırakılırsa xray-core varsayılanı kullanılır. Session ID Length – oluşturulan tanımlayıcıların uzunluğu veya aralığı (ör. 8-16); yalnızca Session ID Table

belirtildiğinde dikkate alınır, minimum değer 0'dan büyük olmalıdır. Önceden kaydedilmiş inbound'lar otomatik olarak taşınır.

- **Inbound: CDN/röle arkasındaki gerçek IP tespiti için Real client IP ön ayarı** – Soket ayarlarında (sockopt) Real client IP seçeneği eklendi – trafik CDN veya röle üzerinden geldiğinde ziyaretçinin gerçek IP'sini tespit etmek için ön ayar (aksi takdirde aracının adresi kaydedilir). Üç seçenek mevcut: Off / direct (işlem yok), Cloudflare CDN (X-Forwarded-For'a güven) ve L4 relay / Spectrum (PROXY) (PROXY protokol başlığını kabul et). Ön ayarlar birbirini dışlar ve seçilen transportun bunları desteklemediği durumlarda uyarı gösterir. Bu alanlar hiçbir zaman abonelikler aracılığıyla istemcilere gönderilmez.
- **gRPC: Trusted X-Forwarded-For başlığı artık dikkate alınıyor** – Trusted X-Forwarded-For başlığı artık gRPC transportunda da dikkate alınıyor (önceden yalnızca WebSocket, HTTPUpgrade ve XHTTP). gRPC inbound'ları için panel artık desteklenmeyen başlık uyarısını göstermiyor.

7. Bağlantı Güvenliği: TLS,XTLS ve REALITY

- **TLS: yeni Verify Peer Cert By Name, Curve Preferences, Master Key Log, ECH Sockopt alanları** – Verify Peer Cert By Name – istemcinin SNI yerine sunucu sertifikasını doğruladığı adlar (virgülle ayrılmış); kaldırılan allowInsecure'un modern yerini alıyor, bağlantılara ve aboneliklere aktarılıyor, sunucuya yazılmıyor. Curve Preferences – TLS anahtar değişimi için eğri kısıtlaması ve sırası (ör. X25519MLKEM768, X25519); boş bırakılırsa varsayılan değerler kullanılır. Master Key Log – Wireshark'ta hata ayıklama için TLS anahtarlarını kaydetme yolu (SSLKEYLOGFILE biçimi); üretimde boş bırakın. ECH Sockopt – ECH yapılandırması almak için soket parametreleri (dialerProxy, Domain Strategy, TCP Fast Open, Multipath TCP).
- **REALITY: fallback hız sınırlaması (Limit Fallback) ve Master Key Log** – Her yön (Upload ve Download) için belirtilir: After Bytes – sınırlamaya başlamadan önce tam hızda geçirilecek bayt miktarı (0 – ilk bayttan itibaren sınırla); Bytes Per Sec – sondaların sunucuyu ücretsiz kanal olarak kullanmasını engellemek için fallback trafik hız tavanı (0 – sınır yok, yönü devre dışı bırakır); Burst Bytes Per Sec – kısa süreli artışlar için rezerv. Ayrıca hata ayıklama için Master Key Log alanı da eklendi (SSLKEYLOGFILE dosyasının yolu).
- **TLS: inbound sertifikasından ve SNI üzerinden Pinned Peer Cert SHA-256 doldurma düğmeleri** – Pinned Peer Cert SHA-256 alanının yanında artık eski rastgele karma düğmesi yerine iki otomatik doldurma düğmesi var. İlki inbound'un kendi sertifikasının SHA-256'sını ekler. İkincisi, belirtilen SNI'ye TLS bağlantısı kurarak canlı sunucu sertifikasının karmasını alır (serverName dolu olmalıdır). Alınan karmalar virgülle ayrılarak alana eklenir ve istemcide sertifika sabitleme için bağlantılara dahil edilir.
- **TLS: yeni inbound sertifikaları için OCSP-stapling varsayılan olarak kapalı** – Yeni inbound'lar için OCSP stapling varsayılan olarak kapalıdır (aralık 0). Bu, OCSP yanıtlayıcısı olmayan sertifikalar (ör. Let's Encrypt) için xray günlüklerindeki hataları ortadan kaldırır. Alan hâlâ mevcut – stapling'i destekleyen sertifikalar için etkinleştirilebilir.
- **REALITY: dest alanıyla uyumluluk (target takma adı)** – REALITY inbound'u dest alanıyla (panelin eski sürümleriyle, API aracılığıyla veya harici araçlarla) oluşturulduysa artık doğru şekilde yükleniyor: dest değeri Target alanına aktarılıyor. Önceden Target boş kalıyor ve yeniden kaydetme REALITY'yi bozuyordu.

8. İstemciler

- **İstemci düzenleyicisinde «Links» sekmesi (harici bağlantılar ve abonelikler)** – Bu sekmede **Add External Link** düğmesiyle üçüncü taraf paylaşım bağlantıları (vless://, vmess://, trojan://, ss://, hysteria2://, wireguard://) ve **Add External Subscription** düğmesiyle uzak abonelik URL'leri eklenir. Bunların tamamı bu istemcinin abonelik çıktısına (raw, JSON ve Clash biçimleri) karıştırılır: bağlantılar

olduğu gibi eklenir, uzak abonelikler ise panel tarafından periyodik olarak indirilir ve yapılandırılmaları kendi yapılandırmalarıyla birleştirilir.

- **«IP Limiti» alanı artık Fail2ban olmadan devre dışı kalıyor** – IP Limiti alanı yalnızca yüklü ve etkin Fail2ban ile çalışıyor. Fail2ban kurulu değilse (veya Windows sistemi ya da sunucuda özellik devre dışıysa) istemci düzenleyicisinin alanı kilitlenir ve üzerine gelindiğinde `x-ui` bash menüsünden Fail2ban kurulmasını öneren ipucu gösterilir. Panel güncellendiğinde, Fail2ban olmayan sunuculardaki istemcilerin kayıtlı IP limiti sıfırlanır; zaten orada uygulanmıyordu.
- **Bağlantısız istemcileri silme, istemci dışa ve içe aktarma** – İstemciler sayfasındaki **Diğer** menüsüne üç işlem eklendi. **İstemcileri Dışa Aktar**, kopyalama ve indirme düğmeleriyle (`clients-export.json`) tüm istemcilerin JSON listesini (`{client, inboundIds}` biçiminde) gösterir. **İstemcileri İçe Aktar** aynı JSON'ı kabul eder: bağlantıları olan istemciler yeniden oluşturulup inbound'lara bağlanır, bağlantısız istemciler ayrı kayıtlar olarak geri yüklenir, mevcut e-posta adresleri üzerine yazılmaz (atlandı olarak işaretlenir). **Bağlantısız İstemcileri Sil**, hiçbir inbound'a bağlı olmayan tüm istemcileri trafiği, IP günlüğü ve harici bağlantılarıyla birlikte siler; işlem geri alınamaz.
- **İstemci IP günlüğü: bağlantı zamanı ve düğüm adı** – İstemci IP günlüğünde (IP Limiti alanının yanındaki görüntüleme düğmesi ve «İstemci Bilgileri» kartında) her kayıt artık IP'nin yanı sıra son erişim zamanını ve bağlantının hangi düğüm üzerinden tespit edildiğini gösteren düğüm etiketini (`@ düğüm_adı`) içeriyor – çok panelli yapılandırmada istemcinin hangi düğüm üzerinden bağlandığı görülebilir.
- **Tek istemci düzenleyicisinde grup etiketini sıfırlama** – Artık tek bir istemcinin düzenleyicisinde **Grup** alanı temizlenip kaydedilirse grup etiketi doğru şekilde kaldırılıyor – önceden istemci yeniden kaydedilene kadar eski grup altında görünmeye devam edebiliyordu.
- **İstemci listesi otomatik güncelleniyor (arka plan yoklaması)** – İstemci listesi artık otomatik güncelleniyor: panel her birkaç saniyede bir güncel sayfayı çekiyor, bu nedenle yeni bağlanan istemciler ve değişen sıralama düzeni manuel yenileme olmadan görünüyor.

10. Abonelikler (Subscription)

- **Yönetilen Hosts: abonelik bağlantılarını host'lara göre geçersiz kılma** – 3.4.0 sürümünde Hosts bölümü (kenar çubuğu menüsü ögesi) eklendi. Her inbound için, inbound'un kendi adresi, portu ve TLS parametreleri yerine abonelik bağlantılarına yerleştirilen bir veya daha fazla Host endpoint'i tanımlanabilir – bu, CDN veya röle üzerinden trafik dağıtmak için kullanışlıdır. Bir host için şunlar belirtilir: Remark ve açıklama, inbound'a bağlantı, Address (boş – inbound adresini devral) ve Port (0 – inbound portunu devral), Security parametreleri (same/tls/none/reality), ayrıca Host header, Path, Mux, Sockopt, Final Mask, abonelik biçimlerinden dışlama (raw/json/clash) ve Clash/mihomo parametreleri. Host'lar inbound içinde sıralanır ve toplu işlemleri destekler.
- **Remark Template, remark modeli oluşturucusunun yerini aldı; {{VAR}} değişkenleri** – Eski profil adı oluşturucu (Inbound/Email/External Proxy seçimi ve ayırıcı) «Remark Template» alanıyla değiştirildi. Bu alanda istediğiniz ad biçimini yazabilir, düğmeyle değişken ekleyebilirsiniz: istemci tanımlaması ({{EMAIL}}, {{INBOUND}}, {{HOST}}, {{ID}}, {{SUB_ID}}, {{COMMENT}}, {{TELEGRAM_ID}}, trafik ({{TRAFFIC_USED}}, {{TRAFFIC_LEFT}}, {{TRAFFIC_TOTAL}}, {{UP}}, {{DOWN}}, {{USAGE_PERCENTAGE}}) ve süre/durum ({{DAYS_LEFT}}, {{TIME_LEFT}}, {{EXPIRE_DATE}}, {{JALALI_EXPIRE_DATE}}, {{STATUS}}, {{STATUS_EMOJI}}). Değişkenler abonelik oluşturulurken her istemci için ayrı ayrı değiştiriliyor, önlizeme mevcut. «|» sembolüyle ayrılan segmentler limitsiz değerler için otomatik gizlenir; kullanım ve süre bilgisi yalnızca istemcinin ilk bağlantısında gösterilir. Alan boş bırakılırsa eski remark modeli kullanılır.
- **İstemci başına harici bağlantılar ve uzak abonelikler (Links sekmesi)** – Burada tek bir istemci için üçüncü taraf paylaşım bağlantıları (Add External Link) ve harici abonelik adresleri (Add External Subscription)

eklenebilir – bunlar istemcinin kendi aboneliğine dahil edilir (RAW, JSON ve Clash biçimleri). Harici abonelikler panel tarafından indirilir ve istemci yapılandırmalarıyla birleştirilir. Bu, kendi inbound'larınızın üzerine istemciye ek sunucular sunmak için kullanışlıdır.

- **Happ: istemcide sunucu ayarlarını gizleme (Hide server settings)** – Abonelik ayarlarının Happ sekmesine «Hide server settings» geçişi eklendi. Etkinleştirildiğinde Happ istemcisinde sunucu parametrelerini görüntüleme ve değiştirme imkânı gizlenir. Seçenek yalnızca Happ istemcisi için geçerlidir.
- **Düğüm adı artık abonelikteki profil adına eklenmiyor** – Düğüm (Node) adı artık abonelikteki profil adına eklenmez. İstemci uygulamasında yalnızca yönetici tarafından belirlenen inbound remarki gösterilir; «@düğüm-adı» gibi dahili bir sonek olmadan.
- **Remark model etiketi Other → External Proxy olarak yeniden adlandırıldı (ardından şablonla değiştirildi)** – Ayrıca belgelemeye gerek yok: remark model ögesi «Other»'ın «External Proxy» olarak yeniden adlandırılması, model oluşturucunun UI'dan kaldırıldığı yeni Remark Template alanına geçişle birleştirildi.
- **Abonelik bağlantısı doğruluğu: SS2022, Shadowrocket, SIP002 obfs, Clash'te XHTTP** – Oluşturulan abonelik bağlantılarının uyumluluğu iyileştirildi: SS2022 kodlaması, Shadowrocket derin bağlantısı, SIP002 biçiminde Shadowsocks+obfs çıktısı (obfs-local eklentisi) ve Clash/Mihomo aboneliklerinde tam XHTTP alan kümesi düzeltildi. Ayrı bir ayar gerekmez – bağlantılar istemciler tarafından daha doğru tanınır.
- **Abonelik remark modeli: «Other» ögesi «External Proxy» olarak yeniden adlandırıldı** – Abonelik ayarlarındaki remark modelinde «Other» ögesi «External Proxy» olarak yeniden adlandırıldı (kaynak – harici proxy remarki). Davranış değişmedi, mevcut ayarlar uyumlu olmaya devam ediyor.
- **Abonelikler: chip'lere tıklayarak remark değişkeni seçme (Remark variable picker)** – Remark Template alanının yanında gruplandırılmış değişken chip'leri mevcut: {{VAR}} değişkenine tıklayınca şablona ekleniyor, üzerine gelince açıklama gösteriliyor. Remark ve host adı alanlarında tek parantez içindeki basitleştirilmiş yazım da kabul ediliyor – {DATA_LEFT}, {EXPIRE_DATE}, {PROTOCOL}, {TRANSPORT} vb.; panel bunu otomatik olarak dahili {...} biçimine dönüştürür.

11. Xray: Yönlendirme, outbounds, DNS ve Uzantılar

- **Yönlendirme ve Outbounds ayrı kenar çubuğu menü öğelerine taşındı** – Bu sürümden itibaren «Giden Bağlantılar» (Outbounds) ve «Yönlendirme» (Routing) ayrı kenar çubuğu menü öğelerine taşındı (hemen «Hosts»'un altında), her birinin kendi adresi var – /outbound ve /routing . Önceden yönlendirme «Xray Yapılandırmaları» alt menüsü içinde açılıyor, outbounds ise Xray sayfasının sekmesi olarak görünüyordu. «Xray Yapılandırmaları» alt menüsünde artık yalnızca şunlar kalıyor: Temel, Dengeleyici, DNS ve Gelişmiş Şablon. /outbound ve /routing doğrudan bağlantıları ve sayfa yenileme beklendiği gibi çalışıyor.
- **Yönlendirme kuralları geçiş anahtarıyla etkinleştirilip devre dışı bırakılabilir** – Tek bir yönlendirme kuralı artık silinmeden bir geçiş anahtarıyla geçici olarak devre dışı bırakılabilir. Kurallar tablosunda «Etkinleştir» geçiş anahtarıyla sütun var, kural formunda «Etkinleştir» alanı da geçiş anahtarı. Devre dışı bırakılan kural nihai Xray yapılandırmasına dahil edilmez. İstatistik hizmeti kuralı (api) devre dışı bırakılamaz – geçiş anahtarı kilitli.
- **Yönlendirme kuralları ve outbounds dışa aktarımı önizleme modal penceresinde açılıyor** – Yönlendirme kuralları ve giden bağlantıların «Dışa Aktar» düğmesi artık dosyayı hemen indirmiyor; «Kopyala» ve «İndir» düğmeleriyle JSON önizlemesi içeren bir modal pencere açıyor. «Yönlendirme» sekmesinde «İçe Aktar» ve «Dışa Aktar» «daha fazla» açılır menüsünde toplanmış (Outbounds sekmesindeki gibi).
- **Tüm giden bağlantıları test etme artık abonelik outbounds'larını da kontrol ediyor; direct/dns artık test edilmiyor** – «Giden Bağlantılar» sayfasındaki «Tümünü Test Et» düğmesi artık aboneliklerden alınan outbounds'ları da kontrol ediyor («aboneliklerden» tablosu) – bu satırlar da sonuçla vurgulanıyor. Bununla

birlikte `freedom` («direct») ve `dns` outbounds'ları artık hiçbir modda test edilmiyor (bunlar proxy değil): test düğmesi bunlarda kullanılamıyor ve «Tümünü Test Et» bunları atlıyor.

- **FinalMask: tekli Length/Delay yerine bölüm bazlı Lengths/Delays dizileri** – Fragment maskesinde (FinalMask) tekli Length ve Delay alanları Lengths ve Delays listesiyle değiştirildi: her segment için ayrı uzunluk aralığı (ör. 100-200) ve ms cinsinden gecikme (ör. 10-20 veya 0) belirtilebilir. Satırlar eklenip silinebilir; önceden kaydedilmiş değerler otomatik olarak aktarılır.
- **Loopback outbound: Sniffing bloğu eklendi** – loopback türündeki outbound'a, inbound'dakiyle aynı parametrelere sahip bir Sniffing bloğu eklendi: etkinleştirme, destOverride, Metadata Only, Route Only ve dışlanan alan adları listesi.
- **Hysteria2 / Salamander: Gecko modu (packetSize) ve Realm maskesi için TLS** – UDP maskesindeki (FinalMask) Hysteria2 özellikleri genişletildi. Salamander maskesine Mode seçici eklendi: Gecko modu, paket uzunluğu analizinden korumak için Min/Max boyutu (1 ile 2048 arasında, varsayılan 512-1200) olan alanlarla rastgele paket dolgusu ekliyor. Realm maskesine isteğe bağlı TLS Config bloğu eklendi: Server Name (SNI), ALPN (h3/h2/http/1.1), Fingerprint ve Allow Insecure.
- **Share bağlantısından outbound içe aktarma, xmux ayarlarını koruyor** – Share bağlantısından outbound içe aktarılırken artık **xmux** çoklayıcısı (XHTTP) ayarları korunuyor: önceden sessizce kayboluyordu. İçe aktarmanın ardından değerler XMUX alt formuna yerleştiriliyor.
- **Abonelik outbounds'larının etiketleri ASCII olmayan karakterleri (Kiril) koruyor** – Aboneliklerden alınan outbounds'ların etiketleri artık ASCII olmayan karakterleri (ör. Kiril harfleri) koruyor ve yalnızca rakamlara indirgenmek yerine okunabilir kalıyor.

12. Düğümler (çok-panel, master/slave)

- **Düğümler: yeni TLS doğrulama modu – Mutual TLS (istemci sertifikası)** – Düğüm formunda TLS doğrulama modu artık dört seçenek sunuyor: Verify (sistem CA), Pin (SHA-256 ile sertifika sabitleme), Skip (doğrulama yok) ve yeni Mutual TLS (istemci sertifikası). Mutual TLS modunda panel, kendi CA'sı tarafından verilen istemci sertifikasıyla kendini düğüme ek olarak doğruluyor; böyle bir düğüm için API token'ı isteğe bağlı hale geliyor. mTLS etkinleştirmek için: düğümde Mutual TLS modunu ayarlayın, Node mTLS bölümünden yönetim panelinin CA'sını kopyalayın, bunu düğümde güvenilir üst CA olarak yapılandırın ve düğümü yeniden başlatın.
- **Düğümler: «Node mTLS» bölümü – panel CA kopyalama ve güvenilir üst CA** – Düğümler sayfasına, paneller arasında karşılıklı TLS kimlik doğrulamasını yapılandırmak için Node mTLS bölümü eklendi. «Bu panelin CA'sını kopyala» düğmesi, panelin kök sertifikasını panoya kopyalar – bunun, panelin istemci sertifikasını doğrulayacak yönetilen düğümlere iletilmesi gerekir. «Güvenilir üst CA» alanı, panelin kendisi bir düğüm olduğunda kullanılır: yönetim panelinin CA'sını buraya yapıştırın, istemci sertifikasını zorunlu kılın ve paneli yeniden başlatın. Karşılıklı TLS isteğe bağlı olarak etkinleştirilir; alanlar boşsa düğümler eski şemayla – yalnızca API token'ıyla çalışır.
- **Panelin düğüme giden bağlantısının yönlendirilmesi (Connection outbound)** – Düğüm formuna «Connection outbound» (giden bağlantı) alanı eklendi. Bir Xray outbound etiketi seçilirse panelin bu düğümün API'sine giden trafiği belirtilen outbound üzerinden akacak (panel, çalışma yapılandırmasına loopback üzerinde bir köprü inbound otomatik olarak ekler ve bunu yeniden başlatma olmadan uygular). Boş değer = doğrudan bağlantı. Listede etiketler «Giden Bağlantılar» ve «Dengeleyiciler» olarak gruplandırılmış; blackhole giden bağlantılar gizlenmiş.
- **Düğümler: panel→düğüm trafiğinin seçili outbound üzerinden yönlendirilmesi («Connection outbound»)** – Düğüm formuna «Connection outbound» alanı eklendi: panelin düğüme yönelik bağlantı trafiğini seçili Xray outbound üzerinden yönlendirmeyi sağlıyor (normal outbounds ve dengeleyiciler mevcut). Panel,

loopback-bridge inbound'u çalışma yapılandırmasına otomatik olarak ekler ve değişiklikleri yeniden başlatma olmadan uygular. Doğrudan bağlantı için alanı boş bırakın.

- **Düğümmler: düğüme inbound'lar bağlıken silme engelleniyor** – Bir düğüm, yalnızca tüm inbound'lar ondan kaldırıldıktan sonra silinebilir. Düğüme hâlâ en az bir inbound bağlıysa panel silme işlemi hatayla reddeder – önce bu inbound'ları ayırın veya silin, ardından düğümü silin.
- **Düğümmler: düğüm sayfasında, düğüme yerleştirilmiş inbound'ların anlık hızı gösteriliyor** – Düğümmler sayfasında, düğüme yerleştirilmiş inbound'lar için artık çevrimiçi istemciler, sayaçlar ve anlık aktarım hızı gösteriliyor. «Tamamlandı» (ended) chip'i yalnızca süresi dolmuş ve trafiği tükenmiş istemcileri sayıyor (devre dışı bırakılanlar artık dahil değil).

14. Telegram Botu

- **Bildirimler: Telegram ve Email (SMTP) kanallarıyla yeni olay veri yolu** – İki dağıtım kanalıyla – Telegram ve Email – olay bildirim sistemi eklendi. Bildirimler sekmesinde olaylar kartlarla gruplandırılmış: Outbound (düşme/kurtarma), Xray Core (beklenmedik sonlanma), Nodes (düğüm erişilemez/kurtarıldı), System (yapılandırılabilir % eşikli yüksek CPU ve bellek yükü), Security (giriş denemesi). Her grubun ana geçiş anahtarı ve seçilen olay sayacı var. Etkinleştirilen olaylar Telegram ve Email için ayrı ayrı yapılandırılıyor; varsayılan olarak «giriş denemesi» ve «yüksek CPU yükü» etkin.
- **Bildirimler: yeni Email/SMTP kanalı ve SMTP sunucu ayarları** – SMTP üzerinden e-posta ile yeni bildirim kanalı eklendi. SMTP ayarları sekmesinde şunlar belirtilir: e-posta bildirimlerini etkinleştirme, SMTP host ve port (varsayılan 587), kullanıcı adı, parola (gizli saklanır), alıcı listesi (virgülle ayrılmış) ve şifreleme türü – none, STARTTLS (varsayılan) veya TLS. «Test e-postası gönder» düğmesi bağlantıyı kontrol eder ve hangi aşamada (bağlantı, kimlik doğrulama, gönderme) hata oluştuğunu gösterir. İkinci sekmede e-posta bildirimlerinin gönderileceği olaylar seçilir.
- **Bildirimler: yüksek bellek yükü uyarısı (RAM eşiği)** – Yüksek CPU yükü uyarılarına yüksek RAM yükü uyarısı eklendi. «System» olaylar grubunda kendi eşik alanıyla (varsayılan %80) «Memory high (%)» eklendi; panel dakikada bir RAM yükünü kontrol eder ve eşik aşıldığında seçili kanallara bildirim gönderir.

15. Coğrafi Veritabanları (geoip / geosite ve özel)

- **Coğrafi veritabanı güncelleme: her dosya için durum ve değişiklik yoksa yeniden başlatmayı atlama** – Coğrafi veritabanı güncellemesi (IR ve RU kümeleri dahil geoip/geosite) artık her dosya için durum gösteriyor: güncellendi, zaten güncel veya yükleme hatası. Xray yeniden başlatması (dolayısıyla aktif bağlantıların kesilmesi) yalnızca en az bir dosya gerçekten güncellendiyse gerçekleşiyor; değişiklik yoksa panel yeniden başlatılmıyor. Aynı davranış x-ui update-all-geofiles komutunda da geçerli.

16. İşletim: Yedekler, Günlükler, Güncelleme, CLI

- **İstemci IP limiti yalnızca fail2ban kuruluysen çalışır; aksi halde alan kilitlenir** – İstemci başına IP sayısı kısıtlaması artık yalnızca sunucuda fail2ban yüklüyse geçerli. Kurulu değilse istemci formundaki ve toplu ekleme işlemindeki «IP Limit» alanı açıklayıcı ipucuyla (Windows'ta ayrı bir mesajla) kullanılamaz hale gelir; bu tür sunucularda önceden ayarlanmış limitler otomatik olarak sıfırlanır, zaten uygulanmıyordu. fail2ban engeli artık TCP ve UDP için de geçerli.
- **Panel kurulum ve güncellemelerinde fail2ban otomatik olarak kuruluyor** – Normal bir sunucuya panel kurulumu ve güncellemesinde fail2ban artık otomatik olarak kuruluyor ve yapılandırılıyor (önceden yalnızca Docker'da oluyordu), böylece IP limiti özelliği kutudan çıkar çıkmaz çalışıyor. Davranışı

XUI_ENABLE_FAIL2BAN ortam değişkeni kontrol eder: değişken ayarlanmamış veya true ise yapılandırma yapılır. Manuel kurulum x-ui setup-fail2ban komutuyla mevcut; fail2ban hatası kurulum veya güncellemeyi durdurmaz.

- **XUI_PORT değişkeniyle panel portunun geçersiz kılınması** – XUI_PORT ortam değişkeni eklendi; web panelinin portunu yalnızca geçerli sürecin çalışması sırasında ayarlar, veritabanındaki kayıtlı webPort değerini değiştirmez. 1 ile 65535 arasındaki değerler geçerlidir; boş, yanlış veya aralık dışı değer yok sayılır (webPort kullanılır), günlüğe uyarı yazılır. Bridge ağı Docker kullanılırken yayımlanan kapsayıcı portu XUI_PORT ile eşleşmeli, ör. XUI_PORT=8080 ve ports: «8080:8080».
- **CLI: -webCert/-webCertKey bayrakları artık setting alt komutunda da geçerli** – CLI'da -webCert ve -webCertKey bayrakları artık x-ui setting alt komutunda da çalışıyor (önceden sessizce yok sayılıyor ve panel HTTPS'siz kalıyordu). Bunları belirterek ayrı bir cert alt komutu çağırmadan web paneli sertifika ve anahtar yolunu hemen ayarlayabilirsiniz.
- **Veritabanı yedek dosyası adı sunucu adresine göre oluşturuluyor** – Veritabanı yedek dosyaları artık sabit x-ui.db / x-ui.dump yerine sunucu adresine göre adlandırılıyor. Tarayıcıdan indirilirken ad, panel adres çubuğundaki adresten alınıyor; yoksa yapılandırılmış web alan adından, o da yoksa genel IP'den (önce IPv4, ardından IPv6). Böylece farklı sunucuların yedekleri kolayca ayırt edilebilir. Uzantı SQLite için .db, PostgreSQL için .dump olarak kalıyor.
- **Yalnızca IPv6 olan host'larda kurulum ve güncelleme desteği** – Kurulum ve güncelleme scriptleri artık yalnızca IPv6 olan sunucularda doğru çalışıyor: sürüm ve yardımcı dosya indirme işlemi artık IPv4 kullanımını zorlamıyor; bu nedenle panel, IPv4 adresi olmayan bir host'a kurulup güncellenebilir.

1. Giriş, Gereksinimler ve Kurulum

1.1. 3X-UI Nedir

3X-UI, [Xray-core](#) sunucuları için açık kaynaklı bir web yönetim panelidir. Panel; tekli bir VPS'ten birden fazla düğümden (node) oluşan dağıtık yapılandırmalara kadar geniş bir yelpazede proxy ve VPN protokollerini dağıtmak, yapılandırmak ve izlemek için birleşik, çok dilli bir web arayüzü sunar.

3X-UI, orijinal X-UI projesinin geliştirilmiş bir çatalıdır. Orijinaline kıyasla daha fazla protokol desteği, artırılmış kararlılık, istemci başına trafik muhasebesi ve çok sayıda kullanışlı özellik eklenmiştir.

Temel özellikler:

- **Farklı protokollerde inbound** – VLESS, VMess, Trojan, Shadowsocks, WireGuard, Hysteria2, HTTP, SOCKS (Mixed), Dokodemo-door / Tunnel, TUN ve **MTPProto** (Telegram proxy'si, 3.3.0 sürümünde eklendi).
- **Modern aktarımlar ve şifreleme** – TCP (Raw), mKCP, WebSocket, gRPC, HTTPUpgrade ve XHTTP; TLS, XTLS ve REALITY ile korunur.
- **Fallback** – Xray'deki fallback mekanizması aracılığıyla tek bir portta birden fazla protokole hizmet verme (örneğin 443'te VLESS ve Trojan).
- **İstemci başına yönetim** – trafik kotaları, son kullanma tarihleri, IP limitleri, "çevrimiçi" durum görüntüleme, tek tıkla davet bağlantıları, QR kodları ve abonelikler.
- **Trafik istatistikleri** – her inbound, istemci ve outbound için; sıfırlama seçeneğiyle.
- **Çoklu düğüm (node) desteği** – tek bir panelden birden fazla sunucuyu yönetme ve ölçeklendirme.
- **Outbound ve yönlendirme** – WARP, NordVPN, özel yönlendirme kuralları, yük dengeleyiciler, proxy zincirleri.
- **Birden fazla çıktı formatına sahip yerleşik abonelik sunucusu.**
- **Uzaktan izleme ve yönetim için Telegram botu.**
- **Yerleşik Swagger belgeleriyle REST API.**
- **Esnek depolama** – SQLite (varsayılan) veya PostgreSQL.
- **13 arayüz dili**, koyu ve açık temalar.
- **Fail2ban entegrasyonu** ile istemci başına IP limitlerinin uygulanması.

Önemli: Proje yalnızca kişisel kullanım için tasarlanmıştır. Yasadışı amaçlarla veya üretim ortamlarında kullanılması önerilmez.

1.2. Desteklenen İşletim Sistemleri ve Mimariler

İşletim Sistemleri

Kurulum betiği, dağıtımı `/etc/os-release` (veya `/usr/lib/os-release`) dosyasındaki `ID` alanına göre belirler. Resmi olarak desteklenenler:

Ubuntu, Debian, Armbian, Fedora, CentOS, RHEL, AlmaLinux, Rocky Linux, Oracle Linux, Amazon Linux, Virtuozzo, Arch, Manjaro, Parch, openSUSE (Tumbleweed / Leap), Alpine ve Windows.

Alpine tabanlı sistemlerde OpenRC servisi (`rc-service` / `rc-update`) kullanılır; diğerlerinde systemd kullanılır. CentOS 7'de paketler `yum` ile, daha yeni sürümlerde ise `dnf` ile kurulur. Dağıtım tanınmazsa betik varsayılan olarak `apt-get` paket yöneticisini kullanmaya çalışır.

İşlemci Mimarileri

Mimari, `uname -m` çıktısına göre belirlenir ve desteklenen değerlerden birine eşlenir:

uname -m Değeri	3X-UI Mimarisi
x86_64 , x64 , amd64	amd64
i*86 , x86	386
armv8* , arm64 , aarch64	arm64
armv7* , arm	armv7
armv6*	armv6
armv5*	armv5
s390x	s390x

Mimari bu listede yer almıyorsa betik «Unsupported CPU architecture!» mesajını gösterir ve kurulumu sonlandırır.

Temel Bağımlılıklar

Panel kurulmadan önce betik, temel paket kümesini otomatik olarak yükler (paket adları dağıtıma göre farklılık gösterir): `cron` / `cronie` / `dcron` , `curl` , `tar` , `tzdata` / `timezone` , `socat` , `ca-certificates` , `openssl` .

1.3. Kurulum Yöntemleri

Yöntem 1. Kurulum Betiği (Önerilen)

Kurulum, root olarak tek bir komutla gerçekleştirilir:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

Betik root yetkisi gerektirmektedir: root olmayan bir kullanıcı tarafından çalıştırıldığında «Please run this script with root privilege» mesajı gösterilir ve hatayla sonlanır.

Kurucunun adım adım yaptıkları:

1. İşletim sistemi ve mimariyi belirler.
2. Temel bağımlılıkları yükler.
3. `x-ui-linux-<arch>.tar.gz` sürüm arşivini indirir ve `/usr/local/x-ui` dizinine açar.
4. `x-ui.sh` yönetim betiğini indirir ve `/usr/bin/x-ui` komutu olarak yükler.
5. `/var/log/x-ui` log dizinini oluşturur.

6. İlk yapılandırmayı başlatır: veritabanı seçimi, kimlik bilgileri oluşturma, port seçimi, isteğe bağlı SSL yapılandırması.
7. Otomatik başlatma servisini yükler ve başlatır (systemd birimi `x-ui.service` veya Alpine için OpenRC init betiği).

Kurulum sırasında veritabanı seçimi. Kurucu şu seçenekleri sunar:

- 1) SQLite (varsayılan; 500'den az istemci için önerilir) – tek bir `/etc/x-ui/x-ui.db` dosyası, yapılandırma gerektirmez.
- 2) PostgreSQL (çok sayıda istemci veya birden fazla node için önerilir). PostgreSQL yerel olarak kurulabilir (`xui` adıyla ayrılmış bir kullanıcı ve veritabanı oluşturulur) ya da mevcut bir sunucunun DSN'si belirtilebilir. Bağlantı parametreleri, dağıtımına bağlı olarak servis ortam dosyasına (`/etc/default/x-ui`, `/etc/conf.d/x-ui` veya `/etc/sysconfig/x-ui`) `XUI_DB_TYPE` ve `XUI_DB_DSN` değişkenleri olarak yazılır.

Örnek: PostgreSQL parametrelerinin servis ortam dosyasına yazılması. PostgreSQL seçilip DSN belirtildikten sonra kurucu ortam dosyasına aşağıdakine benzer satırlar ekler:

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

Burada `xui` kullanıcı adı ve veritabanı adıdır; `127.0.0.1:5432` sunucunun adresi ve portudur; `sslmode=disable` yerel bağlantılar için uygundur (uzak sunucu için genellikle `require` kullanılır).

Belirli (eski) bir sürümün kurulumu. Bir sürüm etiketi açıkça belirtilebilir – kurucu ilgili sürümü indirir:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

Bu şekilde kurulabilecek en düşük sürüm `v2.3.5` 'tir; daha eski bir sürüm belirtilirse «Please use a newer version (at least v2.3.5)» mesajı görüntülenir.

Yöntem 2. Docker

Varsayılan SQLite veritabanıyla başlatma:

```
docker compose up -d
```

Yerleşik PostgreSQL servisiyle başlatmak için `docker-compose.yml` dosyasındaki `XUI_DB_*` satırlarını yorumdan çıkarın ve profile başlatın:

```
docker compose --profile postgres up -d
```

İmaj, IP limitlerini istemci başına uygulamak için Fail2ban içerir (varsayılan olarak etkin). Fail2ban, ihlalcileri iptables aracılığıyla engeller; bu da `NET_ADMIN` yetkisini gerektirir. `docker-compose.yml` dosyasında bu yetki `cap_add` ile zaten sağlanmıştır. `docker run` ile manuel başlatmada bu yetkiler elle eklenmezse engellemeler yalnızca loglanır, uygulanmaz:

Örnek: tam docker run komutu. Panel portunun yönlendirildiği, ağ yetkilerinin verildiği ve veritabanı için kalıcı birimin tanımlandığı minimal seçenek:

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

/etc/x-ui birimi, konteyner yeniden başlatılırken x-ui.db dosyasını korur; aksi takdirde ayarlar ve hesaplar kaybolur.

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

Docker'da panel, konteynerin ana işlemidir: otomatik başlatma, konteyner içindeki bir servis tarafından değil, konteynerin yeniden başlatma politikasıyla (örneğin `restart: unless-stopped`) yönetilir.

1.4. İlk Çalıştırma ve Varsayılan Kimlik Bilgileri

İlk kurulumda (varsayılan kimlik bilgileri henüz kullanılıyorken) kurucu kullanıcı adı, parola, web yolu ve port için **rastgele değerler oluşturur**:

Parametre	Kurulumda nasıl oluşturulur	Not
Kullanıcı adı (Username)	10 karakterli rastgele dize	otomatik oluşturulur
Parola (Password)	10 karakterli rastgele dize	otomatik oluşturulur
Panel web yolu (WebBasePath)	18 karakterli rastgele dize	paneli kök URL'den keşfedilmekten korur
Panel portu (Port)	varsayılan olarak 1024–62000 aralığında rastgele port; istenirse elle belirtilebilir	webPort için "fabrika" değeri 2053 'tür ancak kurucu bunu geçersiz kılar

Kurulumun sonunda betik özet bilgileri gösterir: kullanıcı adı, parola, port, web yolu, API token ve şu biçimde hazır bir giriş bağlantısı (Access URL):

```
https://<alan-adı-veya-IP>:<port>/<web-yolu>
```

SSL sertifikası yapılandırılmamışsa bağlantı `http://` ile başlar ve betik, SSL yapılandırması için bir uyarı gösterir (menü maddesi 19).

Kimlik bilgilerinin değiştirilmesi zorunludur. Giriş adı ve parola rastgele oluşturulduğundan **kurulumdan hemen sonra kaydedilmeleri** gerekir. Bu bilgiler «Reset Username & Password» menü maddesi (aşağıya bakın) veya web arayüzündeki panel ayarlarından her zaman değiştirilebilir. Sıfırlamadan sonra betik şu

hatırlatmayı yapar: «Please use the new login username and password to access the X-UI panel. Also remember them!».

Kurulumdan sonra yönetim menüsünü açmak için `x-ui` komutu kullanılır (bkz. Bölüm 1.6).

1.5. Dosya Konumları

Yol	Açıklama
<code>/usr/local/x-ui/</code>	panel kurulum dizini (<code>x-ui</code> ikili dosyası, <code>x-ui.sh</code> betiği)
<code>/usr/local/x-ui/bin/xray-linux-<arch></code>	Xray-core ikili dosyası (armv5/armv6/armv7'de <code>xray-linux-arm</code> olarak yeniden adlandırılır)
<code>/usr/bin/x-ui</code>	yönetim betiği (<code>x-ui</code> komutu)
<code>/etc/x-ui/x-ui.db</code>	SQLite veritabanı dosyası (varsayılan)
<code>/var/log/x-ui/</code>	panel log dizini
<code>/etc/systemd/system/x-ui.service</code>	systemd servis birimi (Alpine hariç)
<code>/etc/init.d/x-ui</code>	OpenRC init betiği (yalnızca Alpine)
<code>/etc/default/x-ui</code> · <code>/etc/conf.d/x-ui</code> · <code>/etc/sysconfig/x-ui</code>	servis ortam değişkenleri dosyası (yol dağıtımına göre değişir); <code>XUI_DB_TYPE</code> / <code>XUI_DB_DSN</code> buraya yazılır

Veritabanı dizini `XUI_DB_FOLDER` ortam değişkeniyle geçersiz kılınabilir (varsayılan `/etc/x-ui`); Xray ikili dosyaları dizini ise `XUI_BIN_FOLDER` değişkeniyle (varsayılan olarak panel dizinine göre `bin`). Veritabanı dosyasının adı `x-ui.db` 'dir.

Örnek: veritabanını ayrı bir diske taşıma. `x-ui.db` dosyasını `/etc/x-ui` yerine örneğin bağlı bir `/data` diskinde saklamak için servis ortam dosyasına değişkeni ekleyin ve paneli yeniden başlatın:

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

Veritabanının tam yolu `/data/x-ui/x-ui.db` olacaktır.

Temel Ortam Değişkenleri

Değişken	Açıklama	Varsayılan
<code>XUI_DB_TYPE</code>	veritabanı arka ucu: <code>sqlite</code> veya <code>postgres</code>	<code>sqlite</code>
<code>XUI_DB_DSN</code>	PostgreSQL bağlantı dizisi (<code>XUI_DB_TYPE=postgres</code> olduğunda)	–
<code>XUI_DB_FOLDER</code>	SQLite veritabanı dosyasının dizini	<code>/etc/x-ui</code>
<code>XUI_INIT_WEB_BASE_PATH</code>	web panelinin başlangıç URI yolu (yalnızca ilk başlatmada)	<code>/</code>
<code>XUI_DB_MAX_OPEN_CONNS</code>	maksimum açık bağlantı sayısı (PostgreSQL bağlantı havuzu)	–

Değişken	Açıklama	Varsayılan
XUI_DB_MAX_IDLE_CONNS	maksimum boştaki bağlantı sayısı (PostgreSQL bağlantı havuzu)	–
XUI_ENABLE_FAIL2BAN	Fail2ban aracılığıyla IP limitlerini etkinleştir	true
XUI_LOG_LEVEL	loglama düzeyi (debug , info , warning , error)	info
XUI_DEBUG	hata ayıklama modu	false

Örnek: ayrıntılı loglamayı geçici olarak etkinleştirme. Bir sorunu teşhis etmek için log düzeyini debug olarak yükseltin ve servisi yeniden başlatın:

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log      # hata ayıklama logunu görüntüle
```

Teşhisin ardından logların şişmemesi için değeri info olarak geri alın.

Ortam değişkeni aracılığıyla web panelinin başlangıç yolu. XUI_INIT_WEB_BASE_PATH değişkeni, ilk ayar başlatılmasında web panelinin URI yolunu (webBasePath) belirler. Bu, Docker veya systemd aracılığıyla dağıtımda panel giriş yolunu baştan sabitlemek için kullanışlıdır. Değer otomatik olarak normalleştirilir – baştaki ve sondaki eğik çizgiler gerekirse eklenir; boş veya yalnızca boşluklardan oluşan bir değer göz ardı edilir (bu durumda varsayılan / yolu uygulanır). Değişken **yalnızca ilk başlatmayı** etkiler: ayarlar zaten oluşturulmuşsa yol, web arayüzünden veya «Reset Web Base Path» menü maddesinden değiştirilir.

1.6. x-ui Yönetim Komutu (Betik Menüsü)

Kurulumdan sonra x-ui komutu (root olarak çalıştırılır) «3X-UI Panel Management Script» etkileşimli menüsünü açar. Madde seçimi, numarası girilerek yapılır (0–27 aralığı). Pek çok madde, betikler için doğrudan alt komut olarak da kullanılabilir (bkz. Bölüm 1.7).

Menü tematik bloklara ayrılmıştır.

Kurulum ve Güncelleme

- **1. Install** – paneli kurar (install.sh 'ı çalıştırır). Kurulumdan önce panelin henüz kurulmadığı doğrulanır.
- **2. Update** – tüm x-ui bileşenlerini son sürüme günceller. Veriler kaybolmaz; güncellemeden sonra panel otomatik olarak yeniden başlar. Onay gerektirir.
- **3. Update Menu** – paneli yeniden yüklemekten yalnızca yönetim betiğini (x-ui.sh / x-ui komutu) güncel sürüme günceller.
- **4. Legacy Version** – belirtilen (eski) panel sürümünü kurar. Betik bir sürüm numarası ister (örneğin 2.4.0) ve ilgili sürümü indirir.
- **5. Uninstall** – paneli **Xray ile birlikte** tamamen kaldırır. Servis durdurulup devre dışı bırakılır; /etc/x-ui/, /usr/local/x-ui/ dizinleri, servis ortam dosyası ve yönetim betiği silinir. Onay gerektirir (varsayılan «hayır»).

Kimlik Bilgileri ve Ayarlar

- **6. Reset Username & Password** – panel kullanıcı adını ve parolasını sıfırlar. Kendi değerlerinizi girebilir ya da rastgele oluşturulması için boş bırakabilirsiniz (rastgele ad 10 karakter, rastgele parola 18 karakter). Ayrıca yapılandırılmışsa iki faktörlü kimlik doğrulamayı (2FA) devre dışı bırakma seçeneği sunulur. Sıfırlamadan sonra panel yeniden başlar.
- **7. Reset Web Base Path** – panel web yolunu sıfırlar: yeni bir rastgele yol oluşturulur (18 karakter) ve panel yeniden başlar. Önceki yol ele geçirilmişse veya unutulmuşsa kullanılır.
- **8. Reset Settings** – tüm panel ayarlarını varsayılan değerlere sıfırlar. **Kimlik bilgileri (kullanıcı adı ve parola) ve hesap verileri kaybolmaz.** Onay gerektirir; sıfırlamadan sonra panel yeniden başlar.
- **9. Change Port** – web panelinin portunu değiştirir. Bir port numarası istenir (1–65535); ayarlandıktan sonra değişikliğin geçerli olması için yeniden başlatma gerekir.
- **10. View Current Settings** – mevcut ayarları görüntüler (`x-ui setting -show`). Kullanılan veritabanı arkucunu (SQLite veya DSN'de parolası maskelenmiş PostgreSQL) ve hazır erişim bağlantısını (Access URL) gösterir. SSL yapılandırılmamışsa bir IP adresi için Let's Encrypt sertifikası almayı önerir.

Servis Yönetimi

- **11. Start** – panel servisini başlatır. Panel zaten çalışıyorsa yeniden başlatmaya gerek olmadığı bildirilir.
- **12. Stop** – panel servisini durdurur.
- **13. Restart** – panel servisini yeniden başlatır.
- **14. Restart Xray** – panelin kendisini yeniden başlatmadan yalnızca Xray-core çekirdeğini yeniden başlatır (`systemctl reload x-ui` aracılığıyla; Docker'da panel işlemine `USR1` sinyali gönderilerek).
- **15. Check Status** – servis durumunu kontrol eder (`systemctl status x-ui` veya `rc-service x-ui status`).
- **16. Logs Management** – log yönetimi: hata ayıklama logunu görüntüleme (Debug Log, `journalctl` aracılığıyla) ve Alpine dışında tüm logları temizleme (Clear All logs).

Otomatik Başlatma

- **17. Enable Autostart** – işletim sistemi açılışında panelin otomatik başlatılmasını etkinleştirir (`systemctl enable x-ui` veya `rc-update add`).
- **18. Disable Autostart** – işletim sistemi açılışında otomatik başlatmayı devre dışı bırakır.

Docker'da otomatik başlatma, konteynerin yeniden başlatma politikasıyla yönetildiğinden bu maddeler yalnızca ilgili bir ipucu görüntüler.

Güvenlik ve Ağ

- **19. SSL Certificate Management** – acme.sh aracılığıyla SSL sertifika yönetimi: bir alan adı için sertifika alma, iptal etme, zorla yenileme, mevcut alan adlarını görüntüleme, panel için sertifika yollarını belirtme ve bir IP adresi için kısa ömürlü (~6 gün, otomatik yenileme ile) sertifika alma.
- **20. Cloudflare SSL Certificate** – Cloudflare DNS doğrulaması aracılığıyla SSL sertifikası alma.
- **21. IP Limit Management** – istemci başına IP sayısı limitlerini yönetme (Fail2ban tabanlı): engellemeleri görüntüleme ve kaldırma vb.
- **22. Firewall Management** – güvenlik duvarı yönetimi (port açma/kapatma ve kural görüntüleme).

- **23. SSH Port Forwarding Management** – SSH tüneli aracılığıyla yerel makineden panele erişmek için SSH port yönlendirmesi yapılandırma.

Performans ve Bakım

- **24. Enable BBR** – TCP BBR tıkanıklık kontrol algoritmasını etkinleştirme/devre dışı bırakma (Enable BBR / Disable BBR maddeleriyle alt menü).
- **25. Update Geo Files** – geo veritabanlarını güncelleme (`.dat` dosyaları), kaynak seçimiyle: Loyalsoldier (`geoip.dat` , `geosite.dat`), chocolate4u (`geoip_IR.dat` , `geosite_IR.dat`), runetfreedom (`geoip_RU.dat` , `geosite_RU.dat`) veya All (hepsi birden). Güncellemeden sonra panel yeniden başlar.
- **26. Speedtest by Ookla** – Speedtest by Ookla aracılığıyla ağ hızı testi çalıştırma.
- **27. PostgreSQL Management** – yerleşik/bağlı PostgreSQL örneğini yönetme (etkinleştirme ve ilgili işlemler).
- **0. Exit Script** – menüden çıkış.

1.7. `x-ui` Alt Komutları (Etkileşimli Menü Olmadan)

Betiklerde kullanım için `x-ui` komutu doğrudan alt komutları destekler (argümentsiz `x-ui` menüyü açar):

Komut	İşlem
<code>x-ui</code>	yönetim menüsünü aç
<code>x-ui start</code>	paneli başlat
<code>x-ui stop</code>	paneli durdur
<code>x-ui restart</code>	paneli yeniden başlat
<code>x-ui restart-xray</code>	Xray'i yeniden başlat
<code>x-ui status</code>	mevcut servis durumu
<code>x-ui settings</code>	mevcut ayarlar
<code>x-ui enable</code>	işletim sistemi açılışında otomatik başlatmayı etkinleştir
<code>x-ui disable</code>	otomatik başlatmayı devre dışı bırak
<code>x-ui log</code>	logları görüntüle
<code>x-ui banlog</code>	Fail2ban engelleme loglarını görüntüle
<code>x-ui update</code>	paneli güncelle
<code>x-ui update-all-geofiles</code>	tüm geo dosyalarını güncelle
<code>x-ui migrateDB [file]</code>	<code>.db</code> ? <code>.dump</code> dönüşümü (SQLite)
<code>x-ui legacy</code>	eski sürümü kur
<code>x-ui install</code>	paneli kur
<code>x-ui uninstall</code>	paneli kaldır

1.8. SQLite'tan PostgreSQL'e Geçiş

Mevcut bir SQLite kurulumu PostgreSQL'e taşınabilir:

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"  
# ardından /etc/default/x-ui dosyasında XUI_DB_TYPE ve XUI_DB_DSN değerlerini ayarlayın  
ve yeniden başlatın:  
systemctl restart x-ui
```

Orijinal SQLite dosyası dokunulmadan kalır – yalnızca yeni arka ucun çalıştığını doğruladıktan sonra elle silin.

Örnek: PostgreSQL'e geçişin doğrulanması. Geçişten sonra panelin gerçekten yeni arka uçta çalıştığını ayarları görüntüleme komutuyla doğrulayın – çıktıda PostgreSQL belirtilmelidir (DSN'deki parola maskelenir):

```
x-ui settings | grep -i -E 'db|dsn'
```

Panel açılıyor ve hesaplar yerindeyse orijinal `x-ui.db` silinebilir.

2. Panele Giriş ve Erişim Güvenliği

Bu bölüm, 3X-UI paneli yönetici kimlik doğrulamasıyla ilgili her şeyi açıklar: giriş formu, iki faktörlü kimlik doğrulama (TOTP), parola deneme saldırısına karşı koruma, kimlik bilgilerinin değiştirilmesi, gizli yol ve panel portunun değiştirilmesi, oturum ömrü ve LDAP üzerinden senkronizasyon/kimlik doğrulama.

2.1. Giriş Formu

Giriş sayfası, panelin gizli yolunun (`webBasePath`) kökünde sunulur. Kullanıcı zaten oturum açmışsa otomatik olarak `.../panel/` adresine yönlendirilir. Sayfada tema değiştirici, arayüz dili seçimi ve giriş formu bulunur.

Form alanları:

Alan	İpucu/Başlık	Zorunlu	Açıklama
Kullanıcı Adı	«Kullanıcı Adı»	Evet	Yönetici girişi. Boş değer istemci tarafında reddedilir; sunucuda «Kullanıcı adı girin» mesajıyla reddedilir.
Parola	«Parola»	Evet	Yönetici parolası. Boş değer «Parola girin» mesajıyla reddedilir.
2FA Kodu	«2FA Kodu»	Yalnızca 2FA etkinken	Alan yalnızca panelde iki faktörlü kimlik doğrulama etkinleştirilmişse görünür. Kimlik doğrulayıcı uygulamasından alınan 6 haneli kod.

«Giriş Yap» düğmesi formu `POST /login` adresine gönderir.

Davranış ve mesajlar:

- Başarılı girişte «Giriş başarılı» mesajı gösterilir ve `.../panel/` adresine yönlendirme yapılır.
- Hatalı kimlik bilgileri veya yanlış 2FA kodu durumunda sunucu **tek bir** mesaj döndürür: «Geçersiz hesap bilgileri.» (İng.: *Invalid username or password or two-factor code.*). Bu kasıtlıdır – panel neyin yanlış olduğunu (kullanıcı adı, parola veya kod) göstermez; böylece deneme saldırıları kolaylaştırılmaz.
- «2FA Kodu» alanını panel, `POST /getTwoFactorEnable` isteğine göre gösterir veya gizler; bu istek yetkilendirmeden önce mevcut 2FA durumunu döndürür.
- Sunucu oturumu dolmuşsa bir sonraki istekte «Oturum süresi doldu. Lütfen tekrar giriş yapın» mesajı gösterilir ve kullanıcı giriş sayfasına yönlendirilir.

CSRF notu: form gönderilmeden önce istemci bir CSRF belirteci alır (`GET /csrf-token`); `/login` ve `/logout` istekleri CSRF doğrulamasıyla korunur.

Örnek: API üzerinden giriş. 2FA devre dışıyken yalnızca kullanıcı adı ve parola yeterlidir; 2FA etkinken `twoFactorCode` alanı eklenir:

```
# 2FA olmadan
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data 'username=admin&password=ВашПароль'

# 2FA etkinken – 6 haneli kod eklenir
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data 'username=admin&password=ВашПароль&twoFactorCode=123456'
```

Başarı durumunda sunucu `Set-Cookie` başlığıyla bir oturum çerezi döndürür – bu çerez `/panel/api/...` adresine yapılan sonraki isteklerde kullanılmalıdır.

2.2. İki Faktörlü Kimlik Doğrulama (2FA / TOTP)

3X-UI'deki 2FA, **TOTP** standardıyla uygulanmıştır ve herhangi bir kimlik doğrulayıcı uygulamasıyla (Google Authenticator, Aegis, FreeOTP vb.) uyumludur. Parametreler sabit olarak tanımlıdır: algoritma **SHA1**, 6 hane, süre **30** saniye, yayıncı (issuer) `3x-ui`, etiket `Administrator`.

Örnek: QR kodunu şifreleyen otpauth URI'si. Kimlik doğrulayıcı uygulama kamerayla tarama yapamazsa, belirteci bu bağlantıyı kullanarak elle ekleyebilirsiniz (`JBSWY3DPEHPK3PXP` yerine kendi Base32 gizlinizi yazın):

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPEHPK3PXP&issuer=3x-
ui&algorithm=SHA1&digits=6&period=30
```

`algorithm=SHA1`, `digits=6`, `period=30` parametreleri panelin sabit değerleriyle örtüşür – bunları değiştirmeye gerek yoktur.

Ayarlar **Ayarlar** → **Hesap** bölümünde, «İki Faktörlü Kimlik Doğrulama» sekmesinde yer alır.

Öge	Metin	Açıklama
Geçiş anahtarı	«2FA'yı Etkinleştir»	İki faktörlü kimlik doğrulamayı açar/kapatır.
Açıklama	«Güvenliği artırmak için ek bir kimlik doğrulama katmanı ekler.»	Geçiş anahtarının altındaki ipucu.

2FA Nasıl Etkinleştirilir

Geçiş anahtarı açıldığında panel **yerel olarak yeni bir gizli anahtar üretir** – Base32 kodlamasıyla rastgele bir dize (`A–Z` ve `2–7` alfabeti). «İki faktörlü kimlik doğrulamayı etkinleştir» adlı bir pencere açılır ve adım adım yönergeler sunulur:

1. «**Bu QR kodunu kimlik doğrulayıcı uygulamanızda tarayın ya da QR kodunun yanındaki belirteci kopyalayıp uygulamaya yapıştırın**». QR kodunun altında gizli anahtar metin olarak gösterilir – QR koduna tıklandığında gizli anahtar panoya kopyalanır («Kopyalandı» bildirimi çıkar).
2. «**Uygulamadan kodu girin**» – uygulamanın ürettiği 6 haneli kodu girmeniz gerekir. Kod **tarayıcı tarafında** doğrulanır: panel az önce üretilen gizliyle geçerli TOTP'yi hesaplar ve girilen kodla karşılaştırır. Kod yanlışsa «Geçersiz kod»; alan yalnızca tam olarak 6 rakam kabul eder.

Başarılı onayın ardından gizli anahtar ve etkinleştirme bayrağı kaydedilir. Kaydedildiğinde «İki faktörlü kimlik doğrulama başarıyla kuruldu» mesajı gösterilir.

Önemli: ayarlar bölümündeki değişiklikler **«Kaydet»** düğmesiyle uygulanır; genellikle panel yeniden başlatması gerekir («Değişikliklerin geçerli olması için kaydedin ve paneli yeniden başlatın»). 2FA ilk kez etkinleştirildiğinde sunucu ayrıca **tüm etkin oturumları geçersiz kılar** (oturum açma dönemini artırır); bu nedenle ayar uygulandıktan sonra – artık 2FA koduyla – yeniden giriş yapılması gerekir.

2FA Nasıl Devre Dışı Bırakılır

Geçiş anahtarına tekrar tıklandığında «İki faktörlü kimlik doğrulamayı devre dışı bırak» penceresi açılır ve «İki faktörlü kimlik doğrulamayı devre dışı bırakmak için uygulamadan kodu girin.» ipucu görünür. Doğru kod girildiğinde bayrak ve gizli anahtar temizlenir; «İki faktörlü kimlik doğrulama başarıyla kaldırıldı» mesajı gösterilir.

Girişte Kod Doğrulama

Giriş sırasında sunucu kayıtlı gizliyi alır ve geçerli TOTP'yi gönderilen 2FA koduyla karşılaştırır. Eşleşmeme, başarısız giriş olarak değerlendirilir; ancak kullanıcıya yine aynı birleşik «Geçersiz hesap bilgileri.» mesajı gösterilir.

Erişim Kurtarma (recovery)

3X-UI'de ayrı bir «kurtarma kodları» mekanizması **yoktur**. Kimlik doğrulayıcı uygulamasına erişim kaybedilirse panel arayüzü üzerinden giriş kurtarılamaz. Tek yol, sunucudaki veritabanında doğrudan 2FA'yı devre dışı bırakmaktır: ayarlar tablosunda `twoFactorEnable` anahtarını `false` olarak sıfırlayın (gerekirse `twoFactorToken` da temizleyin) ve ardından paneli yeniden başlatın. Bu nedenle 2FA etkinleştirilirken gizli anahtarın (Base32 belirteci) güvenli bir yere kaydedilmesi önerilir.

Örnek: sunucuda 2FA'nın acil olarak devre dışı bırakılması. SSH üzerinden sunucuya erişin, paneli durdurun, ayarlar tablosundaki anahtarları sıfırlayın ve paneli yeniden başlatın:

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE
key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

Bundan sonra giriş yalnızca kullanıcı adı ve parolayla yapılır; 2FA istenirse yeniden kurulabilir.

Kimlik bilgileri değişikliğiyle ilişki: kullanıcı adı/parola değiştirildiğinde (bkz. 2.4) 2FA, eski gizli anahtarın yeni hesapta erişimi engellemesini önlemek amacıyla sunucuda **otomatik olarak devre dışı bırakılır**.

2.3. Giriş Denemelerini Sınırlandırma (login limiter / deneme saldırısına karşı koruma)

Panel, başarısız girişler için yerleşik bir sınırlayıcı içerir (uygulama düzeyinde fail2ban benzeri). Parametreler kod içinde tanımlıdır ve arayüz üzerinden **yapılandırılmaz**:

Parametre	Değer	Amaç
Maksimum başarısız deneme	5	Pencere içinde izin verilen başarısız deneme sayısı.
Sayım penceresi	5 dakika	Başarısız denemelerin biriktirildiği kayan pencere (daha eskiler atılır).
Engelleme süresi (cooldown)	15 dakika	Eşik aşıldıktan sonra anahtarın engellendiği süre.

Nasıl çalışır:

- Engelleme anahtarı «**IP + kullanıcı adı**» çiftinden oluşur (kullanıcı adı küçük harfe dönüştürülür, boşluklar kırpılır). Yani engelleme tüm panele değil, belirli bir adres + kullanıcı adı çiftine uygulanır.
- Her başarısız denemede (yanlış kullanıcı adı/parola veya yanlış 2FA kodu) sayaç artar. **5 dakika** içinde **5** başarısız denemeye ulaşıldığında anahtar **15 dakika** engellenir. Engelleme süresince bu çiftin tüm denemeleri, veriler doğru olsa bile aynı «Geçersiz hesap bilgileri.» mesajıyla anında reddedilir.
- Başarılı giriş sayacı anında sıfırlar** ve bu çiftin engelini kaldırır.
- İstemci IP adresi, güvenirli proxy'ler dikkate alınarak belirlenir (bkz. `trustedProxyCIDRs`): `X-Real-IP` ve `X-Forwarded-For` başlıkları yalnızca istek güvenirli bir adresten geldiyse kabul edilir. Aksi hâlde gerçek bağlantı adresi kullanılır; bu da çıkarılamazsa `unknown` dizesi kullanılır.

Tüm denemeler günlüğe kaydedilir. Başarısızlar için sunucu günlüğüne kullanıcı adı, IP, neden ve engellenmişse `blocked_until` zamanını içeren bir uyarı yazılır. Telegram botu aracılığıyla giriş bildirimi etkinleştirilmişse (`tgNotifyLogin` – «Giriş Bildirimi»), yönetici ek olarak başarılı, başarısız ve engellenmiş denemelerin kullanıcı adı, IP ve zamanını içeren bildirimler alır.

Örnek: Telegram'da giriş bildirimi. `tgNotifyLogin` etkinleştirildiğinde her denemeden sonra yönetici yaklaşık şu içerikteki bir mesaj alır:

```
Уведомление о входе
Пользователь: admin
IP: 203.0.113.45
Время: 2026-06-10 14:32:07
Статус: успешно
```

Engellenen «IP + kullanıcı adı» çifti için durum alanında denemenin sınırlayıcı tarafından reddedildiği belirtilir.

2.4. Yönetici Kullanıcı Adı ve Parolasının Değiştirilmesi

Ayarlar → **Hesap** bölümü, «**Yönetici Kimlik Bilgileri**» sekmesi. Alanlar:

Alan	Metin	Açıklama
Mevcut kullanıcı adı	«Mevcut Kullanıcı Adı»	Geçerli kullanıcı adı. Gerçek kullanıcı adıyla eşleşmezse değişiklik reddedilir.
Mevcut parola	«Mevcut Parola»	Kimliği doğrulamak için geçerli parola.

Alan	Metin	Açıklama
Yeni kullanıcı adı	«Yeni Kullanıcı Adı»	Yeni kullanıcı adı. Boş bırakılamaz.
Yeni parola	«Yeni Parola»	Yeni parola. Boş bırakılamaz.

Değişiklik **«Onayla»** düğmesiyle uygulanır ve `POST /panel/setting/updateUser` adresine gönderilir.

Sunucu mantığı ve mesajları:

- «Mevcut Kullanıcı Adı» gerçekte örtüşmüyorsa veya «Mevcut Parola» yanlışsa – «Yönetici kimlik bilgileri değiştirilirken bir hata oluştu.» mesajı ve «Geçersiz kullanıcı adı veya parola» açıklaması.
- Yeni kullanıcı adı veya yeni parola boşsa – «Yeni kullanıcı adı ve yeni parola doldurulmalıdır» açıklaması.
- Başarı durumunda – «Yönetici kimlik bilgilerinizi başarıyla değiştirdiniz.». Parola bcrypt karması olarak saklanır.

Örnek: API üzerinden kimlik bilgisi değiştirme. İstek geçerli bir oturum çerezi (girişte alınır) ve mevcut kullanıcı adı/parolanın onayını gerektirir:

```
curl -X POST https://panel.example.com:2053/мой-секрет/panel/setting/updateUser \
  -b 'session=BAWA_CECCIOHHAY_COOKIE' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=СтарыйПароль&newUsername=root&newPassword=НовыйСложныйПароль'
```

Başarının ardından mevcut oturum geçersiz kılınır – yeni kimlik bilgileriyle yeniden giriş yapılması gerekir.

Kimlik bilgisi değiştirmenin önemli etkileri:

- **Tüm mevcut oturumlar geçersiz kılınır** (kullanıcının `login_epoch` sayacı artırılır); bu nedenle değiştirmenin ardından panel otomatik olarak çıkış yapar ve giriş sayfasına yönlendirir – yeniden giriş yapılması gerekir.
- Değiştirme sırasında **2FA etkinse, otomatik olarak devre dışı bırakılır** (bayrak ve gizli anahtar sıfırlanır). Kullanıcı adı/parola değişikliğinin ardından iki faktörlü kimlik doğrulamanın yeniden kurulması gerekir.

2FA etkinse form gönderilmeden önce «Kimlik bilgilerini değiştir» adlı bir pencere açılır ve «Yönetici kimlik bilgilerini değiştirmek için uygulamadan kodu girin.» ipucu gösterilir – kimlik bilgileri yalnızca geçerli 2FA kodu onaylanarak değiştirilebilir.

2.5. Gizli Yol (URI yolu / webBasePath) ve Panel Portu

Bu parametreler **Ayarlar** → **Panel** bölümünde yer alır ve panelin «görünmezliğini» ile erişilebilirliğini doğrudan etkiler. Kaydedildikten ve panel **yeniden başlatıldıktan** sonra geçerli olurlar.

Alan	Metin	Varsayılan Değer	Açıklama
Panel portu	«Panel Portu» (<code>panelPort</code>), ipucu «Panelin çalıştığı port»	2053	Web arayüzünün TCP portu.

Alan	Metin	Varsayılan Değer	Açıklama
URI yolu	«URI Yolu» (<code>panelUrlPath</code>), ipucu «/' ile başlamalı ve '/' ile bitmeli»	/	Gizli temel yol (<code>webBasePath</code>). Panel yalnızca bu yoldan erişilebilir (örn. /мой-секрет/).
Panel yönetim IP adresi	«Panel Yönetim IP Adresi» (<code>panelListeningIP</code>), ipucu «Herhangi bir IP'den bağlantıya izin vermek için boş bırakın»	boş	Panelin dinlediği adres. Boş = tüm arayüzler.
Panel alan adı	«Panel Alan Adı» (<code>panelListeningDomain</code>), ipucu «Herhangi bir alan adı ve IP'den bağlantıya izin vermek için boş bırakın.»	boş	Alan adına (Host) göre erişim kısıtlaması.
Panel sertifikası genel anahtar yolu	<code>publicKeyPath</code> , ipucu «/' ile başlayan tam yolu girin»	boş	Panel için HTTPS erişiminde kullanılan TLS sertifikası.
Panel sertifikası özel anahtar yolu	<code>privateKeyPath</code> , aynı ipucu	boş	TLS özel anahtarı.

Temel yolun (`webBasePath`) davranışı:

- Değer otomatik olarak normalleştirilir: / ile başlamıyorsa başa eklenir; / ile bitmiyorsa sona eklenir. Yani gerçekte yol her zaman `/.../` biçimindedir.
- Temel yol panelin kendisine, varlıklara ve oturum çerezine uygulanır (çerez yalnızca bu yol için verilir).

Güvenlik önerileri («Güvenlik Uyarıları» bölümü): yapılandırma «çok açık» olduğunda panel kendi uyarılarını gösterir:

- «Panel düz HTTP üzerinden çalışıyor – üretim ortamı için TLS yapılandırın.»
- «Standart 2053 portu yaygın bilinir – bunu rastgele bir port ile değiştirin.»
- «Varsayılan "/" temel yolu yaygın bilinir – bunu rastgele biriyle değiştirin.»

Başka bir deyişle, üretim sunucusu için **standart dışı bir port, öngörülemez bir URI yolu ve TLS sertifikası** ayarlanmalıdır.

Örnek: üretim ortamı için «gizli» panel yapılandırması. Ayarlar → Panel bölümünde aşağıdakine benzer değerler ayarlayın:

Alan	Değer
Panel portu	34571 (2053 yerine rastgele)
URI yolu	/aXf9Qm2/ (öngörülemez, / ile başlayıp biten)
Panel sertifikası genel anahtar yolu	/etc/letsencrypt/live/panel.example.com/fullchain.pem
Panel sertifikası özel anahtar yolu	/etc/letsencrypt/live/panel.example.com/privkey.pem

Kaydedip yeniden başlattıktan sonra panel yalnızca `https://panel.example.com:34571/aXf9Qm2/` adresinden erişilebilir olur ve güvenlik uyarıları kaybolur.

2.6. Oturum Ömrü (zaman aşımı)

«Oturum Süresi» (`sessionMaxAge`) alanı panel/aralık ayarları içinde yer alır.

Alan	Metin	Varsayılan Değer	Birim	Açıklama
Oturum süresi	«Oturum Süresi», ipucu «Sistemdeki oturum süresi (değer: dakika)»	360	dakika	Yönetici oturum çerezinin yaşam süresi.

Davranış:

- Değer **dakika** cinsinden girilir (varsayılan 360 dakika = 6 saat); çerez yapılandırılırken saniyeye çevrilir.
- Değer **0'dan büyükse** oturum çerezine karşılık gelen `MaxAge` atanır. Bu süre dolduğunda çerez geçerliliğini yitirir ve bir sonraki istekte kullanıcı «Oturum süresi doldu. Lütfen tekrar giriş yapın» mesajıyla karşılaşır.
- Oturum ayrıca kimlik bilgileri değiştirildiğinde veya 2FA ilk kez etkinleştirildiğinde (`login_epoch` mekanizmasıyla, bkz. 2.4 ve 2.2) ve açık çıkış yapıldığında (`POST /logout`) erken geçersiz kılınır.
- Oturum çerezi `HttpOnly` olarak işaretlenir; `SameSite=Lax` politikası uygulanır; `Secure` bayrağı panele doğrudan HTTPS erişiminde ayarlanır.

Zaman aşımının yanı sıra ilgili bir bildirim daha vardır: «Oturum sona erme bildirim gecikmesi» (`expireTimeDiff` , ipucu «Eşik değerine ulaşmadan önce oturum sona erme bildirimi alın (değer: gün)», varsayılan `0`) – önceden uyarı almayı sağlar.

2.7. LDAP (Senkronizasyon ve Kimlik Doğrulama)

LDAP bölümü iki olanak sunar: (1) yerel parola uymadığında yönetici girişini LDAP üzerinden doğrulamak ve (2) istemcilerin durumunu (VLESS bayrağı etkin/devre dışı) dizinden periyodik olarak senkronize etmek.

Girişte nasıl kullanılır: sunucu önce yerel `bcrypt` parola karmasını kontrol eder. **Eşleşmezse** ve LDAP etkinse panel kullanıcıyı dizinde doğrulamaya çalışır: `Bind DN` tanımlıysa servis `bind` işlemi yapılır, ardından filtre ve öznitelikle kullanıcı kaydı aranır ve bulunan DN altında girilen parolayla `bind` denenir. Başarı, girişe izin verir. (Başarılı LDAP kimlik doğrulamasının ardından 2FA etkinse TOTP kodu yine de doğrulanır.)

Bölüm alanları:

Alan	Metin	Varsayılan Değer	Açıklama
LDAP Senkronizasyonunu Etkinleştir	«LDAP Senkronizasyonunu Etkinleştir» (<code>enable</code>)	false	LDAP entegrasyonunun ana anahtarı.
LDAP sunucusu	«LDAP Sunucusu» (<code>host</code>)	boş	LDAP sunucusunun adresi.
LDAP portu	«LDAP Portu» (<code>port</code>)	389	Port. LDAPS için genellikle 636.

Alan	Metin	Varsayılan Değer	Açıklama
TLS Kullan (LDAPS)	«TLS Kullan (LDAPS)» (<code>useTls</code>)	false	Etkinleştirildiğinde sunucu sertifikası doğrulanarak <code>ldaps://</code> şeması kullanılır (doğrulama atlanmaz).
Bind DN	«Bind DN» (<code>bindDn</code>)	boş	İlk bind/arama için servis hesabının DN'i. Boşsa bind yapılmaz (anonim arama).
Bind parolası	ipuçları: «Yapılandırıldı; mevcut parolayı korumak için boş bırakın.» / «Yapılandırılmadı.» / «Yapılandırıldı – değiştirmek için yeni değer girin»	boş	Bind DN için parola. Ayrıca saklanır; eskisini korumak için alan boş bırakılır.
Base DN	«Base DN» (<code>baseDn</code>)	boş	Aramanın yapıldığı alt ağacın kökü (arama özyinelemeli, tüm alt ağaçta yapılır).
Kullanıcı filtresi	«Kullanıcı Filtresi» (<code>userFilter</code>)	(<code>objectClass=person</code>)	Hesap seçimi için LDAP filtresi. Kimlik doğrulama sırasında kullanıcı adı filtreye kaçış karakterleriyle eklenir.
Kullanıcı özniteliği (username/email)	«Kullanıcı Özniteliği (username/email)» (<code>userAttr</code>)	<code>mail</code>	Kullanıcı adı/istemci tanımlayıcısıyla eşleştirilen öznitelik (örn. <code>mail</code> veya <code>uid</code>).
VLESS bayrağı özniteliği	«VLESS Bayrağı Özniteliği» (<code>vlessField</code>)	<code>vless_enabled</code>	İstemcinin VLESS erişiminin etkin olup olmayacağını belirleyen öznitelik.
Genel bayrak özniteliği (isteğe bağlı)	«Genel Bayrak Özniteliği (isteğe bağlı)» (<code>flagField</code>), ipucu «Tanımlıysa VLESS bayrağını geçersiz kılar – örn. <code>shadowInactive</code> .»	boş	Tanımlıysa <code>vless_enabled</code> yerine kullanılır.
Truthy değerler	«Truthy Değerler» (<code>truthyValues</code>), ipucu «Virgülle ayrılmış; varsayılan: <code>true,1,yes,on</code> »	<code>true,1,yes,on</code>	«Etkin» olarak değerlendirilen bayrak özniteliği değerlerinin listesi.
Bayrağı ters çevir	«Bayrağı Ters Çevir» (<code>invertFlag</code>), ipucu «Öznitelik «devre dışı» anlamına geldiğinde etkinleştirin (örn. <code>shadowInactive</code>).»	false	Bayrağın anlamını tersine çevirir.
Senkronizasyon programı	«Senkronizasyon Programı» (<code>syncSchedule</code>), ipucu «Cron benzeri dize, örn. <code>@every 1m</code> »	<code>@every 1m</code>	Cron benzeri biçimde senkronizasyon sıklığı.

Alan	Metin	Varsayılan Değer	Açıklama
inbound etiketleri	«inbound Etiketleri» (<code>inboundTags</code>), ipucu «LDAP senkronizasyonunun otomatik istemci oluşturabileceği veya silebileceği inbound'lar.»	boş	Otomatik işlemlere izin verilen inbound'ları kısıtlar. inbound yoksa: «inbound bulunamadı. Önce bir inbound oluşturun.»
Otomatik istemci oluştur	«Otomatik İstemci Oluştur» (<code>autoCreate</code>)	false	Dizinde yeni bir istemci görünürse belirtilen inbound'larda istemci oluşturur.
Otomatik istemci sil	«Otomatik İstemci Sil» (<code>autoDelete</code>)	false	İstemci dizinden kaybolursa siler.
Varsayılan hacim (GB)	«Varsayılan Hacim (GB)» (<code>defaultTotalGb</code>)	0	Otomatik oluşturulan istemciler için trafik limiti (0 = limitsiz).
Varsayılan süre (gün)	«Varsayılan Süre (gün)» (<code>defaultExpiryDays</code>)	0	Otomatik oluşturulan istemciler için geçerlilik süresi (0 = süresiz).
Varsayılan IP limiti	«Varsayılan IP Limiti» (<code>defaultIpLimit</code>)	0	Eş zamanlı IP sayısı sınırı (0 = sınırsız).

Senkronizasyon bayrağı mantığının ayrıntıları: bayrak özniteliği (`flagField` , varsayılan `vless_enabled`) okunduğunda değer `truthy` değerler listesindeyse «etkin» sayılır; ters çevirme etkinse sonuç tersine döner. Kullanıcı özniteliği (`userAttr`) eşleştirme anahtarı (email/ad) olarak kullanılır – bu özniteliğin değeri olmayan kayıtlar atlanır.

Güvenlik: bind parolalarının ve doğrulanan parolaların açık metin olarak iletilmemesi için **TLS (LDAPS)** etkinleştirilmesi önerilir; Bind DN için yalnızca okuma için gerekli minimum haklara sahip bir hesap kullanılmalıdır.

Örnek: tipik LDAP senkronizasyon yapılandırması (Active Directory). Erişim durumunun `userAccountControl` benzeri bir bayrak özniteliğinde saklandığı ve eşleştirmenin e-postaya göre yapıldığı bir dizin için bölüm alanlarının doldurulması:

Alan	Değer
LDAP sunucusu	<code>ldap.example.com</code>
LDAP portu	<code>636</code>
TLS Kullan (LDAPS)	etkin
Bind DN	<code>CN=svc-3xui,OU=Service,DC=example,DC=com</code>
Base DN	<code>OU=Users,DC=example,DC=com</code>
Kullanıcı filtresi	<code>(objectClass=person)</code>

Alan	Değer
Kullanıcı özniteliği (username/email)	mail
VLESS bayrağı özniteliği	vless_enabled
Truthy değerler	true,1,yes,on
Senkronizasyon programı	@every 5m

Bu yapılandırmayla panel her 5 dakikada bir `0U=Users` alt ağacını tarar, istemcileri `mail` ile eşleştirir ve `vless_enabled` değerine göre VLESS erişimini açar veya kapatır.

3. Genel Bakış / Gösterge Paneli

Gösterge paneli (*Overview*) – panelin başlangıç sayfasıdır. Sunucunun ve Xray sürecinin durumunu gerçek zamanlı olarak gösterir. Tüm ölçümler sunucu tarafından gelir. Arka plan zamanlayıcısı anlık görüntüyü **her 2 saniyede bir** yeniden oluşturur ve WebSocket aracılığıyla tüm açık sekmelere gönderir; bir dakikada bir biriken metrik satırları diske yazılır. `GET /status` HTTP uç noktası son önbelleğe alınmış anlık görüntüyü döndürür.

Aşağıda sayfadaki her ölçüm ve her kontrol öğesi ayrıntılı olarak açıklanmaktadır.

3.1. Veri Toplama Genel İlkeleri

- Anlık görüntü `gopsutil` kütüphanesi tarafından toplanır. Belirli bir ölçüm başarısız olursa alan sıfır kalır ve günlüğe bir uyarı yazılır (`get cpu percent failed` , `get uptime failed` vb.) – bu durum gösterge panelini çökertmez, yalnızca ilgili kutucuk 0/N-A gösterir.
- "Anlık" hızlar (CPU %, ağ, disk G/Ç) mevcut ve önceki anlık görüntü arasındaki farkın saniye cinsinden aralığa bölünmesiyle hesaplanır. Bu nedenle sayfa ilk yüklendiğinde ikinci ölçüm birikene kadar hız değerleri sıfır olabilir.
- Geçmiş, «Sistem Geçmişi» (*System History*) bölümünde incelenebilir – grafikler aşağıda açıklanan veri satırlarına göre oluşturulur (bkz. madde 3.12).

3.2. İşlemci (CPU)

«İşlemci» (*CPU*) kutucuğu mevcut işlemci kullanım yüzdesini ve işlemcinin parametrelerini gösterir.

Ölçüm	Açıklama
İşlemci Kullanımı, %	Son aralıkta kullanılan işlemci zamanının oranı. Göstergenin sıçramasını önlemek için üstel hareketli ortalama (EMA, katsayı $\alpha = 0.3$) ile yumuşatılır. Değer her zaman 0–100 % aralığında sınırlanır. İlk ölçümde 0 döndürülür (başlangıç noktasının başlatılması).
Mantıksal İşlemciler	Hyper-Threading dahil mantıksal çekirdek sayısı.
Fiziksel Çekirdekler	Fiziksel çekirdek sayısı.
Frekans	İşlemcinin temel frekansı (MHz). Gecikmeli olarak sorgulanır ve önbelleğe alınır: ilk başarılı ölçüm kaydedilir, yeniden deneme en fazla 5 dakikada bir yapılır ve sorgunun kendisi 1,5 s zaman aşımıyla sınırlandırılır (bazı sistemlerde frekans sorgusu yavaş yanıt verir).

CPU kullanımı algoritmik olarak şu şekilde hesaplanır: yerel platform uygulaması mevcutsa kullanılır, yoksa işlemci zamanı sayaçlarının deltalarına (busy / total) göre hesaplama yapılır. Guest ve GuestNice zamanları çift saymamak için hariç tutulur.

3.3. Bellek (RAM)

«Bellek» (*RAM*) kutucuğu kullanılanı ve toplamı gösterir. «kullanılan / toplam» ve/veya doluluk yüzdesi olarak gösterilir. Geçmişe yüzde değeri kaydedilir.

3.4. Takas Belleği (Swap)

«Takas Belleği» (*Swap*) kutucuğu kullanılanı ve toplamı gösterir. Takas dosyası/bölümü yapılandırılmamışsa (toplam = 0) ölçüm sıfırdır; swap olmadığında geçmiş satırına 0 yazılır.

3.5. Disk (Storage)

«Disk» (*Storage*) kutucuğu kullanılanı ve toplamı gösterir, ancak yalnızca **kök bölümü** / dikkate alınır. «Disk Kullanımı» (*Disk Usage*) geçmişine doluluk yüzdesi yazılır. Aralık başına sayaç deltası olarak disk G/Ç (okuma / yazma, bayt/s) ayrıca toplanır – geçmişin «Disk G/Ç» sekmesinde gösterilir.

3.6. Sistem Çalışma Süresi (Uptime)

«Sistem Çalışma Süresi» (*Uptime*) ölçümü. Bu, panelin veya Xray'in değil, **tüm sunucunun** başlatılmasından bu yana geçen süredir (saniye cinsinden). Xray sürecinin çalışma süresi ayrıca tutulur (bkz. madde 3.9); panel iş parçacığı sayısı da ayrıca gösterilir (*Threads*).

3.7. Sistem Yüğü (Load average)

«Sistem Yüğü» (*System Load*) bloğu – üç sayıdan oluşan bir dizi: [Load1, Load5, Load15]. Açıklama ipucu: «Son 1, 5 ve 15 dakika için sistem yüğü ortalaması» (*System load average for the past 1, 5, and 15 minutes*). Geçmiş grafiğinin adı «Sistem Yüğü Ortalaması (1 / 5 / 15 dak.)»dır. Geçmiş satırlarına değerler ayrı ayrı yazılır: load1 , load5 , load15 .

Bu standart bir Unix ölçümüdür: yürütme kuyruğundaki ortalama süreç sayısı. Karşılaştırma ölçütü – çekirdek sayısı ile karşılaştırmak: fiziksel çekirdek sayısını sürekli aşan yük aşırı yüklenmeye işaret eder.

3.8. Ağ: Hız ve Toplam Trafik Hacmi

Yalnızca **fiziksel arabirimler** dikkate alınır. Sanal ve tünel arabirimleri hariç tutulur: lo / lo0 ile loopback , docker , br- , veth , virbr , tun , tap , wg , tailscale , zt ile başlayanlar. Değerler kalan tüm arabirimler üzerinde toplanır.

Genel Hız (*Overall Speed*) – aralık başına sayaç deltasından hesaplanan anlık hız:

Ölçüm	Açıklama
Yükleme / gönderme (etiket «Yükleme» / <i>Upload</i>)	Giden hız, bayt/s.
İndirme / alma (etiket «İndirme» / <i>Download</i>)	Gelen hız, bayt/s.

Toplam Trafik Hacmi (*Total Data*) – sistem başlatılmasından bu yana biriken sayaçlar:

Ölçüm	Açıklama
Gönderilen (etiket «Gönderildi» / <i>Sent</i>)	Toplam gönderilen bayt sayısı.
Alınan (etiket «Alındı» / <i>Received</i>)	Toplam alınan bayt sayısı.

Ayrıca paket hızları (paket/s) ve toplam paket sayaçları toplanır – bunlar geçmişin «Ağ Paketleri» (*Network Packets*) sekmesinde gösterilir. Ağa ilişkin geçmiş satırları: `netUp` , `netDown` , `pktUp` , `pktDown` .

3.9. Sunucu IP Adresleri

«Sunucu IP Adresleri» (*IP Addresses*) bloğu IPv4 ve IPv6 adreslerini gösterir. Dış adresler üçüncü taraf hizmetler aracılığıyla belirlenir (`api4.ipify.org` , `ipv4.icanhazip.com` , `v4.api.ipinfo.io/ip` , `ipv4.myexternalip.com/raw` , `4.ident.me` , `check-host.net/ip` IPv4 için ve benzer hizmetler IPv6 için). Liste ilk başarılı yanıt kadar sırayla denir; her istek için zaman aşımı 3 s'dir.

Özellikler:

- Sonuç süreç yaşam süresi boyunca **ön belleğe alınır**: başarıyla belirlenen adres tekrar sorgulanmaz.
- Hiçbir hizmet yanıt vermezse alanda `N/A` kalır. IPv6 için ilk `N/A` durumunda IPv6 istekleri tamamen devre dışı bırakılır; böylece IPv6 içermeyen ağlarda zaman kaybedilmez.
- Yanında adresleri gizlemek/göstermek için bir «göz» düğmesi bulunur – ipucu: «Sunucu IP adreslerinin görünürlüğünü değiştir» (*Toggle visibility of the IP*). Bu yalnızca arayüzdeki görsel bir gizlemedir (örneğin ekran görüntüleri için) ve adreslerin kendisini etkilemez.

3.10. TCP/UDP Bağlantıları

«Bağlantı İstatistikleri» (*Connection Stats*) bloğu sunucudaki aktif TCP ve UDP bağlantılarının toplam sayısını gösterir (tüm sistem genelinde, yalnızca Xray değil). Geçmiş grafiği – «Aktif Bağlantılar (TCP / UDP)» (*Active Connections*), satırlar `tcpCount` , `udpCount` .

3.11. Xray Durumu ve Süreç Yönetimi

«Xray» kartı, Xray-core sürecinin durumunu gösterir ve yönetilmesini sağlar.

Durumlar

Değer	Etiket	Çeviri	Ne Zaman Ayarlanır
<code>running</code>	«Çalışıyor»	<i>Running</i>	Xray süreci çalışıyor.
<code>stop</code>	«Durduruldu»	<i>Stopped</i>	Süreç çalışmıyor ve kayıtlı bir başlatma hatası yok.
<code>error</code>	«Hata»	<i>Error</i>	Süreç çalışmıyor ve bir başlatma hatası kaydedildi. Hata metni «Xray çalıştırılırken hata oluştu» (<i>An error occurred while running Xray</i>) başlıklı açılır pencerede gösterilir.
–	«Bilinmiyor»	<i>Unknown</i>	Durum henüz alınmamışken gösterilir.

Durumun yanında **Xray sürümü** gösterilir.

Kontrol Düğmeleri

- **Durdur (Stop)**. `POST /stopXrayService` çağrısını yapar. Başarı durumunda panel WebSocket üzerinden yeni `stop` durumunu ve «Xray başarıyla durduruldu» (*Xray service has been stopped*) bildirimini gönderir;

hata durumunda – metinle birlikte `error` durumu. Önemli: panel *Xray üzerinden* erişilebiliyorsa Xray'i durdurmak panelle bağlantıyı kesebilir – panele doğrudan bağlantıda sorun yoktur.

- **Yeniden Başlat (Restart).** `POST /restartXrayService` çağrısını yapar. İşlemden önce «xray yeniden başlatılsın mı?» onayı gösterilir ve «xray servisini kaydedilen yapılandırmayla yeniden başlatır» açıklaması yer alır. Başarı durumunda – `running` durumu ve «Xray başarıyla yeniden başlatıldı» (*Xray service has been restarted successfully*) bildirimi. Yeniden başlatma mevcut kaydedilmiş yapılandırmayı uygular – ayarları değiştirdikten sonra kullanın.

Not. Bu fork'ta gösterge paneline tüm kimlik doğrulama türleri için tam Start / Stop / Restart yönetimi eklenmiştir; özgün 3x-ui arayüzünde ayrı bir «başlat» düğmesi yoktur – başlatma yeniden başlatma yoluyla gerçekleştirilir.

Xray Günlüklerini Görüntüleme Düğmesi

Xray kartında Xray günlüklerini görüntüleme düğmesi (*Logs*) bulunur. Bu düğme yalnızca Xray yapılandırmasında `access-log` yapılandırıldığında görünür: yerleşik görüntüleyici tam olarak bu dosyayı okur, dolayısıyla `access-log` olmadan düğme gizlenir. Düğmenin görünürlüğü ayrı bir `accessLogEnable` özelliğine bağlıdır ve artık IP sınırına bağlı değildir – çevrimiçi liste ve IP adresi sınırı `access-log` olmadan da çalışmaya devam eder (bkz. madde 8).

Xray Sürümü Seçimi

«Sürüm Seçimi» (*Version*) bölümü, Xray-core'u başka bir sürüme geçirmenizi sağlar. Sürüm listesi `GET /getXrayVersion` aracılığıyla yüklenir:

- Kaynak – `XTLS/Xray-core` deposunun GitHub API'si (`/releases`). İstekler **15 dakika** boyunca önbelleğe alınır; GitHub başarısız olursa seçici boşalmayacak şekilde en son başarılı liste döndürülür.
- Listeye yalnızca `X.Y.Z` biçimindeki ve **26.4.25'ten daha eski olmayan** sürümler dahil edilir.

İpuçları: «Geçmek istediğiniz sürümü seçin» (*Choose the version you want to switch to.*) ve uyarı «Önemli: eski sürümler mevcut ayarları desteklemeyebilir» (*Choose carefully, as older versions may not be compatible with current configurations.*).

Geçiş: `POST /installXray/:version`. Senaryo:

Örnek. Belirli bir Xray-core sürümüne geçmek (oturum çerezi önceden kimlik doğrulamasıyla alınmış olmalıdır):

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v25.6.8' \
  -b cookie.txt
```

Burada `v25.6.8` – `GET /getXrayVersion` tarafından döndürülen listedeki etikettir. Sürüm bu listede bulunmalıdır, aksi takdirde panel reddetme yanıtı verir.

1. Seçilen sürüm güncel sürüm listesinde doğrulanır (yoksa – red).
2. Xray durdurulur.

3. Mevcut İS ve mimariye uygun `Xray-<os>-<arch>.zip` arşivi GitHub'dan indirilir (amd64/64, arm64-v8a, arm32-v7a/v6/v5, 386/32, s390x desteklenir; Windows için – `xray.exe`). Arşiv ve ikili dosya boyutu 200 MB ile sınırlıdır.
4. İkili dosya atomik olarak değiştirilir (geçici dosya + yeniden adlandırma yoluyla) ve çalıştırılabilir olarak işaretlenir.
5. Xray yeniden başlatılır.

Geçiş öncesinde «Xray Sürümünü Değiştir» (*Do you really want to change the Xray version?*) iletişim kutusu ve «Bu işlem Xray sürümünü #version# olarak değiştirecek» açıklaması gösterilir. Başarı durumunda – «Xray başarıyla güncellendi» (*Xray updated successfully*) bildirimi.

3.12. Panel Güncellemesi (3X-UI)

Panel güncelleme denetimi bloğu. Veriler `GET /getPanelUpdateInfo` aracılığıyla gelir:

Alan	Açıklama
Mevcut Panel Sürümü	Kurulu panelin sürümü.
En Son Panel Sürümü	GitHub'dan alınan en son 3x-ui sürümü.
Güncelleme Mevcut	En son sürümün mevcut sürümden daha yeni olduğunu gösteren bayrak. Güncelleme gerekmiyorsa «Panel güncel» / «Güncellendi» gösterilir.

«**Paneli Güncelle**» (*Update Panel*) düğmesi `POST /updatePanel` işlemini başlatır. İpucu: «Bu işlem 3X-UI'yi en son sürüme güncelleyecek ve panel servisini yeniden başlatacak». Başlamadan önce – «Paneli gerçekten güncellemek istiyor musunuz?» onayı ve «Bu işlem 3X-UI'yi #version# sürümüne güncelleyecek ve panel servisini yeniden başlatacak» metni.

Özellikler ve sınırlamalar:

- Kendi kendini güncelleme yalnızca **Linux'ta** desteklenir (diğer İS'lerde hata döndürülür).
- Güncelleme betiği resmi depodan indirilir (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh` , sınır 2 MB) ve mümkünse `systemd-run` aracılığıyla izole bir şekilde `bash` ile çalıştırılır.
- Başarılı başlatma durumunda «Panel güncellemesi başladı» (*Panel update started*) gösterilir; güncelleme denetimi başarısız olursa – «Panel güncelleme denetimi başarısız oldu». Kurulum sırasında «Kurulum devam ediyor. Sayfayı yenilemeyin» uyarısı gösterilir.

3.13. Coğrafi Dosya Güncellemesi (GeoIP / GeoSite)

Coğrafi veritabanı güncelleme düğmesi/iletişim kutusu `POST /updateGeofile` (tüm dosyalar) veya `POST /updateGeofile/:fileName` (tek dosya) çağrısını yapar. Güncelleme katı bir ad ve kaynak beyaz listesiyle çalışır:

Dosya	Kaynak
<code>geoip.dat</code> , <code>geosite.dat</code>	<code>Loyalsoldier/v2ray-rules-dat</code> (latest)
<code>geoip_IR.dat</code> , <code>geosite_IR.dat</code>	<code>chocolate4u/Iran-v2ray-rules</code> (latest)

Dosya	Kaynak
geoup_RU.dat , geosite_RU.dat	runetfreedom/russia-v2ray-rules-dat (latest)

Davranış:

- Dosya adı doğrulanır: `..`, eğik çizgi, mutlak yollar yasaktır; yalnızca `[a-zA-Z0-9._-]+.dat` izin verilir. Beyaz listede olmayan dosyalar indirilmez.
- Koşullu istek `If-Modified-Since` kullanılır: kaynak sunucuda dosya değişmemişse (HTTP 304) yeniden indirilmez, yalnızca zaman damgası güncellenir.
- İndirme sonrasında Xray **yeniden başlatılır** (yeni veritabanlarını alması için).

Örnek. Yalnızca Rusya coğrafi veritabanlarını güncellemek, diğer dosyalara dokunmadan:

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoup_RU.dat' -b cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b cookie.txt
```

Beyaz listedeki tüm dosyaları aynı anda güncellemek için – dosya adı olmadan `POST /updateGeofile` çağırın.

- İletişim kutuları: tek dosya için «Coğrafi dosyayı gerçekten güncellemek istiyor musunuz?» ve «Bu işlem #filename# dosyasını güncelleyecek», «Tümünü Güncelle» düğmesi için ise «Bu işlem tüm coğrafi dosyaları güncelleyecek». Başarı – «Coğrafi dosyalar başarıyla güncellendi».

3.14. Veritabanı Yedekleme ve Geri Yükleme

«Yedek & Geri Yükle» (*Backup & Restore*) bloğu. Davranış kullanılan VTYS'e (varsayılan olarak SQLite veya PostgreSQL) bağlıdır.

Veritabanı Dışa Aktarma (Yedek)

«Veritabanını Dışa Aktar» / «Yedek» (*Back Up*) düğmesi `GET /getDb` çağrısını yapar. Dosya ek olarak gönderilir:

- SQLite:** önce kontrol noktası yapılır (WAL temizleme), ardından `x-ui.db` dosyası indirilir. İpucu: «Mevcut veritabanınızın yedeğini içeren .db dosyasını indirmek için tıklayın...».
- PostgreSQL:** özel biçimde (`pg_dump --format=custom --no-owner --no-privileges`) `x-ui.dump` dökümü indirilir. Sunucuda PostgreSQL istemci araçları kurulu olmalıdır; aksi takdirde `pg_dump` eksikliği hakkında hata verilir.

Veritabanı İçe Aktarma (Geri Yükleme)

«Veritabanını İçe Aktar» / «Geri Yükle» (*Restore*) düğmesi `POST /importDB` aracılığıyla dosyayı yükler (form alanı `db`). İpucu: «Veritabanını yedekten geri yüklemek için .db dosyasını seçip yüklemek üzere tıklayın».

SQLite için geri alma destekli güvenli senaryo:

- Dosya SQLite biçiminde doğrulanır, geçici dosyaya kaydedilir ve bütünlük denetimi yapılır.

2. Xray durdurulur, mevcut VT kapatılır ve *.backup olarak yeniden adlandırılır (geri dönüş).
3. Yeni dosya çalışan VT konumuna yerleştirilir, başlatma ve göç gerçekleştirilir. Bir şeyler ters giderse geri dönüş yüklenir.
4. Xray yeniden başlatılır.

PostgreSQL için .dump yüklenir (PGDMP imzası doğrulanır) ve pg_restore --clean --if-exists --single-transaction ... aracılığıyla uygulanır. İpucu açıkça uyarır: «Bu işlem mevcut tüm verilerin yerini alır».

Mesajlar: «Veritabanı başarıyla içe aktarıldı», «Veritabanı içe aktarılırken hata oluştu», «...veritabanı okunurken», «...veritabanı alınırken».

Göç Dosyası (SQLite ile PostgreSQL Arasında)

«Göç Dosyasını İndir» (*Download Migration*) düğmesi GET /getMigration çağrısını yapar ve paneli farklı bir VTYS'de çalıştırmak için taşınabilir bir dışa aktarma oluşturur:

- **SQLite** üzerinde x-ui.dump (metin SQL dökümü) indirilir.
- **PostgreSQL** üzerinde x-ui.db indirilir – PostgreSQL verilerinden oluşturulmuş hazır bir SQLite veritabanı.

3.15. Ek Arayüz Öğeleri

- **Çevrimiçi İstemci Göstergesi.** Gösterge paneli online satırını (*Online Clients* / «Çevrimiçi İstemciler») tutar – aktif bağlantısı olan istemci sayısı. Xray çalışırken hesaplanır (aksi takdirde 0) ve aynı 2 saniyelik ritimle geçmişe yazılır. Grafik – «Çevrimiçi» sekmesi.
- **Sistem Geçmişi (Grafikler).** «Grafikler» → «Sistem Geçmişi» düğmesi/bölümü şu sekmeleri içerir: «Bant Genişliği», «Paketler», «Disk G/Ç», «Çevrimiçi», «Yük», «Bağlantılar», «Disk Kullanımı». Veriler GET /history/:metric/:bucket aracılığıyla çekilir; izin verilen toplama aralıkları (bucket, sn): **2, 30, 60, 120, 180, 300, 720, 1440, 2880** (son üçü aralık seçicideki **12h, 24h** ve **48h** ön ayarlarına karşılık gelir), sekmeye en fazla 60 nokta gelir. Metrik döngüsel tamponu verileri **48 saate** kadar saklar; dolayısıyla grafikler (CPU, RAM, trafik, paketler, bağlantılar, disk, çevrimiçi, yük) iki güne kadar olan dönemi kapsar. İzin verilen metrikler: cpu, mem, swap, netUp, netDown, pktUp, pktDown, diskRead, diskWrite, diskUsage, tcpCount, udpCount, online, load1, load5, load15. «Son 2 dakika» etiketi bucket = 2'ye (gerçek zamanlı mod) karşılık gelir.

Örnek. Son ~2 dakikalık CPU yükü satırını almak (bucket = 2 s, en fazla 60 nokta) ve 5 dakikayla toplulaştırılmış aynı satırı almak (bucket = 300 s):

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

Metrik herhangi bir izin verilen değerle değiştirilebilir (mem , netUp , tcpCount , load1 vb.). 2, 30, 60, 120, 180, 300, 720, 1440, 2880 listesi dışındaki bucket değerleri reddedilir.

- **Xray Metrikleri** – Xray bellek tüketimi ve çöp toplama metrikleri (satırlar xrAlloc, xrSys, xrHeapObjects, xrNumGC, xrPauseNs) ve «Gözlemevi» (giden bağlantı durumu) içeren ayrı bir blok. Xray yapılandırmasında

`metrics` bloğu (`listen 127.0.0.1:11111` ,etiket `metrics_out`) tanımlandığında çalışır; aksi takdirde «Xray metrik uç noktası yapılandırılmamış» gösterilir.

Örnek Xray metrik kutucuğunu etkinleştiren blok.Xray ayarları bölümünde aynı anda hem `metrics` (etiketle) hem de bu etiketi dinleyen inbound bulunmalıdır:

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

`127.0.0.1:11111` adresi kasıtlı olarak dışarıya açılmaz – panel bunu yerel olarak sorgular.

- **Karanlık Tema Geçişi.** Gösterge panelinin kendisinde değil, genel menü/başlıkta bulunur. Seçenekler: «Tema» (*Theme*) ile «Koyu» ve «Çok Koyu» (*Ultra Dark*) değerleri. Bu yalnızca görsel bir tema ayarıdır, panelin çalışmasını etkilemez.
- **Gösterge paneli çevresindeki diğer bağlantılar** (menü/alt panelden): «Günlükler», «Yapılandırma» – Xray'in son JSON çıktısını görüntüleme (`GET /getConfigJson`), «Belgeler».

4. Inbounds: oluşturma ve genel parametreler

«**Gelen Bağlantılar**» (İng. *Inbounds*) bölümü, istemcilerin bağlandığı tüm Xray giriş noktalarının listesidir. Her inbound hem "panel" alanlarını (açıklama, trafik limiti, sıfırlama takvimi) hem de ham JSON yapılandırma bloklarını (`settings` , `streamSettings` , `sniffing`) saklar.

Oluşturma işlemi «**Bağlantı Oluştur**» (*Add Inbound*) düğmesiyle, düzenleme ise «**Bağlantıyı Değiştir**» (*Modify Inbound*) düğmesiyle yapılır. Her iki işlem sırasıyla `POST /add` ve `POST /update/:id` API uç noktalarına gönderilir.

Aşağıda formun belirli bir protokolün ayarlarıyla (istemciler, şifreleme, REALITY/TLS) **ilgili olmayan** ve aktarım/akış ayarlarıyla («**Akış**», «**Güvenlik**» sekmeleri) **ilgili olmayan** tüm alanları açıklanmaktadır — bunlar ayrı bölümlerin konusudur.

4.1. Genel form alanları

Remark (Açıklama)

Parametre	Değer
Alan	<code>remark</code>
Tür	dize
Varsayılan	boş

Inbound'un listede ve iletişim kutusu başlıklarında görüntülenen, insan tarafından okunabilir adıdır («Bağlantı "{remark}" silinsin mi?» vb.). Alan etiketi «**Açıklama**»'dır. Xray'in çalışmasını etkilemez, yalnızca yönetim kolaylığı için gereklidir; dışa aktarılan dosya adlarına ve toplu işlem onaylarına eklendiğinden benzersiz ve anlamlı isimler kullanılması önerilir.

Protocol (Protokol)

Parametre	Değer
Alan	<code>protocol</code>
Etiket	« Protokol »
Doğrulama	<code>required,oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun</code>

inbound protokolünün açılır listesi. İzin verilen değerler:

Değer	Açıklama
<code>vmess</code>	
<code>vless</code>	
<code>trojan</code>	

Değer	Açıklama
shadowsocks	
wireguard	
hysteria	Hysteria v2 – streamSettings.version = 2 ile hysteria 'dır, ayrı bir protokol yoktur
http	
mixed	tek portta socks/http
tunnel	
tun	doğrulamayı tarafından kabul edilir, ayrı bir protokol sabiti yoktur

Alan zorunludur (required). Protokol seçimi, hangi istemci ayarı alanlarının ve hangi aktarımın kullanılabileceğini belirler (protokole özgü bölümlere bakınız).

Önemli: kaydedilirken servis streamSettings değerini normalleştirir. Aktarım ayarları yalnızca vmess , vless , trojan , shadowsocks , hysteria protokolleri için bırakılır; diğerleri için (http , mixed , tunnel , wireguard , tun) streamSettings alanı **zorunlu olarak temizlenir**.

tunnel /TPProxy türündeki inbound'larda streamSettings bloğu security anahtarı içermiyorsa (aktarımsız varyant), form streamSettings.security Invalid input doğrulama hatası olmadan açılır ve kaydedilir.

Listen IP (Dinleme IP'si)

Parametre	Değer
Alan	listen
Tür	dize
Varsayılan	boş → Xray tüm IP'lerde dinler (0.0.0.0)

inbound'un bağlantıları kabul ettiği IP adresi. Alan ipucu:

«Tüm IP adreslerini dinlemek için boş bırakın».

Xray yapılandırması oluşturulurken boş değer 0.0.0.0 ile değiştirilir. IP'nin yanı sıra bu alan **Unix soket yolunu** da kabul eder – ipucu:

«TCP bağlantı noktası yerine soket dinlemek için bir Unix soket yolu (örneğin /run/xray/in.sock) veya @ önekli soyut soket adı (örneğin @xray/in.sock) de belirtebilirsiniz; bu durumda bağlantı noktasını 0 olarak ayarlayın».

Böylece alan iki Unix soket biçimini kabul eder: dosya sistemi yolu (/run/xray/in.sock) ve @ önekli soyut soket adı (@xray/in.sock). Her iki durumda da Port değerini 0 olarak ayarlayın.

Bu alan, inbound'u tek bir arayüzle sınırlamak gerektiğinde (örneğin yalnızca Nginx'in arkasında fallback hedefi olarak çalışan inbound için `127.0.0.1`) veya inbound Unix soketinde dinlediğinde değiştirilir.

Örnek. Yalnızca yerel arayüzde dinleyen (Nginx'in arkasındaki tipik fallback hedefi) ve Unix soketinde dinleyen inbound:

```
listen = 127.0.0.1    port = 8443
listen = /run/xray/in.sock    port = 0
```

Port (Bağlantı Noktası)

Parametre	Değer
Alan	port
Etiket	«Bağlantı Noktası»
Doğrulama	gte=0, lte=65535
Varsayılan	– (kullanıcı tarafından belirlenir)

TCP/UDP dinleme bağlantı noktası. 0 ile 65535 arasında değerlere izin verilir. 0 değeri yalnızca Unix soketinde dinlemeyle birlikte kullanılır (yukarıya bakınız).

Kaydedilirken servis bağlantı noktası çakışmasını kontrol eder: iki inbound aynı aktarım (TCP/UDP) için çıkan listen:port değerlerini aynı anda kullanamaz. Aktarım, protokolden ve streamSettings / settings değerinden hesaplanır: örneğin hysteria ve wireguard her zaman UDP kullanır, kcp / quic UDP kullanır, diğerlerin büyük çoğunluğu ise TCP kullanır. Çakışma durumunda kaydetme hatayla reddedilir.

Panel ayrıca **dahili Xray API'sinin ayrılmış bağlantı noktasının** (api etiketi, varsayılan olarak `127.0.0.1` üzerinde `62789`) kullanılmasına izin vermez: dinleme adresi loopback'te bu bağlantı noktasıyla çıkan yerel TCP inbound aynı çakışma hatası ile reddedilir. Gerçek API bağlantı noktası Xray yapılandırma şablonundan okunur (yedek değer `62789`). Düğümlerde (nodes) bu kısıtlama geçerli değildir – bunların kendi Xray'i vardır.

Xray etiketi (Tag , benzersiz), bağlantı noktası ve aktarımdan in-<bağlantı_noktası>-<tcp|udp|tcpudp|any> biçiminde otomatik olarak oluşturulur; bir düğümde dağıtılan inbound için n<nodeId>-öneki eklenir. Çakışma durumunda etiketin sonuna -2 , -3 vb. eklenir. Kullanıcı genellikle etiketi düzenlemez.

Total traffic (Toplam trafik, GB)

Parametre	Değer
Alan	total (bayt cinsinden)
Etiket	«Toplam Kullanım»
Varsayılan	0

inbound'un toplam trafik limiti. Formda değer gigabayt cinsinden girilir, veritabanında bayt cinsinden saklanır. Alan ipucu:

«= Limitsiz. (birim: GB)».

Yani **0 limitsiz anlamına gelir**. Bu, tüm inbound düzeyindeki limittir (tek tek istemcilerin değil); gerçek harcanan trafik **up** (gönderilen) ve **down** (alınan) alanlarında saklanır ve **total** ile karşılaştırılır.

Expiry date / Duration (Bitiş tarihi / süre)

Parametre	Değer
Alan	<code>expiryTime</code> (Unix zaman damgası)
Etiket	« Bitiş Tarihi » (İng. <i>Duration</i>)
Varsayılan	boş / 0

inbound'un geçerlilik süresi. İpucu:

«Sonsuz olması için boş bırakın».

Boş değer (**0**) süresi sınırsız inbound anlamına gelir. Değer Unix zaman damgası olarak saklanır; form hem belirli bir tarih hem de gün cinsinden süre (geçerli andan itibaren göreceli sayım – İng. alan etiketi *Duration*) girmeye olanak tanır.

Enabled (Etkin)

Parametre	Değer
Alan	<code>enable</code>
Etiket	« Etkinleştir » (İng. <i>Enabled</i>)
Varsayılan	oluşturma sırasında belirlenir

inbound'un etkinlik göstergesi. Bu bayrağın listede değiştirilmesi, tam güncelleyin aksine ayrı bir "hafif" `POST / setEnable/:id` uç noktasıyla işlenir – bu, binlerce istemcisi olan bir inbound'da her geçiş tıklamasında tüm `settings` bloğunun (tüm istemcilerin) yeniden serileştirilmemesi için özel olarak yapılmıştır. inbound devre dışı bırakıldığında çalışan Xray'den kaldırılır, etkinleştirildiğinde geri eklenir.

Node / Deploy to (Düğüm / Dağıt)

Parametre	Değer
Alan	<code>nodeId</code>
Etiket	« Şuraya Dağıt », « Yerel Panel »

Parametre	Değer
Varsayılan	boş (yerel panel)

inbound'un fiziksel olarak nerede çalıştığına seçimi: yerel panelde mi yoksa kayıtlı düğümlerden birinde mi. Uygulama detayı: `nodeId = 0`, `nil` olarak normalleştirilir; `0` geçerli bir düğüm id'si değil, form bağlamasının bir eseridir; `nil / 0` yerel panel anlamına gelir. Çevrimdışı bir düğümde inbound kaydedilirken «düğüm yeniden bağlandığında değişiklik senkronize edilecek» bildirimi görünebilir.

Bağlantı adresi stratejisi (Share address strategy)

Parametre	Değer
Alan	strateji + (isteğe bağlı) özel adres
Etiket	«Bağlantı Adresi Stratejisi» (İng. <i>Share address strategy</i>)
Varsayılan	«inbound dinleme adresi» (<i>Inbound listen</i>)

Açılır liste, bu inbound'un **dışa aktarılan paylaşım bağlantılarına ve QR kodlarına** hangi adresin ekleneceğini belirler. Değerler:

Değer	Etiket	Ne eklenir
<code>node</code>	«Düğüm adresi» (<i>Node address</i>)	inbound'un üzerinde çalıştığı düğümün adresi
<code>listen</code>	«inbound dinleme adresi» (<i>Inbound listen</i>)	inbound'un kendi dinleme adresi
<code>custom</code>	«Özel» (<i>Custom</i>)	«Özel Paylaşım Adresi» (<i>Custom share address</i>) alanından alınan kendi adres

«Özel» seçildiğinde «Özel Paylaşım Adresi» alanı görünür; buraya şema ve bağlantı noktası **olmadan** bir ana bilgisayar adı veya IP girilir (değer doğrulanır). «Düğüm adresi» seçeneği listede yalnızca bu inbound'un üzerinde çalışabileceği etkin bir düğüm varsa gösterilir; aksi takdirde gizlenir ve değer «inbound dinleme adresi»'ne döndürülür.

Bu strateji **yalnızca** doğrudan paylaşım bağlantılarını ve QR kodlarını etkiler. Abonelik çıktısını **etkilemez** — orada adres panelin olağan mantığıyla belirlenir.

4.2. Sniffing (Koklama)

«Koklama» sekmesi, ham JSON olarak saklanan Xray yapılandırmasının `sniffing` bloğunu düzenler. Sniffing, Xray'in yönlendirme amacıyla bağlantı içindeki gerçek alan adını/protokolü "gözetlemesine" olanak tanır.

Alt alan	Etiket	Amaç
<code>enabled</code>	(sekme geçişi)	inbound için koklama işlevini etkinleştirir/devre dışı bırakır
<code>destOverride</code>	—	Hedef adresin yakalandığı protokollerin listesi: <code>http</code> , <code>tls</code> , <code>quic</code> , <code>fakedns</code>
<code>metadataOnly</code>	«Yalnızca meta veri»	Yalnızca bağlantı meta verilerini kullanır, yük okumaz

Alt alan	Etiket	Amaç
<code>routeOnly</code>	«Yalnızca yönlendirme»	Koklama sonucunu yalnızca yönlendirme için uygular, hedef adresi değiştirmez
<code>domainsExcluded</code>	«Hariç tutulan alan adları»	Koklamadan hariç tutulan alan adları
(hariç tutulan IP'ler)	«Hariç tutulan IP'ler»	Koklamadan hariç tutulan IP adresleri

- **`destOverride`** – bir dizi koklayıcı: `http` (HTTP Host başlığından alan adını belirler), `tls` (SNI'dan), `quic` (QUIC ClientHello'dan), `fakedns` (FakeDNS havuzuyla eşleştirir). Alan adını belirlemek için genellikle `http` ve `tls` etkinleştirilir.

sniffing bloğu örneği (HTTP ve TLS aracılığıyla alan adını belirle, sonucu yalnızca yönlendirme için kullan, yerel ağa dokunma):

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **`metadataOnly`** – etkinleştirildiğinde Xray ilk paketin içeriğini okumaz ve yalnızca meta verilere dayanır; verilerin "gözetlenemeyeceği" protokolleri bozmamak için kullanışlıdır.
- **`routeOnly`** – koklama sonucu yalnızca yönlendirme kuralları tarafından kullanılır; outbound'daki bağlantı adresi tespit edilen alan adıyla değiştirilmez.

Not: panel `sniffing` değerini opak bir JSON bloğu olarak saklar ve kaydetme sırasında hiçbir şey eklemeyiz – bu onay kutularının varsayılan değerleri istemci uygulama tarafı tarafından oluşturulur. Ham blok, aşağıda açıklanan «inbound JSON» bölümü aracılığıyla düzenlenebilir.

4.3. Allocate (Bağlantı noktası dağıtım stratejisi)

`streamSettings` içindeki `allocate` bloğu, Xray'in dinleme bağlantı noktalarını nasıl dağıttığını yönetir. Bu Xray yapılandırmasının bir parçasıdır; panel bunu `streamSettings /inbound JSON`'un bir parçası olarak saklar ve iletir. Parametreler (Xray-core terminolojisine göre):

Alt alan	Amaç	Değerler / varsayılan
<code>strategy</code>	Bağlantı noktası tahsis stratejisi	<code>always</code> – her zaman belirtilen bağlantı noktasını dinle (varsayılan); <code>random</code> – aralık içinde dinlenen bağlantı noktalarını periyodik olarak değiştir
<code>refresh</code>	<code>random</code> kullanılırken bağlantı noktası değiştirme aralığı (dakika)	dakika cinsinden tam sayı (5 önerilir; minimum 2)

Alt alan	Amaç	Değerler / varsayılan
concurrency	random kullanılırken aynı anda açık tutulacak bağlantı noktası sayısı	tam sayı (varsayılan 3; bağlantı noktası aralığı genişliğinin üçte birini geçemez)

`strategy: always`, inbound'u tek bir bağlantı noktasında tutar (standart mod). `strategy: random`, inbound'un bağlantı noktası aralığında periyodik olarak "zıplaması" gereken engellemeyi önleme senaryoları için gereklidir; bu durumda `refresh` ve `concurrency` anlamlıdır. Bu değerleri yalnızca rastgele bağlantı noktası modunu bilinçli olarak kullanırken değiştirin.

streamSettings içindeki allocate bloğu örneği (rastgele bağlantı noktası modu: 3 bağlantı noktasını açık tut, her 5 dakikada bir değiştir):

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

Bunun çalışması için inbound'un `port` değeri bir aralık olarak ayarlanmalıdır (örneğin `20000-20100`).

4.4. External Proxy (Harici proxy)

«External Proxy» alanı, davet bağlantısı oluşturma ayarlarıyla ilgilidir ve inbound'un `streamSettings` değerinde saklanır. Gerçek `listen:port` inbound yerine istemci bağlantılarına eklenen alternatif harici adreslerin listesini tanımlar (ana bilgisayar/bağlantı noktası, gerektiğinde zorunlu TLS – «Zorunlu TLS» ile).

istemcilerin doğrudan sunucuya değil, harici bir proxy/reverse/CDN aracılığıyla bağlanması gerektiğinde kullanılır: bu durumda paylaşılan bağlantılarda bu ön ucu genel adresi belirtilir. Xray'in bağlantı kabul sürecini etkilemez – bu, oluşturulan bağlantıların yalnızca "kozmetik" düzenlemesidir. İlgili form alanları: «Zorunlu TLS», «Fingerprint», her kaydın etiketleri.

4.5. Fallbacks (Fallback'ler)

«Fallback'ler» bölümü, inbound istemcilerinden hiçbirisiyle eşleşmeyen bağlantılar için yeniden yönlendirme kurallarını tanımlar. TLS aktarımındaki (VLESS/Trojan TCP-TLS) ana inbound için kullanılabilir. `GET /:id/fallbacks` / `POST /:id/fallbacks` uç noktaları aracılığıyla yönetilir.

Bölüm ipucu:

«Bu inbound'daki bir bağlantı hiçbir istemciyle eşleşmediğinde başka bir yere yönlendirilir. Yönlendirme alanlarının (SNI / ALPN / Path / xver) aktarımından otomatik doldurulması için aşağıdan bir alt inbound seçin ya da seçimi boş bırakıp Nginx gibi harici bir sunucuya yönlendirmek için Dest değerini doğrudan girin (örneğin 8080 veya 127.0.0.1:8080). Her alt inbound 127.0.0.1 adresinde `security=none` ile dinlemelidir».

Fallback'ler bölümü yalnızca TLS veya REALITY güvenliğiyle RAW (TCP) üzerindeki VLESS/Trojan inbound için gösterilir. Yeni bir inbound `security=none` ile başlar, bu nedenle bölüm başta mevcut olmayabilir. Bu durumda (VLESS/Trojan, RAW/TCP, güvenlik henüz yapılandırılmamış), bölüm yerine yerleşik bir ipucu görüntülenir: fallback'ler «**Güvenlik**» sekmesinde TLS veya Reality seçildikten sonra kullanılabilir hale gelecektir.

Fallback satırı alanları

Alan	Varsayılan	Açıklama
(alt inbound)	–	Alt inbound seçimi (etiket « Inbound Seçin »). Seçilirse Name/Alpn/Path/Dest alanları aktarımından otomatik doldurulabilir
Name	boş (= herhangi biri)	Ada (SNI/adı) göre eşleşme koşulu. «herhangi biri» etiketi – « herhangi biri »
Alpn	boş	ALPN'ye göre eşleşme koşulu
Path	boş	Yola göre eşleşme koşulu (alt inbound'un WS/HTTP aktarımları için)
Dest	otomatik	Nereye yönlendirileceği. Yer tutucu « otomatik (listen:alt bağlantı noktası) ». Bağlantı noktası (8080) veya <code>host:port</code> (127.0.0.1:8080) belirtilebilir
Xver	0	PROXY protokol sürümü (« Xver »): 0 – devre dışı, 1 veya 2 – ilgili PROXY protokol sürümü
(sıra)	konuma göre	Kural uygulama sırası; « Yukarı »/« Aşağı » düğmeleriyle ayarlanır

Kaydetme mantığı: ana fallback listesinin tamamı atomik olarak değiştirilir. Seçili alt inbound'u (`childId <= 0`) ve tanımlanmış `Dest` değeri olmayan satır **atlanır**. Seçilen alt inbound ana inbound'un id'siyle aynıysa sıfırlanır. Sonuç JSON oluşturulurken: `Dest` boşsa alt inbound'dan `listen:port` olarak hesaplanır; `0.0.0.0 / :: / :::0`, `127.0.0.1` ile değiştirilir; boş `name / alpn / path` alanları çıktı JSON'una dahil edilmez; `xver` yalnızca 0'dan büyükse eklenir.

Sonuç settings.fallbacks örneği (`alpn=h2` olan trafik `/ws` yolundaki WS hedefine gider, geri kalanı 8080 portundaki yerel Nginx'e gider):

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

`name / alpn / path` olmayan son satır, geri kalanı yakalayan "varsayılan" kuraldır.

Fallbacks bölümü düğmeleri ve ipuçları

- «**Fallback Ekle**» – satır ekler; «**Henüz fallback yok**» – boş durum.
- «**Uygun olanları hızlıca ekle**» / «**Tümünü Ekle**» – henüz bağlı olmayan her uygun inbound için bir fallback satırı ekler. Sonuç: «{n} fallback eklendi» veya «Yeni uygun inbound yok».

- **«Alt inbound'dan doldur»** – seçilen alt inbound'un aktarımından yönlendirme alanlarını (SNI/ALPN/Path/xver) yeniden çeker; tamamlandıktan sonra «Alt inbound'dan dolduruldu».
- **«Yönlendirme alanlarını düzenle»** / **«Gelişmiş gizle»** – satırın ayrıntılı alanlarını gösterir/gizler.
- **«Yönlendirme koşulu»** ve **«Varsayılan – geri kalanı yakalar»** etiketleri her satırın tetikleme koşulunu açıklar.

Fallback'ler kaydedildikten sonra sunucu,yeni `settings.fallbacks` değerinin geçerli olması için Xray'i yeniden başlatır.

4.6. Periyodik trafik sıfırlama

«Trafik Sıfırlama» bloğu, inbound trafik sayaçlarının zamanlamaya göre otomatik sıfırlanmasını yapılandırır. Açıklama:

«Belirtilen aralıklarla trafik sayacını otomatik olarak sıfırla».

Parametre	Değer
Alan	<code>trafficReset</code>
Doğrulama	<code>omitempty,oneof=never hourly daily weekly monthly</code>
Varsayılan	<code>never</code>
Eşlik eden alan	<code>lastTrafficResetTime</code> – son sıfırlama zaman damgası (etiket «Son Sıfırlama»)

Açılır liste:

Değer	Etiket
<code>never</code>	«Hiçbir zaman»
<code>hourly</code>	«Saatlik»
<code>daily</code>	«Günlük»
<code>weekly</code>	«Haftalık»
<code>monthly</code>	«Aylık»

Her periyot için ilgili zamanlamaya göre çalışan bir cron görevi kaydedilir (`@hourly` , `@daily` , `@weekly` , `@monthly`). Görev, belirtilen `trafficReset` değerine sahip tüm inbound'ları seçer ve her biri için hem inbound'un kendi sayaçlarını (`up=0` , `down=0`) **hem de** tüm istemcilerinin trafiğini sıfırlar. Yani periyodik sıfırlama hem inbound'u hem de istemcilerini etkiler.

Alan değeri örneği. Sayaçların her ayın birinde sıfırlanması için formda **«Aylık»** seçilir; bu şöyle kaydedilir:

```
{ "trafficReset": "monthly" }
```

`never` değeri (varsayılan), otomatik sıfırlamayı tamamen devre dışı bırakır.

4.7. inbound JSON (gelişmiş)

«inbound JSON Bölümleri» bölümü, inbound'un ham JSON bloklarına doğrudan erişim sağlar. Açıklama:

«Inbound'un tam JSON'u ve settings, sniffing ve streamSettings için ayrı düzenleyiciler».

Kullanılabilir düzenleyiciler:

Sekme	Etiket	Ne düzenler
Tümü	«Tüm alanların tek bir düzenleyicide bulunduğu tam inbound nesnesi»	tüm Inbound nesnesi
Ayarlar	«Xray settings bloğunun sarmalayıcısı»	settings alanı
Sniffing	«Xray sniffing bloğunun sarmalayıcısı»	sniffing alanı
Stream	«Xray stream bloğunun sarmalayıcısı»	streamSettings alanı

Bu alanlar iç içe JSON nesneleri olarak serileştirilir: boş bloklar `null` olarak döndürülür, geçerli JSON olmayan metin ise verilerin kaybolmaması için dizeye sarılır. Kaydetme sırasında ayrıştırma hataları «**Gelişmiş JSON**» önekiyle görüntülenir.

«inbound JSON» görüntüleme penceresi ve inbound içe aktarma penceresi, sözdizimi vurgulamalı tam özellikli bir kod düzenleyici kullanır (sıradan metin alanı yerine): yapılandırma görüntüleme, yalnızca okuma modunda vurgulu şekilde; içe aktarma ise düzenlenebilir modda – bu, okuma ve düzenlemeyi kolaylaştırır.

4.8. inbound işlemleri: QR / Edit / Reset / Delete ve istatistikler

Liste ve inbound kartında şu işlemler kullanılabilir («Menü» menüsü):

Trafik istatistikleri

inbound için toplu trafik görüntülenir: «Gönderilen/alınan» (up / down alanları), «Toplam trafik», «Toplam bağlantı». Kartta ayrıca «Oluşturuldu», «Güncellendi», «Bitiş tarihi» de yer alır.

inbound listesinde her inbound için geçerli trafik hızını (çıkış/indirme) gösteren **Speed** sütunu bulunur; değer, sorgular arası sayaç artışlarından hesaplanır; aynı canlı hız, inbound istatistik penceresinde de gösterilir. Bir sonraki sorgu artış sağlamazsa hız değeri sıfırlanır.

inbound sayfasındaki istemci özetinde durum, «tükenmiş/sona ermiş» önceliğine göre belirlenir: süresi dolmuş veya trafiği tükenmiş (ve otomatik görevin `enable` değerini kaldırdığı) istemciler, gri «Devre Dışı» (Disabled) durumu yerine «Tükenmiş/Sona Ermiş» (Depleted/Ended) durumuna dahil edilir ve iki kez sayılmaz. Sınıflandırma, istemci kartının kendisinde gösterilenle örtüşür ve birden fazla inbound'a bağlı istemcileri doğru şekilde hesaba katar.

QR kodu ve bağlantı kopyalama

- «Ayrıntılar» – bağlantı ve abonelik bağlantılarını genişletir.
- İstemci QR kodu: ipucu «Kopyalamak için QR koduna tıklayın».

- «**Bağlantıyı Kopyala**» (İng. *Copy URL*), «**Bağlantıları Dışa Aktar**».

Edit (Düzenle)

«**Bağlantıyı Değiştir**» – düzenleme formunu açar (`POST /update/:id`). Güncellenirken servis mevcut kaydı yeniden okur, değiştirilen alanları aktarır, gerektiğinde etiketi yeniden oluşturur (eski etiket otomatik oluşturulmuşsa) ve Xray çalışma zamanını senkronize eder. Başarı – «**Bağlantı başarıyla güncellendi**» bildirimi.

Reset Traffic (Trafik Sıfırla)

«**Trafik Sıfırla**» – bu inbound'un `up / down` sayaçlarını sıfırlar (`POST /:id/resetTraffic` , `up=0` , `down=0`) olarak ayarlar). Onay:

«"{remark}" trafiği sıfırlansın mı?» / «Bu bağlantının gönderme/alma sayaçlarını 0'a sıfırlar».

inbound trafiğini sıfırlamak, istemci sayaçlarına **dokunmaz** (bunlar için ayrı «İstemci trafiğini sıfırla» işlemleri vardır). Sıfırlamadan sonra Xray yeniden başlatılır. Başarı – «**Gelen trafik sıfırlandı**» bildirimi. Toplu seçenek de mevcuttur – «**Tüm bağlantıların trafiğini sıfırla**» (`POST /resetAllTraffics`).

Delete (Sil)

«**Bağlantıyı Sil**» (`POST /del/:id`). Onay:

«"{remark}" bağlantısı silinsin mi?» / «Bağlantı ve tüm istemcileri silinecektir. Bu işlem geri alınamaz».

Silme işlemi, inbound'u çalışan Xray'den kaldırır (gerekirse yeniden başlatarak). Başarı – «**Bağlantı başarıyla silindi**» bildirimi. Toplu silme – `POST /bulkDel` , öge bazında raporlama ve en fazla bir Xray yeniden başlatması ile.

inbound istemcileriyle diğer işlemler

Menüde ayrıca şunlar bulunur: «**Klonla**» (yeni bağlantı noktasıyla ve boş istemci listesiyle inbound kopyası), «**Tüm İstemcileri Sil**» (`POST /:id/deleteAllClients` – tüm istemcileri siler, inbound kendisi korunur), «**Devre dışı istemcileri sil**», «**İstemcileri Bağla/Ayr**», «**İçe Aktar**»/«**Bağlantıları Dışa Aktar**» (`POST /import`). İstemci işlemlerinin ayrıntıları, istemciler bölümüne aittir.

5. Protokoller

Bir inbound bağlantısı oluşturulurken ilk olarak **Protokol** («Protocol») seçilir. Protokol; Xray-core'un bu inbound için hangi kimlik doğrulama ve trafik şifreleme yöntemini kullanacağını, **settings** içinde hangi alanların doldurulması gerektiğini ve hangi taşıma katmanlarının (**network**) ile güvenlik türlerinin (TLS / REALITY) destekleneceğini belirler.

Protokol alanı inbound oluşturulurken bir kez ayarlanır ve **düzenleme sırasında değiştirilemez** (düzenleme formunda açılır liste kilitlidir). Protokolü değiştirmek için yeni bir inbound oluşturmak gerekir.

5.1. Desteklenen Protokoller Listesi

Sunucu aşağıdaki **Protocol** alan değerlerini kabul eder:

oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto

3.3.0 sürümünden itibaren listeye **mtproto** değeri (Telegram proxy) eklenmiştir.

Yapılandırmadaki değer	Amaç	İstemci modeli
vless	Temel proxy protokolü (inbound oluştururken varsayılan)	UUID'li istemciler, flow ve kuantum sonrası şifreleme desteği
vmess	Xray'in klasik proxy protokolü	UUID ve security parametrelili istemciler
trojan	Sıradan HTTPS gibi görünen proxy	Parola ile kimlik doğrulayan istemciler
shadowsocks	Shadowsocks proxy (SIP022 / 2022-blake3 dahil)	Tek kullanıcı veya birden fazla kullanıcı (2022)
wireguard	WireGuard inbound	İstemciler yerine peer'lar
hysteria	Hysteria inbound (varsayılan olarak sürüm 2)	auth tokenli istemciler
http	Klasik HTTP proxy (forward proxy)	user/pass hesapları, trafik takibi yok
mixed	Birleşik SOCKS + HTTP proxy	user/pass hesapları
tunnel	Şeffaf yönlendirici (xray dokodemo-door)	İstemci yok
tun	TUN arayüzü (yalnızca mevcut olanları görüntüleme)	İstemci yok
mtproto	Telegram proxy (MTPROTO), 3.3.0'da eklendi; Xray değil, ayrı bir mtg süreci tarafından yönetilir	İstemci yok (gizli anahtar ile erişim)

tun hakkında not: bu değer önceden kaydedilmiş inbound'ların **görüntülenmesi** için uyumluluk amacıyla listede tutulmaktadır; ancak güncel backend sürümünde bu türde inbound oluşturulması önerilmez – destek kullanım dışı sayılmaktadır. Bu türde yeni inbound oluşturmanın bir anlamı yoktur.

Hysteria 2 hakkında not: «hysteria2» adında ayrı bir protokol yoktur. Bu, `streamSettings.version = 2` ayarlı `hysteria` protokolüdür. Paylaşım bağlantıları oluşturulurken `hysteria2://` şeması, akış sürümü 2 olduğunda otomatik olarak seçilir.

Tüm protokoller node'lara dağıtımı desteklemez. Yalnızca şunlar node'lara dağıtılabilir: `vless`, `vmess`, `trojan`, `shadowsocks`, `hysteria`, `wireguard`. `http`, `mixed`, `tunnel`, `tun`, `mtproto` protokolleri yalnızca yerel panelde çalışır.

5.2. Hangi Protokoller TLS / REALITY / Taşıma Katmanı Destekler

Belirli bir güvenlik katmanını veya taşıma türünü etkinleştirme imkânı, protokole ve seçilen ağa (`streamSettings.network`) bağlıdır:

Özellik	Kullanılabildiği protokoller	İzin verilen ağlar (<code>network</code>)
TLS	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> (ayrıca <code>hysteria</code> için her zaman)	<code>tcp</code> , <code>ws</code> , <code>http</code> , <code>grpc</code> , <code>httpupgrade</code> , <code>xhttp</code>
REALITY	<code>vless</code> , <code>trojan</code>	<code>tcp</code> , <code>http</code> , <code>grpc</code> , <code>xhttp</code>
flow (<code>xtls-rprx-vision</code>)	yalnızca <code>vless</code>	yalnızca <code>tcp</code> , <code>security = tls</code> veya <code>reality</code> ile
Stream / taşıma («Akış» sekmesi)	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> , <code>hysteria</code>	–

`http`, `mixed`, `tunnel`, `tun`, `wireguard` protokolleri için taşıma sekmesi mevcut değildir – bunların Xray stream ayarları yoktur.

5.3. VLESS

Amaç: temel modern proxy protokolü. XTLS-Vision (`flow`), REALITY ve VLESS katmanında kuantum sonrası şifrelemeyi (`decryption` / `encryption` alanları) destekler. Yeni inbound'lar için varsayılan olarak kullanılır.

`settings` bloğunun alanları:

Alan	Varsayılan değer	Açıklama
<code>clients</code>	<code>[]</code>	İstemci listesi. Her birinde: <code>id</code> (UUID), <code>email</code> (zorunlu), <code>flow</code> , limitler (<code>limitIp</code> , <code>totalGB</code> , <code>expiryTime</code>), <code>enable</code> , <code>tgId</code> , <code>subId</code> , <code>comment</code> , <code>reset</code>
<code>decryption</code>	<code>none</code>	Sunucu tarafında şifre çözme parametresi. UI etiketi: «Şifre Çözme» (İng. «Decryption»)
<code>encryption</code>	<code>none</code>	Eşleşen şifreleme parametresi (istemci bağlantısına aktarılır). Etiket: «Şifreleme» (İng. «Encryption»)
<code>fallbacks</code>	<code>[]</code>	Fallback listesi (fallback'ler bölümüne bakın); <code>network = tcp</code> ve <code>security = TLS</code> veya REALITY olduğunda kullanılabilir

Alan	Varsayılan değer	Açıklama
testseed	(4 sayı: 900, 500, 900, 256)	«Vision testseed» – XTLS-Vision dolgusu için 4 pozitif tam sayı. Yalnızca <code>xtls-rprx-vision</code> flow'lu istemcilere uygulanır, aksi hâlde göz ardı edilir

flow (`xtls-rprx-vision`)

`flow` , inbound üzerinde değil **istemci** üzerinde ayarlanır ve üç değerden birini alır:

Değer	Anlam
`` (boş)	XTLS-flow yok (varsayılan)
<code>xtls-rprx-vision</code>	XTLS-Vision – TCP+TLS/REALITY üzerinde VLESS için önerilen mod
<code>xtls-rprx-vision-udp443</code>	Aynı Vision, ancak UDP/443 (QUIC) işleme ile

`flow` alanı yalnızca şu koşulların tümü karşılandığında seçilebilir: protokol `vless` , `network = tcp` ve `security = tls` ya da `reality` . Formdaki **Vision testseed** alanı da yalnızca aynı koşullarda görüntülenir.

XHTTP için istisna: kuantum sonrası VLESS kimlik doğrulamasının (`encryption / decryption` , `vlessenc`) etkin olduğu `network = xhttp` üzerindeki VLESS'te, REALITY dahil herhangi bir güvenlik katmanından bağımsız olarak `xtls-rprx-vision` flow'u da geçerlidir. Bu durumda panel, `xtls-rprx-vision` değerini paylaşım bağlantılarına ve aboneliklere (Clash/Mihomo formatı dahil) doğru şekilde aktarır; böylece istemci tam olarak Vision içeren yapılandırmayı alır.

Şifre Çözme / Şifreleme (kuantum sonrası VLESS kimlik doğrulaması)

`decryption` ve `encryption` alanları, VLESS protokolünün kendi düzeyinde kimlik doğrulamasını sağlar (taşıma katmanı TLS/REALITY'den bağımsız olarak). Varsayılan olarak her ikisi de `none` 'dır. Formda yanlarında üç düğme bulunur:

- **X25519 kimlik doğrulaması** (İng. «X25519 auth») – X25519 tabanlı bir `decryption / encryption` çifti oluşturur.
- **ML-KEM-768 kimlik doğrulaması** (İng. «ML-KEM-768 auth») – kuantum sonrası (Post-Quantum) seçenek.
- **Temizle** – her iki alanı tekrar `none` olarak sıfırlar.

Düğmelerin altında «Seçilen: {kimlik doğrulama}» durum satırı görüntülenir; değer, `encryption` dizesinin son segmentine göre belirlenir: boş/ `none` → «None», çok uzun anahtar (>300 karakter) → ML-KEM-768, kısa → X25519, diğer → «Özel».

Teknik olarak düğmeler `GET /panel/api/server/getNewVlessEnc` adresine istek gönderir (anahtarlar `xray vlessenc` ile üretilir) ve **her iki** alanı `mlkem768x25519plus.native.<rtt>.<role>` biçimindeki eşleşen değerlerle doldurur (örneğin `decryption = mlkem768x25519plus.native.600s.server-x25519` , `encryption = mlkem768x25519plus.native.0rtt.client-x25519`). `decryption` parametresi sunucuda kalır, `encryption` ise istemci bağlantısına aktarılır.

Önemli: inbound için Xray yapılandırması oluşturulurken panel gereksiz verileri temizler: `settings` içinde kalan `encryption` (istemci tarafına ait) sunucu yapılandırmasından **çıkarılır**. Sunucuda yalnızca `decryption` kalır.

VLESS ne zaman seçilmeli: özellikle REALITY (kendi sertifikası olmadan) veya TLS + XTLS-Vision kombinasyonunda yeni inbound için varsayılan önerilen seçenektir.

Örnek: tek istemcili ve XTLS-Vision'lı VLESS inbound `settings` bloğu. `flow` alanı istemcide, `decryption` ise sunucuda yer alır:

```
{
  "clients": [
    {
      "id": "d342d11e-d424-4583-b36e-524ab1f0afa4",
      "email": "user1",
      "flow": "xtls-rprx-vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

REALITY kombinasyonu için ilgili `streamSettings` bloğu (Transport sekmesi → Security: REALITY) şöyle görünür:

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<приватный ключ X25519>",
    "shortIds": [ "", "6ba85179e30d4fc2" ]
  }
}
```

5.4. VMess

Amaç: Xray'in klasik proxy protokolü. UUID ile kimlik doğrulama; istemcide yük şifreleme yöntemi (`security`) ayrıca yapılandırılır.

`settings` bloğunun alanları:

Alan	Varsayılan değer	Açıklama
<code>clients</code>	<code>[]</code>	İstemci listesi

Her VMess istemcisinde (ortak `email` , `limit` , `enable` , `tgId` , `subId` , `comment` , `reset` alanlarına ek olarak):

İstemci alanı	Varsayılan değer	Açıklama
<code>id</code>	—	İstemci UUID'si
<code>security</code>	<code>auto</code>	VMess yük şifreleme yöntemi. İzin verilen değerler: <code>aes-128-gcm</code> , <code>chacha20-poly1305</code> , <code>auto</code> , <code>none</code> , <code>zero</code>

`security` değerleri:

- `auto` — Xray platforma göre şifre seçer (önerilir);
- `aes-128-gcm` , `chacha20-poly1305` — sabit AEAD şifreleri;
- `none` — yük şifrelemesi yok (yalnızca TLS üzerinde anlamlıdır);
- `zero` — yük şifrelemesi ve kimlik doğrulaması yok.

Geriye dönük uyumluluk: eski kayıtlarda `security: ""` olabilir — okuma sırasında boş dize `auto` olarak dönüştürülür. Sunucu yapılandırması oluşturulurken VMess istemcilerindeki `security` alanı `settings` 'den **silinir**, çünkü inbound için gerekli değildir.

VMess ne zaman seçilmeli: eski istemcilerle veya mevcut yapılandırmalarla uyumluluk için. Yeni dağıtımlarda genellikle VLESS tercih edilir.

5.5. Trojan

Amaç: sıradan HTTPS trafiğini taklit eden proxy. Parola ile kimlik doğrulama. VLESS gibi fallback'leri destekler ve (`network = tcp` ile) REALITY/TLS kullanılabilir.

`settings` bloğunun alanları:

Alan	Varsayılan değer	Açıklama
<code>clients</code>	<code>[]</code>	İstemci listesi
<code>fallbacks</code>	<code>[]</code>	Fallback listesi (<code>network = tcp</code> ve TLS/REALITY ile kullanılabilir)

Her Trojan istemcisinin temel alanı:

İstemci alanı	Varsayılan değer	Açıklama
<code>password</code>	—	İstemci parolası (zorunlu, en az 1 karakter)
<code>email</code>	—	Benzersiz istemci tanımlayıcısı

Diğer istemci alanları ortaktır (`limitIp` , `totalGB` , `expiryTime` , `enable` , `tgId` , `subId` , `comment` , `reset`).

Trojan ne zaman seçilmeli: 443 numaralı portta HTTPS görünümü gerektiğinde; istenmeyen bağlantılar için web sunucusuna (Nginx) fallback ile birlikte kullanılır.

Örnek: yerel web sunucusuna fallback'li Trojan settings bloğu. Geçerli parolası olmayan istenmeyen bağlantılar, 127.0.0.1:8080 'i dinleyen Nginx'e yönlendirilir:

```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

Fallback için network = tcp ve Security = TLS veya REALITY gerekir; aksi hâlde fallbacks alanı kullanılamaz.

5.6. Shadowsocks

Amaç: hafif Shadowsocks proxy. Hem eski AEAD şifrelerini hem de yeni SIP022 yöntemlerini (2022-blake3-*) destekler. Tek kullanıcı veya çok kullanıcı modda çalışabilir.

settings bloğunun alanları:

Alan	Varsayılan değer	Açıklama
method	2022-blake3-aes-256-gcm	inbound şifreleme yöntemi. UI etiketi: «Şifreleme yöntemi» (İng. «Encryption method»)
password	''	inbound parolası (2022 yöntemleri için seçilen yönteme göre otomatik oluşturulur)
network	tcp,udp	Taşıma. Etiket: «Ağ» (İng. «Network»). Seçenekler: tcp,udp (TCP,UDP), tcp , udp
clients	[]	İstemci listesi
ivCheck	false (kapalı)	«ivCheck» geçişi – IV'nin yeniden kullanımına karşı koruma

Şifreleme yöntemleri (method)

İzin verilen değerler:

Yöntem	Kategori
aes-256-gcm	Eski AEAD
chacha20-poly1305	Eski AEAD
chacha20-ietf-poly1305	Eski AEAD
xchacha20-ietf-poly1305	Eski AEAD
2022-blake3-aes-128-gcm	SS 2022 (önerilir)
2022-blake3-aes-256-gcm	SS 2022 (varsayılan)

Yöntem	Kategori
2022-blake3-chacha20-poly1305	SS 2022, tek kullanıcı

Panel mantığı:

- **2022 yöntemleri** (2022-blake3-*) «SS 2022» olarak değerlendirilir. 2022-blake3-chacha20-poly1305 yöntemi **tek kullanıcılıdır** (çok kullanıcılı desteklenmez); diğer 2022 yöntemleri birden fazla istemciye izin verir. Parola alanı (anahtar uzunluğunu yöneme göre ayarlayan oluştur düğmesiyle) formda yalnızca 2022 yöntemleri için gösterilir.
- **Eski şifreler** (aes-* , chacha20-*) klasik «tek yöntem + tek parola» şemasıyla çalışır.

Xray başlatılmadan önce normalleştirme: eski şifreler için her istemcinin method değeri inbound yöntemiyle eşleşmelidir (aksi hâlde Xray «unsupported cipher method:» hatasıyla düşer). 2022 yöntemlerinde ise tam tersi – istemcideki method alanı **boş** olmalıdır (aksi hâlde Xray «users must have empty method» ile inbound'u reddeder). Panel, yöntem değiştirildiğinde verileri otomatik olarak düzeltir.

Anahtar boyutu değiştiğinde istemci anahtarlarının yeniden oluşturulması: Shadowsocks-2022'de şifreleme yöntemi farklı anahtar boyutuna sahip bir yöneme değiştirildiğinde (örneğin 2022-blake3-aes-256-gcm ile 2022-blake3-aes-128-gcm arasında) panel, inbound kaydedilirken istemci PSK'larını yeni uzunluğa göre otomatik olarak yeniden oluşturur. Aksi hâlde eski anahtarlar eski uzunlukta kalır ve Xray bunları reddeder. Sonuç olarak: etkilenen istemcilerin aboneliği yeniden alması gerekir – eski bağlantılar çalışmaz.

Shadowsocks ne zaman seçilmeli: TLS maskeleyesi olmayan basit dağıtımlar için; modern seçim 2022-blake3 yöntemleridir.

Örnek: 2022-blake3 yöntemi için Shadowsocks settings bloğu (çok kullanıcılı mod). inbound'un kendi parolası (gerekli uzunlukta base64 anahtarı) vardır, her istemcinin kendi parolası bulunur ve istemcideki method alanı **boştur**:

```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlNjQta2V5LWlcmU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zM1lieXRlcy1iYXNlNjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

Eski şifreler için (aes-256-gcm vb.) tersi geçerlidir: inbound için tek parola, istemcinin method değeri ise inbound yöntemiyle eşleşmelidir.

5.7. Dokodemo-door / Tunnel (şeffaf yönlendirici)

Amaç: şeffaf yönlendirici (panelde `tunnel` protokolü, dokodemo-door davranışını uygular). Trafiği alır ve kimlik doğrulama veya istemci olmaksızın belirtilen adrese/porta iletir.

`settings` bloğunun alanları:

Alan	Varsayılan değer	Açıklama
<code>rewriteAddress</code>	(yok)	«Adresi yeniden yaz» (İng. «Rewrite address») – trafiğin yönlendirileceği hedef adres
<code>rewritePort</code>	(yok)	«Portu yeniden yaz» (İng. «Rewrite port») – hedef port (0–65535)
<code>allowedNetwork</code>	<code>tcp,udp</code>	«İzin verilen ağ» (İng. «Allowed network»). Seçenekler: <code>tcp,udp</code> , <code>tcp</code> , <code>udp</code>
<code>portMap</code>	<code>{}</code>	«Port eşleme» – port→port haritası (Record<string,string>)
<code>followRedirect</code>	<code>false</code> (kapalı)	«Redirect'i takip et» (İng. «Follow redirect») – ele geçirilen bağlantıdan özgün hedef adresini kullan

Tunnel için «Transport» sekmesi: bu tür inbound'da «**Transport**» sekmesi mevcuttur ve yalnızca `sockopt` ayarıyla sınırlıdır – bu, **TProxy** modu için yeterlidir (`sockopt.tproxy` aracılığıyla şeffaf proxy/ yönlendirme). Taşıma seçimi açılır listesi (`network`) ve Tunnel için «Security» sekmesi gizlidir, çünkü bu tür TLS/REALITY desteklemez.

Ne zaman seçilmeli: iç servislere şeffaf proxy veya port yönlendirme için.

5.8. SOCKS / HTTP (`mixed` protokolü)

Bu derlemede ayrı bir `socks` protokolü yoktur – SOCKS ve HTTP proxy, `mixed` protokolünde (birleşik SOCKS + HTTP) birleştirilmiştir. Ayrıca ayrı bir saf `http` proxy mevcuttur.

5.8.1. Mixed (SOCKS + HTTP)

`settings` bloğunun alanları:

Alan	Varsayılan değer	Açıklama
<code>auth</code>	<code>password</code>	«Auth» – kimlik doğrulama modu. Seçenekler: <code>password</code> (kullanıcı adı/parola) veya <code>noauth</code> (kimlik doğrulama yok)
<code>accounts</code>	(isteğe bağlı)	«Hesaplar» – user/pass çiftleri listesi. <code>auth = noauth</code> olduğunda yapılandırmaya yazılmaz
<code>udp</code>	<code>false</code> (kapalı)	«UDP» geçişi – SOCKS üzerinden UDP desteği
<code>ip</code>	<code>127.0.0.1</code>	«UDP IP» – UDP ilişkilendirmeleri için yerel adres. Yalnızca <code>udp</code> etkinken gösterilir

Hesaplar «Ekle» düğmesiyle eklenir; eklenirken rastgele kullanıcı adı (8 karakter) ve parola (12 karakter) oluşturulur, bunlar düzenlenebilir.

5.8.2. HTTP (saf proxy)

Amaç: klasik HTTP forward proxy. Xray düzeyinde istemcileri «faturalandırma» amaçlı takip etmez (email/limit yoktur) – yalnızca hesap listesi bulunur.

settings bloğunun alanları:

Alan	Varsayılan değer	Açıklama
accounts	[]	«Hesaplar» – user/pass çiftleri listesi (her iki alan zorunlu)
allowTransparent	false (kapalı)	«Şeffafa izin ver» (İng. «Allow transparent») – istekleri özgün Host başlığıyla ilet

SOCKS/HTTP ne zaman seçilmeli: karmaşık maskeleye olmaksızın yerel veya yardımcı proxy erişimi için. mixed , tek portun hem SOCKS hem de HTTP istemcilerine hizmet vermesi açısından pratiktir.

5.9. WireGuard (inbound)

Amaç: WireGuard inbound. Proxy protokollerinden farklı olarak «istemci» kavramıyla çalışmaz – bunun yerine **peer'lar** (sunucunun kabul ettiği cihazlar) yapılandırılır. Taşıma katmanı ve TLS/REALITY uygulanamaz.

settings bloğunun alanları:

Alan	Varsayılan değer	Açıklama
secretKey	–	Sunucu özel anahtarı (zorunlu). Yanında oluştur düğmesi bulunur; ortak anahtar otomatik olarak gösterilir (yalnızca okunabilir alan)
mtu	(isteğe bağlı)	Arayüz MTU'su
noKernelTun	false (kapalı)	«Çekirdeksiz TUN» (İng. «No-kernel TUN») – çekirdek TUN yerine kullanıcı alanı TUN kullanılır
domainStrategy	(isteğe bağlı)	«Domain Strategy» – etki alanı çözümleme stratejisi: ForceIP , ForceIPv4 , ForceIPv4v6 , ForceIPv6 , ForceIPv6v4
peers	[]	Peer listesi

Her peer'in alanları:

Peer alanı	Varsayılan değer	Açıklama
privateKey	(isteğe bağlı)	İstemcinin özel anahtarı – panelin kullanıcı yapılandırmasını oluşturabilmesi için saklanır (yalnızca inbound peer'larında)

Peer alanı	Varsayılan değer	Açıklama
publicKey	—	Peer'in ortak anahtarı (zorunlu)
preSharedKey (PSK)	(isteğe bağlı)	Ek ön paylaşımlı anahtar
allowedIPs	[]	İzin verilen IP'ler. Yeni peer eklenirken panel otomatik olarak bir sonraki uygun adresi önerir (varsayılan 10.0.0.2/32)
keepAlive	(isteğe bağlı)	«Keep-alive» – bağlantı canlı tutma aralığı
comment	(isteğe bağlı)	«Comment» – peer'in serbest etiket alanı; formda «Peer N» başlığının yanında gösterilir ve paylaşım bağlantısına ile .conf dosyasının remark alanına eklenir

«Peer ekle» düğmesi yeni bir anahtar çifti oluşturur ve bir sonraki allowedIPs değerini ekler. Her peer silinebilir (tek kalan peer silinemez).

Peer'in «Comment» alanı cihazları ayırt etmeye yardımcı olur: metni formda «Peer N» başlığının yanında gösterilir, ayrıca paylaşım bağlantısına ve oluşturulan .conf dosyasının remark alanına eklenir; böylece istemci uygulamasında cihaz kolayca tanınır. Bu alan yalnızca panel tarafından kullanılır – xray-core peer'daki bilinmeyen alanları yok sayar.

Domain Strategy ve Transport Sekmesi

WireGuard inbound'unda peer'lara ek olarak **Domain Strategy** alanı bulunur (etki alanı çözümleme stratejisi: ForceIP , ForceIPv4 , ForceIPv4v6 , ForceIPv6 , ForceIPv6v4). Alan isteğe bağlıdır ve yalnızca ayarlandığında yapılandırmaya yazılır.

Workers alanı (workers , çalışan iş parçacığı sayısı) WireGuard formlarından (hem inbound hem outbound) kaldırılmıştır: xray-core v26.6.22'den itibaren motor bu alanı artık kullanmamakta ve wireguard-go'nun iç mekanizmasına güvenmektedir. Önceden kaydedilmiş yapılandırmalar değişiklik gerektirmeden çalışır – ayırıştırma sırasında alan yalnızca atlanır, geçiş gerekmez.

WireGuard için «**Transport**» sekmesi de mevcuttur – ancak kısıtlı biçimde: yalnızca sockopt ve Finalmask gizleme ayarları yapılandırılabilir. WireGuard her zaman UDP üzerinden dinlediğinden taşıma seçimi açılır listesi (network) gizlidir. Finalmask gürültü kayıtlarında (noise) ayrı bir **Rand Range** alanı (0–255 bayt aralığı, doğrulama ile) bulunur; WireGuard ve Hysteria için gizleme yöntemi olarak **Salamander** kullanılabilir.

WireGuard ne zaman seçilmeli: maskelenen proxy değil, tam anlamıyla WireGuard VPN tüneli gerektiğinde.

5.10. Hysteria (varsayılan olarak v2)

Amaç: QUIC üzerinde Hysteria inbound. Panel varsayılan olarak sürüm 2 ile çalışır. Her istemci UUID/parola yerine auth tokeniyle kimlik doğrular. TLS, Hysteria için her zaman kullanılabilir (5.2'deki özellik tablosuna bakın).

settings bloğunun alanları:

Alan	Varsayılan değer	Açıklama
version	2	Protokol sürümü (en az 1; panel varsayılanı 2)
clients	[]	İstemci listesi

Her istemcinin temel alanı – `auth` (token,zorunlu) ve ortak alanlar (`email` , limitler, `enable` , `tgId` , `subId` , `comment` , `reset`).

Ek parametreler `streamSettings.hysteriaSettings` içinde tanımlanır:

Alan	Değer / seçenekler	Açıklama
version	2 olarak sabit (alan kilitli)	«Sürüm» (İng. «Version»)
udpIdleTimeout	(≥ 1 tam sayı, sn.)	«UDP idle timeout (s)» – UDP boşta kalma zaman aşımı
masquerade	varsayılan olarak kapalı	«Masquerade» – «istenmeyen» isteklerde normal web sunucusu görünümü

`masquerade` etkinleştirildiğinde tür (`type`) seçimi yapılabilir:

- `` – varsayılan (404 sayfası);
- `proxy` – ters proxy («Upstream URL», «Host'u yeniden yaz», «TLS doğrulamayı atla» alanları);
- `file` – izin sunma («Dizin» alanı, örneğin `/var/www/html`);
- `string` – sabit yanıt («Durum kodu», «Body», «Başlıklar» alanları).

Hysteria ne zaman seçilmeli: QUIC taşıma katmanına ve kararsız/mobil bağlantılarda dayanıklılığa ihtiyaç duyulduğunda; maskeleye giriş noktasının gizliliğini artırır.

5.11. MTProto (Telegram proxy)

3.3.0 sürümünde eklendi. Protokol değeri – `mtproto` .

MTProto, Telegram'ın kendi proxy protokolüdür. 3X-UI'de bu tür inbound **Xray tarafından değil, panelin yönettiği ayrı bir `mtg` süreci tarafından** işlenir. Panel etkin MTProto inbound'larını çalışan `mtg` süreçleriyle düzenli olarak karşılaştırır: eksik olanları başlatır, gereksiz olanları durdurur ve `mtg` ölçümlerinden trafik sayaçlarını alır. Bu nedenle bu inbound üzerindeki **trafik takibi** normal protokollerdeki gibi çalışır.

Formdaki resmi ipucu:

«MTProto, Xray değil ayrı bir `mtg` süreci tarafından işlenir. Taşıma ayarları ve istemciler burada geçerli değildir – aşağıdaki bağlantıyı Telegram'da paylaşın.»

Sonuçlar:

- «**Taşıma**» (**Stream Settings**) ve «**İstemciler**» sekmeleri bu inbound için geçerli değildir – erişim istemci listesiyle değil, tek bir gizli anahtarla sağlanır.
- MTProto inbound **yalnızca ana panelde** çalışır; alt node'lara dağıtılmaz (NodeID ayarlı inbound'lar atlanır).
- MTProto için «**Sniffing**» sekmesi gizlidir – bu protokol Xray değil mtg süreci tarafından işlendiğinden sniffing uygulanamaz.

Alanlar. inbound'un settings bölümünde saklanır:

UI'daki alan	Anahtar	Açıklama
Remark	remark	inbound etiketi.
Listen IP	listen	Dinleme IP'si (boş = tüm arayüzler).
Port	port	Proxy portu.
Gizli anahtar	settings.secret	FakeTLS biçiminde erişim gizli anahtarı.
Maske etki alanı (FakeTLS)	settings.fakeTlsDomain	Proxy'nin HTTPS trafiği gibi görüldüğü etki alanı.

Gizli anahtar biçimi (FakeTLS). Panel gizli anahtarı otomatik olarak doğru biçime getirir: sonuç = ee + 32 hex karakter + maske etki alanının hex kodu, yani ee<hex32><hex(fakeTlsDomain)>. ee öneki FakeTLS modunu etkinleştirir; etki alanı (örneğin bilinen bir site) trafiği sıradan HTTPS gibi göstermek için kullanılır. Etki alanını belirtmek yeterlidir – panel gerisini otomatik olarak tamamlar.

Domain-fronting ve gelişmiş mtg seçenekleri

MTProto inbound'unda mtg sürecine ait ek parametreler bulunur. **Domain fronting IP**, **Domain fronting port** ve **Domain fronting PROXY protocol** alanları, mtg 'nin Telegram dışı trafiği nereye göndereceğini belirler (örneğin sahte bir NGINX sitesine): IP boş bırakılırsa DNS üzerinden FakeTLS etki alanı kullanılır, varsayılan port 443 'tür. Ek olarak **Accept PROXY protocol** (dinleyici için), **IP preference** (prefer-ipv6 / prefer-ipv4 / only-ipv6 / only-ipv4) ve **Debug logging** seçenekleri mevcuttur. Her değer yalnızca ayarlandığında mtg-<id>.toml dosyasına yazılır.

Telegram trafiğini Xray üzerinden yönlendirme

«**Route through Xray**» geçişi (varsayılan olarak kapalı) ve isteğe bağlı **Outbound** alanı, Telegram çıkış trafiğini Xray yönlendiricisine bağlamayı sağlar. Etkinleştirildiğinde panel, Xray yapılandırmasına inbound'un kendi etiketiyle yerel bir SOCKS köprüsü ekler ve mtg Telegram trafiğini bu köprü üzerinden gönderir. Bunun ardından trafik, «Routing» sekmesindeki kurallarla eşleştirilebilir ya da **Outbound** alanı aracılığıyla seçilen bir outbound veya dengeleyiciye zorla yönlendirilebilir (alan boşsa yönlendirme kuralları geçerli olur).

Kullanıcıya nasıl paylaşılır. MTProto inbound için panel bir davet bağlantısı oluşturur:

Örnek: FakeTLS gizli anahtarı ve hazır bağlantı. Maske etki alanı www.cloudflare.com ise gizli anahtar ee + 32 hex karakter + etki alanının hex kodu olarak oluşturulur, örneğin:

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

Hazır davet bağlantısı (bu bağlantı ve QR kod Telegram'da kullanıcıya gönderilir):

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

```
tg://proxy?server=<адрес>&port=<порт>&secret=<секрет>
```

(eşdeğeri – <https://t.me/proxy?server=...&port=...&secret=...>). Bu bağlantıyı ve QR kodu Telegram kullanıcısına gönderin – açıldığında proxy uygulamaya hemen eklenir. Bağlantı aynı zamanda abonelik sunucusu üzerinden de sağlanır.

Ne zaman kullanılmalı. Telegram engellerini aşmak için standart yöntem; FakeTLS maskeleyesi (maske etki alanı) trafiği belirtilen siteye yapılan sıradan bir ziyaret gibi gösterir.

5.12. Protokol Seçimi için Hızlı Rehber

- **VLESS** – varsayılan seçim; REALITY veya TLS + XTLS-Vision ile en iyi seçenek, kuantum sonrası kimlik doğrulamayı destekler.
- **Trojan** – web sunucusuna fallback'li HTTPS maskeleyesi.
- **VMess** – eski istemcilerle uyumluluk.
- **Shadowsocks** – TLS olmayan basit proxy; modern seçim `2022-blake3-*` yöntemleridir.
- **Hysteria** – QUIC, zayıf bağlantılarda dayanıklılık.
- **mixed / http** – yardımcı SOCKS/HTTP proxy'leri.
- **WireGuard** – tam VPN tüneli.
- **tunnel** – şeffaf port yönlendirme.
- **MTPROTO** – Telegram engellerini aşmak için proxy (FakeTLS); ayrı `mtg` süreci.

6. Aktarım (Stream Settings)

Aktarım (panel arayüzünde «**Aktarım**» alanı, İngilizce *Transmission*), Xray-core'un inbound içindeki verileri nasıl ilettiğini tanımlar: TLS/Reality üzerinde hangi ağ protokolünün kullanıldığını ve trafiğin nasıl çerçevelendiğini belirtir. Bu parametreler Xray yapılandırmasının `streamSettings` nesnesine kaydedilir ve inbound düzenleyicisindeki aktarım sekmesinden ayarlanır. Şifreleme (TLS / Reality) ayrı bir bölümde ele alınmaktadır; burada yalnızca ağ seçimi ve ağ parametreleri açıklanmaktadır.

6.1. Ağ İletimi Seçimi

Ağ, «**Aktarım**» (`streamSettings.network`) açılır listesinden seçilir. Varsayılan değer `tcp` 'dir (listede **RAW** olarak görünür). Kullanılabilir seçenekler:

Listedeki değer	network alanı	Aktarım
RAW	<code>tcp</code>	Standart TCP (yeni Xray sürümlerinde RAW olarak yeniden adlandırılmıştır), isteğe bağlı HTTP gizlemesiyle
mKCP	<code>kcp</code>	Güvenilir UDP aktarımı mKCP
WebSocket	<code>ws</code>	HTTP(S) üzerinde WebSocket
gRPC	<code>grpc</code>	gRPC tüneli (HTTP/2)
HTTPUpgrade	<code>httpupgrade</code>	HTTP Upgrade
XHTTP	<code>xhttp</code>	XHTTP / SplitHTTP – modern çoğullamalı aktarım

Değer değiştirildiğinde panel, önceki ağın ayarlar bloğunu temizler ve yeni ağın bloğunu şemasındaki varsayılan değerlerle doldurur; bu nedenle alt formun her alanı her zaman anlamlı bir başlangıç değerine sahiptir.

Önemli. Bu panel derlemesinde **HTTP/2 aktarımı (h2) listede yer almamaktadır** – ağ seçenekleri kümesinden çıkarılmıştır; çift yönlü HTTP/2 benzeri tünel için gRPC, modern HTTP maskeli aktarım için XHTTP kullanılmaktadır. **Hysteria** aktarımı (`hysteria`) bu listeden seçilmez: Hysteria protokolüne sabit olarak bağlıdır ve inbound'un kendisi Hysteria protokolüyle oluşturulduğunda otomatik olarak belirir (bkz. madde 6.8).

Aşağıda her ağ ve her alanı ayrı ayrı açıklanmıştır.

6.2. RAW / TCP (`tcpSettings`)

Temel TCP aktarımı. Varsayılan olarak trafik olduğu gibi iletilir; isteğe bağlı olarak sıradan bir HTTP/1.1 alışverişini taklit edecek şekilde gizlenebilir.

Alan	Varsayılan değer	Açıklama
Proxy Protocol (<code>acceptProxyProtocol</code>)	<code>false</code> (kapalı)	Üst akış yük dengeleyici/proxy'den PROXY protokolü başlığını kabul et
HTTP gizlemesi (<code>header.type</code>)	<code>none</code> (kapalı)	Trafiği HTTP/1.1 görünümüne büründürmeyi etkinleştirir

Proxy Protocol

«**Proxy Protocol**» (`acceptProxyProtocol`) düğmesi. Etkinleştirildiğinde Xray, gelen bağlantıda PROXY protokolü başlığı bekler ve gerçek istemci IP adresini bu başlıktan çıkarır. Yalnızca panelin önünde bu başlığı ekleyen bir ters proxy/yük dengeleyici (örneğin `send-proxy` ile HAProxy veya nginx) varsa etkinleştirilir. Varsayılan olarak kapalıdır.

HTTP Gizlemesi (camouflage)

«**HTTP Gizlemesi**» düğmesi. `header` alanını yönetir:

- **Kapalı** → `header.type = "none"` (iletimde `header` alanı tamamen yoktur). Saf TCP.
- **Açık** → `header.type = "http"`. Trafik bir HTTP/1.1 istek ve yanıtı görünümünde çerçevelenir. Etkinleştirildiğinde panel hemen `request` ve `response` alt nesnelerini varsayılan değerlerle doldurur.

HTTP gizlemesi etkinleştirildiğinde taklit istek ve yanıtla ilişkin yapılandırma alanları görünür.

İstek başlığı (`header.request`):

Alan	Anahtar	Varsayılan değer	Açıklama
İstek sürümü	<code>request.version</code>	<code>1.1</code>	İstek başlangıç satırındaki HTTP sürümü
İstek yöntemi	<code>request.method</code>	<code>GET</code>	Taklit edilen isteğin HTTP yöntemi
İstek yolu	<code>request.path</code>	<code>/</code>	Yol(lar). Virgülle ayrılmış değer listesi olarak girilir; iletimde bu bir dize dizisidir. Boş bırakılırsa <code>/</code> atanır
İstek başlıkları	<code>request.headers</code>	<code>{}</code> (boş)	HTTP başlıklarının «Ad/Değer» tablosu. <code>ad</code> → <code>[değerler]</code> eşlemesi olarak saklanır (bir ada birden fazla değer karşılık gelebilir)

Yanıt başlığı (`header.response`):

Alan	Anahtar	Varsayılan değer	Açıklama
Yanıt sürümü	<code>response.version</code>	<code>1.1</code>	Yanıt başlangıç satırındaki HTTP sürümü
Yanıt durumu	<code>response.status</code>	<code>200</code>	Taklit edilen yanıtın HTTP durum kodu
Yanıt nedeni	<code>response.reason</code>	<code>OK</code>	Durum açıklaması (reason-phrase)

Alan	Anahtar	Varsayılan değer	Açıklama
Yanıt başlıkları	<code>response.headers</code>	<code>{}</code> (boş)	Yanıt başlıklarının «Ad/Değer» tablosu (ad → [değerler] eşlemesi)

Başlık alanları satır satır düzenlenir – her satır bir başlık adı (Ad) ve değerini (Değer) tanımlar. Bu parametreler yalnızca trafiğin dışarıdan nasıl görüldüğünü gizlemek için kullanılır; şifrelemeyi etkilemezler. Varsayılan değerler (GET / HTTP/1.1 , 200 OK yanıtı) çoğu senaryo için uygundur – yalnızca belirli bir site/ hizmeti taklit etmek gerektiğinde değiştirilmelidir.

HTTP gizlemesiyle RAW için örnek `streamSettings` :

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
      "response": {
        "version": "1.1",
        "status": "200",
        "reason": "OK"
      }
    }
  }
}
```

İletimde `path` bir dize dizisidir; her başlık ise bir değer dizisidir (Host → ["www.example.com"]).

6.3. mKCP (`kcpSettings`)

mKCP, UDP üzerinde güvenilir bir aktarımdır. Paket kaybı yaşanan ve gecikmesi yüksek kanallarda kullanışlıdır, ancak ek servis trafiği üretir. Tüm varsayılan değerler xray-core'un önerileriyle örtüşmektedir.

Alan	Anahtar	Varsayılan	İzin verilen	Açıklama
MTU	<code>mtu</code>	1350	576–1460	Maksimum paket boyutu (bayt). Parçalanma sorunlarında azaltılır
TTI (ms)	<code>tti</code>	20	10–100	İletim aralığı (ms). Küçük değer daha düşük gecikme, ancak daha yüksek ek yük

Alan	Anahtar	Varsayılan	İzin verilen	Açıklama
Uplink (MB/s)	<code>uplinkCapacity</code>	5	≥ 0	Tahmini yükleme bant genişliği (MB/s)
Downlink (MB/s)	<code>downlinkCapacity</code>	20	≥ 0	Tahmini indirme bant genişliği (MB/s)
CWND çarpanı	<code>cwndMultiplier</code>	1	≥ 1	Tıkanıklık penceresi (congestion window) çarpanı
Maks. gönderme penceresi	<code>maxSendingWindow</code>	2097152	≥ 0	Maksimum gönderme penceresi boyutu

Alan notları:

- **Uplink / Downlink capacity** mKCP'nin kanalı ne kadar agresif kullandığını belirler. Gerçek kanal genişliğine göre ayarlanmalıdır: fazla yüksek değerler gereksiz trafiğe, fazla düşük değerler ise kanalın yetersiz kullanılmasına yol açar.
- **TTI** «gecikme [?] ek yük» dengesini doğrudan etkiler: küçük değerler gecikmeyi azaltır, ancak servis paketi hacmini artırır.
- **MTU**, tek bir mKCP paketinin boyutunu sınırlar; büyük UDP paketlerinin kesildiği veya kaybolduğu kanallarda azaltmak faydalıdır.

Bu panel derlemesinde mKCP'nin «seed» alanı (mKCP gizleme parolası) ve **başlık türü/gizleme** açılır listesi (none , srtp , utp , wechat-video , dtls , wireguard) mKCP alt formunda **ayrı alanlar olarak yer almamaktadır** – aktarım katmanı gizlemesi, mKCP-legacy modu da dahil olmak üzere ilgili bölümde açıklanan genel «FinalMask» mekanizmasına taşınmıştır. «congestion» parametresi de ayrı bir onay kutusu olarak sunulmamaktadır; tıkanıklık kontrolü `cwndMultiplier` ve `maxSendingWindow` üzerinden yönetilmektedir.

6.4. WebSocket (`wsSettings`)

HTTP(S) üzerinde WebSocket aktarımı. CDN'ler ve ters proxy'ler üzerinden iyi geçer, sıradan web trafiğine benzer.

Alan	Anahtar	Varsayılan	Açıklama
Proxy Protocol	<code>acceptProxyProtocol</code>	false	Üst akış proxy'den PROXY protokolü başlığını kabul et (bkz. madde 6.2)
Host	<code>host</code>	"" (boş)	<code>Host</code> HTTP başlığının değeri. CDN/domain fronting üzerinden çalışırken belirtilir
Yol	<code>path</code>	/	WebSocket el sıkışmasının istek satırındaki yol
Heartbeat periyodu	<code>heartbeatPeriod</code>	0	Heartbeat çerçeveleri gönderme aralığı (saniye). 0 heartbeat'i devre dışı bırakır
Başlıklar	<code>headers</code>	{ } (boş)	Ek HTTP el sıkışma başlıkları. «Ad → Değer» düz eşlemesi (dizi değil, yalnızca dize değerleri)

Notlar:

- **Yol**, sunucu (inbound) ve istemci tarafında eşleşmelidir. Bu yol genellikle web sunucusu tarafında giriş noktasını gizlemek için kullanılır.
- **Host**, inbound bir CDN'in arkasında bulunuyorsa veya domain fronting kullanılıyorsa belirtilmelidir; aksi takdirde boş bırakılabilir.
- **Heartbeat periyodu**, etkin olmayan oturumları agresif biçimde kesen proxy/CDN'ler üzerinden bağlantıyı «canlı» tutar. Varsayılan olarak (0) heartbeat kapalıdır.
- RAW'dan farklı olarak WebSocket başlık tablosu «düz» ad → değer biçimini kullanır (başlık başına tek değer satırı).

CDN arkasında WebSocket için örnek streamSettings :

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

host ve path değerleri istemci tarafında da eşleşmelidir; RAW'dan farklı olarak burada başlık değeri dizi değil sıradan bir dizedir.

6.5. gRPC (grpcSettings)

Alan sayısı bakımından en «hafif» aktarım. Trafik gRPC çağrıları içinde tüneller (HTTP/2 üzerinden); gRPC destekleyen CDN'lerle iyi uyum sağlar. Başlık gizlemesi yoktur.

Alan	Anahtar	Varsayılan	Açıklama
Hizmet adı (Service Name)	serviceName	"" (boş)	gRPC hizmet adı (filen tünelin «yolu»). Sunucu ve istemci tarafında eşleşmelidir
Authority	authority	"" (boş)	:authority sözde başlığının değeri (HTTP/2 için Host karşılığı). CDN/domain üzerinden çalışırken belirtilir
Multi Mode	multiMode	false (kapalı)	Tek bağlantı içinde birden fazla paralel gRPC akışının çoğullanmasını etkinleştirir

Notlar:

- **Service Name** – gRPC kanalının temel tanımlayıcısıdır; her iki tarafta da aynı olmalıdır. Boş değer geçerlidir, ancak gizleme amacıyla genellikle rastgele olmayan bir dize kullanılır.

- **Authority**, HTTP/2 çerçevelerinde gönderilen `:authority` değerini etkiler; öncelikle CDN üzerinden proxy'leme yapılırken gereklidir.
- **Multi Mode**, birden fazla mantıksal akışın tek bir fiziksel bağlantı üzerinden iletilmesine olanak tanır; hem sunucu hem de istemci destekliorsa performansı artırmak için etkinleştirilir.

gRPC için örnek `streamSettings` :

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

`serviceName` (burada `GunService`) tünelin «yolu» işlevini görür ve sunucu ile istemci tarafında eşleşmelidir.

6.6. HTTPUpgrade (`httpupgradeSettings`)

HTTP Upgrade mekanizmasına dayalı aktarım (WebSocket gibi, ancak WebSocket protokolü olmadan). Proxy'ler ve CDN'ler üzerinden iyi geçer. Alan kümesi WebSocket ile aynıdır, ancak heartbeat periyodu **yoktur**.

Alan	Anahtar	Varsayılan	Açıklama
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Üst akış proxy'den PROXY protokolü başlığını kabul et
Host	<code>host</code>	<code>""</code> (boş)	<code>Host</code> HTTP başlığının değeri
Yol	<code>path</code>	<code>/</code>	<code>Upgrade</code> başlığıyla HTTP isteğinin yolu
Başlıklar	<code>headers</code>	<code>{}</code> (boş)	Ek HTTP başlıkları. Düz <code>ad</code> → <code>değer</code> eşleşmesi (WebSocket gibi)

Host, **Yol** ve **Başlıklar** alanlarının amacı WebSocket (madde 6.4) ile aynıdır. Heartbeat HTTPUpgrade için öngörülmemiştir – bu WebSocket'e özgü bir özelliktir.

6.7. XHTTP / SplitHTTP (`xhttpSettings`)

XHTTP (diğer adıyla SplitHTTP), xray-core'un modern çoğullamalı HTTP aktarımıdır. Yukarı ve aşağı akışları ayrı HTTP isteklerine böler; bu durum CDN'ler ve bağlantı süresi kısıtlaması olan ortamlar için uygundur. Tüm alanlar aynı anda görünmez: bir kısmı seçilen moda (`mode`) ve etkinleştirilen düğmelere bağlı olarak ortaya çıkar.

Temel alanlar (her zaman görünür)

Alan	Anahtar	Varsayılan	Açıklama
Host	host	"" (boş)	Host HTTP başlığının değeri
Yol	path	/	HTTP isteklerinin temel yolu
Mod (Mode)	mode	auto	İletim modu (aşağıya bakın)
Server Max Header Bytes	serverMaxHeaderBytes	0	Sunucuda istek başlıkları boyutu sınırı (bayt). 0 – xray-core varsayılan değeri
Padding Bytes	xPaddingBytes	100-1000	Boyut analizini zorlaştırmak için rastgele «dolgu» (bayt, min-maks biçimi) aralığı
Başlıklar	headers	{ } (boş)	Ek HTTP başlıkları. Düz ad → değer eşlemesi
HTTP yöntemi Uplink	uplinkHTTPMethod	"" (Varsayılan = POST)	Yukarı yön isteklerinin HTTP yöntemi. Seçenekler: boş (varsayılan POST), POST , PUT , GET (sonuncusu yalnızca packet-up modunda kullanılabilir)
Padding Obfs Mode	xPaddingObfsMode	false (kapalı)	Gelişmiş dolgu gizlemesini etkinleştirir ve ek alanları açar (aşağıya bakın)
No SSE Header	noSSEHeader	false (kapalı)	Content-Type: text/event-stream (SSE) başlığını göndermez. Ara düğümlerden geçişi engelliyorsa etkinleştirilir

«Mod» alanı (mode)

Şu değerleri içeren açılır liste:

Değer	Açıklama
auto	Otomatik mod seçimi (varsayılan)
packet-up	Yukarı yön akışı ayrı HTTP isteklerine bölünür (istek başına bir paket)
stream-up	Yukarı yön akışı tek uzun süreli akış isteğiyle iletilir
stream-one	Tek ortak çift yönlü akış isteği

Mod seçimi hangi ek alanların görüneceğini belirler.

Yalnızca mode = packet-up seçildiğinde görünen alanlar:

Alan	Anahtar	Varsayılan	Açıklama
Maks. arabelleğe alınan yükleme	scMaxBufferedPosts	30	Yukarı yön akışında eş zamanlı arabelleğe alınabilecek maksimum POST isteği sayısı
Maks. yükleme boyutu (bayt)	scMaxEachPostBytes	1000000	Tek bir yukarı yön POST isteğinin maksimum boyutu (bayt)

Alan	Anahtar	Varsayılan	Açıklama
Uplink Data Placement	<code>uplinkDataPlacement</code>	"" (Varsayılan = body)	Yukarı yön verilerinin nereye yerleştirileceği: <code>body</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Uplink Data Key	<code>uplinkDataKey</code>	""	Uplink verileri için anahtar/başlık adı. Yalnızca <code>uplinkDataPlacement</code> belirtilmişse ve <code>body</code> değilse görünür

Yalnızca `mode = stream-up` seçildiğinde görünen alan:

Alan	Anahtar	Varsayılan	Açıklama
Stream-Up Server	<code>scStreamUpServerSecs</code>	20-80	Sunucu tarafı akış bağlantısının tutulma süresi aralığı (saniye, min-maks biçimi)

Dolgu gizleme alanları (`xPaddingObfsMode = açık` olduğunda görünür)

Alan	Anahtar	Varsayılan	Açıklama
Padding Key	<code>xPaddingKey</code>	"" (yer tutucu <code>x_padding</code>)	Dolgu için anahtar adı
Padding Header	<code>xPaddingHeader</code>	"" (yer tutucu <code>X-Padding</code>)	Dolgunun iletildiği HTTP başlığının adı
Padding Placement	<code>xPaddingPlacement</code>	"" (Varsayılan = <code>queryInHeader</code>)	Dolgunun nereye yerleştirileceği: <code>queryInHeader</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Padding Method	<code>xPaddingMethod</code>	"" (Varsayılan = <code>repeat-x</code>)	Dolgu oluşturma yöntemi: <code>repeat-x</code> veya <code>tokenish</code>

Oturum ve sıra yerleştirme (her zaman görünür)

Alan	Anahtar	Varsayılan	Açıklama
Session ID Placement	<code>sessionIDPlacement</code>	"" (Varsayılan = <code>path</code>)	Oturum tanımlayıcısının nerede iletileceği: <code>path</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Session ID Key	<code>sessionIDKey</code>	"" (yer tutucu <code>x_session</code>)	Oturum anahtarı adı. Yalnızca <code>sessionIDPlacement</code> belirtilmişse ve <code>path</code> değilse görünür
Session ID Table	<code>sessionIDTable</code>	"" (yer tutucu <code>Base62</code>)	Oturum tanımlayıcısı oluşturmak için karakter kümesi. Otomatik tamamlamalı açılır listeden önceden tanımlanmış bir değer seçilebilir (<code>ALPHABET</code> , <code>Alphabet</code> , <code>BASE36</code> , <code>Base62</code> , <code>HEX</code> , <code>alphabet</code> , <code>base36</code> , <code>hex</code> , <code>number</code>) ya da keyfi bir ASCII dizisi girilebilir. Boş – <code>xray-core</code> varsayılan değeri
Session ID Length	<code>sessionIDLength</code>	"" (boş)	Oluşturulan tanımlayıcıların uzunluğu veya aralığı (örneğin <code>8-16</code>). Yalnızca <code>Session ID Table</code> belirtildiğinde görünür; minimum değer 0'dan büyük olmalıdır

Alan	Anahtar	Varsayılan	Açıklama
Sequence Placement	seqPlacement	"" (Varsayılan = path)	Paket sıra numarasının nerede iletileceği: path , header , cookie , query
Sequence Key	seqKey	"" (yer tutucu x_seq)	Sıra anahtarı adı.Yalnızca seqPlacement belirtilmişse ve path değilse görünür

Oturum alanları xray-core v26.6.22 sürümüyle yeniden adlandırılmıştır: önceki adları **Session Placement / Session Key** (sessionPlacement / sessionKey) iken artık **Session ID Placement / Session ID Key** (sessionIDPlacement / sessionIDKey) oldu; eski ad çekirdek tarafından artık tanınmamaktadır.

Güncellemeden önce kaydedilmiş inbound'lar yeni anahtarlara otomatik olarak taşınır – yeniden kaydetmeye gerek yoktur.

Öneriler:

- Çoğu kurulum için **Mod = auto** bırakıp **Yol/Host** ayarlamak ve CDN üzerinden çalışırken bunları istemciyle uyumlu hâle getirmek yeterlidir.
- Yerleştirme (*Placement / *Key) ve dolgu gizleme alanları yalnızca belirli anti-DPI/CDN senaryolarına yönelik ince ayar için gereklidir; boş bırakıldığında parantez içinde belirtilen xray-core varsayılan değerleri kullanılır.
- İstemci/outbound tarafına ait parametreler (örneğin tekrarlanan POST aralıkları, öbek boyutları) inbound formunda gösterilmez – sunucu dinleyicisi bunları yok sayar. XMUX çoğullayıcısı ise inbound formunda kullanılabilir (aşağıya bakın).
- **Servis varsayılanları artık yazılmaz.** Panel, XHTTP yapılandırmalarına scMaxEachPostBytes ve scMinPostsIntervalMs servis varsayılanlarını artık yazmamaktadır – xray-core'un iç değerleri uygulanır. Bu, daha önce trafiğin engellenmesine neden olan sabit DPI imzasını (scMinPostsIntervalMs=30 değişmezi) ortadan kaldırır. Önceden kaydedilmiş inbound'larda xray-core varsayılanlarıyla örtüşen değerler bağlantı ve aboneliklerde gösterilmez (inbound'ları yeniden kaydetmeye gerek yoktur); elle girilen değerler korunmaktadır.

XHTTP için örnek streamSettings (auto modu):

```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

Çoğu kurulum için bu dört alan yeterlidir; oturum/sıra yerleştirme ve dolgu gizleme alanları boş bırakılır – bu durumda xray-core varsayılan değerleri kullanılır.

XMUX (bağlantı çoğullaması)

XMUX (`enableXmux`) düğmesi, paralel istekleri küçük bir fiziksel bağlantı havuzuna dağıtan çoğullama katmanını etkinleştirir. Etkinleştirildiğinde altı yapılandırılabilir alan açılır (`xhttpSettings.xmux` içinde saklanır):

Alan	Anahtar	Varsayılan	Açıklama
Max Concurrency	<code>maxConcurrency</code>	16–32	Bağlantı başına maksimum eş zamanlı istek sayısı (<code>min</code> – <code>maks</code> aralığı)
Max Connections	<code>maxConnections</code>	0	Maksimum fiziksel bağlantı sayısı (0 – sınırsız)
Max Reuse Times	<code>cMaxReuseTimes</code>	"" (boş)	Bağlantının kaç kez yeniden kullanılacağı
Max Request Times	<code>hMaxRequestTimes</code>	600–900	Bağlantı başına maksimum istek sayısı (aralık)
Max Reusable Secs	<code>hMaxReusableSecs</code>	1800–3000	Bağlantının yeniden kullanıma uygun olduğu süre (saniye, aralık)
Keep Alive Period	<code>hKeepAlivePeriod</code>	"" (boş)	Bağlantıyı canlı tutmak için keep-alive periyodu

Önemli. **Max Connections** ve **Max Concurrency** aynı anda belirtilemez – xray-core böyle bir yapılandırmayı reddeder. Varsayılan olarak XMUX etkinleştirildiğinde panel **Max Concurrency** = 16–32 değerini atar; **Max Connections** (0 'dan büyük bir değer) belirtirseniz çakışmayı önlemek için panel **Max Concurrency** varsayılan değerini kaldırır.

6.8. Hysteria Aktarımı (`hysteriaSettings`)

Hysteria aktarımı «Aktarım» listesinden seçilmez: inbound Hysteria protokolüyle oluşturulduğunda otomatik olarak etkinleşir ve diğer protokollerden gizlenir (Hysteria protokolünden çıkıldığında ağ zorla `tcp` 'ye döner). Parametreler:

Alan	Anahtar	Varsayılan	Açıklama
Sürüm	<code>version</code>	2 (sabit, alan kilitli)	Hysteria sürümü. Yalnızca Hysteria 2 desteklenmektedir
UDP idle timeout (s)	<code>udpIdleTimeout</code>	60	UDP oturumu boşta kalma zaman aşımı (saniye). İzin verilen aralık 2–600; xray-core başlatmada bu aralık dışındaki değerleri reddeder
Masquerade	<code>masquerade</code>	kapalı (yok)	Dinleyiciyi yoklama yapılırken HTTP/3 sunucusu gibi göstermeyi etkinleştirir

Masquerade etkinleştirildiğinde tür (`type`) seçimi ve buna bağlı alanlar görünür:

- **"" – default (404 page)**: standart 404 sayfası döndürülür (ek alan gerekmez).
- **proxy (reverse proxy)**: harici bir siteye ters proxy.
 - `url` (**Upstream URL**, yer tutucu `https://www.example.com`) – hedef adres;

- `rewriteHost` (**Host'u yeniden yaz**, varsayılan `false`) – Host başlığını değiştir;
- `insecure` (**TLS doğrulamasını atla**, varsayılan `false`) – üst akımın TLS sertifikasını doğrulama.
- **file (serve directory)**: dizinden dosya sunma.
 - `dir` (**Dizin**, yer tutucu `/var/www/html`).
- **string (fixed body)**: sabit HTTP yanıtı.
 - `statusCode` (**Durum kodu**, varsayılan `0`, aralık `0–599`);
 - `content` (**Body**) – yanıt gövdesi;
 - `headers` (**Başlıklar**) – ad → değer eşlemesi.

Masquerade, Hysteria tabanlı inbound'un aktif yoklamalarda sıradan bir HTTP/3 sunucusu gibi görünmesini sağlar; bu da gizliliği artırır. Varsayılan olarak masquerade kapalıdır.

Ters proxy ile örnek hysteriaSettings (masquerade → proxy):

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

Bu durumda yoklama yapıldığında dinleyici `https://www.example.com` adresinden yanıt döndürerek sıradan bir HTTP/3 sitesi gibi davranır.

6.9. İlgili Parametreler

Ağ seçimine ek olarak, aynı sekmede belirli bir aktarımdan bağımsız iki genel blok daha bulunmaktadır (ayrıntılar ilgili bölümlerde):

- **External Proxy** (`externalProxy`) – panel adresinin yerine abonelik bağlantılarına konulan harici adres/port listesi.
- **Sockopt** (`sockopt`) – düşük seviyeli soket seçenekleri (TCP Fast Open, mark, etki alanı stratejisi, şeffaf proxy vb.).

Real client IP (CDN/röle arkasındaki gerçek IP tespiti)

Bir inbound aracı (Cloudflare gibi CDN, L4 tüneli/rölesi veya başka bir panel) arkasında bulunduğunda Xray, gerçek ziyaretçi yerine aracının adresini görür. Bu adres çevrimiçi istemci listesine girer ve istemci başına IP sınırı bu adresten hesaplanır; böylece her ikisi de proxy arkasında işe yaramaz hâle gelir. Gerçek IP'yi yeniden kazanmak için inbound formunun **Sockopt** bölümünde `acceptProxyProtocol` ve `trustedXForwardedFor` ayarlarını bir arada yöneten **Real client IP** ön ayar seçici bulunmaktadır:

Ön ayar	Ne yapar	Ne zaman kullanılır
Off / direct	Her iki alanı da temizler.	inbound'a istemciler doğrudan erişiyor
Cloudflare CDN	<code>sockopt.trustedXForwardedFor = ["CF-Connecting-IP"]</code> ayarlar.	Cloudflare CDN (turuncu bulut) arkasında WebSocket / HTTPUpgrade / XHTTP / gRPC
L4 relay / Spectrum (PROXY)	<code>acceptProxyProtocol = true</code> etkinleştirir.	inbound önünde L4 tüneli/rölesi veya Cloudflare Spectrum

Ön ayarlar birbirini dışlar: birini seçmek diğerinin alanını temizler; bu sayede eski `trustedXForwardedFor` , PROXY protokolü ile elde edilen IP'yi geçersiz kılmaz. Ön ayarın altında ham **Proxy Protocol** düğmesi ve **Trusted X-Forwarded-For** listesi görünür olmaya devam eder – ön ayar bunları sizin yerinize doldurur, gerekirse elle düzenlenir. Seçilen ön ayar mevcut aktarımda desteklenmiyorsa (örneğin mKCP'de PROXY protokolü) form bir uyarı gösterir. Bu alanlar yalnızca sunucu tarafına aittir ve **aboneliklerde asla istemcilere gönderilmez**.

Yalnızca birini kullanın. `acceptProxyProtocol` gerçek IP'yi L4 PROXY protokolü başlığından okurken `trustedXForwardedFor` bunu HTTP istek başlığından okur; bunları manuel olarak karıştırmak yalnızca aracı zinciriniz gerektiriyorsa yapılmalıdır.

- **FinalMask** (`finalmask`) – mKCP eski gizleme de dahil olmak üzere aktarım katmanı gizlemesi için genel mekanizma; ağ alt formlarındaki ayrı «seed»/«header type» alanlarının yerini almaktadır.

7. Bağlantı Güvenliği: TLS, XTLS ve REALITY

Her inbound, transport akışı üzerinden iletimi destekliyorsa (VMess, VLESS, Trojan, Shadowsocks, Hysteria), düzenleyicide «**Güvenlik**» sekmesi bulunur. Bu sekme, transport kanalının nasıl şifrelenip gizleneceğini yapılandırır. Radio düğmeleriyle değiştirilen üç mod mevcuttur:

Mod	Arayüzdeki Etiket	Ne Zaman Kullanılabilir
none	Yok	Her zaman (TLS'nin zorunlu olduğu Hysteria hariç)
tls	TLS	tcp , ws , http , grpc , httpupgrade , xhttp ağılarında VMess/VLESS/Trojan/Shadowsocks için; Hysteria için her zaman
reality	Reality	Yalnızca tcp , http , grpc , xhttp ağılarında VLESS/Trojan için

Protokol Hysteria ise **Yok** düğmesi görüntülenmez (TLS zorunludur). **Reality** düğmesi yalnızca izin verilen protokol ve ağ kombinasyonunda görünür (yukarıdaki tabloya bakın).

Mod değiştirildiğinde panel, `streamSettings` bloğunu tamamen yeniden oluşturur: önceki moddaki `tlsSettings` ve `realitySettings` kaldırılır ve seçilen mod için varsayılan değerler eklenir. Özellikle **Reality** seçildiğinde panel otomatik olarak: yerleşik popüler alan adları listesinden rastgele bir `target` + `serverNames` (SNI) çifti ekler, rastgele `shortIds` üretir ve X25519 anahtar çifti (`privateKey/publicKey`) için sunucudan talepte bulunur.

7.1. Farkları: TLS vs XTLS vs REALITY

- **TLS** – TLS 1.2/1.3 protokolüyle klasik transport şifreleme. Sunucuda geçerli bir sertifika bulunmalıdır (kendi alan adınız + zincir). Trafik normal HTTPS gibi görünür; ancak aktif sansür uygulayıcısı için alan adınıza yönelik tanınabilir bir TLS el sıkışması söz konusudur. SNI'ye göre engelleme yapıldığında veya güvenilir bir sertifika olmadığında bağlantı engellenir ya da hata verir.
- **XTLS (Vision)** – bu, «Güvenlik» listesinde ayrı bir mod değil, TLS **veya** Reality üzerine çalışan bir *flow* mekanizmasıdır. İstemci tarafında inbound'un **Flow** alanına `xtls-rprx-vision` (veya `xtls-rprx-vision-udp443`) yazılarak etkinleştirilir. Vision; `security = tls` veya `security = reality` ile `tcp` ağındaki VLESS için, ayrıca VLESS şifrelemesi (`vlessenc` / ML-KEM) etkinleştirilmiş `xhttp` transport üzerindeki VLESS için kullanılabilir – bu durumda **Flow** alanı da `xtls-rprx-vision` olarak ayarlanabilir ve değer `vless://` bağlantısına doğru şekilde eklenir (`flow=xtls-rprx-vision`). Vision, el sıkışması sonrasında yükü doğrudan aktararak «çift şifrelemeyi» (TLS-in-TLS) azaltır; bu da aktarımı hızlandırır ve gizliliği artırır. Bu nedenle **VLESS + Reality + Flow xtls-rprx-vision** kombinasyonu önerilen modern yapılandırma olarak kabul edilmektedir.
- **REALITY** – kendi sertifikası olmadan çalışan bir gizleme mekanizmasıdır. Sunucu gerçek bir üçüncü taraf sitenin (`target` / `serverNames`) TLS el sıkışmasını «ödünç alır»; bu nedenle gözlemci için bağlantı o siteye yapılan erişimden ayırt edilemez ve hiçbir sertifikaya ihtiyaç duyulmaz. Kimlik doğrulama, X25519 anahtar çifti ve `shortIds` kümesine dayanır. REALITY, SNI'yi gerçek bir harici alan adına yönlendirdiğinden aktif sondaya (`active probing`) ve SNI'ye göre engellemeye karşı dayanıklıdır. Dezavantajı, daha sıkı yapılandırma gereksinimleridir (port içeren doğru `target` , istemciyle anahtar senkronizasyonu).

Kısa seçim kuralı:

- kendi alan adınız ve geçerli sertifikanız varsa, basit HTTPS görünümü yeterliyse → **TLS** (mümkünse Vision ile);
- alan adınız/sertifikanız yoksa veya DPI'ye karşı maksimum gizlilik gerekiyorsa → **REALITY** (VLESS/TCP için Vision ile).

7.2. «Yok» Modu (none)

Transport, TLS sarmalayıcısı olmadan iletilir: `tlsSettings` ve `realitySettings` blokları `streamSettings` 'ten çıkarılır. Modun ek alanı yoktur. Şu durumlarda uygundur:

- inbound yalnızca `127.0.0.1` dinliyor ve fallback hedefi olarak hizmet veriyorsa (panel kuralına göre fallback için alt inbound `security=none` ile `127.0.0.1` dinlemelidir);
- şifreleme/gizleme dış katman tarafından sağlanıyorsa (örneğin panel önündeki Nginx ters proxy);
- dahili ağda kendi şifrelemesine sahip protokol kullanılıyorsa (Shadowsocks).

Dışarıdan erişilebilen inbound'lar için «Yok» modu önerilmez.

Örnek: tcp ağında TLS için streamSettings bloğu (VLESS/Trojan/VMess). **TLS** modu seçilip SNI ve sertifika yolları doldurulduktan sonra sonuç şöyle görünür:

```
"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}
```

7.3. TLS Modu

`tlsSettings` bloğunun alanları. Varsayılan değerler panel şemasından alınmıştır.

Temel Parametreler

Alan (Etiket)	Varsayılan Değer	Açıklama
SNI (<code>serverName</code>)	<code>""</code> (boş)	Server Name Indication – TLS el sıkışmasında sunulan alan adı. Sertifikanın alan adıyla eşleşmelidir. İngilizce yer tutucu: «Server Name Indication».
Cipher Suites (<code>cipherSuites</code>)	<code>""</code> → Otomatik	İzin verilen şifre takımlarının listesi. Varsayılan olarak boşdur – seçim Xray/Go'ya bırakılır (Otomatik seçeneği). Yalnızca şifreleri açıkça kısıtlamanız gerektiğinde değiştirin.
Min/Maks Sürüm (<code>minMaxVersion</code>)	min = <code>1.2</code> , maks = <code>1.3</code>	Minimum ve maksimum TLS sürümleri. Kullanılabilir değerler: <code>1.0</code> , <code>1.1</code> , <code>1.2</code> , <code>1.3</code> . <code>1.2</code> – <code>1.3</code> olarak bırakılması önerilir; minimumu 1.0/1.1'e düşürmek istenmez (eski, güvensiz sürümler).
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code> (formda None = <code>""</code> seçeneği mevcuttur)	İstemci merhaba el sıkışmasının popüler bir tarayıcı gibi görünmesi için taklit edilen TLS parmak izi (uTLS fingerprint). Aşağıdaki listeye bakın. TLS'de listenin ilk ögesi – taklit etmeyi devre dışı bırakan None (<code>""</code>) – mevcuttur.
ALPN (<code>alpn</code>)	<code>["h2", "http/1.1"]</code>	TLS'te pazarlık edilen uygulama katmanı protokollerinin listesi (çoklu seçim). İzin verilen değerler: <code>h3</code> , <code>h2</code> , <code>http/1.1</code> . Varsayılan olarak <code>h2</code> ve <code>http/1.1</code> sunulmaktadır.

Olası **uTLS fingerprint** değerleri (TLS ve REALITY için aynı): `chrome` , `firefox` , `safari` , `ios` , `android` , `edge` , `360` , `qq` , `random` , `randomized` , `randomizednoalpn` , `unsafe` . TLS formunda ek olarak boş **None** seçeneği mevcuttur (parmak izi taklidi uygulanmaz).

Kullanılabilir **Cipher Suites** değerleri (TLS 1.3 ve ECDHE takımları): `TLS_AES_128_GCM_SHA256` , `TLS_AES_256_GCM_SHA384` , `TLS_CHACHA20_POLY1305_SHA256` , `TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA` , `TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA` , `TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA` , `TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA` , `TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256` , `TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384` , `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256` , `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384` , `TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256` , `TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256` .

TLS Anahtarları

Anahtar	Varsayılan	Açıklama
Bilinmeyen SNI'yi Reddet (<code>rejectUnknownSni</code>)	kapalı (<code>false</code>)	Etkinleştirilirse, istemci tarafından sunulan SNI sertifikadaki adla eşleşmediğinde sunucu bağlantıyı keser. Gizliliği artırır (sunucu «yabancı» isteklere yanıt vermez), ancak istemcide SNI'nin tam olarak eşleşmesi gerekir.
System Root'u Devre Dışı Bırak (<code>disableSystemRoot</code>)	kapalı (<code>false</code>)	Güvenilir kök sertifika deposunun sistem deposunu kullanımını devre dışı bırakır.
Oturum Devamı (<code>enableSessionResumption</code>)	kapalı (<code>false</code>)	TLS oturum devamını etkinleştirir (session resumption / session tickets).

Ek TLS Parametreleri (vcn, eğriler, anahtar günlüğü, ECH Sockopt)

Temel TLS ayarlarının altında ek alanlar mevcuttur.

Alan (Etiket)	Varsayılan	Açıklama
Verify Peer Cert By Name (settings.verifyPeerCertByName)	""	İstemcinin sunucu sertifikasını SNI yerine doğruladığı adlar (virgülle ayrılmış). Bu, Xray'de 2026-06-01 sonrasında kaldırılan allowInsecure alanının modern yerine geçer. Yalnızca panel tarafında bir değerdir: xray sunucu yapılandırmasına yazılmaz, ancak istemcinin kendi başına uygulayabilmesi için davet bağlantılarına ve aboneliklere eklenir (vcn=...). Yer tutucu: example.com .
Curve Preferences (curvePreferences)	""	TLS anahtar değişimi eğrilerinin tercihen sıralanmış kısıtlaması (örn. X25519MLKEM768 , X25519). Boşsa Xray-core varsayılanları kullanılır.
Master Key Log (masterKeyLog)	""	SSLKEYLOGFILE formatında TLS master key'lerini kaydetmek için yol (hata ayıklama sırasında Wireshark'ta trafiği çözmek için). Yer tutucu: /path/to/sslkeylog.txt . Üretim ortamında boş bırakın – dosya tüm trafiğin çözülmesine olanak tanır.
ECH Sockopt (echSockopt)	kapalı	Xray'in ECH config list'i sorguladığı bağlantı için soket parametrelili anahtar. Etkinleştirildiğinde şunlar kullanılabilir: Dialer Proxy (dialerProxy – isteği etiketle belirtilen outbound üzerinden yönlendir), Domain Strategy (domainStrategy), TCP Fast Open (tcpFastOpen), Multipath TCP (tcpMptcp). Gerekmedikçe kapalı bırakın.

verifyPeerCertByName , curvePreferences , masterKeyLog ve echSockopt alanları tlsSettings 'in üst düzeyinde yer alır ve yapılandırma kaydedilirken panel alanlarının «kırılmasından» etkilenmez.

Sertifikalar

SSL Sertifikası bölümü (arayüzde «SSL Sertifikası» başlığı) liste olarak yapılandırılır: + düğmesiyle yeni sertifika girişi eklenir, – Sil düğmesiyle kaldırılır (silme düğmesi yalnızca birden fazla giriş olduğunda aktif olur). TLS etkinleştirildiğinde varsayılan olarak bir boş giriş oluşturulur.

Her giriş için giriş modu anahtarı (useFile):

- **Sertifika Yolu** (useFile = true , varsayılan) – sunucudaki dosyalara yol belirtilir:
 - **Genel Anahtar** (certificateFile) – sertifika dosyasının yolu (.crt / .pem);
 - **Özel Anahtar** (keyFile) – özel anahtar dosyasının yolu (.key).
- **Sertifika İçeriği** (useFile = false) – içerik doğrudan alanlara (çok satırlı metin alanları) yapıştırılır:
 - **Genel Anahtar** (certificate) – sertifikanın PEM içeriği;
 - **Özel Anahtar** (key) – anahtarın PEM içeriği.

«Sertifika Yolu» modunun alanlarının altında iki düğme mevcuttur:

- **Panel Sertifikasını Ayarla** – alanlara panelin kendi SSL sertifikasının yollarını ekler. Merkezi paneldeki inbound için panelin sertifikası alınır (`POST /panel/setting/all` → `webCertFile / webKeyFile`); bir düğüme atanan inbound için düğümün kendi sertifikası alınır (`GET /panel/api/nodes/webCert/{nodeId}`), çünkü merkezi panelin yolları düğümde mevcut değildir. Sertifika yapılandırılmamışsa şu uyarı görüntülenir: «Panel için sertifika yapılandırılmamış. Önce Ayarlar'da sertifikayı ayarlayın.» (panelin sertifikası «Ayarlar → Güvenlik» bölümünde yapılandırılır).
- **Temizle** – her iki yolu da siler.

Her sertifika girişinin ek alanları:

Alan	Varsayılan	Açıklama
OCSF Stapling (<code>ocspStapling</code>)	<code>0</code> (kapalı)	OCSF stapling güncelleme aralığı, saniye cinsinden (minimum <code>0</code>). Yeni inbound'lar için varsayılan olarak kapalıdır (<code>0</code>): bu, OCSF responder içermeyen sertifikalar için xray günlüklerindeki hataları önler (örn. OCSF'den vazgeçen Let's Encrypt). Yalnızca stapling destekleyen sertifikalar için etkinleştirin.
Tek Seferlik Yükleme (<code>oneTimeLoading</code>)	kapalı (<code>false</code>)	Etkinleştirilirse, sertifika diskten yalnızca başlangıçta bir kez okunur ve dosya değiştiğinde yeniden okunmaz.
Kullanım Seçeneği (<code>usage</code>)	<code>encipherment</code>	Sertifikanın amacı. İzin verilen değerler: <code>encipherment</code> (şifreleme – normal sunucu sertifikası), <code>verify</code> (doğrulama), <code>issue</code> (sertifika verme – sunucu kendi başına sertifika imzalar/verir).
Build Chain (<code>buildChain</code>)	kapalı (<code>false</code>)	Yalnızca <code>usage = issue</code> durumunda görüntülenir. Sertifika zincirini tamamlar.

Inbound düzenleyicisinde ayrı bir öz imzalı sertifika düğmesi yoktur: panel, inbound için anında öz imzalı sertifika üretmez. Sertifika ya yol/içerikle belirtilir ya da «Panel Sertifikasını Ayarla» düğmesiyle panel ayarlarından çekilir. Panelin kendi SSL sertifikasının verilmesi/alınması (dosya yükleme ve alan adına bağlama dahil) **Ayarlar** → **Güvenlik** bölümünde gerçekleştirilir; burada inbound'lar için ACME/Let's Encrypt uç noktaları bulunmamaktadır.

ECH ve Sertifika Sabitleme (Genişletilmiş TLS Alanları)

Alan	Varsayılan	Açıklama
ECH key (<code>echServerKeys</code>)	<code>""</code>	Encrypted Client Hello sunucu anahtarları.
ECH config (<code>settings.echConfigList</code>)	<code>""</code>	ECH config list (istemci tarafı, bağlantıya eklenir).
Eş sertifikasının SHA-256'sı (<code>settings.pinnedPeerCertSha256</code>)	<code>[]</code>	Eş sertifikasının SHA-256 karmaları (hex dizeler, virgülle ayrılmış). İfade edildiği şekliyle ipucu: «Eş sertifikasının SHA-256 karmaları onaltılık dize biçiminde (örn. <code>e8e2d3...</code>), virgülle ayrılmış. Yalnızca panel için – xray sunucu yapılandırmasına yazılmaz, ancak istemcilerin sertifikayı sabitleyebilmesi için davet bağlantılarına eklenir.»

Düğmeler: **Eş sertifikasının SHA-256'sı** alanının yanında iki otomatik doldurma düğmesi bulunur:

- **Fill from this inbound's certificate** (kalkan simgesi) – bu inbound'un kendi sertifikasının SHA-256 karmasını ekler (`getCertHash` uç noktası üzerinden yerel olarak alınır).
- **Fetch the hash by pinging the SNI (xray tls ping)** (yükleme simgesi) – belirtilen SNI üzerinden TLS bağlantısı kurarak canlı sunucu sertifikasının karmasını alır (sunucuda `getRemoteCertHash` çağrılır). **SNI** (`serverName`) alanı doldurulmuş olmalıdır – aksi hâlde şu ipucu görüntülenir: «*Set the SNI (serverName) first to ping the remote certificate.*»

Alınan karmalar alana eklenir (virgülle ayrılarak) ve istemcinin sertifikayı sabitleyebilmesi için davet bağlantılarına aktarılır.

- **Yeni ECH Sertifikası Al** – mevcut SNI için sunucudan yeni bir ECH çifti talep eder (`POST /panel/api/server/getNewEchCert` , sunucuda `xray tls ech --serverName <SNI>` çalıştırılır); **ECH key** ve **ECH config** alanlarını doldurur.
- **Temizle** – her iki ECH alanını da sıfırlar.

7.4. REALITY Modu

`realitySettings` bloğunun alanları. REALITY, SSL sertifikası kullanmaz: bunun yerine harici bir alan adının ödünç alınan TLS el sıkışması ve X25519 anahtar çifti kullanılır.

Gizleme Parametreleri

Alan (Etiket)	Varsayılan Değer	Açıklama
Göster (<code>show</code>)	kapalı (<code>false</code>)	Xray günlüklerine REALITY hata ayıklama çıktısı. Genellikle kapalı bırakılır.
Xver (<code>xver</code>)	0	Arka uca iletilen PROXY protokolü sürümü (0 – kapalı). Minimum 0 .
uTLS (<code>settings.fingerprint</code>)	chrome	Taklit edilen TLS parmak izi (TLS ile aynı liste, ancak boş None seçeneği yoktur).
Hedef (<code>target</code>)	"" (etkinleştirildiğinde panel rastgele ekler)	Zorunlu alan. REALITY'nin TLS el sıkışmasını ödünç aldığı gerçek alan adı. İfade edildiği şekliyle ipucu: « <i>Zorunlu. Port içermelidir (örn. example.com:443). Port olmadan Xray-core başlamaz.</i> » Panel doğrulaması, portun varlığını ve geçerliliğini kontrol eder; aksi hâlde «REALITY Hedefi zorunludur» / «REALITY Hedefi port içermelidir...» / «REALITY Hedefinde geçersiz port belirtilmiş» hataları görüntülenir. Yanındaki güncelleme düğmesi yerleşik listeden rastgele bir çift ekler.
SNI (<code>serverNames</code>)	[] (hedefle birlikte eklenir)	İzin verilen SNI'lerin listesi (çoklu etiket girişi). Hedef alanındaki alan adıyla eşleşmelidir. Güncelleme düğmesi, rastgele hedefle birlikte SNI'yi de ekler.
Maks. Zaman Farkı (ms) (<code>maxTimediff</code>)	0	İstemci ve sunucu saatleri arasında izin verilen maksimum fark, milisaniye cinsinden (0 – sınırsız). Minimum 0 .
Min. İstemci Sürümü (<code>minClientVer</code>)	""	Minimum Xray istemci sürümü (yer tutucu 25.9.11). Boşsa sınırsız.

Alan (Etiket)	Varsayılan Değer	Açıklama
Maks. İstemci Sürümü (maxClientVer)	""	Maksimum Xray istemci sürümü. Boşsa sınırsız.
Short IDs (shortIds)	[] (etkinleştirildiğinde üretilir)	İstemcileri ayırt eden kısa tanımlayıcıların listesi (hex), çoklu etiket girişi; güncelleme düğmesi rastgele bir küme üretir.
SpiderX (settings.spiderX)	/	Harici siteye erişim taklidi sırasında kullanılan «örümcek» yolu (REALITY'nin istemci tarafı). Davet bağlantısına eklenir.

Hedef (target) ve **SNI** (serverNames) alanları, REALITY etkinleştirildiğinde ve güncelleme düğmesine basıldığında, panelin yerleşik listesinden rastgele bir çiftle doldurulur: `www.amazon.com`, `aws.amazon.com`, `www.oracle.com`, `www.nvidia.com`, `www.amd.com`, `www.intel.com`, `www.sony.com` (her biri :443 portuyla). Kendi sunucunuzun arkasında olmayan güçlü, kararlı bir üçüncü taraf HTTPS sitesi seçin.

Örnek: `tcp` ağında REALITY için `streamSettings` bloğu (VLESS). Sertifika gerekmez – bunun yerine ödünç alınan alan adı ve X25519 anahtar çifti kullanılır:

```
"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": [ "", "6ba85179e30d4fc2" ],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
```

Burada panelin **Hedef** (target) alanı, hazır Xray yapılandırmasındaki `dest` alanına karşılık gelir. Bir REALITY-inbound, destination `dest` anahtarıyla oluşturulduysa (panelin eski sürümleriyle, API aracılığıyla veya harici araçlarla), panel ayrıştırma sırasında `target` boşsa `dest` → `target` olarak normalleştirir; bu nedenle söz konusu inbound doğru şekilde yüklenir, **Hedef** alanı boş kalmaz ve yeniden kaydetme çalışan REALITY'yi bozmaz.

REALITY Anahtarları (X25519)

Alan	Varsayılan	Açıklama
Genel Anahtar (settings.publicKey)	""	X25519 genel anahtarı (istemci bunu kendi yapılandırmasına/bağlantısına ekler).
Özel Anahtar (privateKey)	""	X25519 özel anahtarı (yalnızca sunucuda tutulur).

Anahtarların altındaki düğmeler:

- **Yeni Sertifika Al** – sunucudan yeni bir anahtar çifti talep eder (GET /panel/api/server/getNewX25519Cert ; sunucuda xray x25519 çalıştırılır), **Özel Anahtar** ve **Genel Anahtar** alanlarını doldurur. Bu çift, REALITY modu ilk etkinleştirildiğinde de otomatik olarak üretilir.

Örnek: API aracılığıyla X25519 anahtar çifti alma (form dışında, örneğin bir betik için). İstek, özel ve genel anahtarları döndürür:

```
curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# Yanıt:
# {"success":true,"obj":{"privateKey":"...","publicKey":"..."}}
```

cookie.txt – POST /login ile giriş yapıldıktan sonra elde edilen oturum çerezi dosyası.

- **Temizle** – her iki anahtarı da sıfırlar.

Kuantum Sonrası İmza ML-DSA-65 (mldsa65)

REALITY'nin ek (isteğe bağlı) kuantum sonrası kimlik doğrulama katmanı:

Alan	Varsayılan	Açıklama
mldsa65 Seed (mldsa65Seed)	""	ML-DSA-65 anahtar seed'inin sunucu tarafı.
mldsa65 Verify (settings.mldsa65Verify)	""	Doğrulama değeri (istemci tarafı, bağlantıya eklenir).

Düğmeler:

- **Yeni Seed Al** – yeni bir çift talep eder (GET /panel/api/server/getNewmldsa65 ; sunucuda xray mldsa65 çalıştırılır), **mldsa65 Seed** ve **mldsa65 Verify** alanlarını doldurur.
- **Temizle** – her iki alanı da sıfırlar.

Fallback Hız Sınırı ve REALITY Anahtar Günlüğü

REALITY ayarlarında fallback trafiği için hız sınırı mevcuttur – bu, aktif sondaların sunucuyu ödünç alınan alana ücretsiz bir kanal olarak kullanmasını engeller. Ayar iki yön için ayrı ayrı yapılandırılır: **Limit Fallback Upload** ve **Limit Fallback Download** (limitFallbackUpload / limitFallbackDownload), her birinde aynı alan kümesi bulunur:

Alan (Etiket)	Varsayılan	Açıklama
After Bytes (afterBytes)	0	Sınırlama başlamadan önce tam hızda geçirilecek bayt sayısı. 0 – ilk bayttan itibaren sınırla.
Bytes Per Sec (bytesPerSec)	0	Eşiğin ardından fallback trafiği için bayt/saniye hız tavanı. 0 – sınır yok (bu yönü devre dışı bırakır).
Burst Bytes Per Sec (burstBytesPerSec)	0	Sabit hızın üzerindeki kısa süreli artışlar için tampon (token-bucket boyutu). Bytes Per Sec değerinden küçükse ona yükseltilir.

Aynı yerde **Master Key Log** (masterKeyLog) alanı da bulunur – Wireshark'ta hata ayıklama amacıyla `SSLKEYLOGFILE` formatında TLS master key'lerini kaydetmek için yol; üretim ortamında boş bırakın.

7.5. Yapılandırma için Pratik Öneriler

1. **VLESS + Reality (önerilen):** tcp ağında bir VLESS-inbound oluşturun, «Güvenlik» sekmesinde **Reality**'yi seçin – panel rastgele target /SNI, shortIds ekler ve X25519 anahtarlarını üretir. Gerekirse kendi anahtar çiftiniz için «Yeni Sertifika Al» düğmesine basın. VLESS istemcileri için **Flow** = xtls-rprx-vision (XTLS Vision) etkinleştirin – bu maksimum performans ve gizlilik sağlar.

Örnek: VLESS + Reality + Vision nihai istemci bağlantısı. Panelin böyle bir inbound için oluşturduğu davet bağlantısı şöyle görünür (anahtar/ID değerleri açıklama amaçlıdır):

```
vless://uuid-клиента@1.2.3.4:443?
type=tcp&security=reality&pbk=ПУБЛИЧНЫЙ_КЛЮЧ&fp=chrome&sni=www.nvidia.com&sid=6ba85179e3
0d4fc2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

Burada pbk – X25519 genel anahtarı, sni – **Hedef** alanından ödünç alınan alan adı, sid – **Short IDs**'den biri, flow=xtls-rprx-vision – etkinleştirilmiş XTLS Vision. 2. **Kendi alan adınızla TLS:** TLS'yi seçin, SNI alanına alan adını girin, sertifikayı ekleyin (dosya yollarıyla veya içerikle), ya da alan adı ve sertifika «Ayarlar → Güvenlik» bölümünde zaten yapılandırılmışsa «Panel Sertifikasını Ayarla» düğmesine basın. Normal bir tarayıcı taklidi için **Min/Maks Sürüm** = 1.2 – 1.3 ve **uTLS** = chrome olarak bırakın. 3. Dışarıya açık inbound'lar için **Yok** modunu bırakmayın – yalnızca yerel fallback hedefleri (127.0.0.1) için veya TLS'yi harici proxy sağladığında kullanın. 4. Arayüzden ipucu: çoğu gelişmiş alan için «Varsayılan ayarları bırakmanız önerilir» ipucu gösterilir – bu alanları yalnızca sonuçlarını anlıyorsanız değiştirin.

8. İstemciler

İstemci, bir VPN kullanıcı hesabıdır: bir veya birden fazla inbound'a bağlı, kendi trafik kotası, geçerlilik süresi ve eş zamanlı bağlantı limiti olan kimlik bilgileri (UUID veya şifre) kümesi. Bu fork'ta istemci bağımsız bir varlıktır (`clients` tablosu): aynı istemci aynı UUID/şifre ve ortak trafik sayacını koruyarak birden fazla inbound'a aynı anda bağlanabilir. **İstemciler** bölümü, inbound'dan bağımsız olarak paneldeki tüm hesapları arama, filtreler, sıralama ve toplu işlemlerle birlikte gösterir.

8.1. İstemci alanları

Aşağıda **İstemci ekle** / **İstemciyi düzenle** düzenleyicisinin her alanı açıklanmaktadır.

İstemci formu iki sekmeye ayrılmıştır: **Temel** (email, inbound bağlantısı, limitler, süre, grup, yorum, ters etiket) ve **Kimlik Bilgileri** (UUID/şifre/auth, Flow, VMess Security). Alan etiketlerinde kota **Trafik Limiti (GB)** olarak, süreler ise **Süre (gün)** ve **Otomatik Yenileme (gün)** olarak belirtilmektedir; **Trafik Limiti (GB)** ve **IP Limiti** alanlarının **0** değerinin "sınırsız" anlamına geldiğini açıklayan ipuçları vardır. Mevcut bir istemci düzenlenirken rastgele email oluşturma düğmesi gizlenir ve IP günlüğü düğmesi doğrudan **IP Limiti** alanının yanına taşınarak kaydedilen adreslerin sayısını gösterir.

Alan	JSON anahtarı	Varsayılan	Açıklama
Email	<code>email</code>	– (zorunlu)	İstemcinin benzersiz tanımlayıcısı
UUID	<code>id</code>	oluşturulur	VMess/VLESS için tanımlayıcı
Şifre	<code>password</code>	oluşturulur	Trojan/Shadowsocks için şifre
Yetkilendirme	<code>auth</code>	oluşturulur	Hysteria için şifre
Flow	<code>flow</code>	boş	Flow kontrolü (XTLS), yalnızca VLESS
VMess Security	<code>security</code>	<code>auto</code>	VMess şifreleme yöntemi
IP Limiti	<code>limitIp</code>	0 (limitsiz)	Maksimum eş zamanlı IP sayısı
Toplam gönderilen/alınan (GB)	<code>totalGB</code>	0 (limitsiz)	Trafik kotası
Geçerlilik süresi	<code>expiryTime</code>	0 (süresiz)	Sona erme tarihi
Otomatik yenileme	<code>reset</code>	0 (kapalı)	Trafik sıfırlama periyodu, gün
Telegram kullanıcı ID'si	<code>tgId</code>	0 (yok)	Sayısal Telegram ID'si
Abonelik ID'si	<code>subId</code>	oluşturulur	Abonelik tanımlayıcısı
Grup	<code>group</code>	boş	Mantıksal grupta etiketi
Yorum	<code>comment</code>	boş	Serbest biçimli not
Etkin	<code>enable</code>	<code>true</code>	Hesabın etkin olup olmadığı

Email (tanımlayıcı)

Email alanı, istemcinin temel ve zorunlu tanımlayıcısıdır. Adına karşın bir e-posta adresi olmak zorunda değildir: herhangi bir metin etiketi (kullanıcı adı, numara) kabul edilir. Değer panel genelinde **benzersiz** olmalıdır; dolu bir email ile ikinci bir istemci oluşturma girişimi reddedilir (`email already in use`); `subId` de aynıysa bu durum aynı istemcinin bağlantısı olarak yorumlanır.

Email **boş bırakılamaz** (`client email is required`) ve **boşluk**, `/`, `,`, `\` veya **kontrol karakterleri içeremez** ("Email boşluk, '/', '\ ' veya kontrol karakterleri içeremez"). Email; trafik hesaplamasında, IP günlüğünde, çevrimiçi listede ve işlem adlarında yer alır – sonradan değiştirilmesi önerilmez.

UUID / Şifre / Yetkilendirme (kimlik bilgileri)

Hangi kimlik bilgisi alanının kullanılacağı, istemcinin bağlandığı inbound'un protokolüne bağlıdır. Alan boş bırakılırsa değerler otomatik olarak doldurulur:

- **UUID** (`id` alanı) – **VMess** ve **VLESS** protokolleri için. Belirtilmezse rastgele UUID v4 oluşturulur.
- **Şifre** (`password` alanı) – **Trojan** ve **Shadowsocks** için. Trojan'da varsayılan olarak tirsiz UUID oluşturulur. Shadowsocks'ta inbound şifreleme yöntemine bağlı olarak gerekli uzunlukta Base64 anahtar oluşturulur: `2022-blake3-aes-128-gcm` için 16 bayt, `2022-blake3-aes-256-gcm` ve `2022-blake3-chacha20-poly1305` için 32 bayt; diğer yöntemler için tirsiz UUID. Elle girilen anahtar `2022-blake3` yöntemine uymuyorsa oluşturulan anahtarla değiştirilir.
- **Yetkilendirme** (`auth` alanı) – **Hysteria** için şifre. Varsayılan olarak tirsiz UUID.

Bir istemci farklı protokollerdeki birden fazla inbound'a bağlanabildiğinden, istemci kaydında aynı anda hem UUID hem şifre hem de auth bulunabilir – her protokol kendi alanını kullanır.

Örnek: istemci kimlik bilgilerinin farklı inbound'ların settings bölümünde görünümü. Aynı istemci VLESS inbound'da `id`, Trojan'da `password`, Shadowsocks'ta `password` (Base64 anahtar) ile tanımlanır:

```
// VLESS inbound'un settings.clients bölümünden parça
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-vision" }

// aynı istemci Trojan inbound'da
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// aynı istemci Shadowsocks inbound'da (yöntem: 2022-blake3-aes-256-gcm)
{ "password": "GPy0aA3f7C00az53eaQ8eqMfRDjmbL0h7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow (`flow` alanı) – XTLS akış kontrolü. Yalnızca **VLESS** için geçerlidir ve yalnızca inbound XTLS Vision için yapılandırıldığında: security olarak **tls** veya **reality** kullanılan **TCP** taşıması üzerinde VLESS. Geçerli değer `xtls-rprx-vision` 'dır (geçmişe yönelik `xtls-rprx-vision-udp443` da kabul edilir); boş değer flow'un olmadığını belirtir.

Inbound XTLS flow'u desteklemiyorsa ayarlanan flow, istemci kaydedilirken **sessizce sıfırlanır**: birden fazla inbound'a bağlı aynı istemci için flow yalnızca uygun olduğu inbound'larda uygulanır. Yalnızca VLESS-Vision'ı kasıtlı olarak kullanıyorsanız değiştirmeniz gerekir.

VMess Security

VMess Security (`security` alanı) – VMess için yük şifreleme yöntemi. Varsayılan değer `auto` 'dur (Xray şifrelemeyi kendisi seçer). Geçerli değerler VMess için standart olanlardan ibarettir: `auto` , `aes-128-gcm` , `chacha20-poly1305` , `none` , `zero` . Diğer protokoller için bu alan kullanılmaz.

IP Limiti (eş zamanlı bağlantılar)

IP Limiti (`limitIp` alanı) – istemcinin aynı anda bağlanabileceği maksimum **farklı IP adresi** sayısı. Varsayılan değer `0` olup **sınırsız** anlamına gelir. Pozitif bir değer ayarlandığında panel istemcinin etkin IP'lerini izler ve limit aşıldığında arka plan görevi hesabı devre dışı bırakır. (**3.3.1** sürümünden itibaren IP sayımı Xray çekirdeğinin online-stats API'si aracılığıyla yapılır ve erişim günlüğü **gerektirmez**; eski çekirdek sürümlerinde panel, etkinleştirilmiş olması gereken erişim günlüğünü okumaya geri döner.) Tek bir aboneliğin birçok cihazda paylaşılmasını engellemek için kullanın: örneğin `2` değeri iki cihaza izin verir.

IP Limiti **Fail2ban** aracılığıyla uygulanır; bu nedenle **IP Limiti** alanı yalnızca Fail2ban kurulu ve çalışır durumdayken etkindir (panel durumu `GET /panel/api/server/fail2banStatus` üzerinden kontrol eder). Fail2ban kurulu değilse istemci düzenleyicisindeki alan (ve toplu istemci ekleme formu) kilitlenir ve üzerine gelindiğinde Fail2ban'ı x-ui bash menüsünden yüklemenizi öneren bir ipucu gösterilir ("Fail2ban is not installed, so the IP limit cannot be enforced. Install Fail2ban from the x-ui bash menu to enable this option."); Windows'ta ipucu Fail2ban'ın orada kullanılamayacağını belirtir ("Fail2ban is not available on Windows, so the IP limit cannot be enforced."), sunucuda özellik devre dışıysa ise "The IP limit feature is disabled on this server." mesajı gösterilir. Panel güncellendiğinde Fail2ban'sız sunuculardaki istemcilerin kaydedilmiş IP limiti tek seferlik bir geçişle sıfırlanır; zira orada zaten uygulanmıyordu.

Değer örnekleri. `limitIp: 0` – sınırsız; `limitIp: 1` – aynı anda yalnızca bir cihaz; `limitIp: 3` – en fazla üç farklı IP. Dördüncü etkin IP'de arka plan görevi, siz **IP Limitini Sıfırla** işlemini yapana kadar istemciyi devre dışı bırakır (`enable = false`).

İlgili işlemler: **IP Günlüğü**, istemcinin kaydedilen IP'lerinin listesini gösterir; her kayıt IP'nin yanı sıra son erişim zamanını ve bağlantının kaydedildiği düğüm etiketini (`@ düğüm_adı`) içerir – çok panelli yapılandırmada istemcinin hangi düğüm üzerinden bağlandığı görülür. **IP Limitini Sıfırla**, biriken IP günlüğünü temizleyerek istemcinin doğal kayıt süresinin dolmasını beklemeden yeniden bağlanabilmesini sağlar.

Toplam gönderilen/alınan (GB) – trafik kotası

Toplam gönderilen/alınan (GB) (`totalGB` alanı) – toplam trafik kotası (gönderme + alma). Varsayılan değer `0` , **sınırsız** anlamına gelir. Kota dolduğunda (`up + down >= total`) istemci **tükenmiş** (depleted) sayılır ve devre dışı bırakılır. Kullanıcı arayüzünde genellikle gigabayt cinsinden girilir; veritabanında bayt olarak saklanır.

İstemci listesinde **Trafik** sütunu renkli bir kullanım çubuğu gösterir: kullanılan trafik miktarı, limit etiketi (sınırsızsa ∞ simgesi) ve üzerine gelindiğinde gönderilen/alınan ile kalan miktarların dökümünü içeren ipucu. Aynı kompakt gösterge, telefondaki istemci kartlarında da görünür.

Geçerlilik süresi (Expiry)

Geçerlilik süresi (`expiryTime` alanı) hesabın sona erme anını belirler. Alanın üç modu vardır:

- **Süresiz** – `0` . İstemcinin süresi hiçbir zaman dolmaz.
- **Belirli bir tarih** – pozitif Unix zaman damgası (milisaniye cinsinden). Sona erme anı geldiğinde (`expiryTime <= şimdi`) istemci süresi dolmuş (expired) sayılır ve devre dışı bırakılır. Kullanıcı arayüzünde genellikle tarih seçilerek veya **Süre** alanına **Gün** cinsinden girilerek ayarlanır.
- **İlk kullanımdan sonra başlat** – süreyi kodlayan **negatif** değer. İstemci hiç bayt aktarmadığı sürece süre negatif kalır ("ertelenmiş başlangıç"). İlk trafik sayım döngüsünde panel bunu mutlak tarihe dönüştürür: `şimdi + |süre|` . Bu, istemcinin ne zaman etkinleşeceği önceden bilinmeden örneğin "ilk bağlantıdan itibaren 30 gün" şeklinde satış yapılmasına olanak tanır. Dönüşüm, bağlı tüm inbound'ların aynı son tarihi alması için email başına bir kez gerçekleştirilir.

Süre kodlama örneği. Sabit tarih 1 Mart 2026, 00:00 UTC → `expiryTime: 1772323200000` (milisaniye cinsinden pozitif zaman damgası). "İlk bağlantıdan itibaren 30 gün" → `expiryTime: -2592000000` (negatif değer, $30 \times 24 \times 60 \times 60 \times 1000$); ilk trafik baytında panel bunu `şimdi + 2592000000` ile değiştirir. Süresiz → `expiryTime: 0` .

Otomatik yenileme (istemci trafik sıfırlama periyodu)

Otomatik yenileme (`reset` alanı) – gün cinsinden otomatik yenileme/sıfırlama periyodu. İpucu: "Süre dolunca otomatik yenileme. (0 = devre dışı) (birim: gün)".

- `0` – otomatik yenileme **devre dışı** (varsayılan değer). Süre dolduğunda istemci tükenmiş duruma geçer.
- `> 0` – arka plan görevi süre dolduğunda trafik sayaçlarını sıfırlar (`up = down = 0`), geçerlilik süresini `reset` gün ileriye **kaydırır** (gerekirse yeni süre geleceğe düşene kadar birden fazla periyot atlar) ve gerekirse istemciyi yeniden **etkinleştirir**. Bu, aylık abonelik gibi periyodik abonelikler oluşturur. Otomatik yenileme, **düğüm-nod'lardaki inbound'lara** (`node_id IS NOT NULL`) **uygulanmaz**.

Önemli sonuç: `reset > 0` olan istemciler toplu silme işlemlerindeki "tükenmiş" kavramından **çıkarılır** – trafikleri/süreleri otomatik yenilemeyle sıfırlandığından hesapları silinmeye aday sayılmaz.

Telegram kullanıcı ID'si

Telegram kullanıcı ID'si (`tgId` alanı) – panelin yerleşik Telegram botuna (bildirimler, istatistikleri bağımsız görüntüleme) bağlamak için sayısal Telegram kullanıcı tanımlayıcısı. İpucu: "Sayısal Telegram kullanıcı ID'si (0 = yok)". Varsayılan değer `0` – bağlantı yok. Bu alana göre filtreleme (**Var / Yok**) yapılabilir.

Abonelik ID'si (subId)

Abonelik ID'si (`subId` alanı) – istemcinin **aboneliğe** (subscription) dahil edildiği tanımlayıcı. Aynı `subId` 'ye sahip tüm istemciler tek bir abonelik bağlantısıyla sunulur. Oluşturma sırasında alan boş bırakılırsa panel otomatik olarak rastgele bir `subId` (UUID) **oluşturur**. Değer, farklı email'e sahip istemciler arasında **benzersiz** olmalıdır (`subId already in use`) ve email ile aynı karakter kısıtlamalarına tabidir ("Abonelik ID'si boşluk, '/', '\ ' veya kontrol karakterleri içeremez").

`subId` olmadan istemci için abonelik bağlantısı kullanılamaz ("Bu istemcinin subId'si yok, paylaşım bağlantısı kullanılamaz.").

Links sekmesi (harici bağlantılar ve abonelikler)

Temel ve **Kimlik Bilgileri** sekmelerinin yanı sıra istemci düzenleyicisinde üçüncü bir **Links** sekmesi bulunur (ipucu: "Add third-party share links and remote subscription URLs to include in this client's subscription."). Bu sekmede **Add External Link** düğmesiyle üçüncü taraf paylaşım bağlantıları (`vless://` , `vmess://` , `trojan://` , `ss://` , `hysteria2://` , `wireguard://`), **Add External Subscription** düğmesiyle de uzak abonelik URL'leri (örn. `https://provider.example/sub/...`) eklenir.

Bunların tümü bu istemcinin abonelik çıktısına (raw, JSON ve Clash formatları) dahil edilir: bağlantılar olduğu gibi eklenir, uzak abonelikler ise panel tarafından periyodik olarak indirilir (önbellekleme ve kısa zaman aşımıyla) ve yapılandırmaları yerel olanlarla birleştirilir. Böylece istemcinin tek bir abonelik bağlantısında kendi sunucuların yanı sıra harici yapılandırmalar da sunulabilir.

Grup

Grup (`group` alanı) – ilgili istemcileri bir araya getirmek için mantıksal etiket. İpucu: "İlgili istemcileri gruplamak için mantıksal etiket (örn. ekip, müşteri, bölge). Araç çubuğundan filtrelenebilir.", yer tutucu – "örn. customer-a". Alan isteğe bağlıdır (varsayılan boş). Gruba göre listeyi filtreleyebilir ve toplu işlemler yapabilirsiniz; istemciden etiketi kaldırmak için **Gruptan Çıkar** eylemi kullanılır.

Grubu tek bir istemcinin düzenleyicisinden de kaldırabilirsiniz: **Grup** alanını temizleyip kaydederseniz etiket doğru biçimde kaldırılır ve istemci önceki grup altında görünmez.

Yorum

Yorum (`comment` alanı) – yönetici için serbest biçimli metin notu (varsayılan boş). İçerik aramada yer alır ve filtrelemeye açıktır (**Var / Yok** yorumu).

Etkin

Etkin (`enable` alanı) – hesap etkinlik bayrağı. Varsayılan olarak **etkin** (`true`); oluşturma sırasında bayrak iletilmese bile panel zorla `true` ayarlar. Devre dışı bırakılan istemci (`enable = false`) bağlanamaz ve özetle **devre dışı** (deactive) kategorisinde yer alır. Panel, kotasını tüketen, süresi dolan veya IP limitini aşan istemcileri otomatik olarak devre dışı bırakır.

Yalnızca okunabilir alanlar

İstemci kartında ayrıca servis alanları gösterilir: **Oluşturuldu** (`created_at`) ve **Güncellendi** (`updated_at`) – otomatik doldurulan ve düzenlenemeyen oluşturma ve son değişiklik zaman damgaları. **Ters etiket** (`reverse`) alanı – basit VLESS ters proxy için isteğe bağlı Reverse tag ("İsteğe bağlı Reverse tag").

8.2. Inbound'a bağlama

Her istemci en az bir inbound'a bağlanmalıdır – oluşturma sırasında en az bir tane gereklidir (`at least one inbound is required`). Düzenleyicide bu alan, **Seçin veya birden fazla gelen seçin** ipucuna sahip **Bağlı gelen bağlantılar** alanıdır.

- **Bağla** – istemciyi seçilen inbound'lara ekler (aynı UUID/şifre ve ortak trafik). Mevcut bağlantılar korunur.
- **Bağlantıyı kes** – istemciyi seçilen inbound'lardan çıkarır. İstemci kaydı korunur (tamamen silmek için **Sil** kullanın). İstemcinin bağlı olmadığı çiftler sessizce atlanır.

Birden fazla inbound'a bağlı bir istemci kaydedilirken belirli protokol/taşımayla uyumsuz alanlar (örn. VLESS-Vision dışındaki Flow) her inbound için geçerli değerlere otomatik olarak ayarlanır.

Inbound seçim listesinin üzerinde (istemci formunda, toplu istemci eklemede ve toplu bağlama/bağlantı kesme pencerelerinde) **Tümünü seç** ve **Temizle** düğmeleri bulunur. Bu listelerde her inbound, varsa kendi açıklamasıyla (remark), yoksa inbound etiketiyle gösterilir.

8.3. İstemci üzerinde işlemler

Tek bir istemci için (**İstemci Bilgisi** kartı veya **Eylemler** bağlam menüsü aracılığıyla) şunlar mevcuttur:

Bilgileri, QR kodunu ve bağlantıyı görüntüleme

- **İstemci Bilgisi** – tüm alanları, kullanılan/kalan trafiği (**Kalan**), geçerlilik süresini ve bağlı inbound'ları içeren kart.

API aracılığıyla istemci sorgusu (`GET /panel/api/clients/get/:email`), `client` ve `inboundIds` alanlarının yanı sıra düğüm verilerini de hesaba katan fiilen kullanılan trafiği (gönderilen + alınan) belirten `usedTraffic` 'i de döndürür; bu, kullanımı `totalGB` kotasıyla karşılaştırmayı kolaylaştırır.

- **QR kodu ve Bağlantı** – istemci uygulamasına içe aktarmak için istemci yapılandırma bağlantısı. Desteklenen protokolü olan tüm bağlı inbound'lara göre oluşturulur (`GET /links/:email`). Uygun bağlantı yoksa: "Paylaşılacak bağlantı yok – önce istemciyi desteklenen protokolü bir gelen bağlantıya bağlayın."
- **Abonelik bağlantısı** – `subId` 'ye göre abonelik URL'si (`GET /subLinks/:subId`). Yalnızca istemcinin `subId` 'si varsa ve abonelik hizmeti **Panel Ayarları** → **Abonelik** bölümünde etkinleştirilmişse kullanılabilir (aksi halde "Abonelik hizmeti devre dışı."). Ek olarak **JSON abonelik URL'si** de sunulur.

Trafiği sıfırla

Trafiği sıfırla (`POST /resetTraffic/:email`) belirli bir istemcinin gönderme/alma sayaçlarını (`up` , `down`) sıfırlar. Kota (`totalGB`) ve geçerlilik süresi **etkilenmez** – yalnızca kullanılan miktar sıfırlanır. Bildirim: "Trafik sıfırlandı". İstemci herhangi bir inbound'a bağlı değilse: "Önce bu istemciyi bir gelen bağlantıya bağlayın."

Trafiği sıfırla düğmesi, istemci düzenleme formundan da erişilebilir – **İptal** / **Kaydet** yanında alt panelde (sıfırlamadan önce onay istenir). İstemci trafik tükenmesi nedeniyle devre dışı bırakılmışsa sıfırlama (tek veya toplu) otomatik olarak onu yeniden **etkinleştirir** (`enable = true`) ve bu değişikliği anında düğüm-nod'lara gönderir – ana panelde ve nod'larda istemciyi manuel olarak yeniden etkinleştirmeye gerek kalmaz.

IP Limitini Sıfırla

İstemcinin biriken IP günlüğünü temizler (`POST /clearIps/:email`), eş zamanlı bağlantı limiti aşımından kaynaklanan geçici engeli kaldırır. Bildirim: "Günlük temizlendi".

Sil

Sil (`POST /del/:email`) – istemciyi tamamen siler. Onay iletişim kutusu: başlık "İstemci {email} silinsin mi?", metin "İstemci tüm bağlı gelen bağlantılardan kaldırılacak ve trafik kaydı yok edilecek. Bu işlem geri alınamaz.". Silme, istemciyi **tüm** inbound'lardan kaldırır ve trafik kaydını yok eder. Bildirim: "İstemci silindi".

8.4. Toplu işlemler

İstemci listesinde birden fazla kayıt seçilebilir (**Tümünü seç**, **Tümünü temizle**); sayacı "{count} seçildi" şeklinde gösterilir. Seçilenler üzerinde şunlar yapılabilir:

- **Sil ({count})** (`POST /bulkDel`) – toplu silme. Onay: "{count} istemci silinsin mi?", "Seçilen her istemci tüm bağlı gelen bağlantılardan kaldırılır ve trafik kaydı yok edilir. Bu işlem geri alınamaz.". Bildirim: "{count} istemci silindi", kısmi başarısızlık durumunda "Silindi: {ok}, başarısız: {failed}".
- **Düzenle ({count}) / Ayarlama** (`POST /bulkAdjust`) – seçilen istemcilerin süresini ve/veya kotasını toplu olarak değiştirme. İletişim kutusu "{count} istemciyi düzenle", ipucu "Pozitif değerler ekler, negatif değerler azaltır. Sınırsız süre veya trafiği olan istemciler ilgili alan için atlanır.". Alanlar: **Gün ekle** ve **Trafik ekle (GB)**. Mantık:
 - **Süre:** süresiz istemciler (`expiryTime == 0`) atlanır ("unlimited expiry"); tarihli istemcilerin süresi belirtilen gün sayısı kadar öne alınır; "ilk kullanımdan sonra" modundaki istemcilerde (negatif süre) bekleme süresi ayarlanır. Kalan süreden daha büyük bir azaltma atlanır ("reduction exceeds remaining time/delay window").
 - **Trafik:** sınırsız istemciler (`totalGB == 0`) atlanır ("unlimited traffic"); aksi halde kota belirtilen miktarda değiştirilir, sıfırın altına düşmez.
 - Gün ve trafik hiçbirini belirtilmemişse: "Uygulamadan önce gün veya trafik belirtin.". Bildirim: "Düzenlendi: {count}" / "Düzenlendi: {ok}, atlandı: {skipped}".

Örnek: seçilen istemcileri 30 gün uzatma ve 50 GB ekleme. **Düzenle** iletişim kutusunda **Gün ekle** = 30 , **Trafik ekle (GB)** = 50 girin. Tersine, bir hafta çıkarmak ve kotayı 10 GB kısmak için negatif değerler girin: **Gün ekle** = -7 , **Trafik ekle (GB)** = -10 (sınırsız süre veya kotası olan istemciler ilgili alan için atlanır).

- **Bağla ({count}) / Bağlantıyı kes ({count})** (`POST /bulkAttach / bulkDetach`) – seçilen istemcileri seçilen inbound'lara toplu bağlama/bağlantı kesme. Hedefler yalnızca çok kullanıcı inbound'lar. Bağlantı kesme sonucu: "{detached} bağlantı kesildi, {skipped} atlandı".
- **Abonelik bağlantıları ({count})** – seçilen istemcilerin abonelik ve JSON abonelik URL'lerinin **Tümünü kopyala** düğmesiyle özet tablosu. Hiçbirinde subld yoksa: "Seçilen istemcilerin hiçbirinin Abonelik ID'si yok".
- **Gruba ekle ve Gruptan çıkar** – grup etiketi atama ve kaldırma.

Duruma göre trafik sıfırlama ve silme

- **Tüm istemcilerin trafiğini sıfırla** (POST /resetAllTraffics) – paneldeki **tüm** istemcilerin up / down sayaçlarını sıfırlar. Onay: "Tüm istemcilerin trafiği sıfırlansın mı?" ve "Tüm istemcilerin gönderme/alma sayaçları sıfırlanır. Kotalar ve geçerlilik süreleri etkilenmez. Bu işlem geri alınamaz.". Bildirim: "Tüm istemcilerin trafiği sıfırlandı".
- **Tükenenlerini sil** (POST /delDepleted) – reset = 0 koşuluyla **kotası tükenmiş** (total > 0 and up + down >= total) **veya süresi dolmuş** (expiry_time > 0 and expiry_time <= şimdi) tüm istemcileri siler (otomatik yenilemeli istemcilere dokunulmaz). Onay: "Tükenen istemciler silinsin mi?", "Trafik kotası tükenmiş veya süresi dolmuş tüm istemciler silinir. Bu işlem geri alınamaz.". Bildirim: "{count} tükenen istemci silindi".

Dışa aktarma, içe aktarma ve bağlantısız istemcileri silme

Hiçbir şey seçili değilken **İstemciler** sayfasındaki **Daha fazla** menüsünde üç işlem mevcuttur.

İstemcileri dışa aktar (GET /clients/export), {client, inboundIds} formatındaki tüm istemcilerin JSON listesini kopyalama ve indirme düğmeleriyle (dosya: clients-export.json) birlikte bir görüntüleyicide açar. **İstemcileri içe aktar** (POST /clients/import) aynı JSON'ın yapıştırıldığı ve **Import** düğmesine basıldığı bir düzenleyici açar: inboundIds olan istemciler oluşturulur ve inbound'lara bağlanır, bağlantısız istemciler bağımsız "yalnız" kayıtlar olarak geri yüklenir, mevcut email'ler ise **hiçbir zaman üzerine yazılmaz** – atlananlar listesine eklenir. Bildirimler: "{count} clients imported", "{ok} imported, {failed} skipped".

Bağlantısız istemcileri sil (POST /clients/delOrphans) – tehlikeli bir işlem: hiçbir inbound'a bağlı olmayan tüm istemcileri trafik kayıtları, IP günlükleri ve harici bağlantılarıyla birlikte siler. Onay: "Delete clients without an inbound?", "Removes every client that is not attached to any inbound, along with its traffic record. This cannot be undone.". Bildirim: "{count} unattached clients deleted". İşlem geri alınamaz.

8.5. Arama, filtreler ve sıralama

Listenin üzerinde bir arama çubuğu bulunur ("Email, yorum, sub ID, UUID, şifre, auth ara...") – email, yorum, subId, UUID, şifre ve auth'a göre arama yapar. Sonuç sayacı: "{total} içinden {shown} gösteriliyor".

İstemci listesi otomatik olarak güncellenir: panel birkaç saniyede bir geçerli sayfayı çeker; bu nedenle yeni bağlanan istemciler ve değişen sıralama düzeni manuel yenileme gerekmeksizin görünür (arka plan sorgulamasında yükleme göstergesi yanıp sönmez).

İstemci filtresi paneli; duruma (kategori), protokole, bağlı inbound'a, geçerlilik süresi aralığına, kullanılan trafik aralığına, otomatik yenileme varlığına (**Var/Yok**), Telegram ID ve yorum varlığına ve ayrıca gruba göre filtreleme yapılmasını sağlar. Düğümlü panellerde **Düğümler** çoklu seçimi görünür: liste seçilen düğümlerdeki istemcilerle sınırlandırılabilir; ayrı bir **Yerel panel** seçeneği düğüme bağlı olmayan inbound istemcilerini filtreler (filtre yalnızca düğüm mevcut olduğunda görünür). Sıralama: **Önce eskiler/yeniler, Son güncellenenler, Son çevrimiçi olanlar, Email A→Z / Z→A, En çok trafik, En çok kalan, En yakın süre dolacaklar.**

8.6. Simgeler ve durumlar

Durum önceliği: tükenmiş/süresi dolmuş → devre dışı → yakında dolacak → etkin.

- **Çevrimiçi / Çevrimdışı** – etkin bağlantısı olan (geçerli çevrimiçi listede bulunan) ve **etkin** olan istemci. Çevrimiçi liste ayrı sorgularla güncellenir (/onlines , /onlinesByGuid).
 - **Tükenmiş** (depleted) – kota dolmuş (up + down >= totalGB) **veya** süresi dolmuş (expiryTime <= şimdi). Bu istemci otomatik olarak devre dışı bırakılır ve **Tükenenlerini sil** işlemine tabi olur.
 - **Yakında dolacak / bitecek** (expiring) – süresi dolmaya yakın (eşik aralığından az kaldıysa) **veya** kotası tükenmeye yakın (eşik miktarından az kaldıysa) etkin istemci (eşikler panel ayarlarında belirlenir). İstemci zaten tükenmişse veya devre dışıysa sayılmaz.
 - **Devre dışı** (deactive) – enable = false olan istemci (manuel olarak veya arka plan görevi tarafından devre dışı bırakılmış).
 - **Etkin** (active) – etkin, tükenmemiş, süresi dolmamış ve eşiklerden uzak.
-

9. İstemci Grupları

Bu, 3X-UI'nin bu çatalına özgü bir özelliktir. Orijinal 3x-ui (MHSanaei) projesinde "istemci grubu" kavramı yoktur — burada ayrı bir gruplar tablosu, panel menüsünde **Gruplar** sayfası ve ilgili API yöntemleri eklenmiştir. Yapılandırmayı orijinal 3x-ui'ye taşırsanız, grup etiketi hiçbir yerde dikkate alınmayacaktır.

9.1. İstemci Grubu Nedir ve Ne İşe Yarar

Grup, bir veya birden fazla istemciye atanabilen adlandırılmış bir mantıksal etikettir (label). Yeni bir bağlantı yöntemi oluşturmaz; bir inbound ya da düğüm değildir — yalnızca istemcileri filtrelemek ve toplu olarak işlemek için kullanılan organizasyonel bir etikettir.

Bu çatalın istemci modelindeki temel fikir: **istemci, email ile tanımlanan üst düzey bir varlıktır** (`clients` tablosundaki `email` alanının benzersiz bir dizini vardır). Aynı istemci (aynı email ve aynı kimlik bilgileriyle), farklı protokoller dahil olmak üzere aynı anda birden fazla inbound'da ve hatta birden fazla düğümde yer alabilir. Grup etiketi **istemci başına bir kez** saklanır, bu nedenle otomatik olarak istemcinin tüm inbound bağlantılarına yayılır.

Grup etiketi, gruplama için kullanılan mantıksal bir etikettir:

Katman	Nerede saklanır	Alan
İstemci kaydı (VT)	<code>clients</code> tablosu	<code>group_name</code> (varsayılan olarak boş dize <code>' '</code>)
Grup rehberi (VT)	<code>client_groups</code> tablosu	<code>name</code> (benzersiz dizin, boş olamaz)
inbound ayarları (Xray)	JSON <code>settings.clients[].group</code>	istemcinin üye olduğu her inbound'daki her istemci nesnesine kopyalanır

Pratikte bunun önemi:

- **Birden fazla inbound/düğümde tek istemci.** Bir istemci birden fazla inbound'a (örneğin farklı protokoller veya farklı düğümler) erişim olarak "satılıyorsa", grup onu tek bir bütün olarak yönetmeyi sağlar: trafiği sıfırlamak, silmek, etiketi yeniden adlandırmak — tüm inbound'larında tek bir işlemle.
- **Toplu işlemler ve filtreleme.** **İstemciler** sayfasında liste gruba göre filtrelenebilir; **Gruplar** sayfasında gruptaki tüm üyeler üzerinde toplu işlemler yapılabilir.
- **Büyük istemci parkını organize etme.** `vip`, `trial`, `team-A` gibi etiketler, ayrı inbound'lar oluşturmadan binlerce istemciyi mantıksal kategorilere ayırmaya yardımcı olur.

9.2. Grubun İstemciler, inbound'lar, Düğümler ve Protokollerle İlişkisi

Etiket senkronizasyonu basit olmadığından bu alt bölüm anlaşılması en kritik olanıdır.

Grup, inbound'u değil istemciyi tanımlar. Etiket, istemci kaydında (`clients.group_name`) yaşar. Bir istemci birden fazla inbound'a bağlı olduğunda, herhangi bir grup değişikliğinde panel, bu istemcinin üye olduğu **tüm** inbound'ları gezinir ve Xray ayarlarındaki (`settings.clients[]`) `group` alanını günceller/kaldırır. Teknik

olarak şöyle çalışır: istemcinin email'ine göre üye olduğu tüm inbound'lar bulunur, ardından bu email'e sahip istemci nesnesi her inbound'un JSON ayarlarında düzenlenir. Bu nedenle:

- Grup **protokolden bağımsızdır**. Aynı email, bir inbound'da VLESS istemcisi, başka bir inbound'da Hysteria istemcisi olabilir – grup etiketi yine de tek bir tanedir ve ikisine de uygulanır (her protokolün kimlik bilgileri farklıdır ve ayrı ayrı saklanır).
- Grup **düğümüleri kapsar**. Düğümlere ait inbound'lar, ana panel inbound'larından `nodeId` alanıyla ayırt edilir (ana panel inbound'larında `null / 0` olur). Grup etiketi, istemci nesnelerinin hangi inbound'da – ana panelde mi yoksa düğüm inbound'unda mı – olduğundan bağımsız olarak yayılır; yeter ki o email'e sahip istemci orada bulunsun.

Grup etiketi, düğümlerden gelen senkronizasyona ve ayarların yeniden oluşturulmasına karşı dayanıklıdır. Bu davranış özel olarak tasarlanmıştır:

- Bir düğüm trafik anlık görüntüsü gönderdiğinde, gelen veriler panel VT'sindeki istemcinin yerel `group_name` ve `comment` alanlarını **üzerine yazmaz**. Grup ve yorum, "yalnızca panel" alanları olarak kabul edilir – düğüm bunları yönetmez.
- inbound ayarları yeniden oluşturulurken gelen verilerdeki boş `group` değeri, zaten kaydedilmiş etiketi **sıfırlamaz**. Grup, inbound Xray ayarlarını düzenlemek yerine özellikle **Gruplar** sayfasından yönetilir; bu nedenle normal ayar yeniden oluşturma sırasında "boş grup" "silme" değil, "dokunmama" olarak yorumlanır.

Pratik sonuç: etiketi **kasıtlı olarak temizleyen** tek işlemler, grubu silmek ve istemciyi gruptan açıkça kaldırmaktır (aşağıya bakın). Normal inbound düzenlemesi veya arka planda düğümle senkronizasyon grubu kaybettirmez.

9.3. Grup Rehberi ve "Boş" Gruplar

Sayfadaki grup listesi iki kaynağın birleştirilmesiyle oluşturulur:

- Türetilmiş gruplar (derived)** – istemcilerde gerçekten bulunan tüm boş olmayan `group_name` değerleri, istemci sayısı ile birlikte.
- Kaydedilmiş gruplar (stored)** – `client_groups` tablosundaki kayıtlar.

Bu birleştirme önemli bir etki yaratır: bir grup **hiç istemcisi olmadan** var olabilir. Böyle bir grup, "Grup Ekle" düğmesiyle açıkça oluşturulur (`client_groups` 'a kayıt) ve listede `0` sayacıyla görüntülenir. Bu kayıtlar **boş gruplar** olarak adlandırılır. Liste her zaman büyük/küçük harf duyarsız ada göre sıralanır.

Sayfadaki özet sayacılar:

Alan (RU)	Neyi gösterir
Toplam grup	Toplam grup sayısı (kaydedilmiş ve türetilmiş birlikte).
Gruplu istemciler	Boş olmayan grup etiketine sahip istemci sayısı.
Boş gruplar	İstemcisi olmayan grup sayısı (<code>0</code> sayacı).
Gruptaki istemciler	Belirli bir gruptaki istemci sayısı (tablo sütunu).

9.4. Grup Alanları ve Sütunları

`client_groups` tablosundaki grup kaydı şunları içerir:

Alan	Tür	Varsayılan	Açıklama
Id	int	otomatik artan	Grup kaydının birincil anahtarı.
Name	string	– (zorunlu)	Grup adı. Benzersiz dizin, boş olamaz. Arayüzde – Ad sütunu.
CreatedAt	int64 (ms)	oluşturulma zamanı	Grup kaydının oluşturulma anı.
UpdatedAt	int64 (ms)	değiştirilme zamanı	Son değiştirilme anı.

Sayfadaki tabloda en azından **Ad** ve **Gruptaki istemciler** sütunları ile eylem düğmeleri (aşağıya bakın) görüntülenir.

9.5. Grup Oluşturma

Grup Ekle düğmesi.

Adımlar:

1. **Grup Ekle**'ye tıklayın.
2. Grup adını girin.
3. Onaylayın.

Arka uç davranışı (`POST /panel/api/clients/groups/create` , gövde `{"name": "..."}`):

- Ad baştaki ve sondaki boşluklardan arındırılır. Boş ad, "group name is required" hatasıyla reddedilir.
- Bu adda bir grup zaten varsa – "group already exists" hatası.
- Başarı durumunda `client_groups` 'ta bir kayıt oluşturulur (başlangıçta istemcisiz – boş grup).

Başarı mesajı: «**{name}** grubu oluşturuldu.»

Örnek: API üzerinden boş grup oluşturma. İstemcilerle doldurmadan önce etiket kümesini hazırlayın:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"name": "vip"}'
```

Başarı durumunda yanıt:

```
{ "success": true, "msg": "«vip» grubu oluşturuldu.", "obj": null }
```

Aynı adla tekrarlanan çağrı `"success": false` ve `group already exists` mesajı döndürür.

Boş grup önceden oluşturmak, bir etiket kümesi hazırlayıp ardından "İstemci Ekle..." aracılığıyla istemcilerle doldurmak istediğinizde kullanışlıdır.

9.6. Grubu Yeniden Adlandırma

Yeniden Adlandır düğmesi, iletişim kutusu başlığı – «**{name}** grubunu yeniden adlandır».

Davranış (`POST /panel/api/clients/groups/rename` ,gövde `{"oldName": "...", "newName": "..."}:`

- Her iki ad da baştaki ve sondaki boşluklardan arındırılır. Boş eski ad – "old group name is required" hatası, boş yeni ad – "new group name is required" hatası.
- Yeni ad eskiyle aynıysa – hiçbir şey yapılmaz (`0` istemci etkilenir).
- Aksi takdirde yeniden adlandırma atomik olarak gerçekleştirilir:
 - `client_groups` 'taki kayıt yeniden adlandırılır;
 - `group_name = oldName` olan tüm istemcilerin alanı `newName` olarak güncellenir;
 - etkilenen istemcilerin üye olduğu **tüm inbound'larda** (düğüm inbound'ları dahil) Xray ayarlarındaki `group` değeri eskiden yeniye güncellenir.
- Yeniden adlandırmadan sonra panel Xray'i yeniden başlatma gerektiriyor olarak işaretler ve istemci değişiklik bildirimi gönderir.

Mesajlar:

- Başarı: «**Grup {count} istemci için yeniden adlandırıldı.**»
- Arayüzde ad çakışması: «**{name} adında bir grup zaten mevcut.**»

9.7. Gruba İstemci Ekleme

İstemci Ekle... düğmesi, başlık – «**"{name}" grubuna istemci ekle**».

İletişim kutusundaki ipucu metni:

«Bu gruba eklenecek istemcileri seçin. Mevcut inbound bağlantıları korunur; yalnızca grup etiketi değişir. Bu grupta zaten olan istemciler gösterilmez.»

Eklenecek kimse yoksa «**Eklenecek başka istemci yok.**» görüntülenir.

Davranış (`POST /panel/api/clients/groups/bulkAdd` ,gövde `{"emails": [...], "group": "..."}:`

- Grup adı zorunludur (aksi halde "group name is required" hatası); boş email listesi – işlem hiçbir şey yapmaz.
- Böyle bir grup ne `client_groups` 'ta ne de türetilmiş gruplar arasında yoksa – otomatik olarak oluşturulur.
- Seçilen email'ler için istemcilerin `group_name = group` alanı güncellenir; **istemcilerin inbound bağlantıları değişmez** – yalnızca etiket etkilenir. Ardından bu istemcilerin tüm inbound'larında `group` alanı güncellenir.
- Etkilenen istemci kayıtlarının sayısı döndürülür; Xray yeniden başlatma gerektiriyor olarak işaretlenir.

Başarı mesajı: «**{name} grubuna {count} istemci eklendi.**»

Örnek: tek istekle birden fazla istemciyi grupla etiketleme. İstemciler email ile belirtilir, inbound bağlantıları değişmez:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

`vip` grubu henüz yoksa otomatik olarak oluşturulur. İstekten sonra bu istemcilerin kaydında `group_name = "vip"` ayarlanır ve her inbound'larının Xray ayarlarındaki istemci nesnesi `"group": "vip"` alanını alır:

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. Gruptan İstemci Kaldırma (İstemcileri Silmeden)

İstemci Kaldır... düğmesi, başlık – «**{name}** grubundan istemci kaldır».

İpucu metni:

«Bu gruptan kaldırılacak üyeleri seçin. İstemcilerin kendisi korunur (tam silme için "Gruptaki istemcileri sil" seçeneğini kullanın).»

Davranış (`POST /panel/api/clients/groups/bulkRemove` ,gövde `{"emails": [...]}`): teknik olarak bu, boş grupta "Gruba Ekle" ile aynıdır. Seçilen istemcilerin `group_name` alanı temizlenir ve Xray ayarlarından `group` alanı kaldırılır. İstemcilerin kendisi ve inbound bağlantıları korunur.

Başarı mesajı: «**{name}** grubundan **{count}** istemci kaldırıldı.»

9.9. Grup Trafiğini Sıfırlama

Trafiği Sıfırla düğmesi.

Onay iletişim kutusu:

- Başlık: «**{name}** grubunun trafiği sıfırlansın mı?»
- Metin: «**Bu, gruptaki tüm {count} istemci için up/down değerini sıfırlar.**»

Davranış: gruptaki tüm üyelerin email'leri için trafik tablosundaki `up` ve `down` sıfırlanır ve `enable` alanı `true` olarak ayarlanır (istemci etkinleştirilir). İşlem, bir işlem (transaction) içinde toplu olarak gerçekleştirilir.

Başarı mesajı: «**{count}** istemcinin trafiği sıfırlandı.»

9.10. Grubu Silme ve Gruptaki İstemcileri Silme

Sayfada **temelden farklı iki silme işlemi** vardır – bunları birbirine karıştırmak kolaydır, bu nedenle aradaki fark kritiktir.

9.10.1. Grubu Sil (istemcileri koru)

«**Grubu Sil (istemcileri koru)**» düğmesi.

İletişim kutusu:

- Başlık: «**{name}** grubu silinsin mi?»
- Metin: «**Bu, grubu siler ve {count} istemcideki etiketini temizler. İstemcilerin kendisi silinmez.**»

Davranış (POST /panel/api/clients/groups/delete , gövde {"name": "..."}): grup kaydı client_groups 'tan silinir, tüm istemcilerinin group_name alanı temizlenir ve inbound'larından group alanı kaldırılır. **İstemciler, bağlantıları ve trafiği korunur.** Xray yeniden başlatma gerektiriyor olarak işaretlenir.

Başarı mesajı: «**{count} istemcideki grup etiketi temizlendi.**»

9.10.2. Gruptaki İstemcileri Sil (tam silme)

«**Gruptaki istemcileri sil**» düğmesi.

İletişim kutusu:

- Başlık: «**{name}** grubundaki tüm istemciler silinsin mi?»
- Metin: «**Bu, {count} istemciyi trafik kayıtlarıyla birlikte kalıcı olarak siler. Grup etiketi de temizlenir. Bu işlem geri alınamaz.**»

Bu yıkıcı bir işlemdir: istemcilerin kendisini (email bazında toplu silme, POST /panel/api/clients/bulkDel uç noktası aracılığıyla), trafik kayıtları dahil siler ve böylece onları tüm inbound'lardan kaldırır.

Mesajlar:

- Başarı: «**{count} istemci silindi.**»
- Kısmi sonuç: «**{ok} silindi, {failed} atlandı**»

Grup boşsa, üyeleri üzerindeki eylemler kullanılamaz – «**Bu grupta henüz istemci yok.**» görüntülenir.

9.11. "İstemciler" Sayfasıyla İlişki

Grup etiketi, **Gruplar** sayfasının dışında da görüntülenir ve kullanılır:

- Kompakt istemci kaydında group alanı bulunur, bu nedenle istemci listesinde grup üyeliği görüntülenir.
- İstemci listesi (/panel/api/clients/list/paged), group filtre parametresini kabul eder: virgülle ayrılmış bir veya birden fazla ad geçirilebilir. Eşleştirme alan içinde büyük/küçük harf duyarlı "VEYA" mantığıyla çalışır. Özel durum: filtre grupları listesindeki boş öge, "grupsuz istemciler"i (group alanı boş olanları) temsil eder.
- İstemci sayfası yanıtında, kullanıcı arayüzünün filtre açılır listesini oluşturabilmesi için mevcut tüm grup adlarını içeren groups dizisi döndürülür.

Örnek: istemcileri gruplara göre filtreleme. İstek yalnızca vip veya trial etiketli istemcileri döndürür (birden fazla ad virgülle ayrılır, "VEYA" mantığı):

```
GET /panel/api/clients/list/paged?group=vip,trial
```

Grupsuz istemcileri almak için listeye boş öge geçirin – örneğin `group=` (boş dize) veya `group=vip,` (vip etiketi artı grupsuz istemciler) filtre değeri.

9.12. API Uç Noktaları Özeti

Tüm grup rotaları `/panel/api/clients` altına bağlanmıştır:

Yöntem ve yol	Amaç	İstek gövdesi
GET <code>/panel/api/clients/groups</code>	İstemci sayacıyla grupların listesi	–
GET <code>/panel/api/clients/groups/:name/emails</code>	Gruptaki tüm üyelerin email'leri (email'e göre sıralı)	–
POST <code>/panel/api/clients/groups/create</code>	Boş grup oluştur	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/rename</code>	Grubu yeniden adlandır	<code>{"oldName","newName"}</code>
POST <code>/panel/api/clients/groups/delete</code>	Grubu sil, istemcileri koru (etiket temizleme)	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/bulkAdd</code>	Gruba istemci ekle (email bazında)	<code>{"emails":[...],"group"}</code>
POST <code>/panel/api/clients/groups/bulkRemove</code>	Gruptan istemci kaldır (etiket temizleme)	<code>{"emails":[...]}</code>
POST <code>/panel/api/clients/groups/bulkDel</code>	İstemcileri tam sil ("Gruptaki istemcileri sil" tarafından kullanılır)	<code>{"emails":[...],"keepTraffic"}</code>

Örnek: API üzerinden tipik grup yaşam döngüsü senaryosu.

```
# 1. trial etiketini oluştur
curl -s .../panel/api/clients/groups/create -d '{"name":"trial"}'

# 2. İki istemciye etiket ata
curl -s .../panel/api/clients/groups/bulkAdd -d '{"emails":["u1@example.com","u2@example.com"],"group":"trial"}'

# 3. Tüm üyelerin trafiğini sıfırla (/groups/trial/emails adresinden email al)
curl -s .../panel/api/clients/groups/bulkRemove -d '{"emails":["u2@example.com"]}'

# 4. Grubu sil, istemcileri koru (yalnızca etiket temizleme)
curl -s .../panel/api/clients/groups/delete -d '{"name":"trial"}'
```

1. adım, grup kaydını siler ve istemcilerinin `group_name` alanını temizler; ancak istemcilerin kendisi, bağlantıları ve trafiği korunur. İstemcilerin kendisini kalıcı olarak silmek için bunun yerine `bulkDel` kullanılır.

İstemcilerdeki etiketi değiştiren işlemler (`rename` , `delete` , `bulkAdd` , `bulkRemove`), Xray'i yeniden başlatma gerektiriyor olarak işaretler ve istemci değişiklik bildirimi gönderir.

9.13. Gruba Göre Trafik

3.3.0 sürümünün yeniliği: **Gruplar** bölümündeki (istemci yönetim sekmesindeki "İstemciler" sayfası) grup tablosu artık yalnızca her gruptaki istemci sayısını değil, grubun toplam tüketilen trafiğini de göstermektedir. Sütun «**Kullanılan Trafik**» olarak etiketlenmiştir.

Sütunun Gösterdikleri

Her grup satırı için, gruptaki tüm istemcilerin trafiklerinin toplamı – yani tüm üyelerin up + down (gönderilen + alınan trafik) değerlerinin toplamı – görüntülenir. Bu, "tüm grup toplamda ne kadar indirdi/gönderdi" sorusuna istemcileri tek tek açıp manuel olarak toplamak zorunda kalmadan hızlıca yanıt verir.

Grup tablosunda yanı sıra şunlar da görüntülenir:

Sütun	Anlamı
Ad	Grup adı
İstemciler	Bu gruba etiketlenmiş istemci sayısı (sütun daha önce "Gruptaki istemciler" olarak adlandırılıyordu)
Gönderilen	Gruptaki tüm istemcilerin toplam up değeri (gönderilen trafik)
Alınan	Gruptaki tüm istemcilerin toplam down değeri (alınan trafik)
Kullanılan Trafik	Gruptaki tüm istemcilerin toplam up + down değeri

Gönderilen ve alınan trafik ayrı **Gönderilen** ve **Alınan** sütunlarında görüntülenirken **Kullanılan Trafik** sütunu bunların toplamını gösterir. İstemci sayısı sütunu yalnızca **İstemciler** olarak adlandırılır.

Tablonun üzerindeki özet, tüm gruplardaki toplamaları da gösterir – «**Toplam grup**» ve «**Gruplu istemciler**»; toplam trafik iki karta bölünmüştür: «**Toplam gönderilen / alınan**» (yukarı/aşağı oklar – tüm grupların gönderilen ve alınan trafiği ayrı ayrı) ve «**Toplam trafik**» (grafik simgesi – bunların genel toplamı).

Hesaplama Yöntemi

Hesaplama, istemciler tablosuna trafik muhasebe tablosunun birleştirildiği (LEFT JOIN) tek bir SQL sorgusuyla gerçekleştirilir:

- grup etiketi alanına (group_name) göre istemciler gruplanır, sayıları hesaplanır – bu "Gruptaki istemciler" değeridir;
- trafik, birleştirilmiş client_traffics tablosundan up + down toplamı olarak alınır; yani her istemcinin hem gönderilen (up) hem de alınan (down) baytları toplanır;
- email hem istemciler tablosunda hem de trafik tablosunda benzersiz olduğundan, birleştirme tek bir istemcinin trafiğini iki kez saymaz.

Değerlere ilişkin özellikler:

- **Trafik kaydı olmayan istemciler** üye sayacına dahil edilir ancak toplama 0 ekler, dolayısıyla yeni oluşturulan bir grup 0 trafik gösterir.

- **Boş gruplar** (oluşturulmuş ancak istemcisiz) de sıfır sayaç ve sıfır trafikle listede yer alır: istemci etiketlerinden "türetilen" grupların yanı sıra açıkça kaydedilmiş gruplar da sonuca eklenir ve ardından liste büyük/küçük harf duyarsız ada göre sıralanır.
- Grup etiketi olmayan istemciler (`group_name` boş) hesaba dahil edilmez.

İlgili İşlemler

Grup tablosundan tüm grup üzerindeki işlemler hâlâ kullanılabilir; bunlar arasında «**Trafiği Sıfırla**» da yer alır – seçilen gruptaki tüm istemcilerin `up / down` değerini sıfırlar. Bu sıfırlama işleminden sonra o grup için "Kullanılan Trafik" sütunu `0` gösterir.

10. Abonelikler (Subscription)

Abonelik (subscription), VPN istemcisinin tam yapılandırma setini kendiliğinden indirip periyodik olarak güncellediği tek bir kalıcı bağlantı (URL) sağlayan bir mekanizmadır. Kullanıcıya her inbound için ayrı ayrı bağlantı göndermek yerine, `https://alan:port/sub/<subId>` biçiminde tek bir adres iletilir. Panel bu adres üzerinden söz konusu istemciye bağlı tüm yapılandırmaları anında bir araya getirir ve istemcinin beklediği formatta sunar. Sunucudaki ayarlar değiştiğinde (yeni adres, Reality anahtarlarının döndürülmesi, inbound eklenmesi), istemci bir sonraki otomatik güncellemede herhangi bir kullanıcı müdahalesi gerektiksizin güncel yapılandırmayı alır.

Aboneliği, panel içinde web panelinden bağımsız olarak kendi portunda dinleyen ayrı bir HTTP/HTTPS sunucusu yönetir. Bu tasarım güvenlik amacıyla tercih edilmiştir: abonelik portunu dışarıya açmak için panelin kendi portunu açmak gerekmez.

10.1. subId nedir ve bağlantı nasıl oluşturulur

Bir inbound içindeki her istemcinin `subId` alanı (arayüzde "Abonelik Kimliği" olarak gösterilir) vardır. Bu değer abonelik anahtarı işlevi görür: panel, tüm inbound'larda `subId` değeri istekle eşleşen istemcileri bulur ve yapılandırmalarını tek bir yanıtta birleştirir.

- Birden fazla istemciye (aynı veya farklı inbound'larda) aynı `subId` atanmışsa, bu istemcilerin yapılandırmaları tek bir abonelikte bir araya gelir. Bu, bir kullanıcıya tek bağlantı üzerinden birden fazla sunucu/protokol sunmanın standart yoludur.

Örnek: tek kullanıcı – tek bağlantı üzerinden iki sunucu. İki inbound olduğunu varsayalım (A sunucusunda VLESS ve B sunucusunda Trojan). Kullanıcıya her iki yapılandırmayı tek bağlantıyla sunmak için her iki istemciye aynı `subId` değerini atayın:

```
Inbound 1 (VLESS): email = ivan@vpn, subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn, subId = ivan2025
```

Bu sayede `https://sub.example.com:2096/sub/ivan2025` adresinden panel her iki yapılandırmayı birlikte döndürür. Daha sonra aynı `subId` ile üçüncü bir inbound eklerseniz, yeni bağlantı göndermeye gerek kalmadan bir sonraki otomatik abonelik güncellemesinde kullanıcıya iletilir.

- Bir istemcinin `subId` alanı boşsa, genel erişim bağlantısı paylaşılamaz. Arayüz bunu şu ipucuyla belirtir: "Bu istemcinin subId değeri yok, genel erişim bağlantısı kullanılamaz."

İstemci dış bağlantıları ve abonelikleri (Links sekmesi)

İstemci formunda yalnızca o istemcinin aboneliğine eklenmek üzere ek yapılandırma kaynakları tanımlamanıza olanak tanıyan "Links" sekmesi bulunur (RAW, JSON ve Clash formatları için):

- Add External Link** – üçüncü taraf paylaşım bağlantısı (`vless://`, `trojan://`, `ss://` vb.). Çıktıya olduğu gibi eklenir; JSON/Clash için ayrıca yapılandırmaya ayrıştırılır.
- Add External Subscription** – harici abonelik adresi. Panel bu adresi kendisi indirir (önbellekleme ve kısa zaman aşımı ile) ve alınan yapılandırmaları istemcinin genel listesine ekler.

Bu özellik, istemciye aynı tek bağlantı üzerinden kendi inbound'larınızın yanı sıra ek sunucular sunmak için kullanışlıdır. Uzak abonelik yanıtı çok büyükse artık sessizce kesilmez: panel hata döndürür ve en son başarıyla önbellege alınan değeri kullanmaya devam eder.

- `subId` değeri rastgele belirlenemez: kaydetme sırasında boşluk, `/`, `\` ve denetim karakteri içermediği doğrulanır. İlgili doğrulama ipucu: "Abonelik Kimliği boşluk, `'`, `"` veya denetim karakteri içeremez."

Nihai bağlantı `<taban>/<subPath>/<subId>` biçiminde oluşturulur (abonelik sunucusu ayarları ve "Ters Proxy URI" alanı için ilgili bölüme bakınız). `subId` ile hiçbir istemci bulunamazsa (istemci silindi veya `subId` mevcut değil) sunucu gövdesiz HTTP 404 döndürür. Dahili hata durumunda HTTP 500 döndürülür. VPN istemcileri yalnızca yanıt kodunu dikkate aldığından hata gövdesi kasıtlı olarak boş bırakılmıştır.

Abonelikteki inbound bağlantılarının sırası

Her inbound'un "**Abonelikteki Sıra**" (`subSortIndex`) adlı bir alanı vardır; bu pozitif sayı, söz konusu inbound'un bağlantılarının abonelik çıktısındaki konumunu belirler. Küçük değerler önce gelir; eşit değerlerde oluşturma sırası (id'ye göre) korunur. Sıralama ham metin, abonelik sayfası, JSON ve Clash dahil tüm çıktı formatlarına uygulanır. Bu alan, panelin kendisindeki inbound sırasını etkilemez.

Alan, inbound formunda paylaşım adresi (share address) ayarlarının yanında düzenlenir ve normal kurallar çerçevesinde düğümlere eşitlenir. En az bir inbound'un sırası 1'den farklıysa Inbounds listesinde kompakt bir "**Sıra**" sütunu görünür.

10.2. Abonelik sunucusu ayarları

Tüm abonelik parametreleri, panel ayarlarının "**Abonelik**" sekmesinde yer alır. Aşağıda her parametre ayrıntılı olarak açıklanmıştır; parantez içinde iç ayar anahtarı ve varsayılan değer belirtilmiştir.

Bölüm şu sekmelere ayrılmıştır: "**Panel Ayarları**", "**Bilgi**", "**Profil**", "**Sertifika**", "**Happ**" ve "**Clash / Mihomo**". Abonelik başlığı, destek URL'si, profil sayfası, duyuru ve tema dizini alanları "Profil" sekmesinde; Happ ve Clash/Mihomo yönlendirme kuralları kendi adlarını taşıyan sekmelerde; abonelik güncelleme aralığı ise "Bilgi" sekmesinde bulunur.

Temel parametreler

Alan (UI)	Anahtar	Varsayılan değer	Açıklama
Aboneliği Etkinleştir	<code>subEnable</code>	<code>true</code> (etkin)	Ayrı bir abonelik sunucusu başlatır. İpucu: "Ayrı yapılandırmayla abonelik özelliği". Kapalıysa abonelik sunucusu başlamaz ve hiçbir bağlantı çalışmaz.
Dinleme IP'si	<code>subListen</code>	boş	Abonelik sunucusunun bağlantıları kabul ettiği IP adresi. İpucu: "Tüm IP adreslerini izlemek için varsayılan olarak boş bırakın".
Abonelik Portu	<code>subPort</code>	<code>2096</code>	Abonelik sunucusunun TCP portu. İpucu: "Abonelik hizmetini sunmak için kullanılan port numarası sunucuda kullanılmıyor olmalıdır" – port serbest olmalı, panel veya Xray ile çakışmamalıdır.
URI Yolu	<code>subPath</code>	<code>/sub/</code>	Normal aboneliklerin sunulduğu yol. İpucu: <code>"</code> / <code>"</code> ile başlayıp <code>'</code> / <code>'</code> ile bitmelidir.

Alan (UI)	Anahtar	Varsayılan değer	Açıklama
Dinleme Alanı	subDomain	boş	Aboneliğe erişime izin verilen alan adı (Host doğrulaması). İpucu: "Tüm alan adlarını ve IP adreslerini dinlemek için varsayılan olarak boş bırakın". Belirtilmişse farklı bir Host içeren istekler reddedilir.

Güvenlik notu: /sub/ (ve JSON için /json/) varsayılan yolları yaygın olarak bilinir ve kolayca tahmin edilebilir. Panel şu uyarıyı gösterir: "Varsayılan abonelik yolu '/sub/' geniş çapta bilinmektedir – değiştirin." ve JSON için de benzer bir uyarı. Kendi belirsiz yolunuzu belirlemeniz önerilir.

TLS / Sertifika

Alan (UI)	Anahtar	Varsayılan	Açıklama
Abonelik Sertifikası Genel Anahtar Dosyası Yolu	subCertFile	boş	Sertifika dosyasının tam yolu (.crt / fullchain). İpucu: "/" ile başlayan tam yolu girin".
Abonelik Sertifikası Özel Anahtar Dosyası Yolu	subKeyFile	boş	Özel anahtar dosyasının tam yolu. İpucu: "/" ile başlayan tam yolu girin".

Her iki yol da belirtilmiş ve sertifika başarıyla yüklenmişse abonelik sunucusu **HTTPS** üzerinden çalışır. Alanlar boşsa ya da sertifika okunamazsa sunucu **HTTP**'ye geri döner (hata loga yazılır). Geçerli TLS varlığı temel URL'nin oluşturulmasını da etkiler: 443 portunda TLS kullanıldığında ve 80 portunda TLS kullanılmadığında bağlantıda port numarası atlanır.

Güncelleme aralığı

Alan (UI)	Anahtar	Varsayılan	Açıklama
Abonelik Güncelleme Aralıkları	subUpdates	12	İstemci uygulamasının aboneliği ne sıklıkla (saat cinsinden) yeniden sorgulaması gerektiği. İpucu: "İstemci uygulamasındaki güncellemeler arasındaki aralık (saat cinsinden)".

Değer, Profile-Update-Interval HTTP başlığı aracılığıyla istemciye iletilir; modern istemciler bunu yapılandırmanın otomatik güncelleme periyodu olarak kullanır.

Yanıt formatı ve bilgisi

Alan (UI)	Anahtar	Varsayılan	Açıklama
Şifrele	subEncrypt	true	İpucu: "Abonelikteki döndürülen yapılandırmaları şifrele". Teknik olarak bu şifreleme değil, normal abonelik gövdesinin tamamının Base64 kodlamasıdır (çoğu istemcinin beklediği format). Kapalıyken bağlantılar düz metin olarak, her biri ayrı satırda döndürülür.
Kullanım Bilgilerini Göster	subShowInfo	true	İpucu: "Yapılandırma adının ardından kalan trafiği ve bitiş tarihini görüntüle". Etkinleştirildiğinde her yapılandırmanın adına (remark) kalan trafik () ve geçerlilik süresi (örn. 5D, 3H) etiketleri eklenir; süresi dolmuş/kullanılamaz istemcilerde N/A gösterilir.

Alan (UI)	Anahtar	Varsayılan	Açıklama
Ada E-posta Ekle	subEmailInRemark	true	İpucu: "İstemcinin e-postasını abonelik profil adına ekle." İstemcinin e-posta adresini profil remark'ına ekler.

Remark Şablonu (Remark Template)

Abonelikteki her yapılandırmanın görüntülenen adı (remark), "**Remark Şablonu**" – "**Abonelik Bilgisi**" sekmesindeki "**Not Şablonu**" (remarkTemplate) alanı – kullanılarak oluşturulur. Önceki not modeli oluşturucu (inbound/email/external proxy bölümlerinin ve ayırıcı karakterin ayrı ayrı seçilmesi) arayüzden kaldırılmıştır; artık istediğiniz ad formatını yazıp değişkenler ekleyebilirsiniz. Varsayılan değer `{{INBOUND}}` | `{{TRAFFIC_LEFT}}` | `{{DAYS_LEFT}}`D 'dir. Alan boş bırakılırsa önceki (arayüzden yapılandırılmayan) remark modeli kullanılır.

Değişkenler **Client**, **Traffic** ve **Time & status** bölümleri altında gruplanır ve alanın yanında üzerine gelindiğinde ipucu gösteren tıklanabilir `{{VAR}}` chip'leri olarak gösterilir; tıklama token'ı şablona ekler ve canlı önizleme mevcuttur. Her değişken, abonelik oluşturulurken ilgili istemci için ayrı ayrı değiştirilir. Tek parantezli yazım da kabul edilir (`{DATA_LEFT}` , `{EXPIRE_DATE}` , `{PROTOCOL}` , `{TRANSPORT}` vb.) – panel bunu otomatik olarak dahili `{{...}}` formatına dönüştürür.

Kullanılabilir değişkenler:

- **İstemci tanımlaması:** `{{EMAIL}}` , `{{INBOUND}}` (inbound remark'ının kendisi), `{{HOST}}` (host remark'ı), `{{ID}}` (UUID), `{{SHORT_ID}}` (UUID'nin ilk 8 karakteri), `{{SUB_ID}}` , `{{COMMENT}}` , `{{TELEGRAM_ID}}` , `{{PROTOCOL}}` , `{{TRANSPORT}}` .
- **Trafik:** `{{TRAFFIC_USED}}` , `{{TRAFFIC_LEFT}}` , `{{TRAFFIC_TOTAL}}` (ve tam bayt cinsinden *_BYTES varyantları), `{{UP}}` , `{{DOWN}}` , `{{USAGE_PERCENTAGE}}` .
- **Süre ve durum:** `{{DAYS_LEFT}}` , `{{TIME_LEFT}}` , `{{EXPIRE_DATE}}` (YYYY-MM-DD), `{{JALALI_EXPIRE_DATE}}` (Celali takvimine göre tarih), `{{EXPIRE_UNIX}}` , `{{CREATED_UNIX}}` , `{{RESET_DAYS}}` , `{{STATUS}}` (active / expired / disabled / depleted), `{{STATUS_EMOJI}}` .

Şablon dikey çizgi | ile bölümlere ayrılabilir. Bir değişkenin "sınırsız" değer (∞) ürettiği bölüm – örneğin limitsiz bir istemci için `{{TRAFFIC_LEFT}}` veya `{{DAYS_LEFT}}` – otomatik olarak gizlenir. Ayrıca trafik ve süre bilgisi yinelenen her yapılandırmada görünmemesi için istemcinin yalnızca ilk bağlantısında bir kez gösterilir.

Örnek. `{{EMAIL}}` | `{{TRAFFIC_LEFT}}` | `{{DAYS_LEFT}}`D şablonu, 42 GB kalan trafiki ve 7 günü olan bir istemci için `ivan@vpn 42.00GB 7D` adını, limitsiz istemci için ise yalnızca `ivan@vpn` adını üretir (∞ içeren bölümler atlanır). | Remark Şablonu | remarkTemplate | `{{INBOUND}}` | `{{TRAFFIC_LEFT}}` | `{{DAYS_LEFT}}`D | `{{VAR}}` değişken ikameli, her yapılandırmanın görüntülenen adı (remark) için serbest şablon. Abonelik oluşturulurken her istemci için ayrı ayrı uygulanır. Önceki "not modeli" oluşturucu (inbound/email/external proxy ve ayırıcı seçimi) arayüzden kaldırılmış olup alan boş bırakıldığında yalnızca yedek olarak kullanılır. Ayrıntılar için aşağıdaki "Remark Şablonu (Remark Template)" bölümüne bakın. |

Profil üst verileri (yanıt başlıkları)

Bu dizeler, HTTP yanıt başlıkları aracılığıyla istemciye iletilir ve VPN istemcisinde profil meta verisi olarak görüntülenir. Tümü varsayılan olarak boştur.

Alan (UI)	Anahtar	Başlık	Açıklama
Abonelik Başlığı	subTitle	Profile-Title (Base64 ile)	"VPN istemcisinde görünen abonelik adı". Clash için aynı zamanda Content-Disposition üzerinden içe aktarılan profil adı olarak kullanılır.
Destek URL'si	subSupportUrl	Support-Url	"VPN istemcisinde görüntülenen teknik destek bağlantısı".
Profil URL'si	subProfileUrl	Profile-Web-Page-Url	"VPN istemcisinde görüntülenen sitenizin bağlantısı". Belirtilmezse abonelik isteğinin gerçek URL'si kullanılır.
Duyuru	subAnnounce	Announce (Base64 ile)	"VPN istemcisinde görüntülenen duyuru metni".

Ayrıca her yanıtta istemcinin trafik verilerini içeren Subscription-Userinfo başlığı iletilir: upload , download , total ve expire (saniye cinsinden bitiş zamanı). İstemci bu başlığı kullanarak kalan trafiği ve geçerlilik süresini gösterir.

Yönlendirme (yalnızca Happ istemcisi için)

Alan (UI)	Anahtar	Varsayılan	Açıklama
Yönlendirmeyi Etkinleştir	subEnableRouting	false	"VPN istemcisinde yönlendirmeyi etkinleştirmek için genel ayar. (Yalnızca Happ için)". Routing-Enable başlığıyla iletilir.
Yönlendirme Kuralları	subRoutingRules	boş	"VPN istemcisi için genel yönlendirme kuralları. (Yalnızca Happ için)". Routing başlığıyla iletilir.

| Sunucu Ayarlarını Gizle | subHideSettings | false | "Abonelikteki sunucu ayarlarını gizle (yalnızca Happ için)". Etkinleştirildiğinde Happ istemcisinde sunucu parametrelerini görüntüleme ve değiştirme seçeneği gizlenir. Bu seçenek yalnızca Happ istemcisinde geçerlidir. |

Ters Proxy URI'si

Alan (UI)	Anahtar	Varsayılan	Açıklama
Ters Proxy URI'si	subURI	boş	"Proxy sunucularının arkasında kullanmak üzere abonelik URL'sinin temel URI'sini değiştir".

Alan boşsa panel, temel bağlantı adresini abonelik alanından ve portundan (TLS durumu dikkate alınarak) kendisi oluşturur. Abonelik farklı bir alan adında veya yolda harici bir ters proxy/CDN üzerinden sunuluyorsa bu alana nihai temel URI girilir ve tüm bağlantılar bu URI'den türetilir. JSON (subJsonURI) ve Clash (subClashURI) için ayrı benzer alanlar mevcuttur.

Yalnızca genel subURI belirtilmiş ve JSON ile Clash için ayrı alanlar boş bırakılmışsa bu formatların bağlantıları, abonelik sayfasında sub sunucusunun portu ve http yerine subURI 'deki şema ve host'u devralır; böylece ters proxy adresiyle örtüşürler.

Örnek: ters proxy arkasındaki abonelik. Abonelik 2096 portunda dinliyor ancak dışarıdan `https://cfg.example.com/u/` üzerinden nginx/CDN aracılığıyla erişilebilir durumda. Yanıttaki bağlantıların dahili alan:2096 yerine harici adresten türetilmesi için "Reverse proxy URI" alanına nihai temel URI girilir:

Reverse proxy URI: `https://cfg.example.com/u`

Bu durumda nihai bağlantı `https://cfg.example.com/u/ivan2025` biçimini alır. JSON ve Clash formatları için gerekirse `subJsonURI` ve `subClashURI` alanları aynı yöntemle doldurulur.

10.3. Çıktı formatları

Abonelik, her biri ayrı ayrı etkinleştirilebilen/devre dışı bırakılabilen ve kendi uç noktasına sahip üç bağımsız formatta sunulabilir.

Abonelikteki sunucu adresi ve düğümler

Abonelik bağlantılarındaki sunucu adresi, paneldeki normal bağlantılar ve QR kodlarıyla aynı bağlantı adresi stratejisine göre belirlenir: "listen" – yönlendirilebilir bağlama adresi, "custom" – kullanıcı tarafından belirlenen özel adres (`shareAddr`), "node" (varsayılan) – düğüm adresi. Açık bir strateji belirtilmemiş inbound'lar için abonelik çıktısı değişmez. Bu sayede belirli bir genel IP'ye bağlı bir düğüm inbound'u, istemcilere erişilebilir bir adres sunabilir. Strateji ham metin, JSON ve Clash formatlarına uygulanır.

Düğüm (Node) adı, abonelikteki profil adına (remark) eklenmez: istemci uygulamasında yalnızca yönetici tarafından belirlenen inbound remark'ı gösterilir, `@düğüm-adı` gibi bir iç sonek eklenmez. Çok düğümlü abonelikte aynı adlı kayıtları birbirinden ayırt etmek için remark'ları elle farklı belirleyin ya da kendi Remark'larına sahip yönetilen Host'ları kullanın.

Düğümler arasındaki senkronizasyon kaybı nedeniyle aynı istemci hizmet JSON inbound'unda iki kez yer almışsa abonelik çıktısı, e-postaya göre bu yinelenmeleri üç formatta da otomatik olarak giderir; bu nedenle çıktıda yinelenen profiller görünmez.

Yönetilen Host'lar (Hosts)

Hosts bölümü (yan menü ögesi; Toplam/Etkin/Devre Dışı sayısı ve listeyi gösteren özet sayfa), abonelik bağlantıları için adres geçersiz kılmalarını tanımlar. Her inbound için, istemciye sunulan abonelik bağlantılarında **inbound'un kendi adresi, portu ve TLS parametreleri yerine** ikame edilecek bir veya daha fazla **host** – uç nokta – eklenebilir. Bu özellik, inbound'u değiştirmeksizin trafiği CDN veya röle üzerinden dağıtmak için kullanışlıdır.

Her host için şunlar ayarlanır:

- **Remark** ve açıklama (Description), belirli bir **Inbound'a** bağlama, **Enable** geçiş düğmesi ve düğümlere atama (**Nodes**).
- **Address** (boş – inbound adresi devralınır) ve **Port** (`0` – inbound portu devralınır); **Tags** (yalnızca RAW aboneliğinde dikkate alınır).
- **Security** sekmesi – SN1, parmak izi (fingerprint), ALPN, sertifika sabitleme (pinned-cert), `allowInsecure` ve ECH içeren `same / tls / none / reality`.

- **Advanced** sekmesi – Host başlığı, Path, VLESS rotası, Mux, Sockopt, Final Mask ve host'un ayrı abonelik formatlarından dışlanması (raw / json / clash).
- **Clash (mihomo)** sekmesi – IP sürümü, Mihomo X25519, host karıştırma (Shuffle host).

Host'lar kendi inbound'ları içinde sıralanır ve toplu etkinleştirme, devre dışı bırakma ile silme işlemlerini destekler. Yönetilen Host'lar, önceki External Proxy dizisinin yerini almıştır.

Normal bağlantılar (SUB) – Base64 / düz metin

Temel format, uç nokta `subPath` (varsayılan `/sub/`). Abonelik genel olarak etkinken her zaman açıktır. Xray bağlantılarının listesini (`vless://` , `vmess://` , `trojan://` , `ss://` vb.) – her biri ayrı satırda – döndürür. "Şifrele" (`subEncrypt`) seçeneği etkinken listenin tamamı Base64 ile kodlanır; kapalıyken düz metin olarak sunulur. Bu format neredeyse tüm istemciler tarafından desteklenir (v2rayNG, V2RayTun, Sing-box, NekoBox, Streisand, Shadowrocket, Happ vb.).

Örnek: "Şifrele" kapalıyken yanıt gövdesi. `subEncrypt = false` iken `/sub/` uç noktası düz metin döndürür – her satırda bir bağlantı:

```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tl&...#srvB-ivan
```

`subEncrypt = true` (varsayılan) iken aynı liste tamamen Base64 ile kodlanarak tek satır olarak döndürülür – bu format çoğu istemci tarafından beklenir.

JSON aboneliği (sing-box ve uyumlular)

Uç nokta `subJsonPath` (varsayılan `/json/`), ayrı bir onay kutusuyla etkinleştirilir.

Alan (UI)	Anahtar	Varsayılan	Açıklama
JSON Aboneliği	<code>subJsonEnable</code>	<code>false</code>	"JSON abonelik uç noktasını bağımsız olarak etkinleştir/devre dışı bırak."

sing-box ve türev istemciler (Podkop, OpenWRT sing-box, Karing, NekoBox) tarafından anlaşılır tam JSON yapılandırması döndürür. Bu format için ek parametreler mevcuttur (`subFormats` sekmesi):

- **Mux** (`subJsonMux` , varsayılan boş) – JSON aboneliğinin her akışına eklenen çoğullama (Mux) JSON ayarları. "Tek bağlantıda birden fazla bağımsız veri akışı iletimi".
- **Final Mask** (`subJsonFinalMask` , varsayılan boş) – "JSON aboneliğinin her akışına eklenen xray finalmask (TCP/UDP) maskeleri ve QUIC ayarları. İstemcide güncel bir xray sürümü gerektirir.". Alt alanlarla yapılandırılır: "Paketler" (`packets`), "Uzunluk" (`length`), "Aralık" (`interval`), "Maks. bölme" (`maxSplit`), "Gürültüler" (`noises` : "Tür" / `type` , "Paket" / `packet` , "Gecikme (ms)" / `delayMs` , "Uygula" / `applyTo` , "+ Gürültü" düğmesi) ve ayrıca "Eşzamanlılık" (`concurrency`), "xudp eşzamanlılığı" (`xudpConcurrency`) ve "xudp UDP 443" (`xudpUdp443`).
- **Yönlendirme Kuralları** (`subJsonRules` , varsayılan boş) – JSON yapılandırmasına eklenen genel kurallar.

Clash / Mihomo aboneliği (YAML)

Uç nokta `subClashPath` (varsayılan `/clash/`), ayrı bir onay kutusuyla etkinleştirilir.

Alan (UI)	Anahtar	Varsayılan	Açıklama
Clash / Mihomo Aboneliği	<code>subClashEnable</code>	<code>false</code>	Clash ve Mihomo istemcileri için YAML yapılandırması oluşturmayı etkinleştirir.
Yönlendirmeyi Etkinleştir	<code>subClashEnableRouting</code>	<code>false</code>	"Oluşturulan YAML aboneliklerine genel Clash/ Mihomo yönlendirme kuralları ekle."
Genel Yönlendirme Kuralları	<code>subClashRules</code>	boş	"Her YAML aboneliğinin başına MATCH,PROXY'den önce eklenen Clash/Mihomo kuralları."

Yanıt `application/yaml; charset=utf-8` türüyle döndürülür. "Abonelik Başlığı" (`subTitle`) belirtilmişse Clash istemcisinin içe aktarılan profili bu adla adlandırması için `Content-Disposition` başlığında da (`attachment; filename*=UTF-8''<başlık>`) iletilir.

Oluşturulan bağlantılar ve YAML formatı modern istemciler için güncel tutulur: Shadowsocks-2022 (SS2022) artık `userinfo`'yu Base64 ile kodlamaz; http gizleme içeren Shadowsocks bağlantıları `obfs-local` eklentisiyle SIP002 formatında sunulur; Clash/Mihomo abonelikleri için XHTTP alanlarının tam seti uygulanmıştır. Bu durum için ayrıca herhangi bir ayar gerekmez – bağlantılar istemciler tarafından yalnızca daha doğru şekilde tanınır.

Not: bu derlemede desteklenen üç format şunlardır – normal bağlantılar (Base64/metin), JSON (sing-box uyumlu) ve Clash/Mihomo (YAML). Abonelik sunucusunda ayrı bir Outline formatı mevcut değildir.

10.4. Abonelik bilgi sayfası ve QR kodları

Abonelik bağlantısı bir tarayıcıda açıldığında (ya da URL'ye açıkça `?html=1` veya `?view=html` parametresi eklendiğinde ya da `Accept: text/html` başlığı gönderildiğinde) sunucu ham yanıt yerine görsel bir **abonelik bilgi sayfası** ("Abonelik Bilgisi") sunar. VPN istemcileri HTML talep etmediğinden makine tarafından okunabilir yanıtı almaya devam eder.

Sayfa (Vite ile derlenen tek sayfalı uygulama) şunları gösterir:

- **Abonelik bilgisi** (Descriptions bloğu):
 - "Abonelik Kimliği" – `subId` değeri;
 - "Durum" – "Aktif", "Etkin Değil" veya "Sınırsız". Durum "etkin değil" olarak işaretlenir; istemci devre dışı bırakılmış, trafik limitini tüketmiş veya süresi dolmuşsa;
 - "İndirilen" ve "Yüklenen" – trafik hacimleri;
 - "Toplam Limit" – trafik limiti veya sınırsızsa ∞ ;
 - "Geçerlilik Süresi" – bitiş tarihi veya "Süresiz";
 - kalan trafik ve son çevrimiçi olma zamanı.
 - Tarihler, panel ayarındaki "Calendar Type" (`datepicker` , varsayılan `gregorian`) seçeneğine göre Gregoryen veya Celali takvimine göre görüntülenir.

- **Abonelik bağlantıları:** etkinleştirilen her format için renkli etiket (yeşil **SUB**, mor **JSON**, altın **CLASH**), kopyalama düğmesi ve **QR kodu** düğmesi (açılır pencere, 240 px boyutunda) içeren ayrı bir satır. JSON ve CLASH satırları yalnızca ilgili format ayarlarda etkinleştirilmişse görünür.
- **Bireysel bağlantılar** ("Bağlantıyı Kopyala"): aboneliğe dahil her yapılandırmanın tam listesi; her biri protokol etiketiyle, kopyalama düğmesiyle ve QR koduyla birlikte (post-quantum bağlantılar için QR kodu oluşturulmaz).
- **"Tüm Yapılandırmaları Kopyala" düğmesi** (bireysel bağlantılar listesinin üstünde): tek tıklamayla tüm yapılandırma bağlantılarını (her biri ayrı satırda) panoya kopyalar; tek tek kopyalamaya gerek kalmaz; tamamlanınca "Tüm yapılandırmalar kopyalandı" bildirimi gösterilir.
- **Uygulamalara hızlı aktarım düğmeleri** (platforma göre açılır menüler): Android için – v2box, v2rayNG (derin bağlantı `v2rayng://install-config?url=...`), Sing-box, V2RayTun, NPV Tunnel, Happ (`happ://add/...`); iOS için – Shadowrocket (`flag=shadowrocket` parametresiyle), v2box (`v2box://install-sub?url=...&name=...`), Streisand (`streisand://import/...`), V2RayTun, NPV Tunnel, Happ. Bu düğmeler, abonelik adresi önceden doldurulmuş ilgili uygulamanın derin bağlantısını açar ya da bağlantıyı panoya kopyalar.

Abonelik bilgi sayfası, istemcinin her zaman güncel trafik ve geçerlilik verilerini görmesi için önbellek engelleme başlıklarıyla (`Cache-Control: no-cache`) sunulur.

10.5. Abonelik sayfası için özel şablonlar

3.3.0 sürümünden itibaren standart abonelik açılış sayfası özel bir HTML şablonuyla değiştirilebilir. Varsayılan olarak abonelik adresinden yerleşik sayfa sunulur; ancak kendi şablonunuzu içeren bir dizin belirtirseniz panel bu şablonu işleyerek istemciye ait güncel verileri (trafik, geçerlilik süresi, bağlantılar vb.) içine ekler.

Önemli: panel hazır şablon **sağlamaz**. Depoda yalnızca bir `sub_templates/` dizini ve `sub_templates/README.md` adlı talimat dosyası bulunur; kendi temanızı baştan oluşturmanız gerekir.

Nerede etkinleştirilir

Tema dizini panel ayarlarından belirlenir:

Ayarlar → **Abonelik** → **"Abonelik Bilgisi" bölümü**, **"Abonelik Tema Dizini"** (`subThemeDir`) alanı.

Arayüzdeki alan açıklaması: "Abonelik sayfası için özel şablon (index.html/sub.html) içeren klasörün mutlak yolu (örn. /etc/3x-ui/sub_templates/my-theme/). Varsayılan sayfayı kullanmak için boş bırakın."

Aynı bölümde şablonda erişilebilen değerlere sahip ilgili ayarlar bulunur:

"Abonelik Tema Dizini" alanı açıklamasında, abonelik sayfası için özel tema şablonları oluşturmaya ilişkin belgelere yönlendiren **"Şablon Kılavuzu ↗"** bağlantısı mevcuttur.

- **"Abonelik Başlığı"** (`subTitle`) – istemciye görünen ad;
- **"Destek URL'si"** (`subSupportUrl`) – teknik destek bağlantısı.

Ayar parametresi

Parametre	Varsayılan değer	Amaç
subThemeDir	"" (boş)	HTML şablonunuzu içeren dizinin mutlak yolu. Boş = yerleşik varsayılan sayfa.

Kendi şablonunuzu nasıl uygularsınız

1. Sunucuda bir tema klasörü oluşturun (herhangi bir konumda), örn. `/etc/3x-ui/sub_templates/my-theme/`.
2. İçine `index.html` veya `sub.html` adlı bir HTML dosyası koyun.

Örnek: tema yolu. Sunucudaki nihai dizin yapısı ve ayarlardaki alan değeri:

```
/etc/3x-ui/sub_templates/my-theme/
└─ index.html      (veya sub.html – önceliklidir)
```

Ayarlar → Abonelik → "Abonelik Tema Dizini":
`/etc/3x-ui/sub_templates/my-theme/`

Yol **mutlak** olmalıdır (/ ile başlamalı). Klasörde ne `index.html` ne de `sub.html` varsa panel yerleşik sayfayı sunar. 3. Panelde **Ayarlar** → **Abonelik** bölümünü açın ve "Abonelik Tema Dizini" alanına bu klasörün **mutlak** yolunu girin. 4. Ayarları kaydedin.

Dosya seçimi ve işleme davranışı:

- Dizinde `sub.html` varsa bu dosya kullanılır; yoksa `index.html` alınır. Yani `sub.html`, `index.html` 'ye göre önceliklidir.
- Şablon standart Go `html/template` motoru tarafından işlenir.
- Ayırıştırılan şablon **önbelleklenir** ve yalnızca dosyanın değiştirilme zamanı farklılaştığında diskten yeniden okunur. Bu sayede şablon değişiklikleri paneli yeniden başlatmadan geçerli olur; her istekte okuma/ayırıştırma ek yükü oluşmaz.
- Yanıt tamamen arabelleğe alınır ve ancak tamamlandıktan sonra istemciye gönderilir: şablon çalışma zamanında başarısız olursa kısmen oluşturulmuş (bozuk) sayfa kullanıcıya ulaşmaz.

Varsayılan davranış ve yedek (fallback)

- Alan boş → yerleşik SPA sayfası sunulur (veriler `window.__SUB_PAGE_DATA__` içine eklenir).
- Yol mevcut değil veya bir dizin değil → varsayılan sayfa kullanılır.
- Dizinde ne `index.html` ne de `sub.html` var → loga "subThemeDir set but no usable template found" uyarısı yazılır, varsayılan sayfa sunulur.
- Şablon dosyası mevcut ancak ayırıştırılamıyor → loga "custom template parse failed" hatası yazılır, varsayılan sayfa sunulur.
- Şablon çalışma sırasında hata veriyor → loga "custom template execution failed" yazılır, varsayılan sayfa sunulur.

Yani özel şablondaki herhangi bir sorun aboneliği "bozmaz" — panel her zaman yerleşik sayfaya geri döner. Tüm abonelik sayfaları (hem özel hem de standart), istemcilerin her zaman güncel trafik ve süre verilerini alması için önbellek engelleme başlıklarıyla (`Cache-Control: no-cache, no-store, must-revalidate`) sunulur.

Kullanılabilir şablon değişkenleri

Şablon bağlamına istemcinin abonelik verileri aktarılır. Erişim `{{ .ad }}` söz dizimiyle sağlanır:

Değişken	Tür	Açıklama
<code>{{ .sId }}</code>	string	Abonelik Kimliği (UUID).
<code>{{ .enabled }}</code>	bool	İstemcinin/aboneliğin etkin olup olmadığı.
<code>{{ .download }}</code>	string	Biçimlendirilmiş indirme hacmi (örn. "2.5 GB").
<code>{{ .upload }}</code>	string	Biçimlendirilmiş yükleme hacmi.
<code>{{ .total }}</code>	string	Biçimlendirilmiş toplam trafik limiti.
<code>{{ .used }}</code>	string	Biçimlendirilmiş kullanılan trafik (download + upload).
<code>{{ .remained }}</code>	string	Biçimlendirilmiş kalan trafik.
<code>{{ .expire }}</code>	int64	Geçerlilik süresi — saniye cinsinden Unix zamanı (<code>0</code> = süresiz). <code>JS Date</code> için 1000 ile çarpın.
<code>{{ .lastOnline }}</code>	int64	Son çevrimiçi olma zamanı — milisaniye cinsinden Unix zamanı (<code>0</code> = hiç çevrimiçi olmadı).
<code>{{ .downloadByte }}</code>	int64	Tam bayt cinsinden indirme.
<code>{{ .uploadByte }}</code>	int64	Tam bayt cinsinden yükleme.
<code>{{ .totalByte }}</code>	int64	Tam bayt cinsinden toplam limit.
<code>{{ .subUrl }}</code>	string	Abonelik sayfasının URL'si.
<code>{{ .subJsonUrl }}</code>	string	Abonelik JSON yapılandırmasının URL'si.
<code>{{ .subClashUrl }}</code>	string	Clash/Mihomo yapılandırmasının URL'si.
<code>{{ .subTitle }}</code>	string	Ayarlardan alınan abonelik başlığı (boş olabilir).
<code>{{ .subSupportUrl }}</code>	string	Ayarlardan alınan destek URL'si (boş olabilir).
<code>{{ .links }}</code>	[]string	Yapılandırma dizelerinin listesi (VMess, VLESS vb.). Yineleme: <code>{{ range .links }} ... {{ end }}</code> .
<code>{{ .emails }}</code>	[]string	Aboneliğe ait e-posta listesi.
<code>{{ .datepicker }}</code>	string	Panelin geçerli takvim formatı: <code>gregorian</code> veya <code>jalali</code> ("Takvim Türü" ayarından alınır; boşsa <code>gregorian</code>).

Değişkenlerin bir kısmını kullanan minimal şablon gövdesi örneği:

```

<h1>{{ .subTitle }}</h1>
<p>Kullanılan: {{ .used }} / {{ .total }} (kalan {{ .remained }})</p>
{{ range .links }}<div>{{ . }}</div>{{ end }}

**Örnek: `expire` alanından bitiş tarihi.** `{{ .expire }}` alanı **saniye** cinsinden
Unix zamanıdır; bu nedenle JavaScript'te 1000 ile çarpılır; `0` değeri "süresiz"
anlamına gelir:

```html
<script>
 var exp = {{ .expire }};
 document.write(exp === 0
 ? 'Süresiz'
 : 'Bitiş: ' + new Date(exp * 1000).toLocaleDateString());
</script>

```

Not: {{ .lastOnline }} zaten **milisaniye** cinsindendir – 1000 ile çarpmak gerekmez.

---

## ## 11. Xray: yönlendirme, outbounds, DNS ve uzantılar

**«Xray Ayarları»** bölümü, panelin çekirdeği başlatmak için kullandığı nihai `config.json`'u oluşturduğu Xray-core yapılandırma şablonunun editörüdür. Bölüm şablonuna ilişkin ipucu: **«Xray yapılandırma dosyası şablona göre oluşturulur.»** Inbounds'dan farklı olarak (bunlar ayrı olarak veritabanında saklanır ve yapılandırma oluşturulurken şablona eklenir), diğer her şey – günlükler, yönlendirme, outbounds, DNS, politika, istatistikler – tam olarak burada ayarlanır.

> Önemli: şablon değeri, `xrayTemplateConfig` anahtarı altında veritabanında saklanır. Kaydederken panel, üzerinde bir dizi otomatik dönüşüm uygular (bkz. [11.10](#1110-kaydetme-yeniden-başlatma-ve-otomatik-dönüşümler)). Sözdizimsel olarak hatalı herhangi bir JSON, **«xray template config invalid»** hatası ile reddedilir.

### #### Menüdeki konum: «Çıkışlar» ve «Yönlendirme»

**«Çıkışlar» (Outbounds)** ve **«Yönlendirme» (Routing)** – bunlar ayrı yan menü öğeleridir («Hostlar»ın hemen altında, «Panel Ayarları»nın üstünde), her birinin kendi adresi vardır: `/outbound` ve `/routing`. Bu sayfalara doğrudan bağlantılar ve sayfa yenilemeleri beklendiği gibi çalışır. **«Xray Yapılandırmaları»** alt menüsünde yalnızca şunlar kalır: Temel, Yük Dengeleyici, DNS ve Gelişmiş Şablon. Aşağıdaki açıklamada [11.3](#113-yönlendirme-kuralları-routing) ve [11.4](#114-outbounds-çıkış-bağlantıları) bölümleri «Yönlendirme» ve «Çıkışlar» sayfalarına karşılık gelir.

### ### 11.1. Editör yapısı: sekmeler/modlar

Editör, şablonu görüntülemek için birkaç mod sunar (JSON bölümlerine göre filtreler):

Mod	Ne gösterir
---	---
<b>«Temel»</b>	Temel bölümler (Günlük, temel yönlendirme, ana ayarlar)
<b>«Gelişmiş Şablon»</b>	Tam Xray JSON şablonu
<b>«Tümü»</b>	Tüm bölümler aynı anda

Editördeki mantıksal ayar grupları:

- **«Temel Ayarlar»** (ipucu: **«Bu parametreler genel ayarları açıklar»**).
- **«Günlük»** (bkz. [11.9](#119-günlükler-ve-istatistikler-stats-metrics)).
- **«Temel Bağlantılar»**: engellemeler ve doğrudan rotalar.
- **«Girişler»** (ipucu: **«Belirli istemcilerin bağlanması için yapılandırma şablonunu değiştirme»**).
- **«Çıkışlar»** (bkz. [11.4](#114-outbounds-çıkış-bağlantıları)).
- **«Yük Dengeleyici»** (bkz. [11.5](#115-yük-dengeleyiciler-balancers)).
- **«Yönlendirme»** (ipucu: **«Her kuralın önceliği önemlidir!»**, bkz. [11.3](#113-yönlendirme-kuralları-routing)).
- **«DNS / Fake DNS»** (bkz. [11.6](#116-dns)).

### ### 11.2. Temel Ayarlar (General)

#### #### Freedom Protocol Strategy

Alan	Etiket	Açıklama	Varsayılan
---	---	---	---
`FreedomStrategy`	<b>«Freedom protokol stratejisi ayarı»</b>	Doğrudan (freedom)	

outbound için ağ çıkış stratejisi. İpucu: «Freedom protokolünde ağ çıkış stratejisini ayarlama». `freedom` protokolüne sahip outbound'un `settings` içindeki `domainStrategy` alanını kontrol eder. | Referans şablonunda `direct` freedom-outbound için `domainStrategy` «AsIs»'dir (adres çözülmez, orijinal haliyle iletilir). |

`domainStrategy` için freedom (Xray-core değerleri): `AsIs` (sunucu tarafında etki alanını çözme), ayrıca `UseIP` / `UseIPv4` / `UseIPv6` ailesi ve bunların «zorunlu» `ForceIP\*` varyantları; bu varyantlar çıkış sunucusunu etki alanını çözülemeye ve alınan IP üzerinden bağlanmaya zorlar. Çıkış sunucusunda IPv6 yoksa veya yalnızca IPv4 üzerinden bağlantı gerekiyorsa `UseIPv4` olarak değiştirin.

#### #### Freedom Happy Eyeballs (IPv4/IPv6)

Alan	Etiket	Açıklama
---	---	---
`FreedomHappyEyeballs`	«Freedom Happy Eyeballs (IPv4/IPv6)»	İpucu: «IPv4 ve IPv6'ya sahip çıkış sunucularında kullanışlı olan doğrudan (freedom) çıkış için çift yığın seçimi.» Freedom-outbound için Happy Eyeballs algoritmasını (her iki adres ailesinde eş zamanlı deneme) etkinleştirir.
try delay	(ipucu)	«Başka bir adres ailesini denemeden önceki milisaniye. 150–250 ms iyi bir başlangıç noktasıdır.» Alternatif adres ailesine geçmeden önceki gecikme. Önerilen aralık – 150–250 ms.

#### #### Overall Routing Strategy

Alan	Etiket	Açıklama	Varsayılan
---	---	---	---
`RoutingStrategy`	«Etki alanı yönlendirme stratejisi ayarı»	Yönlendirme için genel DNS çözümleme stratejisi. İpucu: «Genel DNS çözümleme yönlendirme stratejisini ayarlama». `routing.domainStrategy` alanını kontrol eder.   Referans şablonunda `routing.domainStrategy` = «AsIs».	

`routing.domainStrategy`, IP yönlendirme kurallarının etki alanı istekleriyle nasıl eşleştirileceğini belirler: `AsIs` (yalnızca etki alanı kuralları, çözümleme yok), `IPIfNonMatch` (etki alanı kurallarla eşleşmezse – çözümle ve IP kurallarını kontrol et), `IPOnDemand` (bir IP kuralıyla karşılaşıldığında hemen çözümle). IP kurallarının (örn. `geoip:\*`) etki alanı istekleri için çalışması amacıyla genellikle `IPIfNonMatch` gereklidir.

#### #### Outbound Test URL

Alan	Etiket	Açıklama	Varsayılan
---	---	---	---
`outboundTestUrl`	«Çıkış test URL'si»	Outbound test edilirken bağlantıyı kontrol etmek için kullanılan URL. İpucu: «Çıkış bağlantısını kontrol etmek için URL». Şablondan ayrı olarak `xrayOutboundTestUrl` anahtarı altında saklanır.   «https://www.google.com/generate_204»	

Değer temizleme işleminden geçer. Outbound testinin kendisinde ek olarak genel URL olarak doğrulanır – bu, SSRF'ye karşı bir korumadır: kullanıcı, istemci aracılığıyla rastgele (dahili dahil) bir URL geçiremez; test URL'si her zaman sunucu ayarından alınır. Kaydetme/test sırasında boş bir değer, varsayılan `generate\_204` ile değiştirilir.

#### #### Block BitTorrent

Alan	Etiket	Açıklama
---	---	---

```
| `Torrent` | **BitTorrent'i Engelle** | `routing.rules` içine `protocol: ["bittorrent"]` trafiğini `blocked` outbound'una gönderen bir kural ekler. Referans şablonunda bu kural varsayılan olarak mevcuttur. |
```

#### #### Bağlantı Kısıtlamaları (Connection Limits)

İpucu: **«0. düzey kullanıcılar için bağlantı düzeyi politikaları. Xray varsayılan değerini kullanmak için alanı boş bırakın.»** Bu parametreler `policy.levels.0` içine yazılır.

```
| Alan | Etiket | Açıklama | Varsayılan |
|---|---|---|---|
| `connIdle` | **Boşta kalma zaman aşımı** (saniye) | «Belirli sayıda saniye boşta kaldıktan sonra bağlantıyı kapatır. Değerin azaltılması, yoğun sunucularda belleği ve dosya tanımlayıcılarını daha hızlı serbest bırakır (Xray varsayılanı: 300).» | boş → Xray varsayılanı **300** |
| `bufferSize` | **Arabellek boyutu** (KB) | «Bağlantı başına KB cinsinden dahili arabellek boyutu. Az miktarda RAM'e sahip sunucularda bellek kullanımını en aza indirmek için 0 olarak ayarlayın (Xray varsayılan değeri platforma bağlıdır).» Yer tutucu: **otomatik**. | boş → platforma bağlı; `0` – en aza indir |
```

**\*\*Örnek (`policy.levels.0`).** Bu gruptaki alanlar, 0. düzey politikasına yazılır. Az RAM'li yoğun bir sunucuda kaynak serbest bırakmayı şu şekilde hızlandırabilirsiniz:

```
```json
"policy": {
  "levels": {
    "0": {
      "connIdle": 120,
      "bufferSize": 0
    }
  }
}
```

Burada bağlantı, boşta kalmanın ardından varsayılan 300 yerine 120 saniye içinde kapatılır ve `bufferSize: 0` arabellekler için bellek kullanımını en aza indirir. Formda boş bırakılan bir alan JSON'a dahil edilmez – Xray kendi varsayılan değerini uygular.

11.3. Yönlendirme Kuralları (routing)

`routing.rules` kural listesi. **Sıra kritiktir** (*«Her kuralın önceliği önemlidir!»*): kurallar yukarıdan aşağıya değerlendirilir, ilk eşleşen devreye girer. İpucu: *«Sırayı değiştirmek için sürükleyin»*. Sıra kontrol düğmeleri: **İlk, Son, Yukarı Taşı, Aşağı Taşı**.

Her kuralın `type: "field"` değeri vardır. Düğmeler: **Kural Oluştur, Kuralı Düzenle**. Liste alanlarına yönelik ipucu: *«Virgülle ayrılmış öğeler»*.

«Yönlendirme» sayfasında **«Kuralları İç Aktar»** ve **«Kuralları Dış Aktar»** düğmeleri, **«daha fazla»** (more) açılır menüsünde toplanmıştır – tıpkı «Çıkışlar» sayfasında olduğu gibi. **«Kuralları Dış Aktar»** düğmesi dosyayı hemen indirmez, bunun yerine JSON önizlemesi ve **«Kopyala»** ile **«İndir»** düğmelerini içeren bir modal pencere açar: içerik kaydedilmeden önce incelenebilir. «Çıkışlar» sayfasındaki çıkış dış aktarımı da aynı şekilde çalışır.

Route Tester (rota test aracı)

Routing sekmesinde bir **Route Tester** alt sekmesi vardır – bu sekme, gerçek trafik göndermeden belirli bir bağlantının hangi outbound tarafından işleneceğini çalışan Xray'den sorar. Bir etki alanı veya IP, port, ağ (TCP/UDP) ve gerekirse inbound ile yakalanan protokol (`http / tls / quic / bittorrent`) belirtin, ardından **Test Route** düğmesine tıklayın. Karar, doğrudan canlı yönlendirme motorundan alınır.

Yanıta eşleşen outbound gösterilir; bir yük dengeleyici kullanılıyorsa yük dengeleyici etiketi de gösterilir. Hiçbir kural eşleşmezse test aracı, trafiğin varsayılan outbound'a (outbounds listesindeki ilk) gittiğini bildirir. Bu, kurallara güvenmeden önce kural sırasını doğrulamak için kullanışlıdır.

Bireysel bir kuralı etkinleştirme ve devre dışı bırakma

Bireysel bir yönlendirme kuralı, silinmeden bir geçiş düğmesiyle geçici olarak **devre dışı bırakılabilir**. Kurallar tablosunda bir Switch geçiş düğmesiyle **«Etkinleştir»** sütunu bulunur; kural formunda ise **«Etkinleştir»** alanı da bir geçiş düğmesidir. Devre dışı bırakılan bir kural nihai Xray yapılandırmasına dahil edilmez, ancak şablonda saklanır ve istediğiniz zaman yeniden etkinleştirilebilir.

İstatistik servis kuralı (`inboundTag: ["api"] → outboundTag: "api"`) devre dışı bırakılamaz – geçiş düğmesi, panel trafik sayacını bozmamak için kilitlenmiştir (bkz. [11.10](#)).

Kural formu alanları

Form alanı	Etiket (TR)	JSON alanı	Açıklama
Kaynak	Kaynak	<code>source</code>	Kaynak IP adresleri/alt ağları. Virgülle ayrılmış liste.
Kaynak portu	Kaynak portu	<code>sourcePort</code>	Kaynak port(ları).
Hedef	Hedef	<code>domain + ip + port</code>	Hedef etki alanları, IP'ler ve portlar. Etki alanları <code>domain:</code> , <code>full:</code> , <code>regexp:</code> , <code>keyword:</code> örneklerini ve <code>geosite:*</code> değerini destekler; IP'ler <code>geoip:*</code> ve CIDR'ı destekler.
Ağ	–	<code>network</code>	<code>tcp</code> , <code>udp</code> veya <code>tcp,udp</code> .
Protokol	–	<code>protocol</code>	<code>http</code> , <code>tls</code> , <code>bittorrent</code> (sniffing ile belirlenir).
Kullanıcı	Kullanıcı	<code>user</code>	E-posta/kullanıcı tanımlayıcısına göre filtre.
Öznitelikler / Değer	Öznitelikler / Değer	<code>attrs</code>	Eşleştirme için HTTP başlığı öznitelikleri.
VLESS route	VLESS route	–	VLESS için route alanına göre yönlendirme.
Giriş etiketleri	Giriş etiketleri	<code>inboundTag</code>	Kuralın uygulandığı bir veya daha fazla inbound etiketi (yerleşik <code>api</code> ve DNS ayarlarındaki DNS etiketi dahil). Inbound listelerinde inbound'un ayrı bir notu varsa «etiket (not)» olarak, yoksa yalnızca etiket olarak görüntülenir; kaydedilmiş kurallarda yalnızca etiketler saklanmaya devam eder.
Çıkış etiketi	Çıkış etiketi / Çıkış bağlantısı	<code>outboundTag</code>	Eşleşen trafiğin yönlendirileceği yer.

Form alanı	Etiket (TR)	JSON alanı	Açıklama
Yük dengeleyici etiketi	Yük dengeleyici etiketi / Yük Dengeleyici	<code>balancerTag</code>	İpucu: «Trafik yapılandırılmış yük dengeleyicilerden biri aracılığıyla yönlendirir».

`outboundTag` ve `balancerTag` birbirini dışlar: «*balancerTag* ve *outboundTag*'ı aynı anda kullanmak mümkün değildir. Aynı anda kullanılırsa yalnızca *outboundTag* çalışır.» Bir kuralda ya çıkış etiketini ya da yük dengeleyici etiketini belirtin.

Referans şablonunun yerleşik kuralları

Standart `config.json` 'daki `routing` bölümü, bu sırayla üç kural içerir:

1. `inboundTag: ["api"]` → `outboundTag: "api"` – panel istatistik gRPC-API'si için servis kuralı.
2. `ip: ["geoip:private"]` → `outboundTag: "blocked"` – özel aralıkları engelleme.
3. `protocol: ["bittorrent"]` → `outboundTag: "blocked"` – BitTorrent engelleme.

`api` → `api` kuralı, istatistik isteğinin üstteki bir catch-all kuralı tarafından «yutulmaması» için kaydederken her zaman otomatik olarak 0 konumuna yükseltilir (bkz. [11.10](#)).

Kural örneği. Rus sitelere ve özel ağlara giden tüm trafiği doğrudan gönderin (proxy'yi atlayarak), geri kalanını yük dengeleyiciye yönlendirin. Sıra önemlidir: «doğrudan yönlendir» kuralı catch-all'ın üzerinde olmalıdır. `routing.rules` içinde:

```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

IP kurallarının (`geoip:ru`) etki alanı istekleri için de çalışması amacıyla, üst düzey yönlendirmede genellikle `routing.domainStrategy: "IPIfNonMatch"` gereklidir (bkz. [11.2](#)).

Önceden yapılandırılmış yönlendirme grupları (Temel Bağlantılar)

«Temel Bağlantılar» modunda panel, hazır listelerden tipik kurallar oluşturmanıza yardımcı olur:

Grup	Alanlar	İpucu
Protokol/sitelere göre engelleme	—	«İstemcilerin belirli protokollere erişememesi için yapılandırın»
Ülkelere göre engelleme	Engellenen IP adresleri, Engellenen etki alanları	«Bu parametreler, hedef ülkeye bağlı olarak trafiği engeller.»

Grup	Alanlar	İpucu
Doğrudan bağlantılar	Doğrudan IP adresleri, Doğrudan etki alanları	«Doğrudan bağlantı, belirli trafiğin başka bir sunucu üzerinden yönlendirilmeyeceği anlamına gelir.»
IPv4 Kuralları	—	«Bu parametreler, istemcilerin hedef etki alanlarına yalnızca IPv4 üzerinden yönlendirilmesini sağlayacaktır»
WARP Kuralları	—	«Bu seçenekler, trafiği belirli hedefe bağlı olarak WARP üzerinden yönlendirecektir.»
NordVPN Yönlendirme	—	«Bu seçenekler, trafiği belirli hedefe bağlı olarak NordVPN üzerinden yönlendirecektir.»

MTPProto-inbound: Xray aracılığıyla Telegram trafiğini yönlendirme

MTPProto-inbound'un bir «**Route through Xray**» geçiş düğmesi (varsayılan olarak kapalı) ve isteğe bağlı bir **Outbound** seçimi vardır. Etkinleştirildiğinde panel, Xray yapılandırmasına inbound'un etiketine sahip bir döngüsel SOCKS köprüsü ekler ve mtg Telegram trafiğini bu köprü üzerinden yönlendirir. Bundan sonra giden Telegram trafiği yönlendirici tarafından kontrol edilir: Routing sekmesinde inbound etiketi üzerinden normal kurallarla eşleştirilebilir veya **Outbound** alanı aracılığıyla seçilen outbound'a ya da yük dengeleyiciye zorla yönlendirilebilir. Kararın yönlendirme kuralları tarafından verilmesi için **Outbound** alanını boş bırakın.

11.4. Outbounds (çıkış bağlantıları)

outbounds listesi. Düğmeler: **Çıkış Bağlantısı Oluştur**, **Çıkış Bağlantısını Düzenle**. İpucu: «Bu sunucu için çıkış bağlantılarını tanımlamak üzere yapılandırma şablonunu değiştirme».

Referans şablonunda iki zorunlu outbound bulunur:

- `protocol: "freedom", tag: "direct"` – internete doğrudan çıkış (`domainStrategy: "AsIs"` ve `finalRules: [{action: "allow"}]` ile);
- `protocol: "blackhole", tag: "blocked"` – engellenen trafik için «kara delik».

Genel outbound form alanları

Alan	Etiket (TR)	Açıklama
Etiket	Etiket (ipucu: «Benzersiz etiket»)	Outbound'un benzersiz tanımlayıcısı. Yer tutucu: «benzersiz-etiket». Doğrulama: «Etiket zorunludur», «Etiket zaten başka bir çıkış tarafından kullanılıyor».
Protokol	—	Çıkış türü (aşağıya bakın).
Adres / Port	Adres / Port	Bağlantı hedefi. Adres ve port zorunludur.
Üzerinden Gönder	Üzerinden Gönder	Çıkış arayüzünün yerel IP adresi (<code>sendThrough</code>). Yer tutucu: «yerel IP».
Dialer proxy (zincir)	—	İpucu: «Bir proxy zinciri oluşturmak için bu çıkışı başka bir çıkış üzerinden (etikete göre) bağlayın. Doğrudan bağlantı için boş bırakın.» Yer tutucu: «Zincir için çıkış seçin». <code>streamSettings.sockopt.dialerProxy</code> aracılığıyla uygulanır.

Desteklenen outbound protokolleri

Form tarafından desteklenen protokoller:

- **freedom** – doğrudan çıkış. `settings.domainStrategy` , `finalRules` (aşağıya bakın), Happy Eyeballs alanları. Test edilemez («*Outbound has no testable endpoint*»).
- **blackhole** – trafiği atar. **Yanıt türü** alanı. Test edilemez.
- **socks** , **http** – `address / port` içeren `settings.servers[]` listesi; **Yetkilendirme şifresi** alanı.
- **vmess** – `settings.vnext[]` (`address / port`).
- **vless** – `settings.address / settings.port` .
- **trojan** , **shadowsocks** – `settings.servers[]` .
- **wireguard** – `endpoint` içeren `settings.peers[]` ,artı anahtarlar (bkz. [11.7](#)).
- **hysteria** – `settings.address / settings.port` (UDP taşıma).

loopback türündeki outbound için, inbound ile aynı parametrelere sahip bir **Sniffing** bloğu mevcuttur: etkinleştirme, **destOverride**, **Metadata Only**, **Route Only** ve **dışlanan etki alanları** listesi.

UDP maskesinde (FinalMask) **Hysteria2** için ek modlar mevcuttur. **Salamander** maskesinin **Salamander** ve **Gecko** değerlerine sahip bir **Mode** seçici vardır: Gecko modu, **Min/Max** boyut alanlarıyla (`packetSize` , aralık 1–2048, varsayılan 512–1200) rastgele paket dolgusu ekler – bu, paket uzunluğuna göre parmak izini almaya karşı koruma sağlar. **Realm** maskesinde (UDP hole-punching) **Server Name** (SNI), **ALPN** (`h3 / h2 / http/1.1`), **Fingerprint** (uTLS) alanları ve **Allow Insecure** geçiş düğmesiyle isteğe bağlı bir **TLS Config** bloğu bulunur.

Örnek: yukarı SOCKS üzerinden zincir. `upstream` outbound'u harici bir SOCKS5 proxy'sine bağlanır, `chained` ise trafiğini bu outbound üzerinden göndererek (`dialerProxy`) bir zincir oluşturur. `outbounds` içinde:

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

Artık `outboundTag: "chained"` içeren bir yönlendirme kuralı, trafiği `upstream` üzerinden internete gönderecektir.

Paylaşım bağlantısından outbound içe aktarma

Bir outbound, paylaşım bağlantısından (`vless://` , `vmess://` vb.) içe aktarılabilir. İçe aktarma sırasında, bağlantının `extra=` bloğunda iletilen **xmux** (XHTTP) çoğullayıcı ayarları da korunur: içe aktarma sonrasında değerleri, oluşturulan outbound'un **XMUX** alt formuna eklenir.

Mux (çoğullama) alanları

Maks. paralellik, **Maks. bağlantı sayısı**, **Maks. yeniden kullanım sayısı**, **Maks. istek sayısı**, **Maks. yeniden kullanım süresi (saniye)**, **Keep alive süresi**. Bu parametreler, outbound'un mux/XUDP davranışını yapılandırır.

Sockopts (soket ayarları)

Sockopts grubu: **Keep alive aralığı**, **Mark (fwmark)**, **Arayüz**, **Yalnızca IPv6**, **Proxy protokolü kabul et**, **Proxy protokolü**, **TCP kullanıcı zaman aşımı (ms)**, **TCP keep-alive idle (s)**. Zincir dialer-proxy burada da ayarlanır.

Freedom finalRules (özel IP engelleme geçersiz kılma)

Freedom-outbound için bir **Son Kurallar** grubu mevcuttur:

Alan	Etiket	Açıklama
<code>overrideXrayPrivateIp</code>	Xray'deki varsayılan özel IP engelini geçersiz kıl	Özel IP'lere giden çıkışlardaki Xray yerleşik kısıtlamasını kaldırır.
<code>action</code>	Eylem	<code>allow</code> (referans şablonunda olduğu gibi: <code>finalRules: [{action: "allow"}]</code>), <code>redirect</code> (Yönlendirme) veya diğerleri.
<code>blockDelay</code>	Engel gecikmesi (ms)	Bağlantıyı atmadan önceki gecikme.
<code>redirect / fragment</code>	Yönlendirme / Parçalama	Trafik yönlendirme ve parçalama eylemleri.

Fragment maskesi: parça başına Lengths ve Delays

Fragment maskesinde (FinalMask'ta TCP için fragment türü) tek Uzunluk ve Gecikme alanlarının yerini **Lengths** ve **Delays** listeleri almıştır: her segment için ayrı bir uzunluk aralığı (ör. `100-200`) ve milisaniye cinsinden gecikme (ör. `10-20` veya `0`) belirtilebilir. Liste satırları eklenip silinebilir; önceden kaydedilmiş tek değerler otomatik olarak bir elemanlı diziye taşınır.

Diğer form alanları

- **UDP over TCP** ve **UoT Sürümü** – shadowsocks benzeri protokoller için.
- **gRPC başlığı yok**, **Uplink chunk boyutu** – gRPC taşıma parametreleri.
- TLS/uTLS alanları: **Peer adını doğrula**, **Pinned SHA256**, **Short ID**, **Vision testpre**, yer tutucu «sunucu adı».

Çıkışları test etme

Düğmeler: **Test**, **Tümünü Test Et**. Durumlar: **Bağlantı test ediliyor...**, **Test başarılı**, **Test başarısız**, **Çıkış bağlantısı test edilemedi**. Sonuç: **Test sonucu**, milisaniye cinsinden gecikme.

İki mod (ipucu: «TCP: hızlı yalnızca-dial sondası. HTTP: xray üzerinden tam istek.»):

- **TCP** (mode=tcp) — host:port adresine basit dial, tüm uç noktalarda paralel olarak gerçekleştirilir, ~5 s zaman aşımı. Yalnızca TCP erişilebilirliğini kontrol eder, proxy protokolünü doğrulamaz. freedom / blackhole / blocked etiketi için «Outbound has no testable endpoint» döndürür.
- **HTTP** (mode=http veya boş) — geçici bir Xray örneği başlatır, gerçek bir HTTP isteği gönderir (probe URL = sunucu outboundTestUrl), gerçek gecikmeyi ölçer. Yetkili ancak maliyetli mod: global bir kilit tarafından serileştirilir («Another outbound test is already running, please wait»). Tek deneme için zaman aşımı 10 s, sonuç bekleme penceresi 15 s (yavaş veya tünelli kanallar üzerindeki sağlıklı outbound'ların «Başarısız» olarak işaretlenmemesi için artırılmıştır). Başarısızlık durumunda gerçek neden (DNS hatası, bağlantı reddedildi, son tarih dolumu, TLS hatası vb.) panel/Xray günlüğüne yazılır; genel zaman aşımı mesajları bu günlüğe işaret eder.

UDP protokolleri (wireguard , hysteria) ve UDP taşımaları (kcp , quic , hysteria), TCP istenmiş olsa bile her

zaman HTTP modunda test edilir — ham UDP-dial, «canlı» uç noktayı «ölü» olandan ayırt edemez. wireguard için test yapılandırmasında noKernelTun: true zorla ayarlanır.

Toplu kontrol ve aşama ayrımı

Test ve **Tümünü Test Et**, HTTP modunda outbound paketi için ortak bir geçici Xray örneği başlatır, her biri için döngüsel bir SOCKS-inbound kuralı oluşturur ve bu inbound üzerinden paralel olarak gerçek bir HTTP isteği gönderir; **Tümünü Test Et** outbound'ları gruplar halinde kontrol eder. **Tümünü Test Et**, aboneliklerden alınan outbound'ları da kontrol eder (salt okunur «aboneliklerden» tablosu) — satırları da test sonucuyla vurgulanır. freedom («direct») ve dns outbound'ları hiçbir modda test edilmez (bunlar proxy değildir): test düğmeleri kullanılamaz, **Tümünü Test Et** bunları atlar ve sunucu koruması doğrudan API çağrısında bile HTTP testlerini engeller. Başarı/hata durumuna ek olarak açılır pencerede HTTP yanıt durumu ve aşama bazlı süre dökümü gösterilir: **Proxy connect** (proxy'ye bağlantı), **TLS via outbound** (outbound üzerinden TLS) ve **First byte** (ilk bayta kadar geçen süre) — bu, gecikmenin veya arızanın hangi adımda oluştuğunu anlamaya yardımcı olur.

Outbound trafik istatistikleri

Panel, etiketlere göre trafik sayaçlarını (up / down / total) tutar. Sıfırlama düğmesi, belirli bir etiketin veya tüm etiketlerin (tag = "-alltags-") sayaçlarını sıfırlar. **Hesap bilgisi** ve **Çıkış bağlantısı durumu** alanları özet bilgi gösterir.

11.5. Yük Dengeleyiciler (Balancers)

routing.balancers listesi. Düğmeler: **Yük Dengeleyici Oluştur**, **Yük Dengeleyiciyi Düzenle**.

Balancers sekmesinde canlı durum sütunları bulunur: **Live Target**, çalışan Xray'deki yük dengeleyicinin mevcut aktif hedefini gösterir; **Override** ise hedef seçimini manuel olarak geçersiz kılmaya olanak tanır (**Auto (strategy)** değeri seçimi stratejiye döndürür). Durum ayrı bir düğmeyle güncellenir. Yük dengeleyici çalışan Xray'de henüz aktif değilse panel, önce değişiklikleri kaydetmenizi veya Xray'i başlatmanızı önerir.

Alan	Etiket (TR)	Açıklama
Etiket	Etiket (ipucu: «Benzersiz etiket»)	Benzersiz tanımlayıcı. Yer tutucu: «benzersiz yük dengeleyici etiketi». Doğrulama: «Etiket zorunludur», «Etiket zaten başka bir yük dengeleyici tarafından kullanılıyor».
Seçiciler	Seçiciler	Yük dengeleyicinin çıkışı seçtiği outbound etiketlerinin listesi (alt dizeye göre). En az bir tane seçilmelidir: «En az bir çıkış seçin».
Fallback	Fallback	Hiçbir seçici uygun değilse yedek outbound etiketi.
Strateji	Strateji	Seçim algoritması (aşağıya bakın).

Strateji ve gözlem parametreleri

Strateji (`strategy.type`), yük dengeleyicinin seçicilerden nasıl outbound seçeceğini belirler. Xray-core değerleri: `random` (rastgele), `roundRobin` (sırayla), `leastPing` (gözlem sonuçlarına göre minimum gecikme), `leastLoad` (minimum yük). `leastLoad` / `leastPing` için `strategy.settings` parametreleri kullanılır:

Alan	Etiket	Açıklama
<code>expected</code>	Beklenen	Yer tutucu: « <i>optimum düğüm sayısı</i> » – hedef canlı düğüm sayısı.
<code>maxRtt</code>	Maks. RTT	Aday seçiminde kabul edilebilir maksimum RTT üst sınırı.
<code>tolerance</code>	Tolerans	Gecikmeleri/yükleri karşılaştırırken tolerans.
<code>baselines</code>	Baselines	Düğümleri gruplandırmak için gecikme eşik değerleri.
<code>costs</code>	Costs	Bireysel etiketler için ağırlık katsayıları (cost).

Strateji örnekleri. `strategy` bloğu, yük dengeleyicinin içinde (JSON'da `tag` ve `selector` ile birlikte) yer alır:

```
"strategy": { "type": "random" }      // seçicilerden rastgele seçim
"strategy": { "type": "roundRobin" }  // sırayla, dönüşümlü olarak
"strategy": { "type": "leastPing" }   // minimum gecikme (gözlemci gerektirir)
```

`leastLoad` için parametreler `settings` içinde belirtilir:

```
"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regex": false, "match": "proxy-premium", "value": 0.1 },
      { "regex": true, "match": "^proxy-cheap-.+$$", "value": 5 }
    ]
  }
}
```

Nasıl çalışır (örnek). Gözlemcinin çıkışlar için ölçtüğü gecikmelerin şunlar olduğunu varsayalım: A = 250 ms , B = 280 ms , C = 700 ms , D = 1500 ms .Yukarıdaki ayarlarla seçim şu şekilde gerçekleşir:

1. **maxRTT: "1s"** – gecikmesi 1 s üzerindeki çıkışlar elenir: D (1500 ms) elenmiştir. Geride A , B , C kalır.
2. **baselines + expected** – çıkışlar gecikme eşiklerine göre gruplandırılır ve en az expected çıkışın içine girdiği en küçük eşik seçilir. 500ms eşiği A ve B 'yi zaten kapsamaktadır – bu 2 (= expected) değerine eşittir, dolayısıyla { A , B } grubu seçilir. C (700 ms) hızlı çıkışlar yeterliyken seçime girmez («sıcak yedek» olarak kalır).
3. **tolerance: 0.05** – seçilen grup içinde gecikmeleri en fazla %5 farklı olan çıkışlar eşdeğer kabul edilir ve yük aralarında eşit olarak dağıtılır. A (250) ve B (280) yaklaşık %12 (> %5) farklılık gösterir, dolayısıyla diğer koşullar eşit olduğunda daha hızlı olan A tercih edilir; fark %5 dahilinde olsaydı trafik hem A hem B üzerinden giderdi.
4. **costs** – karşılaştırmadan önce bireysel çıkışların «maliyetini» ayarlar: daha küçük value , çıkışı daha çekici kılar; daha büyük ise tersine. Örnekte proxy-premium , 0.1 alır (daha «ucuz» olur ve daha sık tercih edilir); düzenli ifadeyle (regex: true) eşleşen tüm proxy-cheap-* çıkışları 5 alır (daha «pahalı» olur ve en son kullanılır). Bu sayede çıkışları katı şekilde dışlamadan önceliklendirebilirsiniz.

Sonuç: trafik ağırlıklı olarak A üzerinden gider (yakın gecikmeler durumunda B ile eşit olarak paylaşılır), C yedek olarak kalır, D RTT'si maxRTT 'nin altına düşene kadar hariç tutulur.

Gözlemci: observatory ve burstObservatory (leastPing / leastLoad için ölçümler)

leastPing ve leastLoad stratejileri kendi başlarına hiçbir şey ölçmez – her outbound için gecikme ve erişilebilirlik verilerine ihtiyaçları vardır. Bu veriler **gözlemci** (observatory) tarafından toplanır: gözlemci, izlenen her outbound'u periyodik olarak «ping'ler» ve yanıt süresini ve erişilebilirliğini kaydeder. Aynı veriler «Gözlemevi» sekmesinde de gösterilir (durum **Aktif / Erişilemiyor**, «Son aktivite», «Son deneme»).

Panelde gözlemci için ayrı bir form yoktur – blok, Xray yapılandırma editörüne yapılandırmanın üst düzeyine (routing ve outbounds ile aynı seviyede) **manuel olarak** eklenir ve ardından **Xray'in yeniden başlatılması** gerekir.

İki seçenek mevcuttur:

- **observatory** – basit: subjectSelector + probeURL + probeInterval .
- **burstObservatory** – pingConfig aracılığıyla ince ayarlı ping yapılandırmasıyla genişletilmiş; birden fazla çıkış için kullanışlıdır.

burstObservatory bloğu örneği:

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

Alan açıklamaları:

Alan	Ne ayarlar
subjectSelector	Gözlem için outbound etiketi öneklerinin listesi. Xray, etiketleri belirtilen dizilerle başlayan tüm outbound'ları alır. Örnekte WS-SE... , WS-FR... , WS-PL... çıkışları gözlemlenir. Bu etiketlerin yük dengeleyicinin Seçicilerinde seçilenlerle eşleşmesi gerekir.
pingConfig.destination	Gecikme ölçümü için her outbound üzerinden istenen URL. Gövdesiz 204 yanıtı döndüren «hafif» bir sayfa kullanın – örneğin https://www.google.com/generate_204 .Yanıtı kadar geçen süre, ölçülen gecikmedir.
pingConfig.interval	Her outbound'un ne sıklıkta ping'leneceği. Süre dizesi: "1m" – dakikada bir; ayrıca "30s" , "5m" vb. Daha sık – daha taze veriler, ancak daha fazla arka plan trafiği.
pingConfig.connectivity	(isteğe bağlı) Sunucunun temel bağlantısını kontrol etmek için URL. Erişilemezse – sorun sunucunun ağındadır ve gözlemci outbound'u erişilemiyor olarak işaretlemeyiz (yerel arıza durumunda yanlış tetiklemelere karşı koruma). Genellikle 204 yanıtı döndüren bir uç nokta.
pingConfig.timeout	Girişimi başarısız saymadan önce tek bir ping'e bekleme süresi (örn. "5s").
pingConfig.sampling	Her outbound için kaç son ölçümün saklanıp ortalamasının alınacağı. 2 – son iki ping'i dikkate alır (rastgele ani yükselmeleri yumuşatır).

Her şeyi nasıl bağlarsınız:

1. Xray editöründe burstObservatory bloğunu gerekli subjectSelector değerleriyle ekleyin.
2. Yük dengeleyici oluşturun: **Strateji** = leastPing , **Seçiciler** içinde aynı outbound etiketlerini belirtin (WS-SE , WS-FR , WS-PL).
3. Yönlendirme kuralıyla ona trafik yönlendirin (**Yük Dengeleyici Etiketi** alanı, bkz. 11.3).
4. Xray'i yeniden başlatın. «**Gözlemevi**» sekmesinde çıkış durumları görünecek ve yük dengeleyici en hızlı canlı çıkışı seçmeye başlayacaktır.

Bir kuralda aynı anda balancerTag ve outboundTag ayarlanamaz – yalnızca outboundTag çalışır.

11.6. DNS

`dns` bölümü. Etkinleştirme: **DNS'i Etkinleştir** (ipucu: «Yerleşik DNS sunucusunu etkinleştir»).

Genel DNS parametreleri

Alan	Etiket (TR)	JSON	Açıklama / ipucu
<code>tag</code>	DNS etiketi adı	<code>dns.tag</code>	«Bu etiket, yönlendirme kurallarında giriş etiketi olarak kullanılabilir.» DNS isteklerinin kendisini <code>inboundTag</code> aracılığıyla yönlendirmeye olanak tanır.
<code>clientIp</code>	İstemci IP'si	<code>dns.clientIp</code>	«DNS istekleri sırasında sunucuya belirtilen IP konumunu bildirmek için kullanılır» (EDNS Client Subnet).
<code>strategy</code>	İstek stratejisi	<code>dns.queryStrategy</code>	«Genel etki alanı adı çözümleme stratejisi». Değerler: <code>UseIP</code> , <code>UseIPv4</code> , <code>UseIPv6</code> .
<code>disableCache</code>	Önbelleği devre dışı bırak	<code>dns.disableCache</code>	«DNS önbelleğini devre dışı bırakır».
<code>disableFallback</code>	Yedek DNS'i devre dışı bırak	<code>dns.disableFallback</code>	«Yedek DNS isteklerini devre dışı bırakır».
<code>disableFallbackIfMatch</code>	Eşleşme durumunda yedek DNS'i devre dışı bırak	<code>dns.disableFallbackIfMatch</code>	«DNS sunucusunun etki alanı listesi eşleştiğinde yedek DNS isteklerini devre dışı bırakır».
<code>enableParallelQuery</code>	Paralel sorguları etkinleştir	—	«Daha hızlı çözümleme için birden fazla sunucuya paralel DNS sorguları etkinleştir».
<code>useSystemHosts</code>	Sistem Hosts dosyasını kullan	<code>dns.useSystemHosts</code>	«Kurulu sistemdeki hosts dosyasını kullan».

dns bloğu örneği. Google etki alanlarına gönderilen istekler Cloudflare DoH sunucusu üzerinden çözülür; geri kalanı — `1.1.1.1` üzerinden; Google istekleri için yalnızca özel olmayan IP'ler beklenir. Yapılandırmanın üst düzeyinde:

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:!private"]
    },
    "1.1.1.1"
  ]
}
```

Alansız dize sunucu ("1.1.1.1"), diğer tüm etki alanları için varsayılan sunucudur. `dns-inbound` etiketi daha sonra yönlendirme kurallarında `inboundTag` olarak kullanılabilir; bu sayede DNS isteklerinin kendisi gerekli outbound üzerinden yönlendirilebilir.

Eski kayıt önbellegi

Alan	Etiket	Açıklama
<code>serveStale</code>	Eskiye kullan	«Arka planda yenileme sırasında önbellegten eski sonuçları döndür».
<code>serveExpiredTTL</code>	Eski TTL	«Eski önbelleg kayıtları için geçerlilik süresi (saniye); 0 = süresiz».

DNS sunucuları (`dns.servers` listesi)

Düğmeler: **DNS Oluştur**, **DNS'i Düzenle**, **Tümünü Sil** (onay: «Tüm DNS sunucuları listeden silinecektir. Bu işlem geri alınamaz.»). Şablonlar: **Şablon Kullan**, **DNS Şablonları** penceresi; **Aile** önayarı dahil.

Bir DNS sunucu kaydındaki **DNS'i Düzenle** düğmesine (Fake DNS kaydında olduğu gibi) tıklandığında düzenleme penceresi, varsayılan değerler yerine kaydedilmiş sunucu değerlerini doldurur.

DNS sunucu alanları:

Alan	Etiket	Açıklama
<code>address</code>	—	DNS adresi (IP, DoH-URL, <code>localhost</code> , <code>fakedns</code> vb.).
<code>domains</code>	Etki Alanları	Bu sunucunun kullanıldığı etki alanlarının listesi.
<code>expectIPs</code>	Beklenen IP'ler	Yalnızca IP listede yer alıyorsa yanıtı kabul et.
<code>unexpectIPs</code>	Beklenmeyen IP'ler	Belirtilen IP'lere sahip yanıtları at.
<code>skipFallback</code>	Yedek'i Atla	Bu sunucuyu yedek olarak kullanma.
<code>finalQuery</code>	Son Sorgu	Sunucuyu zincirde son olarak işaretle.
<code>timeoutMs</code>	Zaman Aşımı (ms)	Sunucuya istek zaman aşımı.

Hosts (statik kayıtlar)

Hosts grubu (`dns.hosts`). **Host Ekle** düğmesi; boş durum **Host tanımlanmamış**. Alanlar: etki alanı (yer tutucu: «Etki alanı (ör. domain:example.com)») ve değerler (yer tutucu: «IP veya etki alanı – girin ve Enter'a basın»).

DNS Günlükleri

Bkz. 11.9: günlük bölümündeki **DNS Günlükleri** bayrağı (`dnsLog`).

11.7. Fake DNS

`fakedns` bölümü. Düğmeler: **Fake DNS Oluştur**, **Fake DNS'i Düzenle**.

Alan	Etiket	Açıklama
<code>ipPool</code>	IP havuzu alt ağı	Sahte IP'lerin dağıtıldığı CIDR aralığı (ör. <code>198.18.0.0/15</code>).
<code>poolSize</code>	Havuz boyutu	Dairesel havuzda tutulacak adres sayısı.

Fake DNS, inbound üzerinde sniffing ile birlikte kullanılır: çekirdek istemciye sahte bir IP verir, etki alanı[?] IP eşleşmesini hatırlar ve yönlendirme sırasında etki alanını geri yükler. Fake DNS'nin çalışması için `fakedns` adresine sahip bir DNS sunucusunun DNS sunucu listesine eklenmesi gerekir.

Örnek: Fake DNS + DNS sunucu kombinasyonu. Önce sahte adresler havuzunu tanımlarız, ardından etki alanı isteklerinin bu havuzdan IP alması için `fakedns` DNS sunucusunu ekleriz:

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

Ek olarak inbound üzerinde `destOverride: ["fakedns"]` ile sniffing'in etkinleştirilmesi gerekir; aksi takdirde çekirdeğin geri yükleme için gerçek etki alanını alacağı yer olmaz.

11.8. WireGuard / WARP / NordVPN

WireGuard alanları (`wireguard`)

Alan	Etiket	Açıklama
<code>secretKey</code>	Gizli anahtar	Yerel arayüzün özel anahtarı.
<code>publicKey</code>	Genel anahtar	Peer'in genel anahtarı.
<code>psk</code>	Paylaşılan anahtar	PreShared Key (isteğe bağlı).

Alan	Etiket	Açıklama
allowedIPs	İzin verilen IP adresleri	Tünele yönlendirilen aralıklar.
endpoint	Uç nokta	Peer'in host:port adresi.
domainStrategy	Etki alanı stratejisi	WireGuard-outbound için çözümleme stratejisi.

Cloudflare WARP (warp)

Entegrasyon <https://api.cloudflareclient.com/v0a4005> API'sini kullanır (client-version a-6.30-3596). Denetleyici eylemleri (/xray/warp/:action): config , reg , license , data , del .

Adım adım:

1. **WARP hesabı oluştur** → reg : panel özel (privateKey) ve genel (publicKey) anahtarları oluşturur/alır, cihazı Cloudflare'e kaydeder ve access_token , device_id , license_key , private_key (ve client_id) değerlerini warp ayarına kaydeder.
2. **WARP / WARP+ lisans anahtarı** → license : 26 karakterlik WARP+ anahtarını ayarlama (yer tutucu: «26 karakterlik WARP+ anahtarı»). Hata durumunda: «WARP lisansı ayarlanamadı.» Yapılandırma henüz alınmamışsa: «Önce WARP yapılandırmasını alın.»
3. **Hesap bilgisi: Cihaz adı, Cihaz modeli, Cihaz etkin, Hesap türü, Rol, WARP+ verisi, Kota, Kullanım.**
4. **Çıkış ekle** – alınan anahtarlar ve Cloudflare uç noktasıyla WireGuard-outbound oluşturur.
5. **Hesabı sil** → del : kaydedilen WARP verilerini temizler.

NordVPN (nord / nordvpn)

Entegrasyon NordLynx (= WireGuard) kullanır. Denetleyici eylemleri (/xray/nord/:action): countries , servers , reg , setKey , data , del .

Adım adım:

1. **Erişim tokeni** → reg : panel api.nordvpn.com 'dan NordLynx kimlik bilgilerini ister ve nordlynx_private_key 'i çıkarır. private_key ve token 'i nord ayarına kaydeder. Alternatif – setKey : **Özel anahtarı** doğrudan girin (boş olamaz).
2. **Ülke** → countries ülke listesini yükler; **Şehir** (veya **Tüm şehirler**).
3. **Sunucu** → servers seçilen ülkenin sunucularını yükler (countryId , enjeksiyonlara karşı koruma olarak sayı olarak doğrulanır). Filtre: yalnızca **Yük** değeri > 7% olan sunucular gösterilir. Sunucu yoksa: «Seçilen ülke için sunucu bulunamadı». Sunucunun NordLynx genel anahtarı yoksa: «Seçilen sunucu NordLynx genel anahtarını bildirmiyor.»
4. **Çıkış oluşturma/güncelleme**: «NordVPN çıkışı eklendi» / «NordVPN çıkışı güncellendi» bildirim mesajları.

IPv4 önceliği ve kullanıcı alanı TUN

WARP ve NordVPN sihirbazları tarafından oluşturulan WireGuard-outbound'lar, ForceIP yerine domainStrategy: "ForceIPv4v6" kullanır (IPv6 yalnızca konaklarda IPv4'e geri dönüşle IPv4 önceliği) – bu, Cloudflare uç noktasının AAAA kaydı seçildiğinde yarı yapılandırılmış IPv6'ya sahip konaklarda «takılma» el sıkışma sorununu ortadan kaldırır. Bunların yanı sıra bunlar için çekirdek TUN yerine kullanıcı alanı TUN

(`noKernelTun: true`) etkinleştirilmiştir: çekirdek TUN, izinler ve fwmark yönlendirmesi gerektirir ve birçok VPS üzerinde sessizce başarısız olur; oysa panelin yerleşik bağlantı testi her zaman kullanıcı alanı TUN üzerinden test eder – artık gerçek trafik ve test aynı yolu izler. Değişiklik yalnızca yeni eklenen veya sıfırlanan outbound'lar için geçerlidir; zaten kaydedilmiş şablonlar kendi ayarlarını korur.

11.9. Reverse-proxy ve TUN

Reverse (ters proxy)

Xray yapılandırmasının `reverse` bölümü. Outbound formunda **Ters Proxy** türüne geçiş düğmesi bulunur. Düğmeler: **Ters Proxy Oluştur**, **Ters Proxy'yi Düzenle**.

Alan	Etiket	Açıklama
Tür	Tür	Bridge veya Portal – Xray ters proxy'nin iki rolü.
Etki alanı	Etki Alanı	Bridge[?]portal çifti için hizmet etki alanı etiketi.
Etiket / Bağlantı	Etiket / Bağlantı	Bridge ve portal bağlantısı için etiketler.
Reverse Tag	Ters proxy etiketi	İpucu: « <i>Basit VLESS ters proxy için çıkış bağlantısı etiketi. Devre dışı bırakmak için boş bırakın.</i> » Yer tutucu: « <i>çıkış etiketi (boş = devre dışı)</i> ». Basitleştirilmiş VLESS reverse'i uygular.

Outbound formunda ayrıca geri akış alanları da bulunur: **Ters sniffing**, **Çalışanlar**, **Ayrılmış**, **Min. yükleme aralığı (ms)**, **Maks. yükleme boyutu (bayt)**.

TUN (`tun`)

Alan	Etiket	Açıklama	Varsayılan
name	–	« <i>TUN arayüzünün adı.</i> »	xray0
mtu	–	« <i>Maksimum iletim birimi. Veri paketlerinin maksimum boyutu.</i> »	1500
<code>userLevel</code>	Kullanıcı düzeyi	« <i>Bu giriş akışı üzerinden kurulan tüm bağlantılar bu kullanıcı düzeyini kullanacaktır.</i> »	0

11.10. Günlükler ve istatistikler (Stats, metrics)

Günlük (`log`)

İpucu: «*Günlükler sunucu performansını yavaşlatabilir. Yalnızca gerektiğinde ihtiyaç duyduğunuz günlük türlerini etkinleştirin!*» Referans şablon `log` bölümü: `access: "none", error: "", loglevel: "warning", dnsLog: false, maskAddress: ""`.

Alan	Etiket	JSON	Açıklama	Varsayılan
<code>logLevel</code>	Günlük düzeyi	<code>loglevel</code>	« <i>Hata günlükleri için günlük düzeyi...</i> » Değerler: <code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code> , <code>none</code> .	warning

Alan	Etiket	JSON	Açıklama	Varsayılan
accessLog	Erişim günlükleri	access	«Erişim günlüğü dosyasının yolu. «none» özel değeri erişim günlüklerini devre dışı bırakır.»	none
errorLog	Hata günlükleri	error	«Hata günlüğü dosyasının yolu. «none» özel değeri hata günlüklerini devre dışı bırakır.»	"" (varsayılan)
dnsLog	DNS Günlükleri	dnsLog	«DNS sorgu günlüklerini etkinleştir»	false
maskAddress	Adres maskeleye	maskAddress	«Etkinleştirildiğinde gerçek IP adresi günlüklerde maskeleye adresiyle değiştirilir.»	"" (devre dışı)

İstatistikler (stats / policy)

İstatistikler grubu. `policy.system` ve `policy.levels` içinde sayaçları etkinleştirir. Referans şablonunda: `statsInboundUplink: true , statsInboundDownlink: true , statsOutboundUplink: false , statsOutboundDownlink: false ; 0.düzy için – statsUserUplink: true , statsUserDownlink: true .`

Alan	Etiket	Açıklama	Varsayılan
statsInboundUplink	Giriş uplink istatistiği	«Tüm giriş proxy'lerinin çıkış trafiği için istatistik toplamayı etkinleştir.»	true
statsInboundDownlink	Giriş downlink istatistiği	«Tüm giriş proxy'lerinin giriş trafiği için istatistik toplamayı etkinleştir.»	true
statsOutboundUplink	Çıkış uplink istatistiği	«Tüm çıkış proxy'lerinin çıkış trafiği için istatistik toplamayı etkinleştir.»	false
statsOutboundDownlink	Çıkış downlink istatistiği	«Tüm çıkış proxy'lerinin giriş trafiği için istatistik toplamayı etkinleştir.»	false

İstemciler ve inbound'lar için trafik istatistikleri (uplink/downlink), panodaki ve istemcilerdeki trafik gösteriminin temelini oluşturur; devre dışı bırakılması önerilmez. Outbound istatistikleri varsayılan olarak kapalıdır ve yalnızca çıkış etiketlerine göre trafiği izliyorsanız gereklidir.

Metrics

Referans şablonunda `metrics` bölümü (`listen: "127.0.0.1:11111"` , `tag: "metrics_out"`) ve karşılık gelen API `metrics_out` bulunur. Panel, bu listener'ı metrik ve gözlemevi anlık görüntüleri toplamak için kullanır; şablondan `metrics.listen` 'ı ayırıştırır, `/debug/vars` 'ı sorgular ve etiketlere göre gecikme geçmişini toplar. `metrics.listen` adresini/portunu değiştirirseniz panel yeni adrese başvurur; `metrics` bölümünü silmek gözlemevi grafiklerinin toplanmasını devre dışı bırakır.

HTTP modunda outbound testi, rastgele bir portta kendi `metrics` -listener'ına sahip ayrı bir geçici Xray örneği başlatır – bu, ana yapılandırmadaki listener ile aynı değildir.

11.11. Kaydetme, yeniden başlatma ve otomatik dönüşümler

Düğmeler

Düğme	Eylem
Kaydet	POST /xray/update : şablonu + outboundTestUrl doğrular ve kaydeder.
Xray'i Yeniden Başlat	Hizmeti kaydedilmiş yapılandırma ile yeniden yükler. Onay: «Xray'i yeniden başlat?» / «Xray hizmetini kaydedilmiş yapılandırma ile yeniden yükler.»

Bildirim mesajları: başarı – «Xray başarıyla yeniden başlatıldı», «Xray başarıyla durduruldu»; hatalar – «Xray yeniden başlatılırken bir hata oluştu.», «Xray durdurulurken bir hata oluştu.» **Xray Yeniden Başlatma Çıktısı** penceresi, çekirdeğin tanılama çıktısını gösterir.

Değişikliklerin sıcak uygulanması (tam yeniden başlatma olmadan)

Inbound'lar, outbound'lar ve yönlendirme kurallarındaki değişiklikler «canlı» olarak uygulanır: **Kaydet** düğmesine basıldığında panel, eski ve yeni yapılandırma arasındaki farkı hesaplar ve yalnızca değişen parçaları gRPC-API Xray (HandlerService/RoutingService) üzerinden uygular, süreci yeniden başlatmadan. Tam yeniden başlatma yalnızca sıcak yeniden yükleme API'si olmayan bölümler değiştiğinde (log , dns , policy , observatory vb.) otomatik olarak gerçekleştirilir. Bu nedenle Xray sayfasında ayrı bir «Yeniden Başlat» düğmesi gerekmez – **Kaydet** değişiklikleri kendisi uygular. Çekirdeğin gerektiğinde yeniden başlatılması otomatik olarak gerçekleştirilmeye devam eder (abonelik güncellemeleri ve WARP rotasyonunda otomatik yeniden yüklemeye de bakın).

Varsayılan şablona sıfırlama

GET /xray/getDefaultJsonConfig uç noktası, referans şablonu (config.json , ikili dosyaya gömülü) döndürür. Bu sayede yapılandırma fabrika ayarlarına sıfırlanabilir.

Kaydederken otomatik dönüşümler

Xray ayarları kaydedilirken panel şunları gerçekleştirir (bu sırayla):

- Sarmalayıcıları kaldırma** – { "xraySetting": <yapılandırma>, "inboundTags": ..., "outboundTestUrl": ... } gibi sarmalayıcıları kaldırır (aksi takdirde her kaydetmede katmanlar birikirdi). 8 katmana kadar kaldırılır.
- Yapılandırma doğrulama** – JSON, Xray yapılandırma yapısına ayrıştırılır; hata durumunda – «xray template config invalid» ile reddedilir.
- İstatistik kuralı güvencesi** – inboundTag: ["api"] → outboundTag: "api" kuralı, routing.rules içinde zorunlu olarak 0. konuma yükseltilir (veya yoksa eklenir). Bu, panel gRPC istatistik isteğinin üstteki

bir catch-all kuralı tarafından ele geçirilmemesini garanti eder (aksi takdirde proxy çalışırken istemciler sıfır trafik ile çevrimdışı görünebilir).

1. madde nedeniyle `api` → `api` kuralını kaldırmaya veya taşımaya çalışmayın – panel bir sonraki kaydetmede onu yerine geri getirecektir. Bu, bir kullanıcı rotası değil, istatistik altyapısı hizmetidir.

11.12. Abonelikten outbound (otomatik güncellemeyle)

3.3.0 sürümünden itibaren panel, outbound 'ları doğrudan abonelik URL'sinden içe aktarabilir – VPN sağlayıcılarının istemci uygulamaları için sunduğuyla aynı formatta. Abonelikler arka planda düzenli olarak yeniden okunur, bu nedenle sunucudaki outbound kümesi, yapılandırma şablonu manuel olarak düzenlenmeden güncel tutulur.

Bölüm «**Çıkış Abonelikleri**» olarak adlandırılmış olup açıklaması şöyledir: «Uzak abonelik URL'lerinden çıkışları içe aktar (vmess/vless/trojan/ss/...). Etiketler, yük dengeleyicilerde ve yönlendirme kurallarında kullanılmak üzere değişmez kalır. Güncelleme otomatik olarak gerçekleştirilir.» Bölüm, outbound ayar panelinin üzerinde Xray sayfasında yer alır.

Nasıl çalışır

Abonelikler, Xray yapılandırma şablonundan ayrı olarak saklanır. Şablon **hiçbir zaman üzerine yazılmaz**: aboneliklerden alınan outbound 'lar, her Xray yapılandırması oluşturulduğunda nihai yapılandırmaya anında eklenir.

Abonelik ekleme

«Abonelik ekle» formunda şu alanlar mevcuttur:

Alan	Anahtar	Varsayılan	Amaç
Abonelik URL'si	<code>url</code>	– (zorunlu)	Abonelik adresi. Yer tutucu: «https://... (base64 ile kodlanmış bağlantı listesi)». Yalnızca HTTP(S) kabul edilir; adres güvenlik açısından doğrulanır.
Not	<code>remark</code>	boş	İsteğe bağlı etiket (yer tutucu «ör. HK düğümleri»).
Etiket öneki	<code>tagPrefix</code>	<code>subN-</code>	İçe aktarılan outbound etiketlerinin başladığı önek. Boş bırakılırsa panel, <code>sub1-</code> , <code>sub2-</code> vb. biçiminde en küçük boş numarayı kendisi seçer.
Güncelleme aralığı	<code>updateInterval</code>	600 saniye (10 dakika)	Aboneliğin ne sıklıkla yeniden okunacağı. Kullanıcı arayüzünde saat/dakika cinsinden ayarlanır.
Etkin	<code>enabled</code>	evet (<code>true</code>)	Yalnızca etkinleştirilmiş abonelikler yapılandırmaya dahil edilir ve otomatik olarak güncellenir.
Özel adreslere izin ver	<code>allowPrivate</code>	hayır (<code>false</code>)	localhost, LAN ve özel IP'lerdeki URL'lere izin verir. SSRF'ye karşı koruma amacıyla varsayılan olarak kapalıdır – yalnızca güvenilen yerel kaynak için etkinleştirin.

Alan	Anahtar	Varsayılan	Amaç
Manuel çıkışlardan önce	prepend	hayır (false)	Etkinleştirilirse bu aboneliğin outbound 'ları şablonun manuel outbound 'larından önce yerleştirilir ve bunlardan biri varsayılan outbound olabilir. Aksi takdirde sonra eklenir.

«Önizleme» düğmesi (POST /outbound-subs/parse), kaydetmeden önce URL'yi indirip ayrıştırmanıza ve hangi outbound 'ların ve etiketlerin oluşacağını görmeye olanak tanır; bu işlemde veritabanına hiçbir şey yazılmaz. URL'den hiçbir şey tanınmazsa «Bu URL'den çıkış bulunamadı.» mesajı görüntülenir.

Birden fazla aboneliğin genel outbound listesindeki sırası öncelik (priority) ile belirlenir ve yukarı/aşağı okları (POST /outbound-subs/:id/move) ile değiştirilir.

Hangi abonelik biçimleri kabul edilir

URL'deki yanıt gövdesi şu şekilde işlenir:

- İçerik önce **base64** olarak denir (standart ve URL-güvenli varyantlar, otomatik dolgu tamamlama ve boşluk/satır sonu kaldırma ile). Bu base64 ise kodu çözülür; değilse olduğu gibi alınır.
- Ardından gövde satırlara ayrılır. # ile başlamayan her boş olmayan satır bir bağlantı olarak ayrıştırılır. Tanınmayan satırlar (yorumlar, desteklenmeyen protokoller) sessizce atlanır.
- Desteklenen bağlantı şemaları: vmess:// , vless:// , trojan:// , ss:// (Shadowsocks), hysteria2:// / hy2:// , wireguard:// / wg:// .

Yani çoğu sağlayıcıda olduğu gibi «base64 ile kodlanmış bağlantı listesi» biçimindeki normal abonelik uygundur.

Kararlı etiketler

Her bağlantı için kararlı bir «kimlik» hesaplanır (fragment-notunun dışındaki URI çekirdeği; vmess için ps alanı olmadan dahili JSON). «Kimlik → etiket» eşleşmesi korunur ve bir sonraki güncellemede not veya ikincil parametreler değişmiş olsa bile aynı sunucu aynı etiketi alır. Bu, güncellemeler sonrasında yük dengeleyicilerin ve yönlendirme kurallarının çalışmaya devam etmesi için özel olarak yapılmıştır:

- Yük dengeleyicide/kuralda tam etiket aynı sunucuyu işaret etmeye devam edecektir.
- Önek/wildcard seçici (ör. hk-*), aboneliğin daha sonra döndürdüğü yeni sunucuları otomatik olarak algılar – bu, «bir havuza abone olmak» için önerilen yoldur.
- Bir sunucu abonelikten kaybolursa etiketi nihai outbound dizisinden sadece düşer; yük dengeleyicide fallbackTag varsa Xray bunu kullanır.
- Sağlayıcı sunucunun UUID/host/kimlik bilgilerini değiştirirse kimlik değişir – bu yeni bir etikete sahip yeni bir outbound olarak kabul edilir.

Tek bir indirmede etiketler -N soneki ile tekilleştirilir. Abonelik etiketleri ASCII olmayan karakterleri (ör. Kiril) korur ve okunabilir kalır: Unicode harfler ve rakamlar slug'da korunurken noktalama işaretleri kısa çizgiyle değiştirilir – Kiril adlarından gelen etiketler artık yalnızca rakamlara indirgenmez.

Otomatik güncelleme nasıl çalışır

- Abonelik güncelleme arka plan görevi **her 5 dakikada bir** çalışır.
- Her çalışmada etkinleştirilmiş tüm abonelikleri gözden geçirir ve yalnızca kendi aralığı dolmuş olanları günceller: bir abonelik henüz hiç güncellenmemişse veya son güncellemeden bu yana en az kendi `updateInterval` süresi geçmişse güncellenir. Bu sayede görev abonelikleri sık sık kontrol eder, ancak her bir abonelik kendi `updateInterval` değerinden (varsayılan 10 dakika) daha sık okunmaz. Bu durum kullanıcı arayüzünde ilgili ipucuyla yansıtılır.
- Güncelleme: URL, genel URL olarak güvenlik açısından yeniden doğrulanır (özel adresler, abonelikte `allowPrivate` ayarlanmamışsa engellenir), istek `User-Agent: 3x-ui-outbound-sub/1.0` başlığıyla panel proxy istemcisi üzerinden gider. Yönlendirme zinciri 10 atlamayla sınırlıdır ve her atlama da özel adresler açısından doğrulanır (SSRF koruması). HTTP 200 beklenir; aksi takdirde hata kaydedilir.
- Başarılı ayrıştırımdan sonra sonuç kaydedilir, son güncelleme zamanı ayarlanır, hata temizlenir. Hata durumunda hata metni kullanıcı arayüzünde «Son hata» olarak görünür ve önceden alınan `outbound` 'lar geçerliliğini korur.
- En az bir abonelik gerçekten güncellenirse görev, Xray'i yeniden başlatma gerektirecek şekilde işaretler ve arayüzün yeni `outbound` 'ları alması için kullanıcı arayüzü geçersiz kılma gönderir. Xray'in gerçek yeniden yüklenmesi, yöneticinin en yakın 30 saniyelik döngüsünde gerçekleşir.

Tek bir aboneliğin manuel güncellenmesi – «**Şimdi güncelle**» düğmesi (`POST /outbound-subs/:id/refresh`); bu da Xray'i yeniden başlatma gerektirecek şekilde işaretler. Abonelik ekleme, değiştirme, silme işlemleri de Xray yeniden başlatma bayrağını devreye alır (silme işleminde `outbound` 'ları bir sonraki yeniden yüklemede yapılandırmadan düşer). Kullanıcı arayüzü şunu bildirir: «Ekledikten veya güncelledikten sonra çıkışların etkin olması için Xray'i yeniden başlatın (veya bir sonraki otomatik yeniden yüklemeyi bekleyin).»

Xray yapılandırmasına nasıl dahil edilir

Her Xray yapılandırması oluşturulduğunda etkin abonelik `outbound` 'ları iki gruba ayrılır – `prepend` («Manuel çıkışlardan önce» bayrağı) ve diğerleri – ve şablonla birleştirilir: `[abonelik prepend] + [şablon outbound'ları] + [diğer abonelikler]`. Her grup içinde abonelikler önceliğe göre sıralanır. Şablonun manuel `outbound` 'ları bu süreçte etkilenmez; şablonun `outbound` dizisi bir nedenden dolayı ayrıştırılamazsa abonelik `outbound` 'ları buna karıştırılmaz (manuel olanları kaybetmemek için).

İçe aktarılan `outbound` 'lar ayrıca `outbound` panelinde ayrı bir «**Çıkış aboneliklerinden (salt okunur)**» bloğu olarak görüntülenir – bunlar orada düzenlenemez, yönetim yalnızca «Çıkış Abonelikleri» bölümü üzerinden yapılır.

11.13. WARP'ta IP rotasyonu

3X-UI'de bir WARP-outbound kurulabilir – Cloudflare WARP'a giden WireGuard çıkış bağlantısı (Xray yapılandırmasında `warp` etiketi). Panel, Cloudflare sunucularına bir cihaz hesabı kaydeder, WireGuard anahtarlarını ve adreslerini alır ve bunları `warp` etiketli outbound'a ekler. Bu tür bir outbound üzerinden trafik, Cloudflare WARP IP adresi altında internete çıkar. 3.3.0 sürümündeki yenilik – bu çıkış IP'sini WARP hesabını manuel olarak yeniden oluşturmadan manuel olarak veya zamanlamaya göre değiştirebilme imkânı.

Yönetim, **Xray** bölümündeki WARP kartında bulunur («WARP Hesabı Oluştur» düğmesine tıklayıp yapılandırmayı aldıktan sonra; bu işlem tamamlanmadan eylemler kullanılamaz – panel «Önce WARP yapılandırmasını alın» uyarısı gösterir).

IP değiştirildiğinde ne olur

«IP'yi Değiştir» düğmesi IP değişimini başlatır. Mantık:

1. Yeni bir WireGuard anahtar çifti oluşturulur.
2. Yeni anahtarla Cloudflare sunucularına yeni bir WARP cihazı kaydedilir (yeni `device_id` , `access_token` , adresler ve peer verileri).
3. Yeni veriler Xray yapılandırmasının WARP-outbound'una yazılır: `secretKey` , `address` (v4 /32 ve v6 /128), `reserved` (`client_id` 'den), ayrıca peer'in `publicKey` 'i ve `endpoint` 'i güncellenir.
4. Daha önce 26 karakterden uzun bir WARP+ lisans anahtarı ayarlanmışsa yeni hesaba otomatik olarak yeniden yüklenir. Başarısız olursa bu yalnızca günlüklerde bir uyarıdır – IP değişimi iptal edilmez.
5. Başarılı değişim sonrasında yeni outbound'un geçerli olması için Xray yeniden başlatma gerektirecek şekilde işaretlenir.

Başarı durumunda arayüz «WARP IP adresi başarıyla değiştirildi!» mesajını gösterir.

Zamanlamaya göre otomatik rotasyon

WARP kartında bir «**IP adresini otomatik güncelle**» geçiş düğmesi ve «**Aralık (gün)**» alanı bulunur. İpucu: «0 – devre dışı bırak. IP adresini otomatik olarak değiştirir.»

Parametre	Değer
Veritabanı ayarı	<code>warpUpdateInterval</code> (tamsayı, ≥ 0)
Varsayılan değer	0 (otomatik rotasyon devre dışı)
Ölçü birimi	gün
0	otomatik değişimi devre dışı bırakır
> 0	IP'yi her N günde bir değiştir

Aralık kaydedildiğinde `warpUpdateInterval` saklanır ve değer 0'dan büyükse «son güncelleme zamanı» mevcut ana sıfırlanır – aksi takdirde zamanlayıcı IP'yi zaten en yakın tikde değiştirirdi.

Zamanlama, saatte bir çalışan bir arka plan görevi tarafından yürütülür – yani panel, rotasyon zamanının gelip gelmediğini saatte bir kontrol eder. Kontrol algoritması:

- aralık ≤ 0 ise – hiçbir şey yapmaz;
- «son güncelleme zamanı» 0'a eşitse (örn. aralık veritabanına doğrudan yazılarak ayarlanmışsa) – bu ilk çalıştırmadır: görev yalnızca temel zaman damgasını kaydeder ve IP'yi hemen değiştirmez;
- son güncellemeden bu yana `aralık  $\times 24 \times 3600$` saniyeden fazla geçtiyse – aynı IP değişimi gerçekleştirilir, zaman damgası güncellenir ve Xray yeniden başlatma zamanlanır.

Önemli bir ayrıntı: «IP'yi Değiştir» düğmesiyle manual rotasyon da son güncelleme zaman damgasını sıfırlar. Bu nedenle manuel rotasyon sonrasında otomatik aralık sayacı yeniden başlar ve planlı değişim hemen arkasından tetiklenmez.

«Panel proxy'si üzerinden»

3.3.1'de değiştirildi. Ayrı «Panel Ağ Proxy'si» (`panelProxy`) ayarı kaldırıldı. Panelin kendi giden trafiği (WARP API'ye yapılan istekler dahil) artık seçilen **panel trafiği outbound'u** – Xray-outbound veya yük dengeleyici – üzerinden yönlendirilir (bkz. bölüm 13). Aşağıdaki açıklama 3.3.1 öncesi sürümler için geçerlidir.

Cloudflare WARP API'ye yapılan tüm istekler (kayıt, yapılandırma alma, lisans ayarlama, IP değiştirme), doğrudan değil 15 saniyelik zaman aşımli panel HTTP istemcisi üzerinden gider. Bu istemci, panel ayarlarından «**Panel Ağ Proxy'si**» (`panelProxy`) ayarına uymaktadır.

Ayar açıklamasından: proxy, panelin kendi giden isteklerini yönlendirir (coğrafi veri güncellemeleri, Xray/panel sürüm kontrolleri, Telegram ve artık WARP çağrıları) – sunucu filtrelerini aşmak için. `socks5://` veya `http(s)://` biçimindeki adresler kabul edilir; örneğin Xray'in kendi yerel SOCKS girişi. Alan boşsa veya proxy hatalı ayarlanmışsa doğrudan bağlantı kullanılır (davranış bozulmaz).

WARP için faydası: sunucu `api.cloudflareclient.com` 'a doğrudan erişemiyorsa kayıt ve rotasyon daha önce başarısız oluyordu. Artık `panelProxy` içinde çalışan bir proxy (Xray'in kendi inbound'u dahil) belirterek, hem manuel düğmenin hem de planlı rotasyonun çalışmasını garanti eden WARP API erişilebilirliğini sağlayabilirsiniz.

Bu ne zaman kullanışlıdır

- WARP üzerinden giden outbound için giden IP'nin düzenli olarak değiştirilmesi – tek bir adres üzerinden engelleme ve takip riskini azaltır.
- Mevcut Cloudflare adresi kara listelere girmiş veya yavaş çalışıyorsa IP'yi manuel olarak «tazeleyin».
- Cloudflare WARP API'ye doğrudan erişimi olmayan sunucular: istekleri `panelProxy` üzerinden yönlendirmek, kayıt ve rotasyonu çalışır hale getirir.

12. Düğümler (çok panelli yapı, master/slave)

Düğümler bölümü, standart bir 3X-UI kurulumunu, diğer (alt) 3X-UI panellerini uzaktan izleyen ve yöneten **merkezi (master) bir panele** dönüştürür. Her düğüm, kendi sunucusunda ayrı bir 3X-UI kurulumudur; master, kendi HTTP API'si aracılığıyla düğüme bağlanır, durumunu sorgular ve kendisine atanmış inbound'ları ile istemcileri senkronize eder. Bu, **çok panelli yapı** özelliğidir: her panele ayrı ayrı girmek yerine tüm sunucuları tek bir listede görür ve merkezi olarak yönetirsiniz.

Önemli bir ilke: **düğüm bir ajan değil, tam teşekküllü bir 3X-UI panelidir**. Master üzerine hiçbir şey "yüklemes" – yalnızca bir token aracılığıyla API'sine bağlanır. Düğümü listeden silmek yalnızca izlemeyi durdurur; uzak panelin kendisi bu işlemten etkilenmez (ipucu: «Bu işlem düğüm izlemesini durduracak. Uzak panelin kendisi etkilenmeyecek»).

12.1. Liste üstündeki özet

Düğüm tablosunun üzerinde toplu sayaçlar görüntülenir:

Alan	Açıklama
Toplam düğüm	Listedeki toplam düğüm sayısı.
Çevrimiçi	online durumundaki düğüm sayısı.
Çevrimdışı	offline durumundaki düğüm sayısı.
Ortalama gecikme	Düğümlere ortalama gecikme (ping), milisaniye cinsinden.

12.2. Düğüm ekleme ve düzenleme

Düğüm Ekle ve **Düğümü Düzenle** düğmeleri, düğüm alanlarını içeren bir form açar.

Ad, Adres, Port ve **API Token** alanları zorunludur (ipucu: «Ad, adres, port ve API token zorunludur»).

«Kaydet» düğmesine basıldığında (hem eklerken hem de düzenlerken) panel **önce 6 saniyelik zaman aşımıyla düğüme erişilebilirliği kontrol eder**. Düğüm yanıt vermezse kayıt yapılmaz ve hata gösterilir. Yani erişilemeyen bir düğüm eklenemez.

Form alanları

Alan (TR)	Varsayılan	Geçerli değerler	Açıklama
Ad	– (zorunlu)	boş olmayan, benzersiz dize	Düğümün dahili adı. Ad sütununa benzersizlik kısıtı uygulanır – aynı adla iki düğüm oluşturulamaz. Yer tutucu ipucu: örn. de-frankfurt-1 . Kaydedilirken baştaki ve sondaki boşluklar kırılır.
Not	boş	herhangi bir dize	İsteğe bağlı not/düğüm açıklaması. Çalışmayı etkilemez.

Alan (TR)	Varsayılan	Geçerli değerler	Açıklama
Şema	https	http / https	Uzak panele bağlantı protokolü. Boş bırakılır veya geçersiz değer belirtilirse normalizasyon https olarak ayarlar. Düğüm sıradan HTTP üzerinden yanıt verirken şema https olarak ayarlıysa panel anlaşılır bir ipucu döndürür: «the server speaks HTTP, not HTTPS; set the node scheme to http».
Adres	– (zorunlu)	host veya IP	Uzak panelin adresi. Yer tutucu: panel.example.com veya 1.2.3.4. Adres normalize edilir; varsayılan olarak özel/yerel adresler SSRF koruması nedeniyle yasaktır – bkz. «Özel adrese izin ver».
Port	– (zorunlu)	1–65535 tam sayı	Uzak düğümün web paneli portu. Aralık dışındaki değerler reddedilir («node port must be 1-65535»).
Temel yol	/	yol dizesi	Ayarlanmışsa uzak panelin temel yolu (web base path). Normalize edilir: başında ve sonunda / olması garanti edilir (boş değer → /). Panel, sorgulama sırasında buna panel/api/server/status ekler.
API Token	– (zorunlu)	uzak panelin token'ı	Düğüm API'sine erişim için Bearer token. Authorization: Bearer <token> başlığında iletilir. Yer tutucu: «Uzak panelin Ayarlar sayfasından alınan token». İpucu: «Uzak panel, API token'ını Ayarlar → API Token bölümünde gösterir». Yani token'ın düğümün kendisinde oluşturulması gerekir (Ayarlar → API Token), ardından buraya yapıştırılır.
Etkin	true	evet/hayır	Düğüm izleme ve senkronizasyonunu etkinleştirir. Devre dışı bırakılan düğümler arka plan görevleri (heartbeat ve traffic-sync) tarafından sorgulanmaz ve toplu panel güncellemesine dahil edilmez.
Özel adrese izin ver	false	evet/hayır	SSRF korumasını kaldırır ve özel/yerel adres üzerinden düğüme bağlanmaya izin verir. İpucu: «Yalnızca özel ağdaki veya VPN üzerindeki düğümler için etkinleştirin». Yalnızca düğüm gerçekten özel ağdaysa veya VPN üzerinden erişilebiliyorsa etkinleştirin.

Düğüm tarafında token alma ve yeniden oluşturma

Token, uzak panelin **Ayarlar** → **API Token** bölümünden alınır. Orada yeniden oluşturulabilir: **Token'ı yeniden oluştur** düğmesiyle birlikte şu uyarı gösterilir: «Yeniden oluşturma mevcut token'ı geçersiz kılar. Kullanan herhangi bir merkezi panel, güncellenene kadar erişimini kaybeder. Devam edilsin mi?». Yeniden oluşturma'nın ardından master paneldeki eski token çalışmayı durduracaktır – düğüm formunda güncellenmesi gerekir.

Giden bağlantı (Connection outbound)

Connection outbound (Giden bağlantı, outboundTag) alanı, masterın bu düğümün API'ye yaptığı isteklerin sunucudan nasıl çıkacağını belirler. Bir Xray outbound etiketi seçilirse panel'in düğüme yaptığı istekler doğrudan değil, belirtilen outbound üzerinden gider; panel otomatik olarak çalışan yapılandırmaya loopback köprü inbound'u ekler ve yeniden başlatma olmaksızın anında uygular. İpucu: «Route this node's panel API traffic through the selected Xray outbound. A loopback bridge inbound is added to the running config automatically and applied live. Leave empty for a direct connection».

Seçici, panel outbound seçimi gibi çalışır: etiketler **Outbounds** (normal giden) ve **Balancers** (yük dengeleyiciler) olarak gruplandırılır, blackhole outbound'ları listeden gizlenir. Boş değer (yer tutucu «Direct connection») = düğüme doğrudan bağlantı anlamına gelir.

Inbound içe aktarma (senkronize edilecek inbound'ları seçme)

Düğüm formunda **Inbound içe aktar** (`inboundSyncMode`) ayarı bulunur ve iki mod sunar: **Tüm inbound'lar** (`all` , varsayılan) ve **Seçilenler** (`selected`). Varsayılan olarak master, bu düğümün seçili olduğu tüm inbound'ları düğüme senkronize eder; mevcut düğümler «Tüm inbound'lar» modunda çalışmaya devam eder.

Seçilenler modunda alanın altında inbound etiketleri için çoklu seçim alanı görünür. **Inbound'ları yükle** düğmesine basın – master, henüz kaydedilmemiş bağlantı parametrelerini kullanarak düğümden inbound listesini ister (`POST /panel/api/nodes/inbounds` uç noktası) ve etiketleri gösterir; istenenler işaretlenir. Panel yalnızca işaretli etiketleri düğüme senkronize edip dağıtır; düğümden doğrudan mevcut olan diğer inbound'lar dokunulmadan kalır – master bunları silmez ve yönetmez.

Örnek: seçici içe aktarma için düğümden inbound listesi isteme. Gövdede henüz kaydedilmemiş bağlantı parametreleri iletilir; yanıtta düğümden mevcut inbound etiketleri yer alır:

```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. TLS doğrulama (https düğümleri için)

Alan grubu, masterın düğümün HTTPS sertifikasını nasıl doğrulayacağını belirler. Bu ayarlar **yalnızca https şeması için geçerlidir**; `http` düğümleri için yok sayılır.

TLS Doğrulama – açılır liste, ipucu: «Panelin düğümün HTTPS sertifikasını nasıl doğruladığı. Sabitleme veya Atla – otomatik imzalı sertifikalar için (yalnızca https düğümleri)».

Mod (TR)	Değer	Varsayılan	Açıklama
Doğrula (standart CA)	<code>verify</code>	evet (varsayılan)	Güvenilir CA ile sertifika zincirinin standart doğrulaması. Ortak/Let's Encrypt sertifikalı düğümler için uygundur. Tüm <code>http</code> düğümleri için de kullanılır.
Sertifikayı sabitle (SHA-256)	<code>pin</code>	–	CA zinciri doğrulanmaz, ancak düğümün yaprak sertifikasının SHA-256'sı kaydedilen parmak iziyle karşılaştırılır (sabit zamanlı karşılaştırma). Otomatik imzalı sertifikalar için ortadaki adam saldırısından korunmayı sürdürür. Parmak izi alanının doldurulmasını gerektirir.
Doğrulamayı atla	<code>skip</code>	–	Sertifika doğrulama tamamen devre dışı bırakılır. Uyarı: «Doğrulamayı atlamak, ortadaki adam saldırısına karşı korumayı kaldırır – API token ele geçirilebilir. Sertifikayı sabitlemeyi tercih edin».

Yukarıdaki üç moda ek olarak 3.4.0'da dördüncü bir mod eklendi: **Mutual TLS (istemci sertifikası)** (`mtls`), diğerleri gibi yalnızca `https` şeması için kullanılabilir.

Mod (TR)	Değer	Varsayılan	Açıklama
Mutual TLS (istemci sertifikası)	mtls	–	Düğümün sertifikasını doğrulamanın yanı sıra, master kendisini kendi CA'sı tarafından verilen istemci sertifikasıyla düğüme karşı doğrular. Bu modda düğüm için API token isteğe bağlı hale gelir – düğüm masteri sertifikaya göre tanır. Mod seçildiğinde şu ipucu gösterilir: «This node authenticates the panel with a client certificate. Copy this panel's CA from the Node mTLS section onto the node, set its Trusted parent CA, then restart it».

Bir düğüm için mutual TLS'yi etkinleştirmek üzere: düğüm tarafında **Mutual TLS** modunu ayarlayın, yönetici panelin CA'sını **Node mTLS** bölümünden (aşağıya bakın) kopyalayın, bunu düğümde **güvenilen üst CA** olarak yapılandırın ve düğümü yeniden başlatın.

skip, pin veya mtls dışında bir değer seçilirse normalizasyon verify olarak zorlar.

Sertifika sabitleme

Sertifikayı sabitle seçildiğinde şunlar görünür:

- **Sabitlenmiş sertifikanın SHA-256'sı** – giriş alanı. Parmak izi **base64** biçiminde (Xray'den `pinnedPeerCertSha256` biçimi) ya da iki nokta üst üste ile veya onsuz **hex** biçiminde (`openssl -fingerprint` stili) kabul edilir. İpucu: «Düğümün SHA-256 sertifikası base64 veya hex formatında. Şu anda düğümde okumak için «Al» düğmesine basın». Yer tutucu: «SHA-256 base64 veya hex formatında». `pin` seçildiğinde boş veya hatalı parmak izi, kaydetme sırasında doğrulama hatasına neden olur.

Örnek: aynı parmak izinin iki farklı biçimi. Alan her iki varyantı da kabul eder – her ikisi de aynı sertifikayı temsil eder:

```
# base64 (Xray'den pinnedPeerCertSha256 biçimi)
607TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# iki nokta üst üste ile hex (openssl x509 -fingerprint -sha256 stili)
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```

Parmak izi henüz bilinmiyorsa **Al** düğmesine basın – master onu HTTPS üzerinden düğümde okuyup alana yapıştırır.

- **Al** düğmesi – sertifika doğrulama olmaksızın HTTPS üzerinden düğüme bağlanır ve mevcut yaprak sertifikasının SHA-256'sını okur (`POST /certFingerprint` uç noktası), alana yapıştırır. Başarı durumunda – «Düğümün mevcut sertifikası alındı»; başarısız olursa – «Sertifika alınamadı». Yalnızca https düğümleri için kullanılabilir.

Node mTLS (paneller arası karşılıklı TLS kimlik doğrulama)

Düğümler sayfasında ayrı bir **Node mTLS** bölümü bulunur – «panel → düğüm» çağrıları için API token'ına ikinci faktör (istemci sertifikası) ekleyen karşılıklı TLS kimlik doğrulama ayarı. Karşılıklı TLS isteğe bağlıdır; bölüm alanları boşsa düğümler önceki şemayla çalışmaya devam eder – **yalnızca API token ile** (ipucu: «Mutual

TLS adds a client-certificate factor on top of the API token for node-to-node calls. It is opt-in: leave it empty to keep token-only auth»). Bölümde iki işlem bulunur:

- **Bu panelin CA'sını kopyala** (`POST /panel/api/nodes/mtls/ca`) – bu panelin kök sertifikasını (CA) panoya kopyalar. Bu CA'nın yönetilen düğümlere iletilmesi gerekir; böylece panelin istemci sertifikasına güvenirlir; ardından düğümlerde TLS doğrulama modu **Mutual TLS** olarak ayarlanır (ipucu: «Hand this CA to the nodes this panel manages, then set their TLS verification to Mutual TLS»). Kopyalandıktan sonra – «CA certificate copied to clipboard».
- **Güvenilen üst CA** (`Trusted parent CA` , `POST /panel/api/nodes/mtls/trustCA`) – bu panelin kendisinin bir üst (yönetici) panel için düğüm olarak kullanıldığı durumlarda kullanılan alan. Yönetici panelin CA'sını buraya yapıştırın; bu sayede istemci sertifikası zorunlu kılınır ve **Save trust CA** düğmesine basın. Değişiklik **panelin yeniden başlatılmasını** gerektirir (ipucu: «When this panel is itself a node, paste the managing panel's CA here to require its client certificate. Restart the panel to apply»).

12.4. Her düğüm için gösterilen bilgiler

Tablo sütunları ve düğüm kartı alanları (her heartbeat sorgulamasında doldurulan gözlemlenen durum):

Alan (TR)	Açıklama
Durum	<code>online</code> / <code>offline</code> / <code>unknown</code> – aşağıya bakın.
CPU	Uzak sunucunun işlemci yükü yüzde olarak.
Bellek	RAM kullanımı yüzde olarak (<code>current/total*100</code> olarak hesaplanır).
Çalışma süresi	Sunucunun kesintisiz çalışma süresi (saniye cinsinden).
Gecikme	Düğümün son sorgulamaya yanıt süresi (ms).
Son ping	Son başarılı heartbeat zamanı (unix saniyesi; <code>0</code> = «hiç»; yakın değer «az önce» olarak gösterilir).
Xray sürümü	Düğümde çalışan Xray-core sürümü.
Panel sürümü	Düğümdeki 3X-UI sürümü – güncelleme göstergesi için güncel sürümle karşılaştırılır.
(inbound'lar)	Bu düğümde fiziksel olarak barındırılan inbound sayısı.
(istemciler)	Düğümün inbound'larındaki istemci sayısı.
(çevrimiçi)	Şu anda çevrimiçi olan düğüm istemcisi sayısı.
(tükenmiş)	Süresi dolmuş veya trafik limitini aşmış düğüm istemcisi sayısı. Manuel olarak devre dışı bırakılan istemciler bu sayaca dahil değildir.
(hız)	Düğümde barındırılan inbound'lardaki mevcut (anlık) iletim hızı.

Inbound/istemci/çevrimiçi sayaçları, yerel id yerine düğümün kararlı GUID'sine (`panelGuid`) göre düğüme bağlanır – bu sayede alt düğümdeki bir istemci, senkronize edildiği ara düğüme değil tam olarak alt düğüme atfedilir.

Düğümde barındırılan inbound'lar için sayfa çevrimiçi istemcileri, sayaçları ve **anlık iletim hızını** gösterir. Kararlı GUID ile bağlama, aynı `panelGuid` 'e sahip «klonlanmış» düğümleri de doğru şekilde ayırt eder.

Düğüm durumları

Durum	TR	Ne zaman ayarlanır
online	Çevrimiçi	Düğüm <code>panel/api/server/status</code> sorgulamasına <code>success=true</code> yanıtı verdi.
offline	Çevrimdışı	Düğüm yanıt vermedi, HTTP hatası döndürdü, <code>success=false</code> veya tanımsız yanıt döndürdü.
unknown	Bilinmiyor	Düğüm henüz hiç sorgulanmamışken başlangıç değeri.

Başarısız sorgulama durumunda hata metni kaydedilir ve «çevrimdışı» nedenini teşhis etmeye yardımcı olan anlaşılır bir ifadeyle gösterilir.

12.5. Düğüm üzerindeki eylemler

- **Bağlantıyı test et** (`POST /test`) – düğüm formunda, 6 saniyelik zaman aşımıyla henüz kaydedilmemiş parametreler üzerinden bağlantıyı test eder. Sonuç: «Bağlantı başarılı ({ms} ms)» veya «Bağlanılamadı». Kaydetmeden önce adres/port/token/TLS hatalarını ayıklamak için kullanışlıdır.
- **Şimdi kontrol et** («Şimdi kontrol et» düğmesi, `POST /probe/:id`) – kayıtlı bir düğümün planlanmamış sorgulaması; durumu ve metrikleri (CPU/bellek/çalışma süresi/gecikme/sürümler) hemen günceller ve heartbeat kaydeder. Başarısız olursa – «Kontrol başarısız».

Örnek: master API'si üzerinden düğümü test etme ve sorgulama. «Bağlantıyı test et», formdaki henüz kaydedilmemiş parametreleri dener:

```
POST /panel/api/nodes/test
Content-Type: application/json

{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGciOi...", "tlsMode": "verify" }
```

id'si 7 olan kayıtlı düğümün planlanmamış sorgulaması:

```
POST /panel/api/nodes/probe/7
```

- **Paneli güncelle** (`POST /updatePanel` ve `{ids:[...]}` gövdesiyle) – düğümde standart kendi kendini güncelleyiciyi başlatır: düğüm en son 3X-UI sürümünü indirir ve yeniden başlatır. **Seçilenleri güncelle ({count})** düğmesi bunu birden fazla işaretlenmiş düğüm için aynı anda gerçekleştirir. Düğümün yanında bir gösterge görünür: **Güncelleme mevcut** veya **Güncel**, düğümün panel sürümü ile en son sürüm karşılaştırılarak belirlenir.

Örnek: tek istekle birden fazla düğümü güncelleme. Gövdede işaretlenmiş düğümlerin id'leri iletilir; yalnızca etkin ve `online` olanlar güncellenir, diğerleri atlanmış olarak döner.

```
POST /panel/api/nodes/updatePanel
Content-Type: application/json

{ "ids": [3, 7, 12] }
```


Yanıt «Güncelleme 2 düğümde başlatıldı, 1 başarısız» şeklindedir: örneğin düğüm 12 çevrimdışı olduğu için atlanmış olabilir.

- Onay: «{count} düğümü en son sürümüne güncellensin mi? Her seçili düğüm en son sürümü indirip yeniden başlayacak. Yalnızca çevrimiçi etkin düğümler güncellenir».
- **Yalnızca onLine durumundaki etkin düğümler güncellenir.** Devre dışı bırakılan düğüm sonuçlarda «node is disabled», çevrimdışı olan «node is offline» olarak işaretlenir. Sonuç: «Güncelleme {ok} düğümde başlatıldı, {failed} başarısız». Uygun hiçbir düğüm seçilmemişse – «En az bir çevrimiçi etkin düğüm seçin».
- **Set Cert from Panel** (yardımcı, GET /webCert/:id) – düğümde bir inbound oluştururken, dosyaların tam olarak düğümde mevcut olması için **düğümün kendi** web-TLS sertifikasına (merkezi panelin değil) giden yolları eklemeye olanak tanır. Düğümün etkin ve erişilebilir olmasını gerektirir.
- **Düğümü sil** (POST /del/:id) – onay: «"{name}" düğümü silinsin mi? Bu işlem düğüm izlemesini durduracak. Uzak panelin kendisi etkilenmeyecek». Düğüm kaydını ve birikmiş trafik istatistiklerini siler; uzak panel normal çalışmaya devam eder. **Bir düğüm yalnızca tüm inbound'ları ondan kaldırıldıktan sonra silinebilir.** Düğüm hâlâ en az bir inbound bağlıysa (node_id üzerinden), panel «cannot delete node: N inbound(s) still attached to it; detach or delete them first» hatasıyla silme işlemini reddeder – önce bu inbound'ları ayırın veya silin, ardından düğümü silin. Bu, silinen düğüme sarkan başvurusu olan «sahipsiz» inbound'ların oluşmasını önler.

12.6. Metrik geçmişi

Geçmiş düğmesi/grafiği GET /history/:id/:metric/:bucket adresine başvurur. Kullanılabilir metrikler: **cpu** ve **mem** – her başarılı heartbeat'te biriktirilir. Toplama aralığının boyutu (bucket , saniye cinsinden) beyaz listeye sınırlandırılmıştır:

Örnek: geçmiş sorgusu. 60 saniyelik aralıklarla toplanan düğüm 7'nin CPU yük grafiği (en fazla 60 nokta döner):

```
GET /panel/api/nodes/history/7/cpu/60
```

Bellek ve «gerçek zamanlı» mod (2 sn) için sırasıyla .../7/mem/60 ve .../7/cpu/2 . Beyaz liste dışındaki değerler reddedilir («invalid metric» / «invalid bucket»).

Bucket (sn)	Kullanım amacı
2	Gerçek zamanlı mod
30	30 saniyelik aralıklar
60	1 dakikalık aralıklar
120	2 dakikalık aralıklar
180	3 dakikalık aralıklar
300	5 dakikalık aralıklar

En fazla 60 nokta döner. Geçersiz metrik veya bucket reddedilir («invalid metric» / «invalid bucket»).

12.7. Inbound'lar ve istemciler nasıl senkronize edilir

Bir inbound, `node_id` alanı aracılığıyla bir düğüme «ait» olur (inbound düzenleyicide düğüm seçilir):

Örnek: düğüm formundaki token. Token, alt panelden (Ayarlar → API Token) alınır ve masterın **API Token** alanına yapıştırılır. Her sorgulama sırasında master onu başlıkta gönderir:

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

Alt panelde bir **temel yol** (web base path) ayarlıysa, örneğin `/secret/`, master bunu `panel/api/server/status` önüne otomatik ekler → `https://panel.example.com:2053/secret/panel/api/server/status`.

1. **Yapılandırma dağıtımı (reconcile).** Bir düğüme bağlı inbound/istemcide herhangi bir değişiklik olduğunda, düğüm «kirli» olarak işaretlenir. Arka plan görevi, her etkin düğüm için **online durumundaki** düğümlerde değişiklik varsa inbound'ları (`node_id` 'ye göre) düğüme dağıtır ve ardından «kirli» bayrağını sıfırlar. Devre dışı, çevrimdışı veya «kirli» olan bir düğüm «beklemede» sayılır – bağlantı yeniden sağlanana kadar dağıtım ertelenir.

2. **Trafik toplama.** Aynı görev düğümünden trafik anlık görüntüsü ister ve yerel istatistiklere birleştirir. Birleştirilen trafik temel alınarak limit/süre tükenmesi kontrolü yapılır ve gerekirse istemciler devre dışı bırakılır; düğüme göre «tükenmiş» sayacı tam bunu yansıtır. Düğüm erişilemez durumdaysa çevrimiçi istemcileri temizlenir.

Birden fazla panele bağlı istemciler için master, aynı görevde düğümlere bu istemcinin **tüm paneller genelindeki toplam** trafik tüketimini ek olarak dağıtır (düğümdeki ayrı bir tabloda, anahtar master GUID'si; her gönderimde üzerine yazılır, dolayısıyla master tarafındaki sıfırlama da yayılır). Düğümde istemcinin trafiği, yerel veya gönderilen değerlerden büyük olanı gösterir; toplam kota aşıldığında istemci **doğrudan düğümde yerel olarak devre dışı bırakılır** (otomatik devre dışı bırakma sırasında kurulu bağlantıları da kesen Xray yeniden başlatma mekanizması aracılığıyla). Bu, düğümün yalnızca kendi trafik payını görerek toplam limiti tüketen bir istemciye hizmet vermeye devam ettiği durumu ortadan kaldırır. Trafik sıfırlandığında, otomatik yenilendiğinde veya istemci silindiğinde gönderilen sayaçlar temizlenir.

3. **Heartbeat.** Ayrı bir arka plan görevi, tüm **etkin** düğümleri periyodik olarak (eşzamanlılık sınırlı) `panel/api/server/status` üzerinden sorgular, durum/metrik/sürümleri günceller ve web istemcileri varsa güncellenmiş düğüm ağacını WebSocket üzerinden dağıtır.

12.8. Düğüm zincirleri (alt düğümler / geçişli düğümler)

Topoloji düz olmayabilir: bir düğümün kendisi de kendi düğümlerinin masteri olabilir. Bu tür alt paneller **Alt Düğümler** olarak görünür – bunlar doğrudan düğümden alınan **salt okunur projeksiyonlardır**.

- İpucu: «Salt okunur: {parent} üzerinden erişilebilen bağımlı düğüm. Onu {parent} kendi panelinden yönetin». Yani alt düğüm burada düzenlenemez, silinemez veya güncellenemez – tüm işlemler doğrudan üst panelinden yapılır.
- Alt düğümün kimliği GUID'si tarafından belirlenir; bu sayede çevrimiçi istemciler ve inbound'lar, `Node1 → Node2 → Node3` zincirinde bile transit olarak geçtikleri düğüme değil onları barındıran fiziksel düğüme atfedilir (master her doğrudan düğüm üzerinden bir seviye derine «girer»).

- Doğrudan düğüm erişilemez hale gelirse alt düğüm ön belleği temizlenir ve bağlantı yeniden sağlanana kadar alt düğümler ağaçtan kaybolur.

12.9. Düğümler: 3.3.0'daki yenilikler

3.3.0 sürümünde **Düğümler** bölümü üç önemli geliştirme aldı: çok atlamalı (multi-hop) topolojilerde trafik ve çevrimiçi istemcilerin doğru atfedilmesi, düğümler arasında client-IP senkronizasyonu ve düğümün paneli çalışırken Xray çekirdeği çöktüğü durum için ayrı bir durum göstergesi.

1. Multi-hop: alt düğüm zincirinde trafiğin doğru atfedilmesi

Önceden sayaçlar (inbound sayısı, çevrimiçi istemciler, tükenenler) «doğrudan» düğüm düzeyinde hesaplanıyordu. Master → Düğüm1 → Düğüm2 → Düğüm3 gibi bir zincirde, Düğüm2 / Düğüm3 üzerinde fiziksel olarak bulunan her şey, master'a ulaşmayı sağlayan Düğüm1'e yanlışlıkla atfediliyordu. 3.3.0'da atıf gerçek kaynağa göre yapılır.

Nasıl çalışır:

- **Alt düğümler ayrı satırlar olarak görünür.** Her panel kendi doğrudan düğümlerinin listesini yayımlar; yalnızca bilinen Guid 'li düğümler dahil edilir – düğümü bir «atlama» yukarıya atfetmek için kararlı kimlik gereklidir. Master periyodik olarak (heartbeat görevinden) bu listeleri çeker ve ön belleğe alır, ardından doğrudan düğümlere «geçişli» alt düğümleri ekler.
- **Geçişli düğümler salt okunurdur.** Kullanıcı arayüzünde «**Alt Düğüm**» olarak işaretlenir ve şu ipucuyla: «*Salt okunur: {üst} üzerinden erişilebilen bağımlı düğüm. Onu {üst} kendi panelinden yönetin.*» Bu tür satırlarda yönetim düğmeleri bulunmaz – düğüm, doğrudan üst panelinden yönetilir.
- **GUID üzerinden hiyerarşi.** Doğrudan düğümün ParentGuid 'i master'ın kendi GUID'sidir; geçişli düğümünki ise üst düğümün GUID'sidir. Bu sayede ağaç oluşturulur.
- **Sayaçlar için gerçeğin kaynağı – inbound üzerindeki origin_node_guid 'dir.** Bu, söz konusu inbound'u fiziksel olarak tutan düğümün panelGuid 'idir. Inbound düğümle senkronize edilirken ayarlanır ve **sonraki atlamalarda olduğu gibi korunur**; bu nedenle derinlemesine iç içe geçmiş bir inbound, ara düğüme değil gerçek düğüme atfedilir. Bu GUID, inbound sayısı, çevrimiçi istemciler ve tükenmiş istemciler sayaçlarını yeniden hesaplamak için kullanılır. Anahtar seçim mantığı:

Inbound durumu	Kime atfedilir
origin_node_guid ayarlı	Bu GUID'e (gerçek kaynak düğüm)
boş, ancak node_id ayarlı	Düğümün sentetik GUID'i (henüz panelGuid 'ini bildirmemiş eski derleme)
boş ve node_id boş	Master'ın kendi GUID'i (yerel Xray üzerindeki inbound)

Çevrimiçi istemciler de GUID'e göre gruplandırılır; bu nedenle her düğüm satırı yalnızca gerçekten o düğüme bağlı olanları gösterir.

Kullanıcının gördüğü: düz topolojide (doğrudan master altındaki düğümler) hiçbir şey değişmez – GUID ve id 'ye göre sayaçlar örtüşür. Ancak düğüm zinciri oluştuğunda listede «Alt Düğüm» satırları belirir ve her

düğümün inbound/çevrimiçi/tükenmiş sayıları artık transit olarak geçenlerin toplamını değil yalnızca kendi yükünü yansıtır.

2. Düğümler arasında access.log'dan client-IP senkronizasyonu

IP limiti (istemcide `limitIp`), Xray'ın kendi access.log'una yazdığı adreslere dayanır. Önceden her düğüm yalnızca kendisine yapılan bağlantıları görüyordu; bu nedenle «istemci başına en fazla N IP» kısıtlaması kümede çalışmıyordu: bir istemci farklı düğümlere bağlanarak limiti aşabiliyordu. 3.3.0'da gözlemlenen IP'ler tüm küme genelinde senkronize edilir.

Nasıl çalışır:

- Her düğümde bir arka plan görevi access.log'u ayrıştırır; her satırdan IP, istemci e-postası ve zaman damgasını çıkararak yerel tabloya kaydeder (e-posta başına bir kayıt, IP'ler JSON dizisi `{ip, timestamp}` olarak saklanır). `127.0.0.1` ve `::1` yerel adresleri atılır.
- 10 saniyede bir** senkronizasyon, her etkin çevrimiçi düğüm için çift yönlü değişim gerçekleştirir: düğümden IP'leri çeker ve yerel tabloya birleştirir, ardından master'ın genel tablosunu düğüme gönderir.
- Birleştirme, birden fazla düğümde görülen aynı IP'yi **çift saymadan** ve **eski kayıtları yeniden canlandırmadan** eski ve gelen gözlemleri birleştirir: yerel görevle aynı eskime eşiği uygulanır – **30 dakika**. Her IP için en güncel zaman damgası saklanır. Diğer düğümlerden gelen kayıtlar yeni yerel id alır (düğümlerin id uzayları bağımsızdır); aynı e-postanın eşzamanlı eklenmesi yinelenenlere karşı korunur.
- Limit hesaplanırken «canlı» sayılan bir IP, ya geçerli yerel taramada saptanmış ya da senkronize edilmiş veritabanında çok taze zaman damgasına (**2 dakika içinde**) sahiptir. Bu, tüm küme ölçeğinde limiti çalıştıran şeydir; adres başka bir düğümde saptanmış olsa bile. Limit aşıldığında en eski «canlı» IP'ler fail2ban günlüğüne yazılır ve bağlantılar zorla sonlandırılır (Xray API üzerinden istemciyi kaldırma/yeniden ekleme).

Kullanıcının gördüğü: IP sayısı kısıtlaması artık her düğüm için ayrı ayrı değil tüm küme genelinde geçerlidir; panelde istemci için herhangi bir düğümde saptanan IP'ler (30 dakikalık pencere içinde) görünür. Bunun için ayrı bir düğme/ayar yoktur – senkronizasyon, düğümden access.log etkin ve erişilebilir olduğu sürece arka planda otomatik çalışır (limitin kendisi için düğümden Fail2Ban de gereklidir).

3. Ayrı durum göstergesi: düğüm paneli çevrimiçi ancak Xray çöktü

Önceden düğüm durumu özünde «çevrimiçi / çevrimdışı» şeklindeydi. Düğümün paneli yanıt veriyorsa, üzerindeki Xray çekirdeği çalışmasa ve istemciler gerçekte bağlanamasa bile düğüm çevrimiçi sayılıyordu. 3.3.0'da panelin sağlığı ile Xray çekirdeğinin sağlığı ayrıştırıldı.

Nasıl çalışır:

- Düğüm sorgulanırken master, uzak `/panel/api/server/status` yanıtından `xray.state` ve `xray.errorMessage` alanlarını alır ve düğümden saklar. Bu alanlar, çekirdek sağlıklıysa dahi başarılı panel ping'inde doldurulur – tam olarak panel erişilebilirliğini Xray durumundan ayırt etmek için.
- `xray.state` değerleri: `"running"` (çalışıyor), `"stop"` (durduruldu), `"error"` (hata).
- Bu değerler düğüm durumlarına çevrilir. Tanıdık olanlara yenileri eklendi:

Durum anahtarı	Türkçe etiket	Ne zaman gösterilir
online	«Çevrimiçi»	panel yanıt veriyor, Xray çalışıyor (running)
offline	«Çevrimdışı»	panel erişilemez / ping başarısız
unknown	«Bilinmiyor»	durum henüz belirlenmedi
xrayError	«Xray Hatası»	panel çevrimiçi, ancak Xray çekirdeği error durumunda (errorMsg var)
xrayStopped	«Durduruldu»	panel çevrimiçi, ancak Xray durdurulmuş (stop)

- Bu tür durum için kullanıcı arayüzünde **ayrı bir mor gösterge** kullanılır (çevrimiçi için yeşilden ve çevrimdışı için kırmızıdan farklı renk). Mor doğrudan şunu işaret eder: düğüm erişilebilir, sorun ağda veya panelin kendisinde değil, Xray çekirdeğinin kendisindedir.

Kullanıcının gördüğü: çekirdek çöktüğünde yanıltıcı «yeşil» yerine düğüm **mor** renkle «**Xray Hatası**» veya «**Durduruldu**» durumuyla vurgulanır. Bu, düğümün erişilebilirliğini araştırmak yerine düğümdeki Xray'i onarmanın (çekirdeği yeniden başlatma, errorMsg 'ye bakma) gerektiğini hemen gösterir. Aynı xrayState / xrayError , geçişli alt düğümlere de (bkz. madde 1) iletilir; böylece çekirdekteki hatalı durum tüm zincir boyunca görünür.

13. Panel Ayarları

«Ayarlar» bölümü (sayfa başlığı – **Ayarlar**, İng. *Panel Settings*), 3X-UI web panelinin kendi davranışını yönetir: hangi adres ve portu dinlediğini, nasıl korunduğunu, Telegram botu ve harici servislerle nasıl etkileşime girdiğini ve zamanlanmış görevleri hangi saat diliminde yürüttüğünü. Her parametre, veritabanının `settings` tablosunda «anahtar – değer» çifti olarak saklanır; eğer değer veritabanında yoksa, varsayılan değer uygulanır.

Önemli – değişikliklerin uygulanması. Bu sayfadaki herhangi bir değişiklik **Kaydet** (*Save*) düğmesiyle kaydedilmeli, ardından değişikliklerin geçerli olması için panel yeniden başlatılmalıdır. Tam ipucu: «Değişiklikleri kaydedin ve uygulamak için paneli yeniden başlatın.» Kaydedildiğinde «Ayarlar değiştirildi» bildirimi görüntülenir.

13.1. Panelin Kaydedilmesi ve Yeniden Başlatılması

Öge	Amaç
Kaydet (<i>Save</i>)	Form alanlarının tümünü veritabanına yazar (<code>POST /panel/setting/update</code>). Yazılmadan önce değerler doğrulanır – geçersiz adresler, portlar veya yollar reddedilir ve panel hata döndürür.
Paneli Yeniden Başlat (<i>Restart Panel</i>)	Panel web sunucusunu yeniden başlatır (<code>POST /panel/setting/restartPanel</code>). Yeniden başlatma 3 saniyelik gecikmeyle gerçekleşir. İpucu: «Paneli yeniden başlatmak istediğinizden emin misiniz? Onaylayın; yeniden başlatma 3 saniye içinde gerçekleşecektir. Panel erişilemez hale gelirse sunucu günlüğünü kontrol edin». Başarı durumunda – «Panel başarıyla yeniden başlatıldı».
Varsayılanlara Sıfırla (<i>Reset to Default</i>)	Veritabanında kaydedilen tüm ayarları siler; bunun ardından panel varsayılan değerleri kullanır. Yönetici kimlik bilgileri bu işlemle sıfırlanmaz.

Yeniden başlatma, panel sürecine `SIGHUP` sinyali gönderilerek (veya kayıtlı yeniden başlatma kancası aracılığıyla) gerçekleştirilir. Windows'ta sinyal aracılığıyla otomatik yeniden başlatma desteklenmez. **Dinleme parametrelerindeki değişiklikler (IP, port, yol, alan adı, sertifikalar, saat dilimi) yalnızca panel yeniden başlatıldıktan sonra uygulanır.**

13.2. Genel Ayarlar (sekme «Panel» / *General*)

Arayüz Dili (*Language*)

Panel web arayüzünün dili. Kullanılabilir diller: `en-US` (İngilizce), `ru-RU` (Rusça), `zh-CN`, `zh-TW`, `fa-IR`, `ar-EG`, `es-ES`, `id-ID`, `ja-JP`, `pt-BR`, `tr-TR`, `uk-UA`, `vi-VN`. Bu bir görüntüleme ayarıdır ve Xray'in çalışmasını etkilemez.

Takvim Türü (*Calendar Type*)

- **Anahtar:** `datepicker`
- **Varsayılan değer:** `gregorian` (Miladi).

- **Amaç:** tarih seçiminde kullanılan takvim türü (örneğin istemci geçerlilik süresi belirlenirken). İpucu: «Zamanlanmış görevler bu takvime göre yürütülecektir.» Alternatif değer – Farsça (Jalali) takvimi; bu, panelin İranlı kullanıcılar arasında yaygın olması nedeniyle talep görmektedir.

Sayfalama Boyutu (*Pagination Size*)

- **Anahtar:** `pageSize`
- **Varsayılan değer:** 25
- **Geçerli değerler:** 0 ile 1000 arasında tam sayı.
- **Amaç:** tablolardaki (bağlantı/inbound listeleri) sayfa başına satır sayısı. İpucu: «Bağlantı tablosu için sayfa boyutunu belirleyin. Devre dışı bırakmak için 0 ayarlayın» – 0 seçildiğinde sayfalama devre dışı kalır ve tüm kayıtlar tek liste olarak gösterilir.
- **Panel yeniden başlatması gerekmez** (görüntüleme ayarı).

Otomatik Devre Dışı Bırakma Sonrası Xray'i Yeniden Başlat (*Restart Xray After Auto Disable*)

- **Anahtar:** `restartXrayOnClientDisable`
- **Varsayılan değer:** `true`
- **Amaç:** bir istemci otomatik olarak devre dışı bırakıldığında (geçerlilik süresi dolduğunda veya trafik limitine ulaşıldığında) Xray yeniden başlatılarak ilgili istemcinin mevcut bağlantıları kesilir. İpucu: «Bir istemci, geçerlilik süresi veya trafik limiti nedeniyle otomatik devre dışı kaldığında Xray'i yeniden başlatın.» İşlevin kendisi değişmedi – geçiş düğmesi yalnızca «Panel» (*General*) sekmesinde diğer genel ayarlarla birlikte yer almaktadır.

Not Modeli ve Ayırma Karakteri (*Remark Model & Separation Character*)

- **Anahtar:** `remarkModel`
- **Varsayılan değer:** `-ieo`
- **Amaç:** abonelikteki yapılandırma adının (remark) nasıl oluşturulacağını belirler. Dize **ilk karakterden** (ayırıcı) ve ardından **sıra harflerinden** oluşur:
 - `i` – inbound notu (*inbound remark*);
 - `e` – istemci e-postası;
 - `o` – ek etiket (*extra*).

Varsayılan `-ieo` değerinde ayırıcı `-` olup parçaların sırası şöyledir: inbound → e-posta → extra (örneğin `MyInbound-user@mail-extra`). Boş parçalar atlanır. Arayüzdeki «Örnek Not» (*Sample Remark*) alanı, oluşturulan adın önizlemesini gösterir. E-postanın ada dahil edilmesi, abonelik ayarlarındaki «E-postayı ada dahil et» parametresine de bağlıdır (abonelik bölümüne bakın).

Örnek: `remarkModel` değerinin yapılandırma adına etkisi. inbound adının `VLESS-Reality`, istemci e-postasının `alex@vpn` ve ek etiketin `RU` olduğunu varsayalım:

Alan değeri	Sonuç adı (remark)
<code>-ieo</code> (varsayılan)	<code>VLESS-Reality-alex@vpn-RU</code>

Alan değeri	Sonuç adı (remark)
_ie	VLESS-Reality_alex@vpn
-ei	alex@vpn-VLESS-Reality
o (boşluk ayırıcı, yalnızca etiket)	RU

Dizenin ilk karakteri her zaman ayırıcıdır; geri kalan harfler hangi parçaların ve hangi sırada ada dahil edileceğini belirler.

13.3. Panele Erişim: IP, Port, Yol, Alan Adı, Sertifika

Bu grup, panelin ağ giriş noktasını tanımlar. **Buradaki tüm değişiklikler yalnızca panel yeniden başlatıldıktan sonra uygulanır.**

Alan	Anahtar	Varsayılan değer	Açıklama
Panel yönetimi için IP adresi (Listen IP)	webListen	"" (boş)	Web panelinin dinlediği IP. Boş = tüm IP'lerde dinle. İpucu: «Herhangi bir IP'den bağlanmak için boş bırakın». Belirtilmişse, geçerli bir IP adresi olmalıdır (aksi takdirde kaydetme reddedilir).
Panel alan adı (Listen Domain)	webDomain	"" (boş)	İsteği alan adına göre doğrulamak için panel alan adı. Boş = herhangi bir alan adı ve IP'den bağlantıları kabul et. İpucu: «Herhangi bir alan adı ve IP'den bağlanmak için boş bırakın.»
Panel portu (Listen Port)	webPort	2053	Panelin çalıştığı port. İpucu: «Panelin çalıştığı port». 1–65535 geçerlidir. Port serbest olmalıdır; panel ve abonelik servisi aynı IP:port çiftini eş zamanlı kullanamaz.
URI yolu (URI Path)	webBasePath	/	Panel URL'sinin temel yolu (basePath). İpucu: «/' ile başlamalı ve '/' ile bitmelidir». Kaydetme sırasında panel, baştaki ve sondaki / eksikse otomatik olarak ekler. Yolda yasak karakterler reddedilir.

Panel Sertifikası (TLS / HTTPS)

Alan	Anahtar	Varsayılan değer	Açıklama
Panel sertifikası ortak anahtar dosyası yolu (Public Key Path)	webCertFile	""	Sertifika (zinciri) dosyasının tam yolu. İpucu: «/' ile başlayan tam yolu girin».
Panel sertifikası özel anahtar dosyası yolu (Private Key Path)	webKeyFile	""	Özel anahtar dosyasının tam yolu. İpucu: «/' ile başlayan tam yolu girin».

Sertifika/anahtar yollarından **en az biri** belirtilmişse, panel kaydederken «sertifika + anahtar» çiftini yüklemeye çalışır; hata oluşursa (var olmayan dosya, anahtar-sertifika uyumsuzluğu) kaydetme reddedilir. Her iki yol doğru şekilde belirtilmişse panel HTTPS'ye geçer. Her iki alan boşsa panel normal HTTP ile çalışır.

Güvenlik uyarıları (*Security warnings*). Panel, güvensiz yapılandırma tespit ettiğinde «Paneliniz açık olabilir:» uyarı bloğunu gösterir:

- normal HTTP ile çalışma – «üretim ortamı için TLS yapılandırın»;
- standart port 2053 – «rastgele biriyle değiştirin»;
- varsayılan temel yol / – «rastgele biriyle değiştirin»;
- standart abonelik yolu /sub/ ve JSON abonelik yolu /json/ – «değiştirin». Bunlar engelleyici değil, tavsiye niteliğindedir.

13.4. Oturum, Panel Proxy'si ve Güvenilir Proxy'ler (sekme «Proxy ve Sunucu» / *Proxy and Server*)

Oturum Süresi (*Session Duration*)

- **Anahtar:** sessionMaxAge
- **Varsayılan değer:** 360 (dakika, yani 6 saat).
- **Geçerli değerler:** 1 ile 525600 dakika arasında (1 yıl).
- **Amaç:** yöneticinin yeniden giriş yapmadan oturumunun ne kadar süre açık kalacağı. Birim – **dakika**. İpucu: «Sistemdeki oturum süresi (değer: dakika)».

Panel Trafiği için Outbound (*Panel Traffic Outbound*)

- **Anahtar:** panelOutbound
- **Varsayılan değer:** "" (boş = doğrudan bağlantı).
- **Amaç:** panelin **kendi isteklerini** – sürüm kontrolleri ve panel/Xray indirmeleri, Telegram çağrıları, rutin geo-dosyası güncellemeleri – göndereceği Xray **outbound**'unu belirler; böylece sunucu tarafındaki GitHub/Telegram filtrelemesi aşılır. Bu alan bir **açılır liste** olarak sunulur: içinde Xray yapılandırma şablonundaki outbound'lar, outbound aboneliklerindeki outbound'lar ve ayrıca yönlendirme **dengeleyicileri** (ayrı bir grup olarak) listelenir. blackhole türündeki outbound'lar listeden çıkarılmıştır – indirme trafiğini «kara deliğe» yönlendirmenin anlamı yoktur. Tam ipucu: «Panelin kendi isteklerini – sürüm kontrolleri ve panel/Xray indirmeleri, Telegram ve rutin geo-dosyası güncellemeleri – GitHub/Telegram sunucu filtrelemesini atlamak için bu Xray outbound'u üzerinden yönlendirir. Yerel köprü-inbound, çalışan yapılandırmaya otomatik olarak eklenir ve anında uygulanır. Xray'de yerleşik Geodata otomatik güncellemesi etkilenmez; kendine ait bir indirme outbound'u vardır. Doğrudan bağlantı için boş bırakın.»

Nasıl çalışır. Bir outbound seçildiğinde panel, çalışan yapılandırmaya kendiliğinden bir loopback-inbound (panel-egress etiketiyle SOCKS köprüsü) ve panelin kendi HTTP trafiğini seçilen outbound'a yönlendiren bir yönlendirme kuralı ekler. Bir dengeleyici seçilmişse kurala balancerTag eklenir ve panel trafiği katılımcıları arasında dağıtılır. Köprü ve kural, panelin tam yeniden başlatmasına gerek

kalmadan **anında** uygulanır. Doğrudan bağlantı için alanı boş bırakın. Xray'e yerleşik geo-verisi otomatik güncellemesi bu ayardan **etkilenmez** – Xray yönlendirmesi içinde kendi outbound'u vardır.

- **Biçim:** socks5:// (veya socks5h://) ya da http(s):// , gerektiğinde socks5:// user:pass@host:port şeklinde kimlik doğrulamayla. Desteklenen şemalar kesin olarak: socks5 , socks5h , http , https – diğer şemalar geçersiz sayılır ve panel doğrudan bağlantıya geri döner. Tipik örnek – Xray'in kendine ait yerel SOCKS-inbound'u.
- Tam ipucu: «Panelin kendi giden isteklerini (geo güncellemeleri, Xray/panel sürüm kontrolleri, Telegram) GitHub/Telegram sunucu filtrelemesini atlamak için bu proxy üzerinden yönlendirir. socks5:// veya http(s):// kabul eder, örn. Xray'in yerel SOCKS-inbound'u. Doğrudan bağlantı için boş bırakın.»
- Geçersiz bir proxy, kaydetme hatasına neden olmaz – panel yalnızca doğrudan bağlantıyı kullanır ve günlüğe bir uyarı yazar.

Alan değerleri örneği. Sunucuda 10808 portunda yerel bir Xray SOCKS-inbound'u zaten çalışıyorsa, panel isteklerini buradan yönlendirin:

```
socks5://127.0.0.1:10808
```

Kimlik doğrulamalı harici HTTP proxy için:

```
http://user:pass@proxy.example.com:8080
```

Kaydedip yeniden başlattıktan sonra panel, geo veritabanı güncellemelerini, sürüm kontrollerini ve Telegram çağrılarını belirtilen proxy üzerinden gerçekleştirecektir.

Güvenilir Proxy CIDR'leri (*Trusted proxy CIDRs*)

- **Anahtar:** trustedProxyCIDRs
- **Varsayılan değer:** 127.0.0.1/32, ::1/128 (yalnızca yerel makine).
- **Biçim:** virgülle ayrılmış IP adresleri veya CIDR alt ağları listesi (örneğin 10.0.0.0/8, 192.168.1.5). Her öge IP veya CIDR olarak doğrulanır – geçersiz değer kaydedilirken reddedilir.
- **Amaç:** X-Forwarded-Host , X-Forwarded-Proto ve gerçek istemci IP başlığını ayarlamasına izin verilen kaynakları listeler. Tam ipucu: «İletilen host, proto ve istemci IP başlıklarını ayarlamasına izin verilen IP/ CIDR, virgülle ayrılmış.» Panel ters proxy'nin (nginx, Caddy vb.) arkasında çalışıyorsa istemci IP'lerini ve şemayı doğru belirlemek için yapılandırılması gerekir.

Örnek: ters proxy arkasında panel. Nginx aynı makinede bulunuyorsa ve istekleri panele proxy yapıyorsa, yalnızca yerel makineye güveni bırakın (varsayılan değer):

```
127.0.0.1/32, ::1/128
```

Proxy, 10.0.0.0/8 iç ağındaki ayrı bir sunucudaysa, alt ağını ekleyin; aksi takdirde panel iletilen başlıkları yok sayar ve gerçek istemci yerine proxy IP'sini görür:

```
127.0.0.1/32, ::1/128, 10.0.0.0/8
```

Gerçek IP ve şemayı ileten ilgili nginx bloğu örneği:

```
proxy_set_header X-Real-IP      $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

13.5. Telegram Botu (sekme «Telegram Botu» / *Telegram Bot*)

Telegram Botunu Etkinleştir (*Enable Telegram Bot*)

- **Anahtar:** `tgBotEnable`
- **Tür/varsayılan:** mantıksal, `false`.
- **Amaç:** Telegram botunun çalışmasını etkinleştirir. İpucu: «Telegram botu aracılığıyla panel özelliklerine erişim».

Telegram Token'ı (*Telegram Token*)

- **Anahtar:** `tgBotToken`
- **Varsayılan:** `""`.
- **Amaç:** bot token'ı. İpucu: «Token'ı Telegram bot yöneticisi @botfather'dan almanız gerekmektedir».
- **Güvenlik özelliği:** token gizli bir değerdir. Panel ayar okuma yanıtında döndürülmez (alan temizlenir, yalnızca «yapılandırıldı/yapılandırılmadı» bayrağı verilir). Kaydedilirken alan boş bırakılırsa, önceden kaydedilen token **korunur** (silinmez).

Telegram Botu Dili (*Telegram Bot Language*)

- **Anahtar:** `tgLang`
- **Varsayılan:** `en-US`.
- **Amaç:** bot mesajlarının dili (web arayüzü dilinden bağımsız). Kullanılabilir diller listesi, panel dillerinkiyle aynıdır.

Bot Yöneticisi Kullanıcı Kimliği (*Admin Chat ID*)

- **Anahtar:** `tgBotChatId`
- **Varsayılan:** `""`.
- **Biçim:** **virgülle ayrılmış** bir veya birden fazla sayısal Telegram User ID.
- **Amaç:** bildirim alıcıları ve bot aracılığıyla paneli yönetme izni olan yöneticiler. İpucu: «Bir veya birden fazla Telegram bot yöneticisi User ID'si. User ID'yi öğrenmek için @userinfobot kullanın veya botta '/id' komutunu çalıştırın.»

Bildirim Sıklığı (*Notification Time*)

- **Anahtar:** `tgRunTime`
- **Varsayılan:** `@daily` (günde bir kez).

- **Biçim:** **Crontab** biçiminde dize (hem standart cron ifadeleri hem de `@daily`, `@hourly`, `@every 1h` gibi kısaltmalar desteklenir). İpucu: «Bildirim aralığını Crontab biçiminde belirtin». Botun periyodik raporlarını denetler.

Alan değerleri örnekleri.

Değer	Botun rapor gönderme zamanı
<code>@daily</code>	günde bir kez gece yarısı (varsayılan)
<code>@hourly</code>	her saat
<code>@every 6h</code>	her 6 saatte bir
<code>0 9 * * *</code>	her gün 09:00'da
<code>30 8 * * 1</code>	her Pazartesi 08:30'da

Saat, 13.6. bölümündeki «Saat Dilimi» ayarına göre hesaplanır.

SOCKS Proxy (SOCKS Proxy)

- **Anahtar:** `tgBotProxy`
- **Varsayılan:** `""`.
- **Amaç:** botun Telegram'a bağlanması için ayrı SOCKS5 proxy'si. İpucu: «Telegram'a bağlanmak için Socks5 proxy'ye ihtiyacınız varsa, parametrelerini kılavuza göre yapılandırın.» Özellikle bot trafiğine uygulanır (13.4. bölümündeki genel «Panel Ağ Proxy'si»nden farklıdır).

Telegram API Sunucusu (Telegram API Server)

- **Anahtar:** `tgBotAPIServer`
- **Varsayılan:** `""` (standart `api.telegram.org` sunucusunu kullan).
- **Biçim:** `http(s)://...` URL'si; kaydedilirken URL doğruluğu kontrol edilir – geçersiz adres reddedilir. İpucu: «Kullanılacak Telegram API sunucusu. Varsayılan sunucuyu kullanmak için boş bırakın.» Kendi kendinize barındırdığınız Telegram Bot API sunucusu için gereklidir.

Bot Bildirimleri (grup «Bildirimler» / *Notifications*)

Alan	Anahtar	Varsayılan	Açıklama
Veritabanı Yedekleme (Database Backup)	<code>tgBotBackup</code>	<code>false</code>	Rapor ile birlikte Telegram'a veritabanı yedek dosyası gönderir. İpucu: «Veritabanı yedek dosyası ile birlikte bildirim gönder».
Giriş Bildirimi (Login Notification)	<code>tgBotLoginNotify</code>	<code>true</code>	Panele giriş denemesi olduğunda bildir. İpucu: «Birileri panelinize giriş yapmaya çalıştığında kullanıcı adını, IP adresini ve zamanı gösterir.»
Oturum Sona Erme Bildirimi Gecikmesi (Expiration Date Notification)	<code>expireDiff</code>	<code>0</code>	İstemcinin geçerlilik süresi dolmadan kaç gün önce bildirim gönderileceği. <code>0</code> – devre dışı. <code>>= 0</code> geçerlidir. İpucu: «Eşik değerine ulaşılmadan önce oturum sona erme bildirimi al (değer: gün)».

Alan	Anahtar	Varsayılan	Açıklama
Trafik Bildirimi Eşiği (<i>Traffic Cap Notification</i>)	<code>trafficDiff</code>	<code>0</code>	Bildirim için kalan trafik eşiği. İpucu: «Eşiğe ulaşılmadan önce trafik tükenmesi bildirimi al (değer: GB)». <code>0–100</code> geçerlidir.
CPU Yük Eşiği (<i>CPU Load Notification</i>)	<code>tgCpu</code>	<code>80</code>	CPU kullanımı eşiği aşarsa yöneticilere bildir (% cinsinden). <code>0–100</code> geçerlidir. İpucu: «CPU yükü bu eşiği aşarsa Telegram'da yöneticileri bildir (değer: %)».

13.6. Tarih ve Saat (sekme «Tarih ve Saat» / *Date and Time*)

Saat Dilimi (*Time Zone*)

- **Anahtar:** `timeLocation`
- **Varsayılan değer:** `Local` (sunucunun sistem saat dilimi).
- **Biçim:** IANA tz veritabanındaki bölge adı (örneğin `Europe/Moscow`, `UTC`, `Asia/Tehran`).
- **Amaç:** panelin zamanlanmış görevleri (bot raporları, trafik sınırlama/kontrolleri, süreler) yürüttüğü saat dilimi. İpucu: «Zamanlanmış görevler bu saat dilimindeki saate göre yürütülür».
- **Doğrulama:** kaydetme sırasında bölge kontrol edilir – mevcut olmayan bölge reddedilir. Veritabanında sonradan geçersiz bir değer bulunursa panel çalışma zamanında `Local` 'e döner; o da erişilemez durumdaysa `UTC` 'ye döner.

13.7. Harici Trafik ve Xray Davranışı (sekme «Harici Trafik» / *External Traffic*)

Alan	Anahtar	Varsayılan	Açıklama
Harici Trafik Bildirimi (<i>External Traffic Inform</i>)	<code>externalTrafficInformEnable</code>	<code>false</code>	Her trafik güncellemesinde harici API'yi bildir. İpucu: «Her trafik güncellemesinde harici API'yi bildir.»
Harici Trafik Bildirimi URI'si (<i>External Traffic Inform URI</i>)	<code>externalTrafficInformURI</code>	<code>""</code>	Panelin trafik güncellemelerini gönderdiği URL. Kaydedilirken URL doğruluğu kontrol edilir. İpucu: «Trafik güncellemeleri bu URI'ye gönderilir».
Otomatik Devre Dışı Bırakma Sonrası Xray'i Yeniden Başlat (<i>Restart Xray After Auto Disable</i>)	<code>restartXrayOnClientDisable</code>	<code>true</code>	İstemci, süre dolumu veya trafik limiti nedeniyle otomatik devre dışı kaldığında Xray'i yeniden başlat. İpucu: «Bir istemci, geçerlilik süresi veya trafik limiti nedeniyle otomatik devre dışı kaldığında Xray'i yeniden başlatın.» Geçiş düğmesi «Panel» (General) sekmesindedir – bkz. 13.2. bölüm; burada bütünlük amacıyla verilmektedir.

13.8. Diğer: Xray Yapılandırma Şablonu ve Test URL'si

Xray Yapılandırma Şablonu (*xrayTemplateConfig*)

- **Anahtar:** `xrayTemplateConfig`
- **Varsayılan:** derlemeyle birlikte gelen yerleşik (embedded) JSON şablonu.
- **Amaç:** panelin inbound/outbound üzerine inşa ettiği temel Xray-core yapılandırma JSON şablonu. Bu değer, tüm ayarların normal çıktısında **verilmez** ve panel ayarları genel listesinde değil, ayrı Xray yapılandırma sayfasında düzenlenir. Standart varsayılan şablona `GET /panel/setting/getDefaultJsonConfig` üzerinden erişilebilir.

Giden Bağlantı Test URL'si (*xrayOutboundTestUrl*)

- **Anahtar:** `xrayOutboundTestUrl`
- **Varsayılan:** `https://www.google.com/generate_204`
- **Amaç:** giden (outbound) bağlantıların çalışabilirliğini test ederken kullanılan URL. Ayarlanırken HTTP(S) URL'si olarak doğrulanır.

13.9. Yönetici Hesabı ve API Token'ları

Bu parametreler ilgili sekmede («Hesap» / *Authentication*) yer alır ve güvenlik bölümünde ayrıntılı incelenir; burada yalnızca anahtarların kısa özeti verilmektedir.

- **Kimlik bilgisi değiştirme** («Geçerli kullanıcı adı», «Geçerli şifre», «Yeni kullanıcı adı», «Yeni şifre» alanları) ayrı bir `POST /panel/setting/updateUser` isteğiyle kaydedilir. Doğru geçerli kullanıcı adı ve şifre gereklidir; yeni kullanıcı adı ve şifre boş olamaz. Mesajlar: «Yönetici kimlik bilgilerini başarıyla değiştirdiniz.» / «Geçersiz kullanıcı adı veya şifre».
- **İki Faktörlü Kimlik Doğrulama (2FA)** – `twoFactorEnable` (varsayılan `false`) ve gizli `twoFactorToken` anahtarları. Token bir sırdır: 2FA etkinken kaydedilirken boş alan mevcut token'ı silmez. 2FA **ilk kez** etkinleştirildiğinde panel geçerli oturumları geçersiz kılar («giriş çağı» artırılır).
- **API token'ları** ayrı uç noktalarla yönetilir (`/panel/setting/apiTokens...`): liste, oluşturma (`apiTokens/create`), silme, etkinleştirme/devre dışı bırakma. Token'ın kendisi **yalnızca oluşturma sırasında bir kez gösterilir** ve okunabilir biçimde saklanmaz: «Bu token'ı şimdi kopyalayın. Güvenlik nedeniyle okunabilir biçimde saklanmamaktadır ve bir daha gösterilmeyecektir.»

2FA, şifreler, LDAP senkronizasyonu ve abonelik biçimleri (JSON/Clash, fragmentation, noises, mux) ayrıntıları, kılavuzun ilgili ayrı bölümlerine taşınmıştır.

13.10. 3.3.0 Sürümündeki API Değişiklikleri (entegrasyonlar için önemli)

3.3.0 sürümünde sunucu API yollarının yapısı değişti. Panele HTTP üzerinden erişen harici entegrasyonlarınız (betikler, botlar, merkezi paneller, CI görevleri) varsa, bunları **düzeltilmeniz** gerekir, aksi takdirde çalışmayı durduracaklardır.

⚠ BREAKING CHANGE: `/panel/setting/*` ve `/panel/xray/*` uç noktaları `/panel/api` altına taşındı

Daha önce panel ayar yönetimi ve Xray yapılandırması `/panel/setting/*` ve `/panel/xray/*` yolları altında ayrı konumlarda bulunuyordu. Artık her iki küme de ortak `/panel/api` API grubu içinde kayıtlıdır. Eski yollar **tamamen kaldırılmıştır** – bu yollara yapılan istekler 404 döndürecektir.

Bu neden yapıldı: `/panel/api` grubunun tamamı tek bir erişim denetimine tabi olduğundan, bu uç noktalar artık diğer API ile aynı `Authorization: Bearer <token>` başlığını kabul etmektedir. API token'ı tam yönetici erişimi anlamına gelir ve böylece API yüzeyi tamamen tekdüze hale gelmiştir.

Değişmeyen şeyler: web arayüzü sayfaları (SPA rotaları) `/panel/settings` ve `/panel/xray` yerinde kalmıştır – söz konusu olan yalnızca sunucu tarafı API uç noktalarıdır.

Yol eşleştirme tablosu (eski → yeni)

Aşağıdaki tüm yolların ön eki – `/panel/` 'den sonra yalnızca `api/` eklendi.

Eski (≤ 3.2.x)	Yeni (3.3.0)	Yöntem
<code>/panel/setting/all</code>	<code>/panel/api/setting/all</code>	POST
<code>/panel/setting/defaultSettings</code>	<code>/panel/api/setting/defaultSettings</code>	POST
<code>/panel/setting/update</code>	<code>/panel/api/setting/update</code>	POST
<code>/panel/setting/updateUser</code>	<code>/panel/api/setting/updateUser</code>	POST
<code>/panel/setting/restartPanel</code>	<code>/panel/api/setting/restartPanel</code>	POST
<code>/panel/setting/getDefaultJsonConfig</code>	<code>/panel/api/setting/getDefaultJsonConfig</code>	GET
<code>/panel/setting/apiTokens</code>	<code>/panel/api/setting/apiTokens</code>	GET
<code>/panel/setting/apiTokens/create</code>	<code>/panel/api/setting/apiTokens/create</code>	POST
<code>/panel/setting/apiTokens/delete/:id</code>	<code>/panel/api/setting/apiTokens/delete/:id</code>	POST
<code>/panel/setting/apiTokens/setEnabled/:id</code>	<code>/panel/api/setting/apiTokens/setEnabled/:id</code>	POST
<code>/panel/xray/</code>	<code>/panel/api/xray/</code>	POST
<code>/panel/xray/update</code>	<code>/panel/api/xray/update</code>	POST
<code>/panel/xray/getDefaultJsonConfig</code>	<code>/panel/api/xray/getDefaultJsonConfig</code>	GET
<code>/panel/xray/getXrayResult</code>	<code>/panel/api/xray/getXrayResult</code>	GET
<code>/panel/xray/getOutboundsTraffic</code>	<code>/panel/api/xray/getOutboundsTraffic</code>	GET
<code>/panel/xray/resetOutboundsTraffic</code>	<code>/panel/api/xray/resetOutboundsTraffic</code>	POST
<code>/panel/xray/testOutbound</code>	<code>/panel/api/xray/testOutbound</code>	POST
<code>/panel/xray/warp/:action</code>	<code>/panel/api/xray/warp/:action</code>	POST

Eski (≤ 3.2.x)	Yeni (3.3.0)	Yöntem
/panel/xray/nord/:action	/panel/api/xray/nord/:action	POST
/panel/xray/outbound-subs (ve /outbound-subs/*)	/panel/api/xray/outbound-subs (ve /outbound-subs/*)	GET/POST/DELETE

Alt yolların adları, istek gövdeleri ve yanıt biçimleri değişmedi – yalnızca **ön ek** değişti.

Mevcut entegrasyonlar nasıl düzeltilir

1. Betiklerinizde/yapılandırmalarınızda /panel/setting/ ve /panel/xray/ içeren tüm girişleri bulun.
2. Ön eki değiştirin: /panel/ 'den hemen sonra api/ ekleyin (örneğin /panel/setting/all → /panel/api/setting/all).
3. İstek gövdeleri, parametreler ve yanıt biçimi düzenlenmesine gerek yoktur – yalnızca URL değişir.
4. Ayarlar ve Xray yapılandırması artık /panel/api altında olduğundan, bunlara /panel/api/inbounds/* ve diğer uç noktalarla aynı Authorization: Bearer <token> API token'ıyla erişebilirsiniz (ve erişmelisiniz). /panel/api grubunun tamamında etkin olan CSRF middleware'ini unutmayın.

Örnek: API üzerinden tüm ayarların okunması. Eski hali (≤ 3.2.x):

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
-H "Authorization: Bearer <token>"
```

Yeni hali (3.3.0) – /panel/ 'den sonra api/ eklendi:

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
-H "Authorization: Bearer <token>"
```

Benzer şekilde panel yeniden başlatma: POST /panel/api/setting/restartPanel. Eski yol /panel/setting/restartPanel artık 404 döndürecektir.

Tiplendirilmiş API: şemalar ve belgeler (Swagger / OpenAPI)

3.3.0'da OpenAPI spesifikasyonu tamamen tiplendirilmiş hale getirildi. Önceden tiplendirilmiş yanıtlar boş {} nesnesiyle tanımlanıyordu; artık bileşenler ve şemalar (components.schemas) doğrudan veri modellerinden üretilmektedir. Bunun sayesinde:

- Swagger UI gerçek veri modellerini gösterir, kimliksiz yer tutucuları değil.
- Harici üreticiler (openapi-generator vb.) spesifikasyondan istenen dilde hazır istemciler oluşturabilir.
- Her tiplendirilmiş yanıtta belirli bir modele \$ref eklendi ve yanıt örnekleri sunuldu.

API belgelerine nereden bakılır:

- **Yerleşik Swagger sayfası.** Panel menüsünde – «API Belgeleri» ögesi (SPA rotası /panel/api-docs). Burada tüm uç noktalar açıklamalar, istek gövdeleri ve yanıt örnekleriyle etkileşimli olarak listelenmektedir.
- **Ham OpenAPI 3.0 spesifikasyonu** /panel/api/openapi.json adresinden sunulmaktadır. Bu URL doğrudan Postman, Insomnia veya openapi-generator 'a beslenebilir. Spesifikasyon, derleme aşamasında

ikili dosyaya gömülmüştür; panel standart dışı bir `webBasePath` altında çalışırken spesifikasyondaki `servers` alanı geçerli temel yola göre otomatik olarak yeniden yazılır; böylece «Try it out» düğmesi ve harici üreticiler doğru ön eki hedefler.

14. Telegram Botu

3X-UI paneli, sunucunun ve istemcilerin durumu hakkında bildirim almak ve doğrudan mesajlaşma uygulaması üzerinden belirli istemcileri yönetmek için kullanılabilen yerleşik bir Telegram botu içerir. Bot, long polling (sürekli Telegram sorgulama) teknolojisiyle çalıştığından harici bir etki alanına veya açık bir porta ihtiyaç duymaz; yalnızca Telegram sunucularına giden bağlantı yeterlidir.

Bot iki tür muhatap ayırt eder:

- **Yönetici** – Telegram Kullanıcı Kimliği, botun ayarlarında belirtilmiş kullanıcı («Bot yöneticisinin Kullanıcı Kimliği» alanı). Tüm işlevlere erişimi vardır: sunucu istatistikleri, yedekleme, istemci yönetimi, Xray yeniden başlatma.
- **İstemci** – Telegram Kullanıcı Kimliği, gelen bağlantının belirli bir istemcisine bağlanmış (istemcinin `tgId` alanı) herhangi bir kullanıcı. Yalnızca kendi aboneliklerine ait bilgileri görebilir.

Örnek: istemciyi Telegram'a bağlama. Bir kullanıcının kendi abonelik istatistiklerini alabilmesi için sayısal Telegram Kullanıcı Kimliği, istemcinin `tgId` alanına yazılır. İstemcinin JSON ayarlarında bu şu şekilde görünür:

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

Bundan sonra `123456789` Kullanıcı Kimliğine sahip kullanıcı, bota `/usage ivan` komutunu göndererek kendi istatistiklerini görebilir. Yönetici, istemci kartındaki « Telegram Kullanıcısı Ata» düğmesiyle aynı Kimliği atayabilir; JSON'a elle girmek gerekli değildir.

14.1. Botu Etkinleştirme ve Yapılandırma

Tüm bot parametreleri panelde **Ayarlar** → **Telegram Botu** bölümünde ayarlanır. Ayarlar değiştirildikten sonra kaydedilmeli ve panel yeniden başlatılmalıdır; bot, web sunucusu başlangıcında başlatılır.

Alan (UI)	Ayar Anahtarı	Varsayılan Değer	Açıklama
Telegram Botunu Etkinleştir	<code>tgBotEnable</code>	<code>false</code>	Ana anahtar. İpucu: «Telegram botu aracılığıyla panel işlevlerine erişim». Kapalıyken bot başlatılmaz ve bildirim görevleri planlanmaz.
Telegram Tokeni	<code>tgBotToken</code>	(boş)	Bot tokeni. İpucu: «Telegram bot yöneticisi @botfather'dan token almanız gerekir». Boş olmayan bir token olmadan bot başlamaz.
SOCKS Proxy	<code>tgBotProxy</code>	(boş)	Telegram'a bağlanmak için proxy. İpucu: «Telegram'a bağlanmak için Socks5 proxy'ye ihtiyacınız varsa, kılavuza göre parametrelerini yapılandırın».

Alan (UI)	Ayar Anahtarı	Varsayılan Değer	Açıklama
Telegram API Sunucusu	tgBotAPIServer	(boş)	Alternatif Telegram API sunucusu. İpucu: «Kullanılan Telegram API sunucusu. Varsayılan sunucuyu kullanmak için boş bırakın».
Bot Yöneticisinin Kullanıcı Kimliği	tgBotChatId	(boş)	Virgülle ayrılmış bir veya birden fazla yönetici Telegram Kullanıcı Kimliği. İpucu: «Kullanıcı Kimliğini öğrenmek için @userinfobot'u kullanın veya botta /id komutunu gönderin».
Bot'tan Yöneticilere Bildirim Sıklığı	tgRunTime	@daily	Crontab biçiminde periyodik rapor zamanlaması. İpucu: «Crontab biçiminde bildirim aralığını belirtin».
Veritabanı Yedekleme	tgBotBackup	false	İpucu: «Veritabanı yedek dosyasıyla bildirim gönder». Yedeklemeyi periyodik rapora ekler.
Giriş Bildirimi	tgBotLoginNotify	true	İpucu: «Biri panelinize giriş yapmaya çalıştığında kullanıcı adını, IP adresini ve zamanı gösterir».
Bildirim İçin CPU Yük Eşiği	tgCpu	80	Yüzde cinsinden CPU yükü eşiği (0–100 doğrulama). İpucu: «CPU yükü bu eşiği aşarsa Telegram'daki yöneticileri bildir (değer: %). 0 değerinde CPU kontrolü devre dışı bırakılır».
Telegram Bot Dili	—	—	Botun tüm mesajları oluşturduğu dil.

@BotFather Aracılığıyla Token Alma

1. Telegram'da @BotFather ile diyalogu açın.
2. /newbot komutunu gönderin ve talimatları izleyin (bot adı ve bot ile biten benzersiz username).
3. BotFather, 123456789:AA... biçiminde bir token verecektir. Bunu **Telegram Tokeni** alanına yapıştırın.

Yönetici Kullanıcı Kimliği Alma

Kullanıcı Kimliği, hesabın sayısal tanımlayıcısıdır (kullanıcı adı değil). İki yolla öğrenilebilir:

- @userinfobot'a mesaj atın.
- Önceden yapılandırılmış botu başlatın ve /id komutunu gönderin; bot Kimliğinizi döndürecektir.

Elde ettiğiniz sayıyı **Bot Yöneticisinin Kullanıcı Kimliği** alanına girin. Birden fazla yönetici atamak için Kimliklerini virgülle listeleyin (örneğin 11111111, 22222222). Her Kimlik tam sayı olarak doğrulanır; hatalı bir değer botun başlamasında hataya yol açar.

Örnek: «Bot Yöneticisinin Kullanıcı Kimliği» alanı değeri. Tek yönetici – yalnızca sayı:

123456789

Virgülle ayrılmış iki yönetici (boşluk bırakmak zorunda değilsiniz):

123456789,987654321

Her değer tam sayı olmalıdır. @username veya 123 456 gibi (sayının içinde boşluk) bir giriş kabul edilmez – bot başlamaz.

Proxy

socks5:// , http:// ve https:// şemaları desteklenir. Proxy alanı boş bırakılırsa bot, panelin genel proxy'sini kullanmaya çalışır (tanımlanmışsa ve şeması destekleniyorsa). Desteklenmeyen şema veya hatalı sözdizimi içeren URL yok sayılır; bot doğrudan bağlanır. Proxy, sunucudan Telegram API'sine doğrudan erişim engellendiğinde kullanışlıdır.

E-posta Bildirimleri (SMTP)

Telegram'a ek olarak aynı olaylar e-posta ile de alınabilir. Kanal, **Ayarlar** → **E-posta** bölümündeki **SMTP Settings** sekmesinde yapılandırılır:

Alan (UI)	Ayar Anahtarı	Varsayılan Değer	Açıklama
Enable Email Notifications	smtpEnable	false	SMTP üzerinden e-posta bildirimlerinin ana anahtarı.
SMTP Host	smtpHost	(boş)	SMTP sunucu adresi (örneğin smtp.gmail.com).
SMTP Port	smtpPort	587	SMTP sunucu portu.
SMTP Username	smtpUsername	(boş)	SMTP kimlik doğrulama kullanıcı adı. Gönderen (From) adresi olarak da kullanılır.
SMTP Password	smtpPassword	(boş)	SMTP kimlik doğrulama parolası. Gizli olarak saklanır; parola zaten ayarlanmışsa alan «yapılandırıldı» göstergesi gösterir ve mevcut parolayı korumak için boş bırakılabilir.
Recipients	smtpTo	(boş)	Virgülle ayrılmış alıcı listesi (örneğin admin@example.com, ops@example.com).
Encryption	smtpEncryptionType	starttls	Bağlantı şifreleme türü: none (şifreleme yok), starttls (STARTTLS) veya tls (örtük TLS).

Send Test Email düğmesi bir test e-postası gönderir ve sonucu aşamalara göre gösterir: **Connection** (bağlantı), **Authentication** (kimlik doğrulama) ve **Send** (gönderme). Bir sorun varsa tanılama, hangi aşamada hata oluştuğunu belirtir (örneğin «Authentication failed – check username and password» veya «Server requires STARTTLS – change encryption type»), bu da parametrelerin ayarlanmasını kolaylaştırır.

İkinci sekmede (**Notifications**), e-posta ile bildirim gönderilecek olaylar seçilir; Telegram ile aynı grup kartları kullanılır (bkz. «Olay Veriyolu ve Bildirim Seçimi», bölüm 14.5).

Telegram API Sunucusu

Bot varsayılan olarak resmi Telegram API'sine bağlanır. **Telegram API Sunucusu** alanında kendi Bot API sunucunuzun (telegram-bot-api) adresi belirtilebilir. URL güvenlik açısından doğrulanır; engellenen veya hatalı adres reddedilir ve varsayılan sunucu kullanılır.

14.2. Ana Menü ve Düğmeler

Menü **/start** komutuyla çağrılır. Düğmeler, mesaja eklenmiş inline klavyedir; düğme seti yönetici mi yoksa istemci mi olduğunuza göre değişir.

Yönetici Menüsü

Düğme	İşlem
 Sıralı Trafik Kullanım Raporu	Tüm istemcileri trafiğe göre sıralayarak her birinin kullanımını listeler; verisi olmayan «gereksiz» e-postalar « Sonuç yok» olarak işaretlenir.
 Sunucu Durumu	Sunucu özeti (bkz. bölüm 14.5). « Yenile » düğmesi verileri yeniden çizer.
 Tüm Trafiği Sıfırla	Tüm istemcilerin trafik sayaçlarını sıfırlar. «Emin misiniz? » onayı ister, ardından her istemci için « Başarılı» veya « Başarısız» gösterir, sonunda « Tüm istemciler için trafik sıfırlama tamamlandı».
 Veritabanı Yedeği	Veritabanı dosyasını ve config.json dosyasını gönderir (bkz. bölüm 14.6).
 Ban Günlüğü	IP sınırı aşımı nedeniyle yasaklanan IP adreslerinin günlük dosyalarını gönderir.
 Gelen Bağlantılar	Tüm inbound bağlantılarının özeti: Remark, port, trafik, istemci sayısı, bitiş tarihi.
 Yakında Sona Erecek	Trafiği veya süresi yakında dolacak inbound bağlantıları ve istemcilerin listesi (bkz. bölüm 14.5).
 Komutlar	Yönetici komutları için yardım gösterir.
 Çevrimiçi	Çevrimiçi istemcilerin sayısı ve listesi; e-postaya tıklamak istemci kartını açar. « Yenile » düğmesi.
 Tüm İstemciler	inbound seçimini, ardından görüntüleme/yönetim için istemci listesini açar.
 Yeni İstemci	İstemci ekleme sihirbazını başlatır (inbound seçimi → taslak → onay).
Abonelik ayarları / bireysel bağlantılar / QR kodu	Abonelik bağlantısı, bireysel bağlantılar veya QR kodları almak için inbound ve istemci seçimi.

İstemci Menüsü

İstemciye sınırlı sayıda düğme sunulur:

Düğme	İşlem
İstemci İstatistikleri	İstemcinin Telegram Kullanıcı Kimliğine bağlı tüm aboneliklerin verilerini gösterir.
 Komutlar	İstemci komutları için yardım gösterir.

Düğme	İşlem
Abonelik Ayarları	Kendi istemcisini seçer → abonelik bağlantısı.
Bireysel Bağlantılar	Kendi istemcisini seçer → bireysel bağlantılar.
QR Kodu	Kendi istemcisini seçer → QR kodları.

Kullanıcının Telegram Kullanıcı Kimliğiyle bağlı hiçbir istemcisi yoksa bot şu yanıtı verir: « Yapılandırmanız bulunamadı!  Lütfen yöneticiden yapılandırmada Telegram Kullanıcı Kimliğinizi kullanmasını isteyin. Kullanıcı Kimliğiniz: ...». Bu Kimliği yöneticiye iletmek gerekir; yönetici bunu istemci alanına girecektir.

14.3. Bot Komutları

Botta Telegram «/» menüsünde görünen dört kayıtlı komut bulunur:

Komut	Açıklama (menüden)	Erişim	Ne Yapar
/start	Ana menüyü göster	herkes	Karşılama mesajı; yöneticiye ek olarak «  yönetim botuna hoş geldiniz!» ve ana menüyü gösterir.
/help	Bot yardımı	herkes	Genel karşılama mesajı ve menüden bir öğe seçme daveti görüntüler.
/status	Bot durumunu kontrol et	herkes	 « Bot normal çalışıyor» yanıtı verir.
/id	Telegram Kimliğinizi göster	herkes	 « Kullanıcı Kimliğiniz: ... » döndürür. Kendi Kullanıcı Kimliğini öğrenmek için kullanışlıdır.

Kayıtlıların yanı sıra, üç komut-argüman daha işlenir («/» menüsünde görünmez ancak çalışır):

- **/usage [E-posta]** – e-postaya göre istemci arama.
 - **Yönetici** için istemcinin tam kartını gösterir (yönetim düğmeleriyle).
 - **İstemci** için yalnızca belirtilen e-postaya ait kendi aboneliğini gösterir (Telegram Kullanıcı Kimliği bağlantısına göre). Argüman olmadan bot e-posta girmesini ister: « Lütfen aranacak e-postayı belirtin».
- **/inbound [bağlantı adı]** – yalnızca yönetici için. inbound bağlantısını Remark'a göre arar ve tüm istemcilerin istatistikleriyle birlikte parametrelerini gösterir. Argüman olmadan (veya istemci için) – « Bilinmeyen komut».
- **/restart** – yalnızca yönetici için. Xray Core'u yeniden başlatır. Olası yanıtlar: « Xray çekirdeği başarıyla yeniden başlatıldı», « Xray Core çalışmıyor» (çekirdek çalışmıyorsa), « Xray-core yeniden başlatılırken hata oluştu.». /restart 'tan sonra herhangi bir argüman, /restart ipucuyla bilinmeyen komut mesajına yol açar.

Grup sohbetlerinde /komut@botusername biçimindeki komut, yalnızca kullanıcı adı geçerli botun adıyla eşleştğinde işlenir.

Yönetici yardımı («Komutlar» düğmesi):

 Xray Core'u yeniden başlatmak için: /restart
 E-postaya göre istemci aramak için: /usage [E-posta]
 Gelen bağlantıları aramak için (istemci istatistikleriyle): /inbound [bağlantı adı]
 Telegram Kullanıcı Kimliğiniz: /id

İstemci yardımı:

 Abonelik bilgilerinizi görüntülemek için: /usage [E-posta]
 Telegram Kullanıcı Kimliğiniz: /id

14.4. İstemci Yönetimi (Yalnızca Yönetici)

İstemci kartı açıldığında («Tüm İstemciler», «Çevrimiçi», «Yakında Sona Erecek» veya /usage üzerinden), yönetici istemci bilgilerini (e-posta, bağlı inbound bağlantıları, «Etkin» durumu, bağlantı durumu, bitiş tarihi, trafik kullanımı) ve inline yönetim düğmelerini görür:

Düğme	Amaç
 Yenile	İstemci kartını yeniden yükle.
 Trafiği Sıfırla	İstemcinin trafik sayacını sıfırla. «  Trafik sıfırlamayı onaylıyor musunuz?» onayı gerektirir.
 Trafik Sınırı	Trafik sınırı ayarla. Hazır değerler: ☹ Sınırsız (0), 1/5/10/20/30/40/50/60/80/100/150/200 GB veya «  Özel» – yerleşik sayısal klavyeyle sayı girişi (0–9 düğmeleri, «  » – 0'a sıfırla, «  » – son rakamı sil, «Onayla: N»). Değer gigabayt cinsinden ayarlanır.
 Bitiş Tarihini Değiştir	Hazır seçenekler: ☹ Sınırsız, «  Özel», 7/10/14/20 gün ekle, 1/3/6/12 ay ekle. Pozitif sayı süreyi uzatır (mevcut bitiş tarihine gün ekler veya süre dolmuşsa «şimdiye» ekler); 0 süre sınırını kaldırır.
 IP Günlüğü	İstemcinin kayıtlı IP adreslerini gösterir (varsa zaman damgalarıyla). Günlükten «  Yenile» ve «  IP'leri Temizle» (onaylı: «  IP temizlemeyi onaylıyor musunuz?») seçenekleri mevcuttur.
 IP Sınırı	Eş zamanlı IP kısıtlaması. Seçenekler: ☹ Sınırsız (0), 1–10 veya «  Özel» (sayısal klavye).
 Telegram Kullanıcısı Ata	İstemcinin bağlı Telegram Kullanıcı Kimliğini gösterir; bağlantıyı kaldırmaya («  Telegram Kullanıcısını Kaldır» onaylı) izin verir. Yeni kullanıcı bağlama, sistem Telegram kişi seçicisi üzerinden yapılır.
 Etkinleştir/ Devre Dışı Bırak	İstemciyi etkinleştirir veya devre dışı bırakır. «  Kullanıcıyı etkinleştir/devre dışı bırak onaylıyor musunuz?» onayı gerektirir.

Yapılandırmayı değiştiren tüm işlemler (trafik/IP sınırı, bitiş tarihi, Telegram kullanıcısı bağlama/ayırma, etkinleştirme/devre dışı bırakma) gerektiğinde Xray'i yeniden başlatmak üzere işaretler; böylece değişiklikler geçerli olur. Başarılı işlem sonrasında bot « : ...» biçiminde onay gösterir ve kartı yeniden görüntüler.

Sihirbazlardaki herhangi bir sayısal giriş < 9999999 değeriyle sınırlıdır.

14.5. Bildirimler ve Raporlar

Bildirimler tüm yöneticilere (tgBotChatId içindeki tüm Kullanıcı Kimliklerine) gönderilir.

Olay Veriyolu ve Bildirim Seçimi

Bildirimler tek bir olay veriyolu üzerine kurulmuştur; iki dağıtım kanalı vardır — **Telegram** ve **e-posta (SMTP)**. Her kanal için hangi olayların bildirileceği ayrı ayrı seçilir. **Ayarlar** → **Telegram** bölümünde bu **Notifications** sekmesinde, **Ayarlar** → **E-posta** bölümünde aynı adlı sekmede yapılır.

Olaylar kartlar halinde gruplandırılmıştır; her grubun etkin olay sayacına (n/toplam) ve yalnızca bir kısmı seçildiğinde ara duruma sahip bir ana anahtarı bulunur. Mevcut gruplar:

- **Outbound** — «Down» (`outbound.down`) ve «Up» (`outbound.up`): outbound bağlantısı düştü ve geri geldi.
- **Xray Core** — «Crash» (`xray.crash`): Xray çekirdeği anormal sonlandı.
- **Nodes** — «Down» (`node.down`) ve «Up» (`node.up`): düğüm erişilemez hale geldi veya geri döndü.
- **System** — «CPU high (%)» (`cpu.high`) ve «Memory high (%)» (`memory.high`): yüksek işlemci ve RAM kullanımı. Her iki olayın yanında yüzde cinsinden eşik için inline alan bulunur.
- **Security** — «Login attempt» (`login.attempt`): panele giriş denemesi.

Etkin olaylar seti ayrı ayrı saklanır: Telegram için `tgEnabledEvents`, E-posta için `smtpEnabledEvents`. Her iki kanalda varsayılan olarak «Login attempt» ve «CPU high» etkindir (değer `login.attempt,cpu.high`).

Panel Girişi Bildirimi

Giriş Bildirimi onay kutusuyla yönetilir (`tgBotLoginNotify`, varsayılan olarak etkin). Web paneline yapılan her giriş denemesinde yöneticilere şu mesaj gönderilir:

- Başarı durumunda: «  Panele başarıyla giriş yapıldı.» + ana bilgisayar, kullanıcı adı, IP, zaman.
- Başarısızlık durumunda: «  Panele giriş başarısız.» + ana bilgisayar, **neden** (örneğin yanlış ikinci faktörde «2FA Hatası»), kullanıcı adı, IP, zaman.

CPU ve Bellek Yük Aşımı

Panel dakikada bir kez işlemci ve RAM kullanımını kontrol eder. **tgCpu** eşiği > 0 ve bir dakikalık ortalama CPU yükü bu eşiği aşarsa yöneticilere şu mesaj gönderilir: «  İşlemci yükü N%'dir, bu M% eşik değerini aşmaktadır». RAM kullanımı da **tgMemory** eşiğine (varsayılan %80) karşı benzer şekilde kontrol edilir — «Memory high (%)» olayı.

Her iki eşik de **System** grubundaki «CPU high (%)» ve «Memory high (%)» olaylarının yanındaki Notifications sekmesinin inline alanlarında ayarlanır (bkz. «Olay Veriyolu ve Bildirim Seçimi» aşağıda). E-posta kanalı için `smtpCpu` ve `smtpMemory` ayrı anahtarları geçerlidir. Eşik değeri 0 olduğunda ilgili kontrol devre dışı bırakılır.

Periyodik Rapor (Zamanlamaya Göre)

Bildirim Sıklığı alanındaki cron ifadesine göre planlanır (`tgRunTime`, varsayılan `@daily`). Değer boş veya geçersizse `@daily` kullanılır. Rapor şunları içerir:

Zamanlama Oluşturucu

Bot'tan Yöneticilere Bildirim Sıklığı alanı, bir dizeyi elle girerek değil, zamanlama oluşturucusu aracılığıyla ayarlanır. Önce açılır listeden mod seçilir:

- **@every** – **aralıkla tekrarlar** – sayı alanı ve birim seçimi (**Saniyeler / Dakikalar / Saatler**) görünür; sonuç @every 6h biçiminde bir ifade olarak derlenir.
- **@hourly** – **her saat**, **@daily** – **her gün 00:00'da**, **@weekly** – **her hafta**, **@monthly** – **her ay** – karşılık gelen makro olarak kaydedilen hazır ön ayarlar (@hourly , @daily , @weekly , @monthly).
- **Özel (crontab)** – kendi crontab ifadeniz için alan. Panel zamanlayıcısı saniyeler dahil çalışır; bu nedenle özel ifade **6 alandan** oluşur: saniye, dakika, saat, ayın günü, ay, haftanın günü (örneğin **0 30 8 * * *** – her gün 08:30:00'da). Bu moda geçildiğinde, başlangıç noktası olarak alan, geçerli seçimin crontab eşdeğeriyle doldurulur.

Örnek: «Bildirim Sıklığı» (tgRunTime) alanı değerleri. Hem hazır kısaltmalar hem de tam crontab biçimi desteklenir:

Değer	Ne Zaman Tetiklenir
@daily	gün aşırı gece yarısında (varsayılan değer)
@hourly	her saat
@every 6h	her 6 saatte bir
0 9 * * *	her gün saat 09:00'da
0 9 * * 1	her Pazartesi saat 09:00'da
0 */12 * * *	her 12 saatte bir (00:00 ve 12:00'da)

Crontab alan sırası: dakika, saat, ayın günü, ay, haftanın günü.

1. « Zamanlanmış raporlar: » satırı ve geçerli tarih/saat.
2. **Sunucu Durumu** (aşağıya bakın).
3. inbound bağlantıları ve istemciler için «Yakında Sona Erecek» bloğu.
4. Telegram Kullanıcı Kimliği bağlı istemcilere kişisel bildirimler – yönetici olmayan her istemciye trafiği veya süresi yakında dolacak aboneliklerinin listesi gönderilir (devre dışı bırakılanlar dahil).
5. **Veritabanı Yedekleme** (tgBotBackup) etkinse – yöneticilere veritabanı yedeği.

Sunucu Durumu şunları içerir: ana bilgisayar, 3X-UI ve Xray sürümü, IPv4/IPv6, çalışma süresi (gün cinsinden), ortalama yük (Load1/2/3), RAM (geçerli/toplam), çevrimiçi istemci sayısı, TCP/UDP bağlantı sayaçları, toplam ağ trafiği (↑/↓) ve Xray durumu.

«Yakında Sona Erecek» şunları gösterir:

- inbound bağlantıları için: devre dışı bırakılan ve «yakında dolacak» bağlantı sayısı, ardından bu bağlantıların listesi (Remark, port, trafik, bitiş tarihi);
- istemciler için: aynı bilgiler artı istemci kartları ve e-postalarıyla düğmeler (tıklandığında istemci kartı açılır).

«Yakında dolacak» eşikleri genel panel ayarlarından alınır: trafik marjı (GB cinsinden) ve süre marjı (gün cinsinden). Trafik sınırına kalan miktarı eşikten az VEYA bitiş tarihine kalan gün sayısı eşikten az olan bir inbound bağlantısı/istemci «dolmakta olan» olarak kabul edilir.

14.6. Yedekleme ve Günlükler

- **Veritabanı Yedeği** (« Veritabanı Yedeği» düğmesi veya periyodik rapordaki onay kutusu): bot yedekleme zamanını, veritabanı dosyasını (`x-ui.db` veya PostgreSQL için `x-ui.dump`) ve Xray yapılandırma dosyası `config.json` 'u gönderir.
- **Ban Günlüğü** (« Ban Günlüğü» düğmesi): IP sınırı aşımı nedeniyle yasaklanan IP adreslerinin geçerli ve önceki günlük dosyalarını gönderir. Boş dosyalar gönderilmez.

14.7. Çalışma Özellikleri

- **Uzun mesajlar** parçalara bölünür (~2000 karakter eşiği); inline klavye son parçaya eklenir.
- **Paralellik**: komutlar ve düğme tıklamaları eş zamanlı olarak işlenir (en fazla 10 eş zamanlı işleyicili havuz).
- **Gönderim güvenilirliği**: bağlantı hatalarında mesajlar üstel geri çekilmeyle yeniden gönderilir (1s/2s/4s, en fazla 3 deneme).
- **Önbelleğe alma**: «Sunucu Durumu» verileri önbelleğe alınır; böylece sık «Yenile» tıklamaları sistemi yormaz.
- **Bot yeniden başlatma**: ayarlar kaydedilip panel yeniden başlatıldığında önceki sorgulama döngüsü düzgün biçimde durdurulur ve yeni bir döngü başlatılır; aynı anda yalnızca bir güncelleme alma örneği çalışır.

15. Coğrafi Veritabanları (geoip / geosite ve Özel Kaynaklar)

Coğrafi veritabanları, Xray-core'un trafiği ülkeye göre (IP aralıkları) veya alan adı kategorisine göre yönlendirmek ve filtrelemek için kullandığı ikili .dat dosyalarıdır. Panel, hem standart geo-dosya setini hem de URL ile belirtilen rastgele kullanıcı tanımlı kaynakları indirip güncelleyebilir. Tüm dosyalar, Xray ikili dosyasının yanındaki bin dizininde saklanır (varsayılan yol bin ; XUI_BIN_FOLDER ortam değişkeniyle değiştirilebilir).

15.1. geoip.dat ve geosite.dat Nedir?

- **geoip.dat** – «IP adresi → ülke/bölge kodu» eşleme veritabanıdır. Yönlendirme kurallarında `geoip:<kod>` biçiminde kullanılır; örneğin `geoip:ru` , `geoip:cn` ve özel etiket `geoip:private` (özel/yerel ağlar). Kısaca «bu IP hangi ülkede?» sorusunu yanıtlar.
- **geosite.dat** – «alan adı → kategori/liste» eşleme veritabanıdır. `geosite:<kategori>` biçiminde kullanılır; örneğin `geosite:category-ads-all` (reklam alan adları), `geosite:google` , `geosite:ru` . Kısaca gruplandırılmış alan adı listeleridir.

Bu dosyalar, «Rus IP'lerine/alan adlarına giden tüm trafik doğrudan gitsin, geri kalanı outbound üzerinden geçsin» gibi kurallar oluşturmak için gereklidir. Kuralların kendisi Xray yönlendirme bölümünde tanımlanır; coğrafi veritabanları yalnızca bu kurallara veri sağlar. Güncel geo-dosyaları olmadan `geoip: / geosite:` referanslı kurallar çalışmaz ya da güncel olmayan listelere dayanır.

Örnek: «Rus alan adları ve IP'leri doğrudan» kuralı. Yönlendirme bölümündeki bu kural, Rus kaynaklarına giden tüm trafiği `direct` etiketli outbound'a yönlendirir:

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. Standart Geo-Dosyalar ve Güncelleme

Panel, sabit kodlanmış indirme kaynaklarıyla birlikte altı standart dosyadan oluşan sabit bir «izin listesi» (allowlist) içerir. Güncelleme işlemi `POST /panel/api/server/updateGeofile/:fileName` (veya dosya adı belirtilmeden – tümünü aynı anda güncellemek için) aracılığıyla gerçekleştirilir.

Örnek: API üzerinden tek dosya ve tüm dosyaların güncellenmesi. Yalnızca `geoip_RU.dat` dosyasını güncellemek için:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/
geoip_RU.dat' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Tek bir istekle altı standart dosyanın tamamını güncellemek için (dosya adı belirtilmez):

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Başarılı yanıt:

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

Dosya adı	Kaynak (sürüm deposu)
geoip.dat	github.com/Loyalsoldier/v2ray-rules-dat (geoip.dat)
geosite.dat	github.com/Loyalsoldier/v2ray-rules-dat (geosite.dat)
geoip_IR.dat	github.com/chocolate4u/iran-v2ray-rules (geoip.dat)
geosite_IR.dat	github.com/chocolate4u/iran-v2ray-rules (geosite.dat)
geoip_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geoip.dat)
geosite_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geosite.dat)

Standart dosyaların güncellenmesine ilişkin özellikler:

- **Tek dosya güncelleme düğmesi.** İndirmeden önce onay iletişim kutusu gösterilir: «Geo-dosyayı gerçekten güncellemek istiyor musunuz?» ile «Bu işlem #filename# dosyasını güncelleyecek.» açıklaması (*Do you really want to update the geofile? This will update the #filename# file.*). Başarı durumunda «Geo-dosyalar başarıyla güncellendi» (*Geofile updated successfully*) bildirimi görünür.
- **«Tümünü Güncelle»** (*Update all*) düğmesi altı dosyanın tamamını indirir. Onay: «Bu işlem tüm geo-dosyaları güncelleyecek.» (*This will update all geofiles.*).
- **Koşullu indirme.** Yerel dosya zaten mevcutsa, isteğe dosyanın değiştirilme zamanını içeren `If-Modified-Since` başlığı eklenir. Sunucudan `304 Not Modified` yanıtı gelmesi dosyanın değişmediği anlamına gelir – dosya yeniden indirilmez, yalnızca dosya zaman damgası güncellenir.
- **Dosya adı güvenliği.** Yalnızca allowlist'teki adlar kabul edilir; ad `..`, yol ayırıcıları `/` ve `\`, mutlak yollar içermemeli ve `^[a-zA-Z0-9._-]+\\.dat$` şablonuyla eşleşmelidir. Liste dışındaki her ad «Invalid geofile name» hatasıyla reddedilir.
- **Xray yeniden başlatma.** Geo-dosyaları indirildikten sonra Xray-core, güncellenmiş veritabanlarını yeniden okumak üzere yeniden başlatılır. Yeniden başlatma başarısız olursa hata iletisine ilgili satır eklenir.

Geo-Veritabanlarını Komut Satırından Güncelleme (x-ui)

Geo-veritabanları panel olmadan da güncellenebilir – `x-ui` etkileşimli menüsünden (geo-dosya güncelleme seçeneği) veya etkileşimsiz `x-ui update-all-geofiles` komutuyla. Her dosya seti (geoip/geosite, IR ve RU setleri dahil) için ayrı bir durum görüntülenir: «güncellendi», «zaten güncel» veya «indirme hatası». İndirme başarısız olduğunda yanlış bir başarı iletisi yazdırılmaz. Xray yeniden başlatma (dolayısıyla etkin bağlantıların kesilmesi) yalnızca en az bir dosya gerçekten güncellendiğinde gerçekleşir; hiçbir dosya değişmediyse (tamamı `304 Not Modified` döndürdüyse) panel ve Xray yeniden başlatılmaz.

15.3. Xray Aracılığıyla Geo-Verilerinin Otomatik Güncellenmesi (Geodata Auto-Update)

Rastgele URL'lerden ek `.dat` kaynakları, panel araçlarıyla değil Xray-core'un yerel `geodata` bölümü aracılığıyla eklenir. İlgili bölüm, Xray güncellemeleri modal penceresinde yer alır (Kontrol Paneli → Xray güncellemeleri, `xrayUpdates`) – bu «Geodata Otomatik Güncelleme» (*Geodata Auto-Update*) sekmesidir. Panel burada yalnızca Xray yapılandırma şablonundaki `geodata` anahtarını düzenler; dosyaların indirilmesi, doğrulanması ve canlı yeniden yüklenmesi Xray çekirdeğinin kendisi tarafından yapılır.

Bölümün üst kısmında ipucu gösterilir: «Xray bu dosyaları zamanlamaya göre indirir ve yeniden başlatmadan sıcak olarak yeniden yükler. URL'ler HTTPS olmalıdır. Xray dosyayı güncelleyebilmeden önce dosyanın bin klasöründe zaten mevcut olması gerekir.» (*Xray downloads these files on schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it.*).

Bölüm Alanları

- **Zamanlama (cron)** (*Schedule (cron)*) – 5 alanlı bir cron dizesi; varsayılan değer `0 4 * * *` (her gün 04:00'da). Kaydedilirken dizinin tam olarak 5 alan içerdiği doğrulanır; aksi takdirde «Cron 5 alan içermelidir, örn. `0 4 * * *`» hatası görüntülenir.
- **Outbound üzerinden indir (isteğe bağlı)** (*Download through outbound (optional)*) – Xray'in dosyaları indireceği mevcut outbound etiketlerinin (abonelik outbound'ları dahil) listesini gösteren açılır menü; `blackhole` protokollü outbound'lar listeye dahil edilmez. Alan boş bırakılabilir – bu durumda doğrudan bağlantı kullanılır. Bu seçim, panelin kendi istekleri için kullanılan outbound'dan (bkz. §11) bağımsızdır: `geodata` otomatik güncellenmenin indirme için kendi ayrı outbound'u vardır.
- **Dosya listesi** – her satır bir «URL + Dosya adı» (*File name*) çifti tanımlar. URL `https://` ile başlamalıdır (aksi takdirde «Her dosya için HTTPS URL gereklidir.»). Dosya adı yol ve ayırıcı içermeyen sade biçimde belirtilmelidir – yalnızca `^[A-Za-z0-9._-]+$` karakterleri (aksi takdirde «Dosya adı sade olmalıdır, örneğin `geosite_custom.dat` (yol içermeyen).»). URL girildiğinde panel, yolun son segmentinden dosya adını otomatik olarak doldurmaya çalışır. «Dosya Ekle» (*Add file*) düğmesi yeni satır ekler, çöp kutusu düğmesi satırı siler.

Liste boşsa şu ipucu gösterilir: «Yapılandırılmış dosya yok. Yönlendirme kurallarında dosyalara `ext:geosite_custom.dat:category` biçiminde başvurun.» (*No files configured. Reference files in routing rules as ext:geosite_custom.dat:category.*).

Kaydetme

«Kaydet ve Xray'i Yeniden Başlat» (*Save & Restart Xray*) düğmesi «Geodata ayarları kaydedilsin mi?» onayıyla birlikte «Xray yapılandırma şablonu güncellenecek ve Xray yeniden başlatılacak.» (*Save geodata settings? This updates the Xray config template and restarts Xray.*) açıklamasını gösterir. Kaydedildikten sonra `geodata` anahtarı yapılandırma şablonuna yazılır (`POST /panel/api/xray/update`) ve Xray yeniden başlatılır (`POST /panel/api/server/restartXrayService`). Dosya listesi boşsa `geodata` anahtarı şablondan kaldırılır.

Önemli özellikler:

- **Dosya bin dizininde zaten mevcut olmalıdır.** Xray yalnızca başlatma sırasında `bin` klasöründe zaten bulunan `.dat` dosyalarını günceller. Bu nedenle yeni bir özel dosya önce `bin` dizinine elle yerleştirilmeli

(veya en azından gerekli adla boş/eski bir sürüm oluşturulmalı), ardından Xray dosyayı zamanlamaya göre güncel tutmaya başlar.

- **Sıcak yeniden yükleme.** Planlı indirmeden sonra Xray, işlemi tamamen yeniden başlatmadan güncellenmiş veritabanlarını yeniden okur.
- **Uyumluluk.** Önceden indirilen geo-dosyaları (hem standart hem de özel) değişiklik olmaksızın `ext:` söz dizimiyle yönlendirme kurallarında çalışmaya devam eder.

Liste boşsa şu ipucu görüntülenir: «Henüz özel geo kaynağı yok – oluşturmak için Ekle'ye tıklayın» (*No custom geo sources yet – click Add to create one*).

Tablo Sütunları ve Kaynak Alanları

Alan (UI)	JSON	Varsayılan değer	Açıklama
Tür (<i>Type</i>)	<code>type</code>	– (zorunlu)	Kaynak türü: yalnızca <code>geosite</code> veya <code>geoip</code> . Sonuç dosyasının adını belirler.
Takma ad (<i>Alias</i>)	<code>alias</code>	– (zorunlu)	Kaynağın kısa tanımlayıcısı. Dosya adı, tür ve takma addan oluşturulur.
URL (<i>URL</i>)	<code>url</code>	– (zorunlu)	<code>.dat</code> dosyasına doğrudan bağlantı (http / https).
Etkin (<i>Enabled</i>)	–	–	Listedeki kaynağın etkinlik durumu.
Güncellenme zamanı (<i>Last updated</i>)	<code>lastUpdatedAt</code>	<code>0</code>	Son başarılı güncellemenin zamanı (Unix zaman damgası; <code>0</code> – henüz güncellenmedi).
Yönlendirme (ext:...) (<i>Routing (ext:...)</i>)	–	–	Yönlendirme kuralları için hazır dize: <code>ext:<dosya.dat>:tag</code> .
Eylemler (<i>Actions</i>)	–	–	«Düzenle», «Sil», «Şimdi Güncelle» düğmeleri.

Veritabanında ek dahili alanlar da saklanır: `localPath` (bin dizinindeki gerçek dosya yolu), `lastModified` (sunucudan gelen `Last-Modified` başlık değeri, koşullu indirme için kullanılır), `createdAt` ve `updatedAt`.

Dosya Adlandırma

Sonuç dosyasının adı, tür ve takma addan otomatik olarak oluşturulur:

- tür `geoip` → `geoip_<alias>.dat`;
- tür `geosite` → `geosite_<alias>.dat`.

Örneğin `geosite` türüyle `myads` takma adına sahip bir kaynak `geosite_myads.dat` dosyasını oluşturur.

Örnek: API üzerinden kaynak ekleme. `myads` takma adıyla özel bir reklam alan adı listesini `geosite` kaynağı olarak eklemek için:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
-H 'Cookie: 3x-ui=<session-cookie>' \
-H 'Content-Type: application/json' \
-d '{
  "type": "geosite",
  "alias": "myads",
  "url": "https://example.com/lists/myads.dat"
}'
```

Panel, dosyayı `bin` dizinine `geosite_myads.dat` olarak indirecek, kaydı saklayacak ve Xray'i yeniden başlatacaktır.

Düğmeler ve Eylemler

- **Ekle (Add)** – «Kaynak Ekle» (*Add custom geo*) formunu açar. Kaydetme düğmesi – «Kaydet» (*Save*). API: `POST /add`.
- **Düzenle (Edit)** – «Kaynağı Düzenle» (*Edit custom geo*) formu. API: `POST /update/:id`. Tür veya takma ad değiştirildiğinde eski dosya silinir, yeni dosya yeniden indirilir.
- **Sil (Delete)** – «Bu özel geo kaynağını silmek istiyor musunuz?» (*Delete this custom geo source?*) onayı. Kaydı veritabanından ve `.dat` dosyasını siler. API: `POST /delete/:id`. Başarı durumunda: «Özel geo dosyası «» silindi».
- **Şimdi Güncelle (Update now)** – belirli bir kaynağı yeniden indirir ve zaman damgasını günceller. API: `POST /download/:id`. Başarı durumunda: «Geofile «» güncellendi».
- **Tümünü Güncelle** – tüm özel kaynakları aynı anda günceller. API: `POST /update-all`. Tamamen başarılı olduğunda: «Tüm özel geo kaynakları güncellendi» (*All custom geo sources updated*). En az bir kaynak güncellenemezse işlem kısmen başarısız sayılır ve «Bir veya daha fazla özel geo kaynağı güncellenemedi» (*One or more custom geo sources failed to update*) iletilisiyle başarılı ve başarısız kaynaklar yanıtta listelenir.

Eylemlerden herhangi birinin ardından (ekleme, düzenleme, silme, güncelleme, başarıların olduğu toplu güncelleme) Xray-core yeniden başlatılır.

Adım Adım: Kaynak Ekleme

1. «Ekle» düğmesine tıklayın.
2. «Tür» alanında `geosite` veya `geoip` seçin.
3. «Takma ad» alanına bir tanımlayıcı girin (yalnızca küçük Latin harfleri, rakamlar, `-` ve `_`; yer tutucu ipucu: `a-z 0-9 _ -`).
4. «URL» alanına `.dat` dosyasına doğrudan bağlantıyı girin (`http://` veya `https://` ile başlamalıdır).
5. «Kaydet» düğmesine tıklayın. Panel dosyayı hemen `bin` dizinine indirecek, kaydı saklayacak ve Xray'i yeniden başlatacaktır.

15.4. Doğrulama ve Kısıtlamalar

Kaynak oluşturma ve değiştirme sırasında katı kontroller uygulanır. Hata iletileri:

Koşul	İleti (TR)	İleti (EN)
Tür <code>geosite</code> / <code>geoip</code> değil	Tür <code>geosite</code> veya <code>geoip</code> olmalıdır	<i>Type must be geosite or geoip</i>

Koşul	İleti (TR)	İleti (EN)
Takma ad boş	Takma ad gereklidir	<i>Alias is required</i>
Takma adadaki geçersiz karakterler (<code>^[a-z0-9_-]+\$</code> eşleşmesi yok)	Takma ad geçersiz karakterler içeriyor	<i>Alias must match allowed characters</i>
Takma ad rezerve edilmiş	Bu takma ad rezerve edilmiş	<i>This alias is reserved</i>
URL boş	URL gereklidir	<i>URL is required</i>
URL ayrıştırılamıyor	Geçersiz URL	<i>URL is invalid</i>
Şema http/https değil	URL http veya https kullanmalıdır	<i>URL must use http or https</i>
Boş/geçersiz host veya SSRF koruması tarafından engellendi	Geçersiz URL hostu	<i>URL host is invalid</i>
«Tür + takma ad» tekrarı	Bu takma ad bu tür için zaten kullanımda	<i>This alias is already used for this type</i>
Kaynak bulunamadı	Kaynak bulunamadı	<i>Custom geo source not found</i>
İndirme hatası	İndirme başarısız	<i>Download failed</i>

Formdaki ipuçları (istemci tarafı doğrulama): «Takma ad: yalnızca a-z, rakamlar, - ve _» (*Alias may only contain lowercase letters, digits, - and _*) ve «URL http:// veya https:// ile başlamalıdır» (*URL must start with http:// or https://*).

Ek teknik kısıtlamalar:

- **Rezerve edilmiş takma adlar.** Standart dosyalarla çakışan takma adlar kullanılamaz. Rezerve edilenler (büyük/küçük harf duyarsız karşılaştırma, tire alt çizgiye eşdeğer sayılır): `geoip` , `geosite` , `geoip_ir` , `geosite_ir` , `geoip_ru` , `geosite_ru` .Örneğin `geosite-ru` , `geosite_ru` olarak reddedilir.
- **SSRF koruması.** URL hostu IP'ye çözümlenir; özel/iç bir adrese işaret ediyorsa indirme engellenir (kullanıcı «Geçersiz URL hostu» görür). Bu, panelin iç servislere erişim için kullanılmasını önler.
- **Yol geçişi koruması.** Dosyanın nihai yolu sembolik bağlantılar çözümlendikten sonra `bin` dizini içinde kalmalıdır; dışına çıkma girişimleri reddedilir.
- **Minimum dosya boyutu.** İndirilen dosya ancak 64 bayttan küçük değilse geçerli sayılır; çok küçük dosyalar indirme hatasıyla reddedilir.
- **Proxy ve koşullu indirme.** Panel ayarlarında proxy tanımlanmışsa indirme onun üzerinden gerçekleşir; diğer durumlarda SSRF güvenli aktarımla doğrudan bağlantı kullanılır. Standart dosyalarda olduğu gibi `If-Modified-Since / 304 Not Modified` uygulanır (değişmemiş dosyalar yeniden indirilmez). İndirme zaman aşımı 10 dakika, URL erişilebilirlik testi (HEAD, gerekirse kısmi GET) 12 saniyedir.

15.5. Panel Başlangıcında Otomatik Kontrol

Panel başlatıldığında tüm özel kaynakları tarar ve her biri için yerel dosyanın varlığını ve bütünlüğünü kontrol eder (dosya yok, izin veya 64 bayttan küçük). Dosya eksik veya bozuksa kaynak test edilir ve yeniden indirme denenir. Bu, `bin` dizininin yeniden kurulumdan veya kaybolmadan sonra özel geo-dosyalarının otomatik olarak geri yükleneceğini garanti eder.

15.6. Yönlendirme Kurallarında Geo-Veritabanlarının Kullanımı

Xray yönlendirme kurallarında geo-veritabanları `domain / ip` gibi alanlarda örnekler aracılığıyla kullanılır:

- **geoip:** IP veritabanları için — `geoip:<kod>`. Örnekler: `geoip:ru`, `geoip:cn`, `geoip:private`. `geoip.dat` dosyasından alınır (veya kural belirli bir dosyaya işaret ediyorsa `geoip_RU.dat` vb.).
- **geosite:** Alan adı veritabanları için — `geosite:<kategori>`. Örnekler: `geosite:category-ads-all`, `geosite:google`, `geosite:ru`. `geosite.dat` dosyasından alınır.

Örnek: geosite üzerinden reklam engelleme. Tüm reklam alan adlarını «kara deliğe» gönderen kural (`blocked` etiketli ve `blackhole` protokollü bir outbound varsayılır):

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

Özel dosyalar için `ext:` harici dosya söz dizimi kullanılır. UI'daki ipucu: «Yönlendirme kurallarında değer sütununu `ext:dosya.dat:etiket` biçiminde kullanın (etiketi değiştirin).» (*In routing rules use the value column as `ext:file.dat:tag` (replace tag).*). Biçim:

```
ext:<dosya_adı.dat>:<etiket>
```

burada `<dosya_adı.dat>`, `geoip_<alias>.dat` veya `geosite_<alias>.dat`; `<etiket>` ise dosya içindeki belirli liste/kategoridir. Panel «Yönlendirme (ext:...）」 sütununda `ext:geosite_myads.dat:tag` biçiminde hazır bir şablon gösterir — `tag` yerine gerekli etiketi yazmak yeterlidir. Bu tür bir dosyanın adı «Geodata Otomatik Güncelleme» bölümünde (bkz. §15.3) «Dosya adı» alanında belirlenir — örneğin `geosite_custom.dat`; kurallarda `ext:geosite_custom.dat:category` biçiminde başvurulur.

Örnek: özel dosyaya dayalı kural. `myads` takma adıyla `geosite` türünde bir kaynak eklenmiş ve `.dat` dosyası içindeki liste `ads` etiketiyle işaretlenmişse yönlendirme kuralı şöyle görünür:

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

IP kaynağı için (tür `geoip`, takma ad `mycorp`, etiket `office`) alan `"ip": ["ext:geoip_mycorp.dat:office"]` olacaktır.

16. İşletim: Yedekler, Loglar, Güncelleme, CLI

Bu bölüm panelin günlük bakımını kapsar: veritabanı yedekleme ve geri yükleme, panel ve Xray loglarını görüntüleme, servisleri yeniden başlatma ve durdurma, Xray ile panelin güncellenmesi, periyodik görevler (cron) ve panelin kaldırılması. İşlemlerin bir kısmı web arayüzünden yapılır («Dashboard» ve «Panel Ayarları» sayfalarındaki sekmeler), diğerleri ise sunucudaki `x-ui` konsol menüsünden gerçekleştirilir.

16.1. Veritabanı Yedekleme ve Geri Yükleme

Panelin tüm verileri (inbound'lar, istemciler, gruplar, düğümler, ayarlar) tek bir veritabanında saklanır. Yedek yönetimine «**Dashboard**» sayfasındaki «**Yedek**» sekmesinden, blok başlığı «**Yedek ve Geri Yükleme**» olan bölümden erişilir.

Panel iki farklı veritabanı motorunu destekler ve yedek davranışı buna göre farklılık gösterir:

- **SQLite** (varsayılan seçenek) – veriler `x-ui.db` dosyasında saklanır.
- **PostgreSQL** – panel PostgreSQL üzerinde çalışıyorsa blokta şu bilgi gösterilir:

«Bu panel PostgreSQL üzerinde çalışmaktadır. «Yedek» işlemi bir `pg_dump` arşivi (.dump) indirir; «Geri Yükleme» ise bunu `pg_restore` aracılığıyla geri yükler. Sunucuda PostgreSQL istemci araçlarının (`pg_dump` ve `pg_restore`) kurulu olması gerekir.»

Dışa Aktarma (Yedek Oluşturma)

«**Veritabanını Dışa Aktar**» düğmesi (İng. **Back Up**) yedek dosyasını cihazınıza indirir.

Veritabanı Motoru	Dosya Adı	Sunucuda Ne Olur
SQLite	<code>x-ui.db</code>	Önce WAL checkpoint yapılarak dosyanın son kayıtları içermesi sağlanır, ardından dosya tümüyle okunup indirmeye sunulur
PostgreSQL	<code>x-ui.dump</code>	<code>pg_dump</code> çalıştırılır ve arşiv indirmeye sunulur

Arayüzdeki ipuçları:

- **SQLite**: «Mevcut veritabanınızın yedeğini içeren .db dosyasını cihazınıza indirmek için tıklayın.»
- **PostgreSQL**: «Mevcut veritabanınızın PostgreSQL dumpını (.dump) cihazınıza indirmek için tıklayın.»

Teknik olarak dışa aktarma, `GET /panel/api/server/getDb` isteğidir. Ekte dosyanın adı sunucu tarafından motora göre (`Content-Disposition`) belirlenir.

Yedek dosyasının adı `x-ui.db` / `x-ui.dump` şeklinde sabit değil, sunucu adresine göre oluşturulur. Tarayıcıdan indirildiğinde adres çubuğundaki panel adresinden (istek host'u) alınır; yoksa yapılandırılmış web domain'inden, o da yoksa sunucunun genel IP'sinden (önce IPv4, sonra IPv6) türetilir; hiçbiri yoksa `x-ui` kullanılır. Bu sayede farklı sunuculardan alınan yedekler kolayca ayırt edilir. Uzantı SQLite için `.db` ,

PostgreSQL için `.dump` olarak kalır; Telegram üzerinden alınan yedekler de aynı domain/IP adlandırmasını kullanır.

Örnek: API üzerinden yedek indirme. Aynı dışı aktarmayı konsol üzerinden de yapabilirsiniz – örneğin otomatik yedekleme betiği için. Kimlik doğrulamalı oturum (giriş cookie'si) gereklidir:

```
# 1) Giriş yapıp oturum cookie'sini kaydediyoruz
curl -s -c cookies.txt \
  -d 'username=admin&password=admin' \
  https://panel.example.com:2053/panel/login

# 2) Veritabanı dosyasını indiriyoruz (ad sunucu tarafından belirlenir: x-ui.db veya x-
ui.dump)
curl -s -b cookies.txt -OJ \
  https://panel.example.com:2053/panel/api/server/getDb
```

Panel temel yol (Web Base Path) ile açılıyorsa URL'ye eklenmesi gerekir: `...:2053/<base_path>/panel/api/server/getDb`.

İçe Aktarma (Geri Yükleme)

«Veritabanını İçe Aktar» düğmesi (İng. Restore) dosya seçimini açar ve geri yükleme için sunucuya yükler (POST /panel/api/server/importDB ,form alanı db).

Arayüzdeki ipuçları:

- SQLite: «Veritabanını yedekten geri yüklemek için cihazınızdan bir .db dosyası seçip yüklemek üzere tıklayın.»
- PostgreSQL: «PostgreSQL veritabanını geri yüklemek için bir .dump dosyası seçip yüklemek üzere tıklayın. Bu işlem mevcut tüm verilerin yerini alır.»

SQLite için içe aktarma süreci (atomik ve geri alınabilir yapısını anlamak önemlidir):

1. Yüklenen dosya format açısından kontrol edilir – geçerli bir SQLite veritabanı olmalıdır; değilse «Invalid db file format» hatası döner.
2. Dosya geçici `x-ui.db.temp` konumuna kaydedilir ve bütünlük kontrolünden geçirilir.
3. Veritabanı değiştirilmeden önce **Xray durdurulur**.
4. Mevcut veritabanı yedek olarak `x-ui.db.backup` adıyla yeniden adlandırılır (geri dönüş için).
5. Geçici dosya çalışma veritabanının yerine taşınır, şema başlatması ve migrasyonları çalıştırılır, ardından inbound migrasyonu yapılır.
6. **Herhangi bir adım başarısız olursa** geri alma devreye girer: önceki veritabanı `x-ui.db.backup` 'tan geri yüklenir ve Xray eski verilerle yeniden başlatılır.
7. Başarı durumunda yedek dosya silinir ve **Xray geri yüklenen verilerle otomatik olarak yeniden başlatılır**.

İşlem sonucunda arayüz iletileri:

Sonuç	Metin
Başarı	«Veritabanı başarıyla içe aktarıldı»
İçe aktarma hatası	«Veritabanı içe aktarılırken bir hata oluştu»

Sonuç	Metin
Dosya okuma hatası	«Veritabanı okunurken bir hata oluştu»

Geri yükleme mevcut verilerin tamamının yerini alır. Xray işlem sırasında kısa süreliğine durduğundan içe aktarma süresince mevcut istemci bağlantıları kesilir.

Motorlar Arasında Geçiş Dosyası (SQLite ⇌ PostgreSQL)

Normal yedekten bağımsız olarak «**Geçiş Dosyasını İndir**» (Download Migration , istek GET /panel/api/server/getMigration) işlevi mevcuttur. Bu işlev başka bir veritabanı motoruna geçiş için taşınabilir bir dosya oluşturur:

Mevcut Motor	İndirilen	Dosya Adı	Amaç
SQLite	Taşınabilir SQL dumpı (metin)	x-ui.dump	PostgreSQL'i verilerinizle başlatmak için
PostgreSQL	PostgreSQL verilerinden oluşturulmuş SQLite veritabanı	x-ui.db	Paneli tekrar SQLite'a geçirmek için

İpuçları:

- SQLite'ta: «SQLite veritabanınızın taşınabilir .dump (SQL metin) dışa aktarımını indirmek için tıklayın.»
- PostgreSQL'de: «PostgreSQL verilerinizden oluşturulmuş ve paneli SQLite üzerinde çalıştırmaya hazır SQLite veritabanını (.db) indirmek için tıklayın.»

SQLite için .db ⇌ .dump dönüşümü CLI'dan x-ui migrateDB [file] komutuyla da yapılabilir (bkz. bölüm 16.7).

Telegram Botu Üzerinden Yedek

Telegram botu yapılandırıldıysa (bkz. bildirimler bölümü), bot yedek kopyayı doğrudan yönetici sohbetine gönderebilir. Telegram üzerinden yedek **iki dosya** içerir: veritabanının kendisi (x-ui.db veya PostgreSQL'de x-ui.dump) ve Xray yapılandırması config.json . İletinin önüne « Yedekleme zamanı: ...» satırı eklenir.

Telegram'dan yedek almanın iki yolu vardır:

1. **İstek üzerine.** Bot menüsündeki « **Veritabanı Yedeği**» düğmesi — bot dosyaları hemen mevcut sohbete gönderir.
2. **Raporla birlikte otomatik.** Bot ayarlarında «Veritabanı yedeğini rapor bildirimleriyle birlikte gönder» açıklamalı «**Veritabanı Yedekleme**» (Database Backup) anahtarı bulunur. Bu seçenek etkinleştirildiğinde her periyodik rapor gönderiminde bot, raporu tüm yöneticilere gönderdikten sonra yedeği de ekler. Rapor gönderme periyodu bot'un cron zamanlamasıyla belirlenir (bkz. bölüm 16.6). Bot, Telegram limitlerini aşmamak için dosyalar ve yöneticiler arasında bekleme yapar.

Bot üzerinden yedek yalnızca bot çalışırken gönderilir; PostgreSQL'de sunucuda `pg_dump` kurulu olması da gerekir.

16.2. Logları Görüntüleme

Panelde birbirinden bağımsız iki log görüntüleyici bulunur; her ikisi de «Dashboard»daki «**Loglar**» sekmesinden açılır. Her pencere yenilenebilir (başlıktaki «yenile» simgesi) ve gösterileni `x-ui.log` dosyasına indirebilir (indirme simgesi düğme).

Panel (Uygulama / Syslog) Logları

Panel log penceresi (`POST /panel/api/server/logs/{count}`). Kontrol öğeleri:

Öge	Varsayılan Değer	Açıklama
Satır sayısı	20	Açılır liste: 10 / 20 / 50 / 100 / 500
Seviye	Info	Minimum seviye: Debug / Info / Notice / Warning / Error
SysLog (onay kutusu)	kapalı	Logların nereden alınacağı: uygulama arabelleğinden mi yoksa sistem günlüğünden mi

Davranış **SysLog** onay kutusuna göre değişir:

- **Kapalı (varsayılan):** Loglar, panelin dahili döngüsel arabelleğinden seçilen seviyeye göre filtrelenerek alınır. Kayıtlar seviyeyle (DEBUG / INFO / NOTICE / WARNING / ERROR) ve kaynakla birlikte gösterilir: **X-UI:** – panelin kendi iletileri, **XRAY:** – Xray'den yönlendirilen iletiler.
- **Açık:** Panel sunucuda `journalctl -u x-ui --no-pager -n <count> -p <level>` komutunu çalıştırır; yani `x-ui` servisinin sistem günlüğünü gösterir. İzin verilen satır sayısı 1 ile 10000 arasındadır; seviye syslog değerlerini kabul eder (`emerg/0` , `alert/1` , `crit/2` , `err/3` , `warning/4` , `notice/5` , `info/6` , `debug/7`). Windows'ta SysLog modu desteklenmez – onay kutusunu kaldırıp uygulama loglarını kullanmanız gerektiğine dair bir uyarı gösterilir. `systemd/servis` kullanılamıyorsa `journalctl` başlatma hatası iletileri görüntülenir.

Örnek: Aynı günlüğü sunucu konsolundan okuma. Panel kullanılamıyorsa (örneğin başlamıyorsa) sistem günlüğü doğrudan okunabilir – SysLog modunda panelin çalıştırdığı komutun ta kendisidir:

```
# warning ve üzeri seviyede son 100 satır
journalctl -u x-ui --no-pager -n 100 -p warning

# günlüğü gerçek zamanlı izleme
journalctl -u x-ui -f
```

Bu penceredeki seviye **çıkırtı** filtreler. Konsola/syslog'a hangi minimum seviyenin yazıldığı panel loglama seviyesiyle belirlenir (ortam değişkeni, varsayılan `Info` ; dosyaya panel her zaman `DEBUG` seviyesinde yazar).

Xray Logları (Erişim Günlüğü)

Xray erişim günlüğü için ayrı bir pencere (`POST /panel/api/server/xraylogs/{count}`). Xray erişim günlüğü satırlarını ayrıştırarak tablo şeklinde gösterir: **Date, From, To, Inbound, Outbound, Email**.

Öge	Varsayılan Değer	Açıklama
Satır sayısı	20	10 / 20 / 50 / 100 / 500
Filtre	boş	Alt dize metin araması (Enter'a basılınca uygulanır)
Direct (onay kutusu)	açık	Doğrudan bağlantıları göster (freedom-outbound üzerinden geçen trafik)
Blocked (onay kutusu)	açık	Engellenmiş bağlantıları göster (blackhole-outbound'a giden trafik)
Proxy (onay kutusu)	açık	Proxy trafiğini göster

Olay türü, log satırındaki giden bağlantı etiketine göre otomatik olarak belirlenir: freedom etiketlerine eşleşme → «DIRECT» (yeşil), blackhole → «BLOCKED» (kırmızı), diğerleri → «PROXY» (mavi). `api` → `api` satırları ve boş satırlar atlanır.

Bu pencerenin kayıt göstermesi için Xray'de dosya yoluna sahip **erişim günlüğü** etkinleştirilmiş olmalıdır (`none` değil) – aşağıya bakın. Erişim logu devre dışıysa veya dosya erişilemezse pencere boş kalır («No Record...»).

16.3. Xray Loglama Seviyesi ve Yapılandırması

Xray'in kendi günlükleme parametreleri «Xray Yapılandırmaları» sayfasındaki «Log» bloğunda ayarlanır; bu blokta şu uyarı bulunur:

«Loglar sunucu performansını yavaşlatabilir. Yalnızca gerekli durumlarda ihtiyaç duyduğunuz log türlerini etkinleştirin!»

Alan	Çeviri	Varsayılan Değer	Açıklama
Loglama seviyesi (<code>LogLevel</code>)	Log Level	warning	Xray hata loglarının ayrıntı düzeyi. Geçerli değerler: <code>debug</code> , <code>info</code> , <code>notice</code> , <code>warning</code> , <code>error</code> . İpucu: «Kaydedilmesi gereken bilgiyi belirten hata günlükleri için günlük düzeyi.»
Erişim logları (<code>accessLog</code>)	Access Log	none	Erişim günlüğü dosya yolu. <code>none</code> özel değeri erişim loglarını devre dışı bırakır. İpucu: «Erişim günlüğü dosyasının yolu. «none» özel değeri erişim loglarını devre dışı bırakır.»
Hata logları (<code>errorLog</code>)	Error Log	boş (varsayılan yol)	Hata logları dosya yolu; <code>none</code> devre dışı bırakır. İpucu: «Hata günlüğü dosyasının yolu. «none» özel değeri hata loglarını devre dışı bırakır.»
DNS logları (<code>dnsLog</code>)	DNS Log	false (kapalı)	DNS sorgu günlüğünü etkinleştir. İpucu: «DNS sorgu günlüklerini etkinleştir.»

Alan	Çeviri	Varsayılan Değer	Açıklama
Adres maskeleyme (<code>maskAddress</code>)	Mask Address	boş (kapalı)	Etkinleştirildiğinde gerçek IP adresi loglarda otomatik olarak maskeleyme adresiyle değiştirilir. İpucu: «Etkinleştirildiğinde gerçek IP adresi loglarda maskeleyme adresiyle değiştirilir.»

Varsayılan olarak «**Erişim logları**» = **none** olduğundan «Xray Logları» penceresi (bölüm 16.2) başlangıçta boştur. Çalışır hale getirmek için burada erişim logu yolunu girin ve Xray'i yeniden başlatın.

Dikkat: boş erişim logu yalnızca bu pencereyi etkiler. «Dashboard»daki çevrimiçi istemciler listesi ve istemci formundaki IP sayısı sınırı **erişim loguna bağlı değildir** – panel, çevrimiçi istemcileri ve IP adreslerini Xray çekirdeğinin online-stats API'si (bağlantı istatistikleri) aracılığıyla belirler. Bu API'nin bulunmadığı eski çekirdek sürümlerinde panel otomatik olarak eski yönteme (erişim logu okuma) geri döner ve bu durumda IP sınırı için burada erişim logu yolu hâlâ gereklidir.

IP sayısı sınırı ve fail2ban. İstemcideki IP sayısı sınırı (istemci formundaki ve toplu ekleme sırasındaki «IP Limit» alanı) sunucuda yalnızca **fail2ban** kuruluysa uygulanır – limiti aşan adresleri fail2ban engeller. Panel fail2ban varlığını kontrol eder (GET /panel/api/server/fail2banStatus); yoksa «IP Limit» alanı açıklayıcı bir ipucuyla (Windows'ta ayrı bir iletiyle) devre dışı bırakılır; bu tür sunucularda daha önce ayarlanmış limitler, hiçbir etkisi olmadığı için otomatik olarak sıfırlanır. fail2ban engellemesi hem TCP hem de UDP için geçerlidir. Normal sunucularda fail2ban artık panel kurulumu ve güncellemesi sırasında otomatik olarak kurulup yapılandırılır (bkz. bölüm 16.5).

Örnek: «Xray Logları» penceresinin kayıt göstermesini sağlayan Log bloğu. Xray JSON yapılandırmasında bu şöyle görünür:

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

Asıl yapılması gereken `"access": "none"` değerini bir dosya yoluyla değiştirmektir (örneğin `"./access.log"`). Kaydedip Xray'i yeniden başlattıktan sonra «Xray Logları» penceresindeki tablo satırlarla dolacaktır.

16.4. Xray Yönetimi: Durdurma ve Yeniden Başlatma

Xray'in durumu «Dashboard»daki Xray kartından yönetilir. Mevcut durum şu değerlerden biriyle gösterilir:

Çalışıyor (Running), **Durduruldu** (Stopped), **Bilinmiyor** (Unknown), **Hata** (Error). Hata durumunda «Xray başlatılırken hata oluştu» araç ipucu kullanılabilir.

Düğme	Çeviri	Uç Nokta	İşlem
Durdur	Stop	POST /panel/api/server/stopXrayService	Xray işlemini durdurur. Başarı durumunda «Xray service has been stopped» uyarı bildirimi gösterilir.
Yeniden Başlat	Restart	POST /panel/api/server/restartXrayService	Xray'i mevcut yapılandırmayı uygulayarak yeniden başlatır (veya başlatır). Başarı durumunda «Xray service has been restarted successfully» bildirimi gösterilir.

Her iki işlemten sonra panel WebSocket üzerinden yeni durumu yayınlar; bu nedenle «Dashboard»daki durum sayfa yenilenmeden güncellenir. İşlem hatayla tamamlanırsa Xray durumu «Hata» olur ve hata metni bildirim eklenir.

Manuel yeniden başlatmanın yanı sıra panel, Xray'in yeniden başlatılması gerekip gerekmediğini (her 30 saniyede bir arka plan görevi) ve işlemin çöküp çökmediğini (saniyede bir kontrol) kendiliğinden kontrol eder – bkz. bölüm 16.6.

16.5. Paneli Yeniden Başlatma ve Güncelleme

Paneli Yeniden Başlatma

«Panel Ayarları» sayfasında «**Paneli Yeniden Başlat**» (Restart Panel , POST /panel/api/setting/restartPanel) eylemi bulunur. Onaylandığında panel **3 saniye sonra** yeniden başlatılır.

İletiler:

- Onay: «Paneli yeniden başlatmak istediğinizden emin misiniz? Onaylayın; yeniden başlatma 3 saniye içinde gerçekleşecek. Panel erişilemez hale gelirse sunucu logunu kontrol edin.»
- Başarı: «Panel başarıyla yeniden başlatıldı».

Teknik olarak Linux'ta yeniden başlatma, panel işlemine SIGHUP sinyali gönderilerek (veya kayıtlı kanca aracılığıyla) gerçekleştirilir. Windows'ta SIGHUP gönderimi desteklenmez.

Panel Otomatik Güncellemesi (Update Panel)

«Dashboard»da «**Paneli Güncelle**» (Update Panel) işlevi mevcuttur – 3X-UI'yi doğrudan web arayüzünden en son sürüme güncelleme.

Güncelleme öncesinde panel sürümleri karşılaştırır (GET /panel/api/server/getPanelUpdateInfo) ve GitHub'dan 3x-ui'nin son sürümünü sorgular:

Alan	Çeviri
Mevcut panel sürümü	Current panel version
En son panel sürümü	Latest panel version
Panel güncel / «Güncel»	Panel is up to date / Up to date – yeni sürüm yoksa gösterilir

Güncellemeyi başlatmak için `POST /panel/api/server/updatePanel` . Onay iletişim kutusu:

- «Paneli gerçekten güncellemek istiyor musunuz?»
- «Bu işlem 3X-UI'yi #version# sürümüne güncelleyecek ve panel servisini yeniden başlatacaktır.»

Başlatmanın ardından «Panel update started» açılır bildirimi görüntülenir; sürüm kontrolü başarısız olursa «Panel update check failed» iletisi gösterilir.

Sunucuda ne olur: Otomatik güncelleme **yalnızca Linux'ta** desteklenir (diğer işletim sistemlerinde «panel web update is supported only on Linux installations» hatası döner). Panel, GitHub'dan (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`) resmi `update.sh` betiğini indirir ve ayrı bir işlemde çalıştırır: tercihen ayrı bir systemd biriminde `systemd-run` aracılığıyla (`x-ui-web-update-<timestamp>`), systemd yoksa ayrı bağımsız bir işlem olarak. Betik tamamlandığında bileşenleri günceller ve panel servisini yeniden başlatır. Çalıştırmak için `bash` gereklidir.

Güncelleme sırasında betik yeni rastgele bir web paneli temel yolu (Web Base Path) oluşturduysa `x-ui` servisi otomatik olarak yeniden başlatılır; böylece yeni yol hemen devreye girer. (Yeniden başlatma olmadan sunucu eski yolu sunmaya devam eder ve yeni adres manuel yeniden başlatmaya kadar erişilemez olurdu.)

Düğümelerde (node'larda) aynı 3x-ui paneli, `POST /panel/api/nodes/updatePanel` aracılığıyla merkezi olarak güncellenir – bkz. düğümler bölümü.

fail2ban Otomatik Kurulumu

İstemcilerdeki IP sayısı sınırının (bölüm 16.3) kutudan çıkar çıkmaz çalışması için, normal sunucularda panel kurulumu ve güncellemesinde `fail2ban` artık otomatik olarak kurulup yapılandırılır (önceden bu yalnızca Docker görüntüsünde gerçekleşiyordu). Davranış `XUI_ENABLE_FAIL2BAN` ortam değişkeniyle yönetilir: değişken tanımlanmamışsa veya `true` ise yapılandırma yapılır. Manuel çalıştırma `x-ui setup-fail2ban` komutuyla mümkündür. `fail2ban` yapılandırmasındaki bir hata panel kurulumunu veya güncellemesini durdurmaz.

Yalnızca IPv6 Olan Sunucularda Kurulum ve Güncelleme

`install.sh` ve `update.sh` betikleri artık yalnızca IPv6'ya sahip sunucularda da düzgün çalışır: sürüm, `x-ui.sh` betiği ve servis dosyaları indirme işlemleri artık zorla IPv4 (`curl -4`) kullanmaz; mevcut protokolü alır. Bu sayede panel IPv4 adresi olmayan bir sunucuya da kurulup güncellenebilir.

XUI_PORT Değişkeniyle Panel Portunun Geçersiz Kılınması

Web panelinin dinleme portu `XUI_PORT` ortam değişkeniyle geçersiz kılınabilir – bu değişken yalnızca mevcut işlemin çalışma süresi boyunca geçerlidir ve veritabanındaki kayıtlı `webPort` değerini **değiştirmez**. `1` ile `65535` arasındaki değerler geçerlidir; boş, hatalı veya aralık dışı değerler loga uyarı yazılarak yok sayılır (`webPort` kullanılır). Bu, özellikle Docker'da dağıtım sırasında kullanışlıdır: bridge ağı kullanılırken konteynerin yayımlanan portu `XUI_PORT` ile eşleşmelidir – örneğin `XUI_PORT=8080` ve `ports: "8080:8080"`.

Xray-core Güncelleme ve Sürüm Değiştirme

«Dashboard»da panel güncellemesinden bağımsız olarak Xray-core sürümü de yönetilebilir.

- **Xray Güncellemeleri** (`Xray Updates`) / **Sürüm Seçimi** (`Version`) – mevcut sürümlerin açılır listesi. İpuçları: «İstediğiniz sürümü seçin» ve «Önemli: eski sürümler mevcut ayarları desteklemeyebilir» uyarısı.
- Sürüm kurma/değiştirme – `POST /panel/api/server/installXray/{version}`. İletişim kutusu: «Xray sürümünü değiştir» / «Xray sürümünü değiştirmek istediğinizden emin misiniz?». Başarı durumunda «Xray başarıyla güncellendi».

Örnek: Xray-core sürümünü API isteğiyle değiştirme. Sürüm, XTLS/Xray-core'daki yayın etiketiyle belirtilir (`v` örneğiyle). Örneğin `v1.8.24` sürümüne geçmek için:

```
curl -s -b cookies.txt -X POST \
  https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

(`cookies.txt` – bölüm 16.1'deki örnekteki cookie dosyası.) Kurulumdan sonra Xray seçilen sürümle otomatik olarak yeniden başlatılır.

Sunucuda sürüm değiştirilirken önce Xray durdurulur, GitHub'dan (XTLS/Xray-core) istenen sürümün arşivi indirilir, ikili dosya çıkarılıp değiştirilir; ardından arşiv/ikili dosyanın kontrol boyutları doğrulanarak Xray yeniden başlatılır.

16.6. Periyodik Görevler (cron)

Panel başlatıldığında bir dizi arka plan görevi kaydeder. Zamanlamaları sabittir (Telegram raporu zamanlaması ve LDAP senkronizasyonu dışında UI'dan yapılandırılmaz). Aşağıda işletimle ilgili görevler yer almaktadır.

Görev	Zamanlama	Amaç
Xray çalışma denetimi	her 1 s	Xray işleminin çalışıp çalışmadığını kontrol eder
Xray yeniden başlatma gereksinimi denetimi	her 30 s	Yapılandırma değiştirilmiş olarak işaretlendiyse yeniden başlatır
Xray trafik toplama	her 5 s (başlatmadan 5 s sonra başlar)	inbound/istemci trafiğini muhasebeleştirir
İstemci IP denetimi	her 10 s	Loga göre IP limitini kontrol eder
Düğüm heartbeat ve trafik senkronizasyonu	her 5 s	Düğümlemlerle (node'larla) iletişim kurar

Görev	Zamanlama	Amaç
Log temizleme	günlük (@daily)	IP-limit loglarını ve kalıcı erişim logunu temizler; mevcut logu *.prev.log olarak döndürür
Periyoda göre trafik sıfırlama	@hourly , @daily , @weekly , @monthly	Karşılık gelen otomatik sıfırlama periyodu ayarlı inbound'ların (ve istemcilerinin) trafik sayaçlarını sıfırlar
Telegram raporu	bot ayarlarında belirlenir (varsayılan @daily)	Yöneticilere rapor gönderir; seçenek etkinse veritabanı yedeğini ekler (bölüm 16.1)
Telegram hash deposu sıfırlama	her 2 d	Yalnızca bot etkinken
Telegram için CPU yükü kontrolü	her 10 s	Yalnızca CPU eşiği > 0 ayarlandığında

Ek bilgiler:

- **Periyodik trafik sıfırlama** yalnızca karşılık gelen otomatik sıfırlama modu (saatlik/günlük/haftalık/aylık) seçilmiş inbound'lar için devreye girer. Görev, inbound'un kendisinin ve tüm istemcilerinin trafiğini sıfırlar.
- **Sona erme ve tükenme denetimi.** Süresi dolmuş veya trafik limiti aşılmış istemcilerin devre dışı bırakılması trafik muhasebesi kapsamında gerçekleştirilir: expiry_time dolmuş veya hacmi tükenmiş istemciler işaretlenerek devre dışı bırakılır; gerektiğinde bir sonraki dönem hesaplanır (döngüsel limitler ve «ilk kullanımdan itibaren sayım» modu için). «Dashboard»da ve listelerde bu durum «Süresi Doldu»/«Tükendi»/«Yakında Bitecek» olarak yansır.
- **Telegram'a otomatik yedek** – rapor görevinin yan etkisidir; yalnızca yedek için ayrı bir cron zamanlaması yoktur. Bu nedenle otomatik yedek sıklığı bot rapor sıklığıyla aynıdır.

16.7. Konsol Menüsü ve CLI (x-ui)

Sunucuda panel x-ui komutuyla yönetilir. Argümansız çalıştırıldığında «3X-UI Panel Management Script» interaktif menüsü açılır; argümanla çalıştırıldığında belirli bir alt komut çalışır. İşletimle ilgili menü öğeleri:

Menü No	Öge	İşlem
1	Install	Paneli kur (install.sh indirir ve çalıştırır)
2	Update	Veri kaybetmeden tüm x-ui bileşenlerini son sürüme güncelle; ardından otomatik yeniden başlatma
3	Update Menu	Yalnızca x-ui menü betiğini güncelle
4	Legacy Version	Girilen numaraya (örneğin 2.4.0) göre belirtilen (eski) panel sürümünü kur
5	Uninstall	Paneli ve Xray'i tamamen kaldır (bkz. 16.8)
6	Reset Username & Password	Yönetici kullanıcı adını/parolasını sıfırla
7	Reset Web Base Path	Web paneli temel yolunu sıfırla
8	Reset Settings	Ayarları varsayılan değerlere sıfırla

Menü No	Öge	İşlem
9	Change Port	Panel portunu değiştir
10	View Current Settings	Mevcut ayarları görüntüle
11–13	Start / Stop / Restart	Panel servisini başlat, durdur, yeniden başlat
14	Restart Xray	Yalnızca Xray'i yeniden başlat
15	Check Status	Mevcut servis durumu
16	Logs Management	Logları görüntüle ve temizle (aşağıya bakın)
17–18	Enable / Disable Autostart	İşletim sistemi başlangıcında servis otomatik başlatmayı etkinleştir/devre dışı bırak
25	Update Geo Files	Coğrafi dosyaları güncelle (GeoIP/GeoSite)
27	PostgreSQL Management	PostgreSQL yönetimi

CLI'da Log Yönetimi (16. öge)

«Logs Management» alt menüsünde:

- **Debug Log** – servis günlüğünün akışını görüntüle: `journalctl -u x-ui -e --no-pager -f -p debug` (Alpine'de – `/var/log/messages` üzerinde `grep`).
- **Clear All logs** – sistem günlüğünü temizle: `journalctl --rotate + journalctl --vacuum-time=1s`, ardından servis yeniden başlatılır. (Alpine'de kullanılamaz.)

Doğrudan x-ui Alt Komutları

Mevcut tüm alt komutlar:

Komut	Açıklama
<code>x-ui</code>	Yönetim menüsünü aç
<code>x-ui start</code>	Paneli başlat
<code>x-ui stop</code>	Paneli durdur
<code>x-ui restart</code>	Paneli yeniden başlat
<code>x-ui restart-xray</code>	Xray'i yeniden başlat
<code>x-ui status</code>	Mevcut durum
<code>x-ui settings</code>	Mevcut ayarları göster
<code>x-ui enable</code>	İşletim sistemi başlangıcında otomatik başlatmayı etkinleştir
<code>x-ui disable</code>	Otomatik başlatmayı devre dışı bırak
<code>x-ui log</code>	Logları görüntüle
<code>x-ui banlog</code>	Fail2ban engelleme loglarını görüntüle

Komut	Açıklama
<code>x-ui setup-fail2ban</code>	IP limiti için fail2ban'ı kur ve yapılandır (bkz. 16.5)
<code>x-ui update</code>	Paneli güncelle
<code>x-ui update-all-geofiles</code>	Tüm coğrafi dosyaları güncelle (ardından yeniden başlatma yapılır)
<code>x-ui migrateDB [file]</code>	<code>.db</code> $\Rightarrow$ <code>.dump</code> veritabanı dönüşümü (SQLite)
<code>x-ui legacy</code>	Eski bir sürüm kur
<code>x-ui install</code>	Paneli kur
<code>x-ui uninstall</code>	Paneli kaldır

`x-ui update` komutu resmi `update.sh` betiğini indirir ve çalıştırır (bölüm 16.5'teki web güncellemesiyle aynıdır); onay ister: «This function will update all x-ui components to the latest version, and the data will not be lost.» Tamamlanınca panel otomatik olarak yeniden başlatılır.

setting alt komutundaki `-webCert` / `-webCertKey` bayrakları. Web panelinin sertifika ve özel anahtar yolları doğrudan `x-ui setting -webCert <yol> -webCertKey <yol>` alt komutuyla ayarlanabilir – bu bayraklardan herhangi birinin belirtilmesi ilgili yolu kaydeder (ayrı `cert` alt komutuyla aynı şekilde) ve panel hemen HTTPS'ye geçer.

CLI Üzerinden API Token Alma

CLI üzerinden API tokeni alma komutu (menü öğesi/ `x-ui` komutu) daha önce verilmiş tokeni göstermez. API tokenleri yalnızca hash biçiminde saklandığından mevcut bir token düz metin olarak alınamaz. Tokenler zaten yapılandırılmışsa komut sayılarını bildirir, tokenleri panelden yönetmenizi önerir (**Settings** → **API Tokens**, bkz. API tokenleri bölümü) ve `cli-fallback-<timestamp>` adıyla yeni bir yedek token üretip arayüze girmeden CLI'nın kullanışlı kalmasını sağlayacak şekilde ekrana yazar.

16.8. Paneli Kaldırma

Kaldırma işlemi CLI'dan yapılır – **5 (Uninstall)** menü öğesi veya `x-ui uninstall` komutu. Kaldırma öncesinde onay istenir (varsayılan «hayır»): «Are you sure you want to uninstall the panel? xray will also be uninstalled!».

Onaylandığında betik:

1. Servisi durdurur ve otomatik başlatmayı devre dışı bırakır (`systemctl stop/disable x-ui` ,Alpine'de `-rc-service / rc-update`),servis birim dosyasını siler ve systemd yapılandırmasını yeniden yükler.
2. Veri ve uygulama izinlerini (`/etc/x-ui/` , kurulum dizini) ve servis env dosyasını siler (`/etc/default/x-ui` , `/etc/conf.d/x-ui` veya `/etc/sysconfig/x-ui` – dağıtıma göre).
3. `x-ui` betiğinin kendisini siler ve «Uninstalled Successfully.» iletilisiyle birlikte yeniden kurulum komutunu gösterir.

Kaldırma geri alınamaz: panelle birlikte Xray ve tüm veriler (veritabanı dahil) silinir. Veriler gerekebilirse önceden veritabanı dışa aktarması yapın (bölüm 16.1).

16.9. x-ui migrateDB Komutu

3.3.0 sürümünden itibaren `x-ui.sh` yönetim betiği, panel veritabanını SQLite ikili `.db` formatı ile taşınabilir metin dumpı `.dump` (düz SQL metni) arasında dönüştürmek için dahili `x-ui` ikili dosyasını (`x-ui migrate-db`) sarmalayan `migrateDB` alt komutunu almıştır.

Komut Ne Yapar

Komut iki yönde çalışır; yön **otomatik olarak** giriş dosyasına göre belirlenir:

Yön	Nasıl Adlandırılır	Ne Olur
<code>.db → .dump</code>	dump (dışa aktarma)	İkili SQLite veritabanı metin SQL dosyasına aktarılır
<code>.dump → .db</code>	restore (geri yükleme)	Metin SQL dosyasından ikili SQLite veritabanı yeniden oluşturulur

Arka planda betik panel ikilisini çağırır:

- dışa aktarma: `x-ui migrate-db --src <giriş> --dump <çıkış>`
- geri yükleme: `x-ui migrate-db --restore <giriş> --out <çıkış>`

Çağırma Sözdizimi

```
x-ui migrateDB [file.db|file.dump] [output]
```

- [file.db|file.dump]** – giriş dosyası (birinci argüman). Belirtilmezse panelin varsayılan kurulu veritabanı kullanılır: `/etc/x-ui/x-ui.db`.
- [output]** – çıkış dosyasının yolu (ikinci argüman). İsteğe bağlıdır: belirtilmezse ad giriş dosyasının yanında otomatik olarak seçilir (aşağıya bakın).

Örnekler:

```
x-ui migrateDB                                # /etc/x-ui/x-ui.db -> /etc/x-ui/x-ui.dump
olarak dışa aktar
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db        # dumptan .db oluştur
```

Yön Nasıl Belirlenir

Betik giriş dosyasının uzantısına bakar:

- `*.db`, `*.sqlite`, `*.sqlite3` → **dump** modu (metne dışa aktarma);
- `*.dump`, `*.sql` → **restore** modu (veritabanı oluşturma).

Uzantı tanınmazsa betik dosyanın ilk 16 baytını okur: SQLite format 3 imzası ikili veritabanı anlamına gelir (dump modu), aksi hâlde dosya dump olarak kabul edilir (restore modu).

İkinci argüman belirtilmediğinde çıkış dosyası adı:

- dış aktarmada – giriş dosyasıyla aynı ad, .dump uzantısıyla;
- geri yüklemeye – aynı ad, .db uzantısıyla.

Koruyucu Denetimler ve Davranış

- **İkili dosya varlığı.** x-ui ikili dosyası bulunamazsa veya çalıştırılmaz değilse «x-ui binary not found ... Is the panel installed?» hatası gösterilir.
- **Sürümdeki özellik desteği.** Betik, ikili dosyanın migrate-db --dump/--restore özelliğini desteklediğini denetler (x-ui migrate-db -h aracılığıyla). Desteklemiyorsa önce x-ui update komutuyla paneli güncellemeniz önerilir.
- **Giriş dosyasının varlığı.** Giriş dosyası yoksa hata ve sözdizimi satırı yazdırılır.
- **Çıkışın üzerine yazma.** Çıkış dosyası zaten varsa onay istenir (varsayılan «hayır»); onay verilmezse işlem iptal edilir. Geri yüklemeye eski çıkış dosyası önceden silinir.
- **«Canlı» veritabanı koruması.** Panel çalışırken /etc/x-ui/x-ui.db varsayılan veritabanına geri yükleme yapılmaya çalışıldığında işlem reddedilir; önce paneli durdurmanız (x-ui stop) veya farklı bir çıkış yolu seçmeniz istenir. Bu, çalışan bir servisin aktif veritabanının üzerine yazılmasını önler.
- Veritabanı oluşturma başarısız olursa yarım kalan çıkış dosyası silinir.

Neden Kullanılır

- **Yedek.** Metin .dump – insan tarafından okunabilir; sürüm kontrol sistemlerinde depolamak ve veritabanı içeriğini fark olarak incelemek için uygundur.
- **Taşıma.** Dump makineler arasında taşınabilir ve SQLite dosya formatı sürüm farklılıklarına karşı dayanıklıdır – yeni bir sunucuda çalışan bir .db oluşturulabilir.
- **Tanımlama.** .dump dosyasından SQLite araçları el altında olmasa bile panel yapısı ve verileri incelenebilir.

Etkileşimli Mod

Doğrudan çağrının yanı sıra dönüştürme etkileşimli menüden de yapılabilir. PostgreSQL alt menüsünde (x-ui → PostgreSQL ile çalışma bölümü) **9. Convert SQLite .db <-> .dump** ögesi bulunur: giriş dosyasının yolunu (varsayılan /etc/x-ui/x-ui.db) ve çıkış dosyasının yolunu sorar (otomatik adlandırma için boş bırakılabilir); yön ise CLI modunda olduğu gibi otomatik belirlenir.

Bu belge 3X-UI kaynak koduna dayanılarak hazırlanmıştır. Arayüzün herhangi bir ögesi kullandığınız sürümde farklıysa panel davranışı ve UI'daki ipuçları önceliklidir.